

Τεχνική τεκμηρίωση κώδικα

Διπλωματική εργασία: Πρόβλεψη της ηλεκτρικής κατανάλωσης κτηρίων με χρήση τεχνικών μηχανικής μάθησης

Το παρόν αρχείο αποτελεί συνοδευτική τεχνική τεκμηρίωση του κώδικα που αναπτύχθηκε στο πλαίσιο της διπλωματικής εργασίας. Περιλαμβάνει συνοπτική και αναλυτική περιγραφή των βασικών αρχείων Jupyter Notebook (.ipynb) και Python (.py) που χρησιμοποιήθηκαν για την προεπεξεργασία δεδομένων, τη διερευνητική ανάλυση, την εκπαίδευση μοντέλων μηχανικής μάθησης, την αξιολόγηση αποτελεσμάτων και την ανάπτυξη της εφαρμογής πρόβλεψης μέσω API και γραφικής διεπαφής χρήστη.

Η τεκμηρίωση οργανώνεται με βάση τα δύο σύνολα δεδομένων που χρησιμοποιήθηκαν στην εργασία, το IDEAL και το PLEGMA. Για κάθε σύνολο δεδομένων παρουσιάζονται τα βασικά αρχεία υλοποίησης, ο ρόλος τους στη συνολική ροή της μεθοδολογίας και η σύνδεσή τους με τα αντίστοιχα στάδια της διπλωματικής εργασίας.

Η αναπαραγωγή της συνολικής διαδικασίας προϋποθέτει την ύπαρξη των αντίστοιχων συνόλων δεδομένων, την εκτέλεση των scripts με τη σωστή σειρά και τη διατήρηση ή κατάλληλη προσαρμογή των διαδρομών αρχείων που χρησιμοποιούνται στον κώδικα. Ορισμένα τμήματα της υλοποίησης χρησιμοποιούν σταθερά μονοπάτια για την πρόσβαση σε δεδομένα, την αποθήκευση ενδιάμεσων αρχείων και τη φόρτωση εκπαιδευμένων μοντέλων. Για τον λόγο αυτό, η μη τήρηση της προτεινόμενης δομής φακέλων ενδέχεται να οδηγήσει σε σφάλματα κατά την εκτέλεση.

ΠΕΡΙΕΧΟΜΕΝΑ

IDEAL. Σύντομη περιγραφή αρχείων ipynb.....	2
Προεπεξεργασία.....	2
Εξαγωγή χαρακτηριστικών - Exploratory Data Analysis (EDA)	3
Baseline Μοντέλα	4
Machine learning scripts	5
IDEAL. Σύντομη περιγραφή αρχείων py.....	9
Αξιολόγηση αποτελεσμάτων IDEAL demo.....	16
PLEGMA. Σύντομη περιγραφή αρχείων ipynb	20
Προεπεξεργασία.....	20
EDA_PLEGMA	23
Baselines_PLEGMA	23
ML Scripts.....	25
PLEGMA. API, UI.....	30
Σύντομη περιγραφή αρχείων py.....	30
Evaluation Scripts (ipynb).....	32
Τεχνική περιγραφή υλοποίησης συστήματος πρόβλεψης IDEAL.....	34
Τεχνική ανάλυση κώδικα για την μηχανική μάθηση	34
Εκπαίδευση Μοντέλων και Παραγωγή Artifacts.....	36
Τεχνική ανάλυση του API και του Gradio UI.....	72
Υλοποίηση API- UI.....	77

IDEAL. Σύντομη περιγραφή αρχείων ipynb

Προεπεξεργασία

IDEAL_Preproccession

Παίρνει τις ωριαίες μετρήσεις του IDEAL (σε csv.gz) και φτιάχνει ένα ενιαίο αρχείο hourly.CSV ανά σπίτι, που περιέχει:

consumption_Wh (κατανάλωση από electric mains), internal_temperature (μέση εσωτερική θερμοκρασία από όλα τα room_temperature sensors), external_temperature (κενό προς το παρόν).

Το house233 αποκλείστηκε λόγω μεγάλης έλλειψης δεδομένων τα οποία εμφανίστηκαν με την εκτέλεση του επόμενου script.

IDEAL_NaN_reports

Κάνει ποσοτικό έλεγχο ποιότητας δεδομένων μετά την προεπεξεργασία και παράγει συγκεντρωτικό report NaN ανά σπίτι για όλα τα αρχεία hourly.CSV του IDEAL.

IDEAL_NaN_filler

Συμπληρώνει τις ελλείπουσες τιμές της εσωτερικής θερμοκρασίας στα ωριαία δεδομένα του IDEAL ανά σπίτι. Για σπίτια με πολλά κενά εφαρμόζεται αρχικά εποχική συμπλήρωση από το ίδιο χρονικό στιγμιότυπο του επόμενου έτους, ώστε να διατηρηθεί η εποχικότητα, και στη συνέχεια χρονική παρεμβολή για τα εναπομείναντα NaN. Για σπίτια με λίγα κενά εφαρμόζεται μόνο χρονική παρεμβολή.

IDEAL_metadata_aggregations

Συγκεντρώνει τα μεταδεδομένα σε επίπεδο δωματίου του συνόλου IDEAL σε χαρακτηριστικά σε επίπεδο κατοικίας, υπολογίζοντας συνολικό και μέσο εμβαδό, αριθμό δωματίων, εξωτερικών παραθύρων και τοίχων. Τα εμβαδά κανονικοποιούνται στην κλίμακα m² και αποθηκεύονται σε ένα ενιαίο αρχείο μεταδεδομένων ανά σπίτι.

IDEAL_import_ext_temp

Αντλεί ωριαία δεδομένα εξωτερικής θερμοκρασίας για την περιοχή του IDEAL από το Open-Meteo Archive API και τα αποθηκεύει σε ενιαίο αρχείο CSV. Αντλήθηκαν ιστορικά ωριαία δεδομένα θερμοκρασίας μόνο από την περιοχή του Εδιμβούργου καθώς τα περισσότερα σπίτια είναι από εκείνη την περιοχή.

IDEAL_merge_ext_temp

Φορτώνει τα ωριαία δεδομένα εξωτερικής θερμοκρασίας και τα ενώνει με τα συμπληρωμένα hourly datasets κάθε σπιτιού μέσω timestamp. Η κενή στήλη εξωτερικής θερμοκρασίας αντικαθίσταται με την πραγματική, γίνεται στρογγυλοποίηση και αποθηκεύονται τα τελικά αρχεία ανά σπίτι.

IDEAL_Master_Dataset

Συνενώνει τα τελικά ωριαία datasets όλων των κατοικιών σε ένα ενιαίο master αρχείο, προσθέτοντας αναγνωριστικό κατοικίας και διατηρώντας τη χρονική σειρά ανά σπίτι.

IDEAL_Merge_aggrmetadata_home

Δημιουργεί έναν ενιαίο πίνακα μεταδεδομένων ανά κατοικία, συνδυάζοντας βασικά χαρακτηριστικά από το home.csv με συγκεντρωτικά (aggregated) χαρακτηριστικά δωματίων, συσκευών και δημογραφικών.

IDEAL_Calendar_features_plus_Training_Set

Δημιουργεί το τελικό ωριαίο dataset για μοντελοποίηση, ενώνοντας τα χρονοσειριακά δεδομένα κατανάλωσης και θερμοκρασιών με στατικά μεταδεδομένα ανά κατοικία μέσω του home_id. Στη συνέχεια παράγει ημερολογιακά χαρακτηριστικά (ώρα, ημέρα εβδομάδας, Σ/Κ) και δείκτη αργίας για τη Σκωτία, εκτελεί βασικούς ελέγχους πληρότητας/ορθότητας του merge και αποθηκεύει το τελικό σύνολο δεδομένων σε αρχείο CSV για χρήση στην εκπαίδευση μοντέλων.

Εξαγωγή χαρακτηριστικών - Exploratory Data Analysis (EDA)

EDA1_new

Το script υλοποιεί μια ολοκληρωμένη διερευνητική ανάλυση (EDA) της ωριαίας ηλεκτρικής κατανάλωσης στο dataset IDEAL, δίνοντας ιδιαίτερη έμφαση στη μείωση της μεροληψίας και στην ενίσχυση της στατιστικής αξιοπιστίας των αποτελεσμάτων. Για τον σκοπό αυτό, αξιοποιούνται τόσο pooled aggregations όσο και home-weighted aggregations, ώστε κάθε κατοικία να συνεισφέρει ισότιμα στα τελικά συμπεράσματα, ανεξάρτητα από το πλήθος των διαθέσιμων μετρήσεων.

Η ανάλυση περιλαμβάνει τη μελέτη του ημερήσιου προφίλ κατανάλωσης, τη σύγκριση μεταξύ καθημερινών και Σαββατοκύριακου, τη διερεύνηση της σχέσης εξωτερικής θερμοκρασίας και κατανάλωσης μέσω ομαδοποίησης τιμών θερμοκρασίας σε διαστήματα του 1°C, με χρήση μέσης και διάμεσης τιμής, καθώς και την εξέταση της χωρικής διαφοροποίησης της κατανάλωσης με χρήση διαγραμμάτων κουτιού (boxplots) ανά περιοχή.

EDA_hometype_new

Το συγκεκριμένο script εστιάζει στη διερεύνηση της κατανάλωσης με βάση τον τύπο κατοικίας. Αρχικά υπολογίζεται η μέση ωριαία κατανάλωση για κάθε κατηγορία κατοικίας, ενώ στη συνέχεια συγκρίνονται τα ημερήσια προφίλ μεταξύ καθημερινών και Σαββατοκύριακου ανά τύπο κατοικίας.

Επιπλέον, εξετάζεται η σχέση εξωτερικής θερμοκρασίας και κατανάλωσης με ομαδοποίηση ανά 1°C και υπολογισμός τόσο της μέσης όσο και της διάμεσης τιμής. Τέλος, μελετάται η κατανομή της μέσης ημερήσιας κατανάλωσης ανά κατοικία μέσω boxplots, διατηρώντας σκόπιμα τις ακραίες τιμές (outliers) ώστε να αποτυπωθεί η πραγματική διασπορά των δεδομένων.

EDA_urban_class_new

Το script αυτό πραγματοποιεί διερευνητική ανάλυση με βάση τη μεταβλητή urban_rural_class, χρησιμοποιώντας το τελικό ωριαίο dataset. Αρχικά γίνεται καθαρισμός της μεταβλητής (συγχώνευση κατηγοριών 3+ σε μία ενιαία κατηγορία), διατηρώντας μόνο έγκυρες τιμές.

Στη συνέχεια, παράγονται συγκρίσεις της μέσης ωριαίας κατανάλωσης ανά κατηγορία, αναλύεται η διαφοροποίηση μεταξύ καθημερινών και Σαββατοκύριακου, εξετάζεται η σχέση κατανάλωσης και εξωτερικής θερμοκρασίας (με bins 1°C και χρήση mean/median), ενώ τέλος παρουσιάζεται η κατανομή της μέσης ημερήσιας κατανάλωσης ανά κατοικία σε kWh/ημέρα.

EDA_residents_new

Στο συγκεκριμένο script η ανάλυση επικεντρώνεται στον αριθμό των ενοίκων της κατοικίας. Χρησιμοποιείται το τελικό ωριαίο dataset και επιλέγονται μόνο οι κατοικίες με διαθέσιμη πληροφορία για τη μεταβλητή residents.

Αρχικά υπολογίζεται η μέση ωριαία κατανάλωση ανά ομάδα ενοίκων με home-weighted προσέγγιση, ώστε να διασφαλίζεται ισότιμη συμβολή κάθε κατοικίας. Στη συνέχεια, εξετάζεται η μέση ημερήσια κατανάλωση ανά σπίτι, με χρήση boxplots για την αποτύπωση της κατανομής.

EDA ανά εμβαδό κατοικίας (EDA_floor_area_new)

Το script αυτό διερευνά τη σχέση κατανάλωσης και μεγέθους κατοικίας. Αρχικά φορτώνεται το τελικό ωριαίο dataset και επιλέγονται μόνο οι κατοικίες με διαθέσιμη πληροφορία για το συνολικό εμβαδό (total_floor_area_m2).

Οι κατοικίες κατηγοριοποιούνται σε Small, Medium και Large, και στη συνέχεια υπολογίζεται η μέση ωριαία κατανάλωση για κάθε κατηγορία. Παράλληλα, εξετάζεται η μέση ημερήσια κατανάλωση ανά κατοικία μέσω boxplots, επιτρέποντας την κατανόηση της διαφοροποίησης μεταξύ διαφορετικών μεγεθών κατοικιών.

Baseline Μοντέλα

IDEAL_Naive

Το συγκεκριμένο script υλοποιεί ένα απλό αλλά ιδιαίτερα σημαντικό baseline μοντέλο, βασισμένο στην υπόθεση persistence. Για κάθε κατοικία, η κατανάλωση σε χρονική στιγμή t προβλέπεται ως ίση με την κατανάλωση της ίδιας ώρας της προηγούμενης ημέρας ($t-24$ ώρες), ελέγχοντας παράλληλα ότι η χρονική απόσταση είναι ακριβώς 24 ώρες.

Η απόδοση του μοντέλου αξιολογείται με τους δείκτες MAE, RMSE και MAPE, τόσο συνολικά όσο και ανά κατοικία, αποτελώντας σημείο αναφοράς για τα πιο σύνθετα μοντέλα.

IDEAL_Naive_baseline2and3

Το script αυτό υλοποιεί δύο επιπλέον naive baselines βασισμένα σε μέσους όρους, τα οποία λειτουργούν ως πιο «στατιστικά» σημεία αναφοράς για σύγκριση με τα μοντέλα μηχανικής μάθησης.

Στο Baseline 02, η πρόβλεψη βασίζεται στον μέσο όρο κατανάλωσης ανά ώρα, υπολογισμένο από όλα τα σπίτια και όλες τις ημέρες. Στο Baseline 03, η προσέγγιση επεκτείνεται με διάκριση μεταξύ καθημερινών και Σαββατοκύριακου, δημιουργώντας διαφορετικούς μέσους όρους.

Και τα δύο μοντέλα αξιολογούνται με MAE, RMSE και MAPE, ενώ τα αποτελέσματα αποθηκεύονται τόσο συνολικά όσο και ανά κατοικία, μαζί με τα αντίστοιχα αρχεία προβλέψεων.

IDEAL_Naive_baseline4

Το τελευταίο baseline script υλοποιεί μια πιο εξατομικευμένη προσέγγιση, όπου η πρόβλεψη γίνεται σε επίπεδο κατοικίας. Για κάθε σπίτι, η κατανάλωση προβλέπεται ως ο μέσος όρος κατανάλωσης της ίδιας ώρας, υπολογισμένος αποκλειστικά από τα ιστορικά δεδομένα του συγκεκριμένου σπιτιού.

Το script είναι βελτιστοποιημένο ως προς τη μνήμη, χρησιμοποιώντας κατάλληλους τύπους δεδομένων και υπολογισμούς. Η αξιολόγηση γίνεται και εδώ με MAE, RMSE και MAPE, τόσο συνολικά όσο και ανά κατοικία, και τα αποτελέσματα αποθηκεύονται σε ξεχωριστά αρχεία.

Machine learning scripts

RF_for_API2

Το συγκεκριμένο script χρησιμοποιείται για την τελική εκπαίδευση και αποθήκευση των μοντέλων Random Forest που αξιοποιούνται από το API της εφαρμογής πρόβλεψης. Η παρούσα υλοποίηση δεν περιορίζεται στην απλή πειραματική εκπαίδευση, αλλά είναι προσανατολισμένη στη δημιουργία των τελικών αρχείων που απαιτούνται για τη λειτουργική χρήση των μοντέλων από την εφαρμογή. Περιλαμβάνει δύο διακριτά μοντέλα: ένα μοντέλο για το σενάριο με διαθέσιμο ιστορικό κατανάλωσης και ένα μοντέλο για το σενάριο χωρίς διαθέσιμο ιστορικό κατανάλωσης.

Το μοντέλο με ιστορικό κατανάλωσης αξιοποιεί εκτεταμένα ιστορικά χαρακτηριστικά, όπως προηγούμενες τιμές κατανάλωσης, κυλιόμενους μέσους όρους και στατιστικά προηγούμενης κατανάλωσης ανά κατοικία. Επιπλέον, χρησιμοποιεί το `home_id` ως κατηγορικό χαρακτηριστικό, καθώς και κατηγοριοποίηση της κατοικίας με βάση το επίπεδο κατανάλωσης. Για λόγους υπολογιστικής αποδοτικότητας, η εκπαίδευση του συγκεκριμένου μοντέλου πραγματοποιείται σε αντιπροσωπευτικό δείγμα του συνόλου εκπαίδευσης. Τα στατιστικά χαρακτηριστικά κάθε κατοικίας υπολογίζονται αποκλειστικά από το training τμήμα των δεδομένων, ώστε να αποφεύγεται η διαρροή πληροφορίας από το test set.

Το μοντέλο χωρίς ιστορικό κατανάλωσης χρησιμοποιείται όταν δεν υπάρχουν προηγούμενες μετρήσεις κατανάλωσης για την κατοικία. Για τον λόγο αυτό δεν χρησιμοποιεί `home_id`, ιστορικές μεταβλητές, προφίλ κοντινότερων κατοικιών ή προφίλ συμπεριφοράς. Αντίθετα, βασίζεται μόνο σε βασικά χρονικά χαρακτηριστικά, στην εξωτερική θερμοκρασία και σε στατικά χαρακτηριστικά του νοικοκυριού, όπως το εμβαδόν, ο αριθμός κατοίκων και ο τύπος κατοικίας. Στην τελική εκπαίδευση του συγκεκριμένου μοντέλου χρησιμοποιούνται όλα τα διαθέσιμα δεδομένα πριν από την περίοδο ελέγχου, συμπεριλαμβανομένων των περιόδων επικύρωσης και βαθμονόμησης. Με αυτόν τον τρόπο, το μοντέλο που αποθηκεύεται για χρήση από το API έχει εκπαιδευτεί στο μεγαλύτερο δυνατό διαθέσιμο σύνολο δεδομένων.

Το script εφαρμόζει χρονικό διαχωρισμό των δεδομένων σε σύνολο εκπαίδευσης, σύνολο επικύρωσης, σύνολο βαθμονόμησης και σύνολο ελέγχου. Παράλληλα, αφαιρεί κατοικίες που έχουν χαρακτηριστεί προβληματικές, δημιουργεί τα απαραίτητα χρονικά και ιστορικά χαρακτηριστικά, εφαρμόζει λογαριθμικό μετασχηματισμό στη μεταβλητή στόχο και αξιολογεί τα μοντέλα με κοινές μετρικές απόδοσης. Οι μετρικές αυτές περιλαμβάνουν το μέσο απόλυτο σφάλμα, τη ρίζα του μέσου τετραγωνικού σφάλματος, το μέσο απόλυτο ποσοστιαίο σφάλμα, το συμμετρικό μέσο απόλυτο ποσοστιαίο σφάλμα, τη μεροληψία σε Wh και το μέσο ημερήσιο απόλυτο σφάλμα σε kWh.

Για το μοντέλο με ιστορικό κατανάλωσης παράγεται επίσης μια προαιρετική διορθωμένη εκδοχή της πρόβλεψης με βάση την κατηγορία κατανάλωσης της κατοικίας. Η βασική πρόβλεψη παραμένει η κύρια έξοδος του μοντέλου, ενώ η διορθωμένη εκδοχή αποθηκεύεται ως εναλλακτική στήλη πρόβλεψης. Στόχος αυτής της πρόσθετης δυνατότητας είναι να εξεταστεί αν μπορεί να μειωθεί το ημερήσιο σφάλμα ή η συστηματική υποεκτίμηση/υπερεκτίμηση σε ορισμένες κατηγορίες κατανάλωσης.

Μετά την ολοκλήρωση της εκπαίδευσης, το script αποθηκεύει όλα τα απαραίτητα αρχεία για τη μετέπειτα χρήση των μοντέλων από το σύστημα πρόβλεψης. Για κάθε σενάριο αποθηκεύονται το εκπαιδευμένο μοντέλο, ο μηχανισμός προεπεξεργασίας, το αρχείο ρυθμίσεων χαρακτηριστικών, τα μεταδεδομένα εκπαίδευσης και οι προβλέψεις του test set. Για το μοντέλο με ιστορικό κατανάλωσης αποθηκεύονται επιπλέον τα στατιστικά ανά κατοικία, τα όρια των κατηγοριών κατανάλωσης και ο μηχανισμός βαθμονόμησης ανά κατηγορία.

Τέλος, δημιουργείται το συνολικό αρχείο `training_summary.json`, το οποίο συγκεντρώνει τις βασικές ρυθμίσεις της εκπαίδευσης, τις διαδρομές των παραγόμενων αρχείων και τις τελικές πληροφορίες των μοντέλων. Με αυτόν τον τρόπο, το `script` εξασφαλίζει συνέπεια ανάμεσα στη φάση εκπαίδευσης και στη φάση πρόβλεψης, ενώ εντάσσει το Random Forest μοντέλο στη συνολική λειτουργική αρχιτεκτονική της εφαρμογής.

XGB_for_API2

Το συγκεκριμένο `script` χρησιμοποιείται για την τελική εκπαίδευση και αποθήκευση των μοντέλων XGBoost που προορίζονται να αξιοποιηθούν από το API και τη γραφική διεπαφή της εφαρμογής. Η βασική ιδιαιτερότητα της συγκεκριμένης υλοποίησης είναι ότι τα μοντέλα έχουν σχεδιαστεί για γενική χρήση από τη διεπαφή, χωρίς να απαιτείται από τον χρήστη να γνωρίζει ή να παρέχει κάποιο συγκεκριμένο `home_id`. Το `home_id` χρησιμοποιείται μόνο εσωτερικά κατά την προετοιμασία των δεδομένων, κυρίως για τον σωστό υπολογισμό των ιστορικών χαρακτηριστικών και για την αξιολόγηση των αποτελεσμάτων ανά κατοικία, αλλά δεν εισάγεται ως χαρακτηριστικό στο τελικό μοντέλο. Το `script` εκπαιδεύει δύο διαφορετικές εκδοχές μοντέλου XGBoost. Η πρώτη αφορά το σενάριο με διαθέσιμο ιστορικό κατανάλωσης (`with_history_generic`). Σε αυτή την περίπτωση, το μοντέλο αξιοποιεί χρονικά, στατικά, περιβαλλοντικά και ιστορικά χαρακτηριστικά. Τα ιστορικά χαρακτηριστικά περιλαμβάνουν προηγούμενες τιμές κατανάλωσης, κυλιόμενους μέσους όρους, κυλιόμενη τυπική απόκλιση, ελάχιστες και μέγιστες τιμές προηγούμενων χρονικών παραθύρων, καθώς και πρόσθετες μεταβλητές που εκφράζουν διαφορές ή λόγους μεταξύ ιστορικών τιμών. Τα χαρακτηριστικά αυτά υπολογίζονται ξεχωριστά για κάθε κατοικία κατά τη φάση της εκπαίδευσης, ώστε να διατηρείται η χρονική συνέχεια των δεδομένων και να αποφεύγεται η ανάμειξη μετρήσεων από διαφορετικές κατοικίες.

Παρότι το `home_id` χρησιμοποιείται για την ορθή δημιουργία των ιστορικών μεταβλητών, δεν χρησιμοποιείται ως είσοδος του μοντέλου. Επίσης, το συγκεκριμένο μοντέλο δεν αξιοποιεί στατιστικά χαρακτηριστικά αποκλειστικά συνδεδεμένα με γνωστή κατοικία ούτε κατηγοριοποίηση κατανάλωσης που θα προϋπέθετε άμεση ταυτοποίηση της κατοικίας. Με αυτόν τον τρόπο, το μοντέλο παραμένει κατάλληλο για χρήση από έναν νέο χρήστη της εφαρμογής, ο οποίος μπορεί να παρέχει πρόσφατο ιστορικό κατανάλωσης, αλλά δεν αντιστοιχεί απαραίτητα σε κάποια κατοικία του αρχικού συνόλου δεδομένων.

Η δεύτερη εκδοχή αφορά το σενάριο χωρίς διαθέσιμο ιστορικό κατανάλωσης (`no_history_optional`). Στην περίπτωση αυτή, το μοντέλο βασίζεται σε στατικά χαρακτηριστικά της κατοικίας, χρονικά χαρακτηριστικά, εξωτερικές και εσωτερικές περιβαλλοντικές μεταβλητές, καθώς και σε πρόσθετα χαρακτηριστικά που προκύπτουν από προφίλ συμπεριφοράς και από προφίλ παρόμοιων κατοικιών. Τα προφίλ συμπεριφοράς υπολογίζουν αναμενόμενα επίπεδα κατανάλωσης με βάση ομάδες κατοικιών που έχουν παρόμοια στατικά χαρακτηριστικά, όπως τύπο κατοικίας, περιοχή, αριθμό κατοίκων, εμβαδόν και αριθμό ηλεκτρικών συσκευών. Παράλληλα, η μέθοδος κοντινότερων γειτόνων χρησιμοποιείται για την εύρεση παρόμοιων κατοικιών στο `training set`, ώστε να εξαχθούν ενδεικτικά πρότυπα κατανάλωσης χωρίς να απαιτείται πραγματικό ιστορικό από τον χρήστη.

Πριν από την εκπαίδευση, το `script` φορτώνει το τελικό ενοποιημένο σύνολο δεδομένων του IDEAL, αφαιρεί κατοικίες που έχουν χαρακτηριστεί προβληματικές και δημιουργεί τα απαραίτητα χρονικά χαρακτηριστικά. Τα χαρακτηριστικά αυτά περιλαμβάνουν την ώρα, την ημέρα της εβδομάδας, τον μήνα, την ημέρα του μήνα, την εβδομάδα του έτους, ένδειξη Σαββατοκύριακου και ένδειξη αργίας. Επιπλέον, δημιουργούνται κυκλικά χρονικά χαρακτηριστικά, ώστε να αποτυπώνεται καλύτερα η περιοδικότητα της κατανάλωσης μέσα

στην ημέρα, την εβδομάδα και το έτος. Η μεταβλητή στόχος μετασχηματίζεται λογαριθμικά, ώστε η εκπαίδευση να πραγματοποιείται σε πιο σταθερό αριθμητικό χώρο και να μειώνεται η επίδραση πολύ μεγάλων τιμών κατανάλωσης.

Ο χρονικός διαχωρισμός των δεδομένων γίνεται σε τέσσερα τμήματα: σύνολο εκπαίδευσης, σύνολο επικύρωσης για πρόωρη διακοπή της εκπαίδευσης, σύνολο βαθμονόμησης και σύνολο ελέγχου. Οι τελευταίες 30 ημέρες χρησιμοποιούνται ως test set, ώστε η αξιολόγηση να προσομοιώνει ρεαλιστικά την πρόβλεψη μελλοντικής κατανάλωσης. Η πρόωρη διακοπή της εκπαίδευσης χρησιμοποιείται για να περιοριστεί η υπερπροσαρμογή του XGBoost, ενώ η περίοδος βαθμονόμησης αξιοποιείται για την προαιρετική διόρθωση της ημερήσιας κλίμακας των προβλέψεων.

Για το μοντέλο με ιστορικό κατανάλωσης παράγεται τόσο η βασική πρόβλεψη όσο και μια προαιρετική ημερήσια βαθμονομημένη εκδοχή. Η βασική πρόβλεψη παραμένει η κύρια έξοδος του μοντέλου, ενώ η βαθμονομημένη εκδοχή εφαρμόζει έναν συνολικό διορθωτικό συντελεστή που υπολογίζεται από την περίοδο βαθμονόμησης. Ο σκοπός αυτής της δεύτερης εκδοχής είναι να μειωθεί πιθανή συστηματική υποεκτίμηση ή υπερεκτίμηση της συνολικής ημερήσιας κατανάλωσης, χωρίς όμως να αντικαθιστά την κύρια πρόβλεψη.

Η αξιολόγηση των μοντέλων πραγματοποιείται με κοινές μετρικές, ώστε τα αποτελέσματα να είναι συγκρίσιμα με τα υπόλοιπα μοντέλα της εργασίας. Συγκεκριμένα, υπολογίζονται το μέσο απόλυτο σφάλμα, η ρίζα του μέσου τετραγωνικού σφάλματος, το μέσο απόλυτο ποσοστιαίο σφάλμα, το συμμετρικό μέσο απόλυτο ποσοστιαίο σφάλμα, η μεροληψία σε Wh και το μέσο ημερήσιο απόλυτο σφάλμα σε kWh. Με αυτόν τον τρόπο αξιολογείται τόσο η ωριαία ακρίβεια των προβλέψεων όσο και η ικανότητα του μοντέλου να εκτιμά σωστά τη συνολική ημερήσια κατανάλωση.

Μετά την ολοκλήρωση της εκπαίδευσης, το script αποθηκεύει τα απαραίτητα συνοδευτικά αρχεία σε ξεχωριστούς φακέλους για το μοντέλο `with_history_generic` και το μοντέλο `no_history_optional`. Τα αρχεία αυτά περιλαμβάνουν το εκπαιδευμένο μοντέλο, τον μηχανισμό προεπεξεργασίας, τις ρυθμίσεις των χαρακτηριστικών, τα μεταδεδομένα εκπαίδευσης, τις προβλέψεις του test set και το συνολικό αρχείο `training_summary.json`. Με αυτόν τον τρόπο εξασφαλίζεται ότι το inference pipeline μπορεί να φορτώσει τα μοντέλα και να εφαρμόσει την ίδια ακριβώς διαδικασία προεπεξεργασίας που χρησιμοποιήθηκε κατά την εκπαίδευση.

LGBM_for_API2

Το συγκεκριμένο script χρησιμοποιείται για την τελική εκπαίδευση και αποθήκευση των μοντέλων LightGBM που προορίζονται να αξιοποιηθούν από το API και τη γραφική διεπαφή της εφαρμογής. Η βασική ιδιαιτερότητα της συγκεκριμένης υλοποίησης είναι ότι τα μοντέλα έχουν σχεδιαστεί για γενική χρήση από τη διεπαφή, χωρίς να απαιτείται από τον τελικό χρήστη να γνωρίζει ή να παρέχει κάποιο συγκεκριμένο `home_id`. Το `home_id` χρησιμοποιείται μόνο εσωτερικά κατά την προετοιμασία των δεδομένων, κυρίως για τον σωστό υπολογισμό των ιστορικών χαρακτηριστικών και για την αξιολόγηση των αποτελεσμάτων ανά κατοικία, αλλά δεν εισάγεται ως χαρακτηριστικό στο τελικό μοντέλο. Το script εκπαιδεύει δύο διαφορετικές εκδοχές μοντέλου LightGBM. Η πρώτη αφορά το σενάριο με διαθέσιμο ιστορικό κατανάλωσης (`with_history_generic`). Σε αυτή την περίπτωση, το μοντέλο αξιοποιεί χρονικά, στατικά, περιβαλλοντικά και ιστορικά χαρακτηριστικά. Τα ιστορικά χαρακτηριστικά περιλαμβάνουν προηγούμενες τιμές κατανάλωσης, κυλιόμενους μέσους όρους, κυλιόμενη τυπική απόκλιση, ελάχιστες και μέγιστες τιμές προηγούμενων χρονικών παραθύρων, καθώς και πρόσθετες μεταβλητές που εκφράζουν διαφορές ή λόγους μεταξύ ιστορικών τιμών. Τα χαρακτηριστικά αυτά υπολογίζονται ξεχωριστά για κάθε κατοικία κατά τη φάση της εκπαίδευσης, ώστε να

διατηρείται η χρονική συνέχεια των δεδομένων και να αποφεύγεται η ανάμειξη μετρήσεων από διαφορετικές κατοικίες.

Παρότι το `home_id` χρησιμοποιείται για τη σωστή δημιουργία των ιστορικών μεταβλητών, δεν χρησιμοποιείται ως είσοδος του μοντέλου. Επίσης, το συγκεκριμένο μοντέλο δεν αξιοποιεί στατιστικά χαρακτηριστικά αποκλειστικά συνδεδεμένα με γνωστή κατοικία ούτε κατηγοριοποίηση κατανάλωσης που θα προϋπέθετε άμεση ταυτοποίηση της κατοικίας. Με αυτόν τον τρόπο, το μοντέλο μπορεί να χρησιμοποιηθεί από έναν νέο χρήστη της εφαρμογής, ο οποίος διαθέτει πρόσφατο ιστορικό κατανάλωσης, χωρίς όμως να αντιστοιχεί απαραίτητα σε κάποια κατοικία του αρχικού συνόλου δεδομένων.

Η δεύτερη εκδοχή αφορά το σενάριο χωρίς διαθέσιμο ιστορικό κατανάλωσης (`no_history_optional`). Στην περίπτωση αυτή, το μοντέλο βασίζεται σε στατικά χαρακτηριστικά της κατοικίας, χρονικά χαρακτηριστικά, εξωτερικές και εσωτερικές περιβαλλοντικές μεταβλητές, καθώς και σε πρόσθετα χαρακτηριστικά που προκύπτουν από προφίλ συμπεριφοράς και από προφίλ παρόμοιων κατοικιών. Τα προφίλ συμπεριφοράς υπολογίζουν αναμενόμενα επίπεδα κατανάλωσης με βάση ομάδες κατοικιών που έχουν παρόμοια στατικά χαρακτηριστικά, όπως τύπο κατοικίας, περιοχή, αριθμό κατοίκων, εμβαδόν και αριθμό ηλεκτρικών συσκευών. Παράλληλα, η μέθοδος κοντινότερων γειτόνων χρησιμοποιείται για την εύρεση παρόμοιων κατοικιών στο σύνολο εκπαίδευσης, ώστε να εξαχθούν ενδεικτικά πρότυπα κατανάλωσης χωρίς να απαιτείται πραγματικό ιστορικό από τον χρήστη.

Πριν από την εκπαίδευση, το script φορτώνει το τελικό ενοποιημένο σύνολο δεδομένων του IDEAL, αφαιρεί κατοικίες που έχουν χαρακτηριστεί προβληματικές και δημιουργεί τα απαραίτητα χρονικά χαρακτηριστικά. Τα χαρακτηριστικά αυτά περιλαμβάνουν την ώρα, την ημέρα της εβδομάδας, τον μήνα, την ημέρα του μήνα, την εβδομάδα του έτους, ένδειξη Σαββατοκύριακου και ένδειξη αργίας. Επιπλέον, δημιουργούνται κυκλικά χρονικά χαρακτηριστικά, ώστε να αποτυπώνεται καλύτερα η περιοδικότητα της κατανάλωσης μέσα στην ημέρα, την εβδομάδα και το έτος.

Η μεταβλητή στόχος μετασχηματίζεται λογαριθμικά, ώστε η εκπαίδευση να πραγματοποιείται σε πιο σταθερό αριθμητικό χώρο και να μειώνεται η επίδραση πολύ μεγάλων τιμών κατανάλωσης. Ο χρονικός διαχωρισμός των δεδομένων γίνεται σε τέσσερα τμήματα: σύνολο εκπαίδευσης, σύνολο επικύρωσης για πρόωρη διακοπή της εκπαίδευσης, σύνολο βαθμονόμησης και σύνολο ελέγχου. Οι τελευταίες 30 ημέρες χρησιμοποιούνται ως test set, ώστε η αξιολόγηση να προσομοιώνει ρεαλιστικά την πρόβλεψη μελλοντικής κατανάλωσης.

Για το μοντέλο με ιστορικό κατανάλωσης παράγεται τόσο η βασική πρόβλεψη όσο και μια προαιρετική ημερήσια βαθμονομημένη εκδοχή. Η βασική πρόβλεψη παραμένει η κύρια έξοδος του μοντέλου, ενώ η βαθμονομημένη εκδοχή εφαρμόζει έναν συνολικό διορθωτικό συντελεστή που υπολογίζεται από την περίοδο βαθμονόμησης. Ο σκοπός αυτής της δεύτερης εκδοχής είναι να μειωθεί πιθανή συστηματική υποεκτίμηση ή υπερεκτίμηση της συνολικής ημερήσιας κατανάλωσης, χωρίς όμως να αντικαθιστά την κύρια πρόβλεψη.

Η αξιολόγηση των μοντέλων πραγματοποιείται με κοινές μετρικές, ώστε τα αποτελέσματα να είναι συγκρίσιμα με τα υπόλοιπα μοντέλα της εργασίας. Συγκεκριμένα, υπολογίζονται το μέσο απόλυτο σφάλμα, η ρίζα του μέσου τετραγωνικού σφάλματος, το μέσο απόλυτο ποσοστιαίο σφάλμα, το συμμετρικό μέσο απόλυτο ποσοστιαίο σφάλμα, η μεροληψία σε Wh και το μέσο ημερήσιο απόλυτο σφάλμα σε kWh. Με αυτόν τον τρόπο αξιολογείται τόσο η ωριαία ακρίβεια των προβλέψεων όσο και η ικανότητα του μοντέλου να εκτιμά σωστά τη συνολική ημερήσια κατανάλωση.

Μετά την ολοκλήρωση της εκπαίδευσης, το script αποθηκεύει τα απαραίτητα συνοδευτικά αρχεία σε ξεχωριστούς φακέλους για το μοντέλο `with_history_generic` και το μοντέλο `no_history_optional`. Τα αρχεία αυτά περιλαμβάνουν το εκπαιδευμένο μοντέλο, τον

μηχανισμό προεπεξεργασίας, τις ρυθμίσεις των χαρακτηριστικών, τα μεταδεδομένα εκπαίδευσης, τις προβλέψεις του test set και το συνολικό αρχείο training_summary.json. Με αυτόν τον τρόπο εξασφαλίζεται ότι το σύστημα πρόβλεψης μπορεί να φορτώσει τα μοντέλα και να εφαρμόσει την ίδια διαδικασία προεπεξεργασίας που χρησιμοποιήθηκε κατά την εκπαίδευση.

Feature_Importance_unified

Το παρόν script διαβάει τα αποθηκευμένα μοντέλα και τις αντίστοιχες λίστες χαρακτηριστικών από τους φακέλους των Random Forest, XGBoost και LightGBM μοντέλων. Για κάθε συνδυασμό μοντέλου και σεναρίου υπολογίζει τη σημασία των χαρακτηριστικών, την κανονικοποιεί σε ποσοστά και αποθηκεύει τα αποτελέσματα σε αρχεία CSV, JSON και PNG για οπτικοποίηση των σημαντικότερων μεταβλητών. Επιπλέον, δημιουργεί ενιαίο συγκεντρωτικό αρχείο με όλες τις τιμές importance ώστε να είναι δυνατή η άμεση σύγκριση μεταξύ των μοντέλων και των δύο σεναρίων.

home100_as_history

Το παρόν script εξάγει τις ωριαίες τιμές κατανάλωσης ενός επιλεγμένου σπιτιού (home100) από το αρχικό σύνολο δεδομένων και κρατά μόνο τις εγγραφές ενός συγκεκριμένου έτους. Στη συνέχεια αναδιαμορφώνει τη χρονοσειρά σε πλήρες ωριαίο πλέγμα, συμπληρώνει τυχόν ελλυπίες τιμές με forward fill και backfill, και μετασχηματίζει τα timestamps ώστε να αντιστοιχούν σε ένα διαφορετικό έτος, διατηρώντας όμως τον ίδιο μήνα, ημέρα και ώρα. Τέλος, αποθηκεύει το παραγόμενο αρχείο σε CSV για χρήση ως ιστορικό υπόδειγμα ή βοηθητικό dataset σε επόμενες αναλύσεις.

IDEAL. Σύντομη περιγραφή αρχείων py

history_store

Το συγκεκριμένο script χρησιμοποιείται για τη φόρτωση, τον καθαρισμό και την ανάκτηση ιστορικών δεδομένων κατανάλωσης από ένα αρχείο CSV. Το αρχείο αυτό αποτελεί την πηγή ιστορικού που μπορεί να χρησιμοποιηθεί από το API και τη γραφική διεπαφή της εφαρμογής, όταν ο χρήστης επιλέγει πρόβλεψη με διαθέσιμο ιστορικό κατανάλωσης.

Το script έχει σχεδιαστεί ώστε να λειτουργεί χωρίς να απαιτείται home_id. Αυτό είναι σημαντικό για την τελική γενική εφαρμογή, καθώς ο χρήστης δεν χρειάζεται να αντιστοιχεί σε κάποια γνωστή κατοικία του αρχικού συνόλου δεδομένων. Αντίθετα, αρκεί να παρέχει ένα αρχείο με ωριαίες τιμές κατανάλωσης και αντίστοιχες χρονικές σημάνσεις.

Το προεπιλεγμένο αρχείο ιστορικού είναι το history_store.csv και αναμένεται να περιέχει τουλάχιστον δύο στήλες: τη στήλη timestamp, η οποία δηλώνει τη χρονική στιγμή της μέτρησης, και τη στήλη consumption_Wh, η οποία δηλώνει την ωριαία κατανάλωση σε Wh. Το script μπορεί επίσης να αναγνωρίσει εναλλακτικές ονομασίες στηλών, όπως time, datetime ή date για τη χρονική στήλη και consumption, energy_Wh, Wh, value ή y για τη στήλη κατανάλωσης. Με αυτόν τον τρόπο γίνεται πιο ευέλικτο ως προς τη μορφή του αρχείου που μπορεί να δώσει ο χρήστης.

Κατά τη φόρτωση του αρχείου, το script μετατρέπει τις χρονικές τιμές σε κατάλληλη μορφή ημερομηνίας και ώρας, μετατρέπει την κατανάλωση σε αριθμητική μορφή και αφαιρεί γραμμές με μη έγκυρες ή κενές τιμές. Στη συνέχεια, οι χρονικές σημάνσεις

κανονικοποιούνται σε ωριαία βάση. Αν υπάρχουν πολλαπλές εγγραφές για την ίδια ώρα, διατηρείται η τελευταία διαθέσιμη τιμή. Επιπλέον, οι αρνητικές τιμές κατανάλωσης μηδενίζονται, καθώς δεν θεωρούνται έγκυρες στο πλαίσιο της συγκεκριμένης εφαρμογής. Η βασική λειτουργία του script είναι να επιστρέφει ένα συνεχόμενο ιστορικό παράθυρο κατανάλωσης πριν από την ημερομηνία πρόβλεψης. Από προεπιλογή, το παράθυρο αυτό έχει διάρκεια 168 ωρών, δηλαδή μίας εβδομάδας, ώστε να είναι συμβατό με τα μοντέλα που απαιτούν πρόσφατο ιστορικό κατανάλωσης. Για παράδειγμα, αν η ημερομηνία στόχου είναι μια συγκεκριμένη ημέρα στις 00:00, το script προσπαθεί να επιστρέψει τις 168 προηγούμενες ωριαίες τιμές κατανάλωσης.

Αν το ακριβές ζητούμενο χρονικό παράθυρο υπάρχει πλήρες στο αρχείο, τότε επιστρέφεται αυτό. Αν όμως το ζητούμενο παράθυρο είναι ελλιπές και έχει ενεργοποιηθεί η εναλλακτική επιλογή, το script επιστρέφει το πιο πρόσφατο διαθέσιμο συνεχόμενο τμήμα ιστορικού με το απαιτούμενο πλήθος ωρών. Με αυτόν τον τρόπο, η εφαρμογή μπορεί να συνεχίσει να λειτουργεί ακόμη και όταν το αρχείο δεν περιέχει ακριβώς τις μετρήσεις πριν από την ημερομηνία στόχου, αρκεί να υπάρχει κάποιο άλλο επαρκές συνεχόμενο ιστορικό τμήμα.

Το script παρέχει επίσης δυνατότητα επιστροφής του ιστορικού είτε ως απλή λίστα τιμών είτε ως πίνακα δεδομένων με χρονικές σημάνσεις και τιμές κατανάλωσης. Η δεύτερη μορφή είναι χρήσιμη για το API και το σύστημα πρόβλεψης, επειδή διατηρεί μαζί την πληροφορία του χρόνου και της κατανάλωσης. Μαζί με τα δεδομένα επιστρέφονται και μεταδεδομένα, όπως η πηγή του ιστορικού, η αρχή και το τέλος του χρονικού παραθύρου και το πλήθος των ωρών που χρησιμοποιήθηκαν.

Τέλος, το script περιλαμβάνει βοηθητική συνάρτηση πληροφόρησης για την κατάσταση του αρχείου ιστορικού. Η συνάρτηση αυτή επιστρέφει πληροφορίες όπως η διαδρομή του αρχείου, αν το αρχείο υπάρχει, το πλήθος των διαθέσιμων γραμμών, η πρώτη και η τελευταία χρονική σήμανση, καθώς και αν το αρχείο μπορεί να καλύψει το προεπιλεγμένο ιστορικό παράθυρο των 168 ωρών. Ελέγχει επίσης αν υπάρχουν χρονικά κενά μεταξύ των εγγραφών, κάτι που είναι σημαντικό για την αξιοπιστία του ιστορικού.

model_registry

Το συγκεκριμένο script λειτουργεί ως κεντρικό μητρώο των διαθέσιμων μοντέλων της εφαρμογής. Ο ρόλος του είναι να συγκεντρώνει σε ένα σημείο όλες τις βασικές πληροφορίες για τα μοντέλα που μπορούν να χρησιμοποιηθούν από το API και τη γραφική διεπαφή, ώστε το σύστημα να γνωρίζει ποιο μοντέλο πρέπει να φορτώσει σε κάθε σενάριο πρόβλεψης.

Η παρούσα έκδοση του μητρώου έχει σχεδιαστεί για τη γενική έκδοση της εφαρμογής, όπου ο τελικός χρήστης δεν παρέχει `home_id`. Για τον λόγο αυτό, το registry αποφεύγει μοντέλα που απαιτούν γνωστή κατοικία του αρχικού dataset και περιλαμβάνει μόνο μοντέλα που μπορούν να χρησιμοποιηθούν είτε με βασικά χαρακτηριστικά κατοικίας είτε με πρόσφατο ιστορικό κατανάλωσης που δίνει ο χρήστης.

Το script ορίζει δύο βασικά σενάρια λειτουργίας. Το πρώτο είναι το `no_history`, το οποίο αντιστοιχεί σε πρόβλεψη χωρίς διαθέσιμο πρόσφατο ιστορικό κατανάλωσης. Το δεύτερο είναι το `with_history`, το οποίο αντιστοιχεί σε πρόβλεψη με χρήση πρόσφατων ωριαίων τιμών κατανάλωσης. Παράλληλα, διατηρούνται παλαιότερες ονομασίες, όπως `coldstart`, `cold_start` και `withhistory`, ώστε να παραμένει συμβατό το σύστημα με προηγούμενες εκδόσεις του API ή της διεπαφής.

Για το σενάριο χωρίς ιστορικό κατανάλωσης, το προεπιλεγμένο μοντέλο είναι το Random Forest. Το μοντέλο αυτό βασίζεται σε απλά χρονικά, μετεωρολογικά και στατικά χαρακτηριστικά της κατοικίας και δεν απαιτεί ιστορικές μεταβλητές, `home_id`, προφίλ συμπεριφοράς ή προφίλ παρόμοιων κατοικιών. Με αυτόν τον τρόπο αποτελεί την κύρια επιλογή όταν ο χρήστης δεν διαθέτει προηγούμενες μετρήσεις κατανάλωσης.

Για το σενάριο με ιστορικό κατανάλωσης, το προεπιλεγμένο μοντέλο είναι το LightGBM. Το μοντέλο αυτό αξιοποιεί πρόσφατες τιμές κατανάλωσης μέσω ιστορικών χαρακτηριστικών και κυλιόμενων στατιστικών, χωρίς όμως να απαιτεί ή να χρησιμοποιεί `home_id`. Ως ελάχιστο απαιτούμενο ιστορικό ορίζονται οι 168 ώρες, δηλαδή μία εβδομάδα ωριαίων μετρήσεων. Έτσι, το μοντέλο μπορεί να χρησιμοποιηθεί από έναν νέο χρήστη, αρκεί να παρέχει πρόσφατο ιστορικό κατανάλωσης.

Εκτός από τα προεπιλεγμένα μοντέλα, το registry περιλαμβάνει και προαιρετικά μοντέλα σύγκρισης. Για το σενάριο με ιστορικό κατανάλωσης διατίθενται προαιρετικά μοντέλα XGBoost και Random Forest, ενώ για το σενάριο χωρίς ιστορικό διατίθενται προαιρετικά μοντέλα LightGBM και XGBoost. Τα προαιρετικά no-history μοντέλα μπορούν να αξιοποιούν προφίλ συμπεριφοράς και προφίλ παρόμοιων κατοικιών, χωρίς όμως ο χρήστης να χρειάζεται να εισαγάγει κάποιο πραγματικό `home_id`.

Για κάθε μοντέλο, το μητρώο αποθηκεύει πληροφορίες όπως το αναγνωριστικό του μοντέλου, το όνομα εμφάνισης, την οικογένεια αλγορίθμου, τον τύπο του μοντέλου, το σενάριο που υποστηρίζει, τη διαδρομή των αποθηκευμένων αρχείων, καθώς και το αν πρόκειται για προεπιλεγμένο ή προαιρετικό μοντέλο. Επιπλέον, καταγράφει αν το μοντέλο απαιτεί ιστορικό κατανάλωσης, αν χρησιμοποιεί ιστορικές μεταβλητές, αν χρησιμοποιεί κυλιόμενα χαρακτηριστικά, αν απαιτεί `home_id` και αν είναι ορατό στη γραφική διεπαφή.

Το script περιλαμβάνει επίσης βοηθητικές συναρτήσεις για την κανονικοποίηση των επιλογών του χρήστη. Για παράδειγμα, αν ο χρήστης ή το API δώσει μια παλαιότερη ονομασία όπως `coldstart`, αυτή μετατρέπεται εσωτερικά στο νεότερο `no_history`. Αντίστοιχα, ονομασίες όπως `withhistory` ή `recent_history` μετατρέπονται στο `with_history`. Με αυτόν τον τρόπο διατηρείται η συμβατότητα με παλαιότερες εκδόσεις, ενώ η εσωτερική λογική του συστήματος παραμένει καθαρή και ενιαία.

Μία ακόμη σημαντική λειτουργία του script είναι η επιλογή του κατάλληλου προεπιλεγμένου μοντέλου. Όταν δεν δοθεί συγκεκριμένο μοντέλο από τον χρήστη, το σύστημα επιλέγει αυτόματα το κατάλληλο μοντέλο με βάση το σενάριο πρόβλεψης. Έτσι, για πρόβλεψη χωρίς ιστορικό επιλέγεται το προεπιλεγμένο Random Forest, ενώ για πρόβλεψη με ιστορικό επιλέγεται το προεπιλεγμένο LightGBM.

Το registry παρέχει επίσης συναρτήσεις για τη λίστα των διαθέσιμων μοντέλων, για τη δημιουργία επιλογών που μπορούν να εμφανιστούν στη γραφική διεπαφή και για τον έλεγχο της συμβατότητας ενός μοντέλου με το επιλεγμένο σενάριο. Αν, για παράδειγμα, ζητηθεί μοντέλο που δεν υποστηρίζει το συγκεκριμένο mode ή που απαιτεί `home_id`, το script επιστρέφει κατάλληλο σφάλμα και αποτρέπει τη λανθασμένη χρήση του.

Τέλος, το script περιλαμβάνει βοηθητική συνάρτηση ελέγχου των διαδρομών των αποθηκευμένων αρχείων. Η συνάρτηση αυτή ελέγχει αν υπάρχουν οι φάκελοι των μοντέλων και αν έχουν αποθηκευτεί τα βασικά αρχεία που απαιτούνται για τη λειτουργία τους, όπως το εκπαιδευμένο μοντέλο, ο μηχανισμός προεπεξεργασίας και το αρχείο ρυθμίσεων χαρακτηριστικών.

inference

Το συγκεκριμένο script αποτελεί τον βασικό μηχανισμό παραγωγής προβλέψεων της εφαρμογής. Ο ρόλος του είναι να φορτώνει τα κατάλληλα αποθηκευμένα μοντέλα και τα αντίστοιχα συνοδευτικά αρχεία τους, να δημιουργεί τα απαραίτητα χαρακτηριστικά εισόδου και να επιστρέφει την πρόβλεψη της κατανάλωσης για το επόμενο 24ωρο.

Η επιλογή του κατάλληλου μοντέλου γίνεται μέσω του `model_registry.py`, το οποίο καθορίζει ποιο μοντέλο χρησιμοποιείται για κάθε σενάριο. Έτσι, για το σενάριο χωρίς ιστορικό κατανάλωσης επιλέγεται το αντίστοιχο προεπιλεγμένο μοντέλο, ενώ για το

σενάριο με ιστορικό κατανάλωσης επιλέγεται το προεπιλεγμένο μοντέλο που αξιοποιεί πρόσφατες ωριαίες τιμές κατανάλωσης.

Το script υποστηρίζει δύο βασικές λειτουργίες πρόβλεψης. Η πρώτη είναι η πρόβλεψη χωρίς ιστορικό κατανάλωσης (no_history). Σε αυτή την περίπτωση, η πρόβλεψη βασίζεται στα χαρακτηριστικά που μπορεί να δώσει ο χρήστης, όπως το εμβαδόν της κατοικίας, ο αριθμός κατοίκων, ο τύπος κατοικίας, η κατηγορία περιοχής, η εξωτερική θερμοκρασία και, όπου είναι διαθέσιμη, η εσωτερική θερμοκρασία ή ο αριθμός ηλεκτρικών συσκευών. Για κάθε ώρα της ημέρας στόχου δημιουργούνται τα αντίστοιχα χρονικά χαρακτηριστικά, όπως ώρα, ημέρα εβδομάδας, μήνας, ένδειξη Σαββατοκύριακου και ένδειξη αργίας.

Η δεύτερη λειτουργία είναι η πρόβλεψη με ιστορικό κατανάλωσης (with_history). Σε αυτή την περίπτωση, εκτός από τα βασικά χαρακτηριστικά της κατοικίας και του χρόνου, χρησιμοποιούνται και πρόσφατες ωριαίες τιμές κατανάλωσης. Το ιστορικό μπορεί να δοθεί είτε ως λίστα τιμών είτε ως αρχείο CSV είτε ως πίνακας δεδομένων. Το script ελέγχει ότι υπάρχει επαρκές ιστορικό πριν από την ημερομηνία στόχου και το μετατρέπει σε κανονική ωριαία μορφή. Αν υπάρχουν μικρά κενά στο ιστορικό, αυτά μπορούν να συμπληρωθούν με παρεμβολή, εφόσον η συνολική κάλυψη του απαιτούμενου χρονικού παραθύρου είναι επαρκής.

Στο σενάριο με ιστορικό, η πρόβλεψη γίνεται αναδρομικά για τις 24 ώρες της ημέρας στόχου. Αυτό σημαίνει ότι για την πρώτη ώρα χρησιμοποιείται το πραγματικό διαθέσιμο ιστορικό, ενώ η πρόβλεψη της πρώτης ώρας προστίθεται στη συνέχεια στο ιστορικό και χρησιμοποιείται για τον υπολογισμό των χαρακτηριστικών της επόμενης ώρας. Η διαδικασία αυτή επαναλαμβάνεται μέχρι να παραχθούν οι 24 ωριαίες προβλέψεις. Με αυτόν τον τρόπο, το script μπορεί να δημιουργήσει χαρακτηριστικά υστέρησης και κυλιόμενα στατιστικά για ολόκληρο το 24ωρο πρόβλεψης, ακόμη και όταν οι μελλοντικές πραγματικές τιμές δεν είναι γνωστές.

Το script περιλαμβάνει επίσης βοηθητικές συναρτήσεις για τη δημιουργία χρονικών χαρακτηριστικών και για τη σωστή προετοιμασία των δεδομένων πριν περάσουν στον μηχανισμό προεπεξεργασίας του μοντέλου. Ο μηχανισμός αυτός έχει αποθηκευτεί κατά τη φάση εκπαίδευσης και εξασφαλίζει ότι οι αριθμητικές και κατηγορικές μεταβλητές μετατρέπονται με τον ίδιο ακριβώς τρόπο και κατά τη φάση πρόβλεψης. Έτσι διατηρείται συνέπεια ανάμεσα στην εκπαίδευση και στην τελική χρήση του μοντέλου.

Για τα προαιρετικά μοντέλα χωρίς ιστορικό που χρησιμοποιούν προφίλ συμπεριφοράς ή προφίλ παρόμοιων κατοικιών, το script μπορεί να δημιουργήσει εσωτερικά τα απαιτούμενα πρόσθετα χαρακτηριστικά. Σε αυτή την περίπτωση χρησιμοποιείται ένα εικονικό αναγνωριστικό κατοικίας μόνο για τις βοηθητικές διαδικασίες του κώδικα, χωρίς όμως να ζητείται πραγματικό home_id από τον χρήστη και χωρίς αυτό να αποτελεί χαρακτηριστικό του τελικού μοντέλου.

Ιδιαίτερη λειτουργία του script είναι η προαιρετική προσαρμογή της πρόβλεψης με βάση το πρόσφατο ιστορικό κατανάλωσης. Η προσαρμογή αυτή χρησιμοποιεί το ωριαίο προφίλ των τελευταίων επτά ημερών και εφαρμόζει ήπια διόρθωση στο σχήμα της 24ωρης πρόβλεψης. Η διόρθωση δεν αλλάζει αυθαίρετα το συνολικό ημερήσιο επίπεδο της πρόβλεψης, αλλά μεταβάλλει προσεκτικά την κατανομή της κατανάλωσης μέσα στη διάρκεια της ημέρας, με βάση τη σταθερότητα που παρουσιάζει το πρόσφατο ιστορικό ανά ώρα. Για λόγους ασφαλείας, η ένταση της διόρθωσης περιορίζεται από μέγιστο συντελεστή, ώστε το μοντέλο να παραμένει η κύρια πηγή της πρόβλεψης.

Το script μπορεί επίσης να εφαρμόσει προαιρετική ημερήσια βαθμονόμηση, εφόσον υπάρχει αποθηκευμένος αντίστοιχος διορθωτικός συντελεστής στα συνοδευτικά αρχεία του μοντέλου. Η λειτουργία αυτή χρησιμοποιείται μόνο όταν ζητηθεί κατάλληλη μορφή βελτιστοποίησης και έχει ως στόχο τη διόρθωση πιθανής συστηματικής υποεκτίμησης ή υπερεκτίμησης της συνολικής ημερήσιας κατανάλωσης.

Η έξοδος του script είναι ένας πίνακας με 24 γραμμές, μία για κάθε ώρα της ημέρας στόχου. Ο πίνακας περιλαμβάνει τη χρονική σήμανση, την προβλεπόμενη κατανάλωση σε Wh, το μοντέλο που χρησιμοποιήθηκε, το σενάριο πρόβλεψης και την εκδοχή της πρόβλεψης. Στο σενάριο με ιστορικό προστίθενται επίσης πληροφορίες σχετικά με την πηγή του ιστορικού, την κάλυψη των διαθέσιμων δεδομένων και το αν εφαρμόστηκε προσαρμογή με βάση το πρόσφατο ιστορικό.

Τέλος, το script παρέχει μία ενιαία συνάρτηση πρόβλεψης, η οποία δέχεται ως είσοδο το επιθυμητό σενάριο και καλεί εσωτερικά την κατάλληλη διαδικασία. Με αυτόν τον τρόπο, το API μπορεί να χρησιμοποιεί ένα κοινό σημείο εισόδου τόσο για προβλέψεις χωρίς ιστορικό όσο και για προβλέψεις με ιστορικό. Συνολικά, το inference.py συνδέει τα εκπαιδευμένα μοντέλα με τη λειτουργική εφαρμογή και εξασφαλίζει ότι η παραγωγή προβλέψεων γίνεται με τρόπο συνεπή, οργανωμένο και συμβατό με τα τελικά αρχεία εκπαίδευσης.

api_app

Το συγκεκριμένο script υλοποιεί το βασικό API της εφαρμογής πρόβλεψης ηλεκτρικής κατανάλωσης. Αποτελεί το ενδιάμεσο επίπεδο ανάμεσα στη γραφική διεπαφή και στο σύστημα πρόβλεψης, καθώς δέχεται τα στοιχεία που εισάγει ο χρήστης, επιλέγει το κατάλληλο μοντέλο, προετοιμάζει τα απαιτούμενα δεδομένα και επιστρέφει την τελική πρόβλεψη σε οργανωμένη μορφή.

Η επιλογή του μοντέλου γίνεται μέσω του model_registry.py, ενώ υποστηρίζονται δύο βασικά σενάρια πρόβλεψης: το no_history, όταν δεν υπάρχει διαθέσιμο πρόσφατο ιστορικό κατανάλωσης, και το with_history, όταν ο χρήστης διαθέτει πρόσφατες ωριαίες τιμές κατανάλωσης. Για λόγους συμβατότητας, το API εξακολουθεί να αναγνωρίζει και παλαιότερες ονομασίες, όπως coldstart, cold_start και withhistory.

Το API περιλαμβάνει βασικά endpoints για την πρόβλεψη, τη λίστα των διαθέσιμων μοντέλων, τον έλεγχο λειτουργίας της εφαρμογής και την πληροφόρηση σχετικά με το διαθέσιμο ιστορικό. Το endpoint /predict είναι το κύριο σημείο εισόδου για την παραγωγή πρόβλεψης. Το endpoint /models επιστρέφει τα διαθέσιμα μοντέλα και τις επιλογές που μπορούν να εμφανιστούν στη διεπαφή. Το endpoint /health ελέγχει την κατάσταση του συστήματος και των αποθηκευμένων μοντέλων, ενώ το /history/info παρέχει πληροφορίες για τη μορφή του αρχείου ιστορικού που μπορεί να χρησιμοποιηθεί στο σενάριο με ιστορικό κατανάλωσης.

Κατά την εκτέλεση ενός αιτήματος πρόβλεψης, το script αρχικά κανονικοποιεί το σενάριο πρόβλεψης και την επιλογή βελτιστοποίησης. Στη συνέχεια, επιλύει το κατάλληλο μοντέλο, είτε με βάση τη ρητή επιλογή του χρήστη είτε αυτόματα μέσω της επιλογής auto. Αν δεν δοθεί συγκεκριμένο μοντέλο, χρησιμοποιείται το προεπιλεγμένο μοντέλο για το αντίστοιχο σενάριο. Για λόγους αποδοτικότητας, τα μοντέλα και τα συνοδευτικά αρχεία τους αποθηκεύονται προσωρινά στη μνήμη μετά την πρώτη φόρτωση, ώστε να μη χρειάζεται να φορτώνονται εκ νέου σε κάθε αίτημα.

Το script διαχειρίζεται επίσης τις εξωτερικές θερμοκρασίες που απαιτούνται για την πρόβλεψη. Αν ο χρήστης δώσει πλήρες ωριαίο προφίλ θερμοκρασίας 24 τιμών, το API χρησιμοποιεί απευθείας αυτές τις τιμές. Αν δεν δοθεί τέτοιο προφίλ, επιχειρείται ανάκτηση πρόγνωσης θερμοκρασίας από το Open-Meteo με βάση την πόλη και την ημερομηνία στόχου. Αν αυτό δεν είναι δυνατό, το API μπορεί να δημιουργήσει ένα απλό ωριαίο προφίλ θερμοκρασίας με βάση την ελάχιστη και τη μέγιστη θερμοκρασία που παρέχει ο χρήστης. Με αυτόν τον τρόπο, η εφαρμογή παραμένει λειτουργική ακόμη και όταν δεν είναι διαθέσιμη αυτόματη μετεωρολογική πληροφορία.

Για το σενάριο με ιστορικό κατανάλωσης, το API μπορεί να δεχθεί το ιστορικό με τρεις τρόπους. Ο πρώτος είναι άμεση λίστα ωριαίων τιμών κατανάλωσης. Ο δεύτερος είναι διαδρομή προς αρχείο CSV μίας κατοικίας, το οποίο περιέχει τις στήλες timestamp και consumption_Wh. Ο τρίτος είναι η χρήση του προεπιλεγμένου αρχείου ιστορικού ως εναλλακτική λύση, κυρίως για λόγους συμβατότητας με προηγούμενες εκδόσεις. Σε κάθε περίπτωση, ελέγχεται ότι το διαθέσιμο ιστορικό καλύπτει το ελάχιστο απαιτούμενο χρονικό διάστημα, το οποίο από προεπιλογή είναι 168 ώρες.

Το αίτημα πρόβλεψης περιλαμβάνει επίσης βασικά χαρακτηριστικά της κατοικίας, όπως εμβαδόν, αριθμό κατοίκων, τύπο κατοικίας, κατηγορία περιοχής και προαιρετικά αριθμό ηλεκτρικών συσκευών. Επιπλέον, μπορεί να περιλαμβάνει εσωτερική θερμοκρασία, πλήρες προφίλ εξωτερικής θερμοκρασίας ή ελάχιστη και μέγιστη θερμοκρασία για δημιουργία απλού ημερήσιου προφίλ. Το API ελέγχει την εγκυρότητα αυτών των εισόδων και επιστρέφει κατάλληλα μηνύματα σφάλματος όταν λείπουν απαραίτητα στοιχεία.

Μετά την προετοιμασία των εισόδων, το script καλεί τη βασική συνάρτηση πρόβλεψης από το inference.py. Η συνάρτηση αυτή αναλαμβάνει τη δημιουργία των χαρακτηριστικών, τη χρήση του κατάλληλου μοντέλου και την παραγωγή των 24 ωριαίων προβλέψεων. Στο σενάριο με ιστορικό, μπορεί επίσης να εφαρμοστεί προαιρετική προσαρμογή της πρόβλεψης με βάση το πρόσφατο ιστορικό κατανάλωσης. Η προσαρμογή αυτή ελέγχεται από παραμέτρους όπως το πλήθος ημερών ιστορικού που θα χρησιμοποιηθούν και ο μέγιστος συντελεστής διόρθωσης.

Η απάντηση του API περιλαμβάνει το μοντέλο που χρησιμοποιήθηκε, το σενάριο πρόβλεψης, την ημερομηνία στόχου, τη συνολική ημερήσια κατανάλωση σε kWh, την πηγή των θερμοκρασιακών δεδομένων, την πηγή του ιστορικού, καθώς και αναλυτικές ωριαίες προβλέψεις. Για κάθε ώρα επιστρέφεται η χρονική σήμανση και η προβλεπόμενη κατανάλωση σε Wh. Όταν εφαρμόζεται προσαρμογή με βάση το ιστορικό, επιστρέφονται επιπλέον διαγνωστικές πληροφορίες, όπως η αρχική πρόβλεψη του μοντέλου, το ιστορικό ωριαίο προφίλ, ο συντελεστής προσαρμογής και ο λόγος για τον οποίο εφαρμόστηκε ή δεν εφαρμόστηκε η διόρθωση.

Συνολικά, το api_app.py αποτελεί το λειτουργικό επίπεδο που συντονίζει όλα τα υπόλοιπα μέρη της εφαρμογής. Συνδέει το μητρώο μοντέλων, το σύστημα πρόβλεψης, το αρχείο ιστορικού, την ανάκτηση θερμοκρασιών και τη γραφική διεπαφή. Με αυτόν τον τρόπο, επιτρέπει στην εφαρμογή να παράγει προβλέψεις με ενιαίο και οργανωμένο τρόπο, τόσο για περιπτώσεις χωρίς ιστορικό κατανάλωσης όσο και για περιπτώσεις όπου ο χρήστης διαθέτει πρόσφατες μετρήσεις.

ui_app

Το συγκεκριμένο script υλοποιεί τη γραφική διεπαφή της εφαρμογής πρόβλεψης ηλεκτρικής κατανάλωσης με χρήση της βιβλιοθήκης Gradio. Ο ρόλος του δεν είναι η εκπαίδευση των μοντέλων ούτε η άμεση εκτέλεση της λογικής πρόβλεψης, αλλά η παροχή ενός εύχρηστου περιβάλλοντος μέσα από το οποίο ο χρήστης μπορεί να εισάγει τα στοιχεία της κατοικίας, να επιλέγει σενάριο και μοντέλο πρόβλεψης, να δίνει προαιρετικά ιστορικό κατανάλωσης και να βλέπει τα αποτελέσματα σε πίνακες και διαγράμματα.

Η επιλογή του κατάλληλου μοντέλου γίνεται αυτόματα ή μέσω των διαθέσιμων επιλογών που επιστρέφει το API. Για το σενάριο χωρίς ιστορικό κατανάλωσης χρησιμοποιείται ως προεπιλογή το αντίστοιχο Random Forest μοντέλο, ενώ για το σενάριο με ιστορικό κατανάλωσης χρησιμοποιείται ως προεπιλογή το αντίστοιχο LightGBM μοντέλο. Παράλληλα, ο χρήστης μπορεί να επιλέξει και προαιρετικά μοντέλα σύγκρισης, όπως XGBoost ή Random Forest, ανάλογα με το διαθέσιμο σενάριο.

Στην αρχή του script ορίζονται βασικές ρυθμίσεις για την επικοινωνία με το API και για την τοπική εκτέλεση της διεπαφής. Η διεπαφή επικοινωνεί με το FastAPI backend μέσω της διεύθυνσης <http://127.0.0.1:8000>, ενώ το περιβάλλον Gradio εκτελείται τοπικά στη θύρα 7860. Επιπλέον, ορίζονται φάκελοι για την αποθήκευση αρχείων εξόδου, όπως τα CSV των προβλέψεων, καθώς και φάκελος για τα αρχεία ιστορικού που επιλέγει ή ανεβάζει ο χρήστης.

Το script περιλαμβάνει επίσης ειδικές ρυθμίσεις συμβατότητας για τη σωστή λειτουργία του Gradio. Συγκεκριμένα, ορίζονται μεταβλητές περιβάλλοντος ώστε η τοπική επικοινωνία με το API να μη διακόπτεται από ρυθμίσεις proxy και απενεργοποιείται η αποστολή analytics του Gradio. Επιπλέον, εφαρμόζεται βοηθητική διόρθωση συμβατότητας για εκδόσεις του Gradio 4.44.x, ώστε να αποφεύγονται σφάλματα που σχετίζονται με τον τρόπο που ερμηνεύονται ορισμένα JSON schema.

Η διεπαφή δίνει στον χρήστη τη δυνατότητα να επιλέξει αν η πρόβλεψη θα γίνει χωρίς ιστορικό ή με ιστορικό κατανάλωσης. Όταν ενεργοποιείται η πρόβλεψη με ιστορικό, εμφανίζονται οι αντίστοιχες επιλογές για την παροχή πρόσφατων ωριαίων τιμών κατανάλωσης. Το ιστορικό μπορεί να δοθεί είτε με επιλογή αρχείου CSV, είτε με απευθείας εισαγωγή διαδρομής αρχείου, είτε με χειροκίνητη εισαγωγή τιμών κατανάλωσης σε μορφή λίστας. Το αρχείο CSV θεωρείται ότι αφορά μία μόνο κατοικία και πρέπει να περιέχει τις στήλες timestamp και consumption_Wh.

Μία σημαντική λειτουργία του script είναι η επιλογή αρχείου ιστορικού μέσω του συστήματος αρχείων του υπολογιστή. Όταν ο χρήστης επιλέγει ένα CSV, το αρχείο αντιγράφεται σε σταθερό φάκελο του έργου, ώστε να μπορεί να το διαβάσει αξιόπιστα και το FastAPI backend. Στη συνέχεια, η διαδρομή του αντιγραφμένου αρχείου εμφανίζεται στο αντίστοιχο πεδίο της διεπαφής και χρησιμοποιείται κατά την πρόβλεψη με ιστορικό.

Η διεπαφή υποστηρίζει επίσης διαφορετικούς τρόπους παροχής θερμοκρασιακών δεδομένων. Ο χρήστης μπορεί να δώσει χειροκίνητα πλήρες ωριαίο προφίλ εξωτερικής θερμοκρασίας 24 τιμών ή, εναλλακτικά, να δώσει ελάχιστη και μέγιστη θερμοκρασία της ημέρας. Αν δεν δοθούν χειροκίνητες τιμές, το API μπορεί να επιχειρήσει να αντλήσει πρόγνωση από το Open-Meteo. Με αυτόν τον τρόπο, η εφαρμογή παραμένει ευέλικτη και μπορεί να λειτουργήσει τόσο με αυτόματα όσο και με χειροκίνητη εισαγωγή καιρικών δεδομένων.

Για το σενάριο με ιστορικό κατανάλωσης, το script περιλαμβάνει ειδική λειτουργία ανάλυσης της σταθερότητας του πρόσφατου ιστορικού. Η λειτουργία αυτή εξετάζει το ιστορικό πριν από την ημερομηνία στόχου και υπολογίζει δείκτες όπως πληρότητα δεδομένων, μέση ημερήσια κατανάλωση, διακύμανση ημερήσιας κατανάλωσης, σταθερότητα ωριαίου προφίλ και σταθερότητα αιχμών. Με βάση αυτά τα στοιχεία, προτείνει μια ενδεικτική μέγιστη τιμή για τον συντελεστή προσαρμογής του history-aware correction. Έτσι, ο χρήστης μπορεί είτε να κρατήσει χειροκίνητη τιμή είτε να χρησιμοποιήσει την προτεινόμενη τιμή που προκύπτει από την ανάλυση του ιστορικού.

Η διεπαφή περιλαμβάνει ακόμη δυνατότητα προαιρετικής συμπεριφορικής προσαρμογής της πρόβλεψης. Ο χρήστης μπορεί να δηλώσει συγκεκριμένες ώρες ή χρονικά διαστήματα της ημέρας στα οποία αναμένει αυξημένη ή μειωμένη κατανάλωση και να ορίσει έναν συντελεστή μεταβολής. Η προσαρμογή αυτή εφαρμόζεται μετά την παραγωγή της βασικής πρόβλεψης και λειτουργεί ως εργαλείο σεναριακής διερεύνησης, όταν ο χρήστης γνωρίζει ότι η ημέρα στόχος θα διαφέρει από τη συνηθισμένη συμπεριφορά της κατοικίας.

Το script παρέχει τρεις βασικές ενέργειες. Η πρώτη είναι η απλή πρόβλεψη, όπου χρησιμοποιείται ένα επιλεγμένο μοντέλο ή το αυτόματα προεπιλεγμένο μοντέλο για το αντίστοιχο σενάριο. Η δεύτερη είναι η σύγκριση μοντέλων, όπου παράγονται προβλέψεις από τα διαθέσιμα μοντέλα Random Forest, XGBoost και LightGBM, ώστε ο χρήστης να μπορεί να δει τις διαφορές μεταξύ τους. Η τρίτη είναι η συνδυασμένη πρόβλεψη, όπου οι

προβλέψεις των τριών μοντέλων συνδυάζονται με σταθμισμένο μέσο όρο. Τα βάρη του συνδυασμού μπορούν να παραμείνουν στις προεπιλεγμένες τιμές ή να οριστούν χειροκίνητα από τον χρήστη.

Σε κάθε ενέργεια, το script δημιουργεί το κατάλληλο αίτημα προς το API, στέλνοντας το επιλεγμένο μοντέλο, το σενάριο πρόβλεψης, την ημερομηνία στόχου, τα χαρακτηριστικά της κατοικίας, τις θερμοκρασίες και, όταν απαιτείται, το ιστορικό κατανάλωσης. Στη συνέχεια λαμβάνει την απάντηση του API, μετατρέπει τις προβλέψεις σε πίνακα δεδομένων και δημιουργεί σύνοψη με τις βασικές πληροφορίες του αποτελέσματος.

Η έξοδος της διεπαφής περιλαμβάνει αναλυτική περίληψη της πρόβλεψης, πίνακα με τις ωριαίες προβλέψεις, πίνακα με τις θερμοκρασίες, πίνακα σύγκρισης μοντέλων και διάγραμμα της ωριαίας κατανάλωσης. Για την τρέχουσα ημέρα, το script μπορεί επίσης να υπολογίσει την υπολειπόμενη προβλεπόμενη κατανάλωση από την επόμενη πλήρη ώρα μέχρι το τέλος της ημέρας. Επιπλέον, όταν ο χρήστης το επιλέγει, τα αποτελέσματα μπορούν να αποθηκευτούν σε αρχείο CSV για μελλοντική χρήση ή τεκμηρίωση.

Τα διαγράμματα δημιουργούνται με τη βιβλιοθήκη Matplotlib και παρουσιάζουν την προβλεπόμενη κατανάλωση ανά ώρα. Στην απλή πρόβλεψη εμφανίζεται η καμπύλη του επιλεγμένου μοντέλου, ενώ στη σύγκριση και στη συνδυασμένη πρόβλεψη μπορούν να εμφανιστούν οι καμπύλες των διαφορετικών μοντέλων ή του σταθμισμένου συνδυασμού τους. Με αυτόν τον τρόπο ο χρήστης μπορεί να ερμηνεύσει πιο εύκολα όχι μόνο τη συνολική ημερήσια κατανάλωση, αλλά και την κατανομή της μέσα στη διάρκεια της ημέρας.

Συνολικά, το ui_app.py αποτελεί το ανώτερο επίπεδο της λειτουργικής αρχιτεκτονικής της εφαρμογής. Ενώ τα υπόλοιπα scripts διαχειρίζονται την εκπαίδευση, την επιλογή μοντέλων, το ιστορικό κατανάλωσης, την ανάκτηση θερμοκρασιών και την παραγωγή προβλέψεων, το συγκεκριμένο αρχείο μετατρέπει όλες αυτές τις λειτουργίες σε ένα διαδραστικό περιβάλλον χρήστη. Με αυτόν τον τρόπο, το ερευνητικό αποτέλεσμα της εργασίας παρουσιάζεται ως ολοκληρωμένη εφαρμογή επίδειξης, στην οποία ο χρήστης μπορεί να πραγματοποιεί προβλέψεις, να συγκρίνει μοντέλα, να εφαρμόζει προαιρετικές διορθώσεις και να εξάγει τα αποτελέσματα χωρίς άμεση αλληλεπίδραση με τον κώδικα.

__init__

Το αρχείο `__init__.py` χρησιμοποιείται για τον ορισμό ενός φακέλου ως πακέτου (package) της Python, επιτρέποντας την εισαγωγή (import) των επιμέρους modules που περιέχει. Αν και το αρχείο είναι κενό, η ύπαρξή του είναι απαραίτητη για τη σωστή οργάνωση του κώδικα και τη λειτουργία της δομής του έργου. Συγκεκριμένα, το αρχείο αυτό επιτρέπει, την αναγνώριση του φακέλου ως Python package, τη χρήση σχετικών εισαγωγών (relative imports) μεταξύ αρχείων και την καλύτερη οργάνωση και επεκτασιμότητα της εφαρμογής.

Αξιολόγηση αποτελεσμάτων IDEAL demo

New_histories_creation

Το συγκεκριμένο script χρησιμοποιείται για τη δημιουργία αρχείων ιστορικού κατανάλωσης για επιλεγμένες κατοικίες του συνόλου δεδομένων IDEAL, με σκοπό την αξιολόγηση των προβλέψεων του συστήματος UI/API στο σενάριο με χρήση ιστορικών δεδομένων. Αρχικά, φορτώνεται το τελικό ωριαίο σύνολο δεδομένων IDEAL_final_hourly_dataset.csv, το οποίο περιλαμβάνει τις ωριαίες τιμές κατανάλωσης ανά κατοικία. Στη συνέχεια, εφαρμόζεται έλεγχος μοναδικότητας ώστε για κάθε κατοικία

και χρονική στιγμή να υπάρχει μία μόνο εγγραφή. Σε περίπτωση διπλότυπων εγγραφών, η τιμή κατανάλωσης υπολογίζεται ως ο μέσος όρος, ώστε να αποφευχθεί πιθανός διπλός υπολογισμός.

Για κάθε επιλεγμένη κατοικία ορίζεται μία συγκεκριμένη ημερομηνία-στόχος από το test set του Ιουνίου 2018. Το script εξάγει το ιστορικό κατανάλωσης των 14 ημερών που προηγούνται αυστηρά της ημερομηνίας πρόβλεψης, δηλαδή ένα παράθυρο 336 ωρών, καθώς και τις πραγματικές τιμές κατανάλωσης της ημέρας-στόχου. Το ιστορικό αποθηκεύεται σε αρχείο history_store.csv, το οποίο περιλαμβάνει μόνο τις στήλες timestamp και consumption_Wh, ώστε να είναι άμεσα συμβατό με το περιβάλλον UI/API. Παράλληλα, οι πραγματικές τιμές της ημέρας πρόβλεψης αποθηκεύονται σε αρχείο actual_target_day.csv, το οποίο χρησιμοποιείται μεταγενέστερα για τη σύγκριση των προβλέψεων με τις πραγματικές μετρήσεις.

Επιπλέον, για κάθε περίπτωση δημιουργείται αρχείο μεταδεδομένων case_metadata.json, στο οποίο καταγράφονται πληροφορίες όπως η κατοικία, η ημερομηνία πρόβλεψης, το χρονικό διάστημα του ιστορικού, το πλήθος των αναμενόμενων και διαθέσιμων ωρών, η πληρότητα των δεδομένων, καθώς και βασικά στατιστικά μεγέθη κατανάλωσης. Το script ελέγχει επίσης αν υπάρχουν ελλείπουσες ώρες τόσο στο ιστορικό παράθυρο όσο και στην ημέρα-στόχο, ώστε να διασφαλιστεί ότι τα αρχεία αξιολόγησης είναι πλήρη και κατάλληλα για χρήση.

Τέλος, δημιουργείται ένα συνολικό αρχείο σύνοψης evaluation_cases_summary.csv, το οποίο συγκεντρώνει τις βασικές πληροφορίες όλων των επιλεγμένων κατοικιών και ημερομηνιών αξιολόγησης. Με τον τρόπο αυτό, το script οργανώνει με ενιαίο και αναπαραγώγιμο τρόπο τα απαραίτητα αρχεία για την πειραματική αξιολόγηση των προβλέψεων του συστήματος.

Ext_temp_for_UI

Το συγκεκριμένο script χρησιμοποιείται για τη δημιουργία των θερμοκρασιακών εισόδων που απαιτούνται κατά την αξιολόγηση του IDEAL UI/API. Για κάθε επιλεγμένη κατοικία και ημερομηνία-στόχο εξάγει τις 24 ωριαίες τιμές εξωτερικής θερμοκρασίας της ημέρας πρόβλεψης, ενώ για την εσωτερική θερμοκρασία χρησιμοποιεί τη διάμεσο τιμή των αντίστοιχων ωρών από τις προηγούμενες 7 ημέρες, ώστε να αποφεύγεται διαρροή πληροφορίας από την ημέρα-στόχο. Τα αποτελέσματα αποθηκεύονται σε αρχεία temperature_inputs.csv και temperature_inputs_for_ui.txt, ώστε να μπορούν να χρησιμοποιηθούν άμεσα στο UI, ενώ παράλληλα δημιουργούνται αρχεία μεταδεδομένων και συνοπτικό αρχείο ελέγχου για την τεκμηρίωση των παραγόμενων θερμοκρασιακών εισόδων.

New_Evaluation_June

Το συγκεκριμένο script χρησιμοποιείται για την αξιολόγηση των προβλέψεων που παράγονται από το περιβάλλον UI/API για επιλεγμένες κατοικίες του συνόλου δεδομένων IDEAL. Η αξιολόγηση πραγματοποιείται σε επίπεδο μεμονωμένης κατοικίας και ημέρας πρόβλεψης, συγκρίνοντας τις προβλεπόμενες ωριαίες τιμές κατανάλωσης με τις πραγματικές μετρήσεις της αντίστοιχης ημέρας.

Αρχικά, το script ορίζει τον βασικό φάκελο εργασίας C:/IDEAL_Programming/New_Evaluation και τη λίστα των κατοικιών που συμμετέχουν στην αξιολόγηση. Για κάθε κατοικία αναζητά το αρχείο actual_target_day.csv, το οποίο περιέχει τις πραγματικές τιμές κατανάλωσης της ημέρας-στόχου, καθώς και τα αρχεία προβλέψεων pred_no_history.csv και pred_with_history.csv, τα οποία αντιστοιχούν στο

σενάριο χωρίς χρήση ιστορικών δεδομένων και στο σενάριο με χρήση ιστορικών δεδομένων αντίστοιχα.

Το script περιλαμβάνει βοηθητικές συναρτήσεις για την αυτόματη αναγνώριση στηλών χρόνου, πραγματικής κατανάλωσης και προβλέψεων. Με αυτόν τον τρόπο μπορεί να διαχειριστεί αρχεία με ελαφρώς διαφορετική ονοματολογία στηλών. Επίσης, πραγματοποιείται μετατροπή τιμών από kWh σε Wh όταν αυτό εντοπίζεται από το όνομα της στήλης, ώστε όλες οι συγκρίσεις να γίνονται στην ίδια μονάδα μέτρησης. Οι στήλες που αντιστοιχούν σε βάρη ensemble αγνοούνται, καθώς δεν αποτελούν προβλέψεις κατανάλωσης, ενώ οι προβλέψεις ensemble που έχουν εξαχθεί από το UI διατηρούνται όπως είναι, χωρίς επανυπολογισμό.

Για κάθε κατοικία, το script συγχωνεύει τις πραγματικές τιμές με τις προβλέψεις όλων των διαθέσιμων μοντέλων, δηλαδή Random Forest, XGBoost, LightGBM και ensemble. Η σύγκριση γίνεται ανά ώρα του 24ώρου, δημιουργώντας ενιαίο αρχείο `hourly_comparison_all_models.csv`. Παράλληλα, δημιουργούνται και ξεχωριστά αρχεία σύγκρισης για το σενάριο χωρίς χρήση ιστορικών δεδομένων και για το σενάριο με χρήση ιστορικών δεδομένων.

Στη συνέχεια υπολογίζονται βασικές μετρικές αξιολόγησης για κάθε μοντέλο και σενάριο. Συγκεκριμένα, υπολογίζονται τα MAE, RMSE, MAPE και sMAPE σε επίπεδο ωριαίων τιμών, καθώς και το συνολικό πραγματικό και προβλεπόμενο ημερήσιο φορτίο σε kWh. Επιπλέον, υπολογίζεται το απόλυτο ημερήσιο σφάλμα σε kWh, το ποσοστιαίο ημερήσιο σφάλμα, η μεροληψία της πρόβλεψης και η απόκλιση ως προς την τιμή και την ώρα της μέγιστης κατανάλωσης. Οι μετρικές αυτές αποθηκεύονται στο αρχείο `evaluation_metrics.csv` για κάθε κατοικία.

Το script παράγει επίσης γραφήματα σύγκρισης πραγματικής και προβλεπόμενης κατανάλωσης. Δημιουργείται ένα συνολικό γράφημα με όλες τις προβλέψεις των μοντέλων, καθώς και ξεχωριστά γραφήματα για κάθε σενάριο αξιολόγησης. Τα γραφήματα αυτά επιτρέπουν την οπτική αξιολόγηση της ικανότητας των μοντέλων να ακολουθούν το ωριαίο προφίλ κατανάλωσης της πραγματικής ημέρας.

Για κάθε κατοικία δημιουργείται επιπλέον αρχείο `evaluation_metadata.json`, το οποίο καταγράφει τα αρχεία εισόδου, τα αρχεία εξόδου, τις στήλες προβλέψεων που χρησιμοποιήθηκαν, το χρονικό διάστημα της αξιολόγησης και τον κανόνα αγνόησης των στηλών που περιέχουν βάρη ensemble. Με αυτόν τον τρόπο διατηρείται πλήρης τεκμηρίωση της διαδικασίας αξιολόγησης.

Τέλος, το script συγκεντρώνει τα αποτελέσματα όλων των κατοικιών σε συνολικά αρχεία εξόδου. Το αρχείο `all_homes_evaluation_metrics.csv` περιλαμβάνει όλες τις μετρικές ανά κατοικία, μοντέλο και σενάριο. Το αρχείο `summary_by_scenario_and_model.csv` παρουσιάζει συγκεντρωτική σύνοψη ανά σενάριο και μοντέλο, υπολογίζοντας μέσες τιμές των βασικών μετρικών για όλες τις κατοικίες. Επιπλέον, το αρχείο `all_homes_hourly_comparison.csv` περιλαμβάνει τη συνολική ωριαία σύγκριση όλων των κατοικιών.

Συνολικά, το script αποτελεί το βασικό εργαλείο πειραματικής αξιολόγησης των προβλέψεων του IDEAL UI/API, καθώς επιτρέπει τη συστηματική σύγκριση των μοντέλων τόσο στο σενάριο χωρίς χρήση ιστορικών δεδομένων όσο και στο σενάριο με χρήση ιστορικών δεδομένων, σε επίπεδο μεμονωμένων κατοικιών και πραγματικών ημερών πρόβλεψης.

Create Seasonal Histories

Το συγκεκριμένο script χρησιμοποιείται για τη δημιουργία αρχείων αξιολόγησης για επιλεγμένες εποχικές περιπτώσεις του συνόλου δεδομένων IDEAL. Η λειτουργία του είναι

αντίστοιχη με το script δημιουργίας αρχείων ιστορικού για τις βασικές κατοικίες αξιολόγησης (New_histories_creation), με τη διαφορά ότι εδώ οι ημερομηνίες-στόχοι έχουν επιλεγεί ώστε να αντιπροσωπεύουν διαφορετικές εποχές του έτους.

Extrnl_temps_Seasonal

Το συγκεκριμένο script χρησιμοποιείται για τη δημιουργία των θερμοκρασιακών εισόδων που απαιτούνται στις εποχικές περιπτώσεις αξιολόγησης του IDEAL UI/API. Αποτελεί αντίστοιχη διαδικασία με το script δημιουργίας θερμοκρασιακών εισόδων για τις βασικές κατοικίες αξιολόγησης (Ext_temp_for_UI), με τη διαφορά ότι εδώ εφαρμόζεται στους φακέλους των εποχικών case studies που έχουν ήδη δημιουργηθεί στον φάκελο C:/IDEAL_Programming/New_Evaluation/SEASONAL.

Evaluation_Seasonal

Το συγκεκριμένο script χρησιμοποιείται για την αξιολόγηση των προβλέψεων που παράγονται από το IDEAL UI/API στις εποχικές περιπτώσεις μελέτης. Η διαδικασία είναι αντίστοιχη με το βασικό script αξιολόγησης των προβλέψεων για τις επιλεγμένες κατοικίες (New_Evaluation_June), με τη διαφορά ότι εδώ η αξιολόγηση εφαρμόζεται σε εποχικά case studies, τα οποία έχουν οργανωθεί στον φάκελο C:/IDEAL_Programming/New_Evaluation/SEASONAL.

PLEGMA. Σύντομη περιγραφή αρχείων `ipynb`

Προεπεξεργασία

NaNstats4

Το συγκεκριμένο script υλοποιεί συστηματικό έλεγχο των ελλিপών τιμών στα δεδομένα του PLEGMA dataset, τόσο για τα ηλεκτρικά δεδομένα όσο και για τα περιβαλλοντικά χαρακτηριστικά κάθε κατοικίας. Για κάθε σπίτι, το script διαβάζει όλα τα διαθέσιμα αρχεία των φακέλων `Electric_data` και `Environmental_data`, αγνοώντας τα αρχεία μεταδεδομένων συσκευών που δεν περιέχουν μετρήσεις.

Στο τμήμα των ηλεκτρικών δεδομένων εστιάζει ειδικά στη μεταβλητή `P_agg`, εντοπίζοντας συνεχόμενα χρονικά διαστήματα στα οποία εμφανίζονται ελλείπουσες τιμές. Αντίστοιχα, στα περιβαλλοντικά δεδομένα εξετάζει κάθε διαθέσιμη μεταβλητή ξεχωριστά και καταγράφει τα χρονικά διαστήματα στα οποία παρατηρούνται NaN. Για τον εντοπισμό αυτό χρησιμοποιείται ειδική βοηθητική συνάρτηση, η οποία εντοπίζει την αρχή και το τέλος κάθε συνεχόμενου κενού διαστήματος.

Τα αποτελέσματα αποθηκεύονται ανά κατοικία σε αρχεία αναφοράς CSV, τα οποία περιλαμβάνουν πληροφορίες για τον φάκελο προέλευσης, το αρχείο, τη μεταβλητή και το χρονικό εύρος κάθε κενού. Παράλληλα, το script παράγει διαγράμματα διασποράς (scatter plots) για την οπτικοποίηση της παρουσίας NaN στον χρόνο, τόσο για τη μεταβλητή `P_agg` όσο και για τα περιβαλλοντικά χαρακτηριστικά. Στα διαγράμματα αυτά αποτυπώνεται και το ποσοστό των ελλিপών τιμών, διευκολύνοντας τη γρήγορη εκτίμηση της ποιότητας των δεδομένων κάθε κατοικίας.

Final_Dataset

Το συγκεκριμένο script υλοποιεί τη δημιουργία μιας τελικής, καθαρισμένης εκδοχής του PLEGMA dataset, στην οποία έχουν συμπληρωθεί οι ελλειπείς τιμές τόσο στα ηλεκτρικά όσο και στα περιβαλλοντικά δεδομένα. Η διαδικασία ξεκινά από τον αρχικό φάκελο `Clean_Dataset` και παράγει έναν νέο φάκελο εξόδου `Clean_Dataset_Final`, από τον οποίο έχουν ήδη αποκλειστεί οι κατοικίες `House_06`, `House_08` και `House_12`, λόγω εκτεταμένων προβλημάτων πληρότητας.

Για τα ηλεκτρικά δεδομένα, το script επικεντρώνεται στη μεταβλητή `P_agg`. Οι ελλείπουσες τιμές της συμπληρώνονται με βάση παρόμοιες χρονικές θέσεις από το ίδιο σπίτι, χρησιμοποιώντας ως κριτήρια την ημέρα της εβδομάδας, την ώρα και το λεπτό της μέτρησης. Όταν υπάρχουν διαθέσιμες αντίστοιχες τιμές, χρησιμοποιείται ο μέσος όρος τους· διαφορετικά εφαρμόζεται ως fallback ο συνολικός μέσος όρος της μεταβλητής. Με τον τρόπο αυτό διατηρείται όσο το δυνατόν περισσότερο η χρονική και συμπεριφορική δομή της κατανάλωσης.

Για τα περιβαλλοντικά δεδομένα, εφαρμόζεται γραμμική παρεμβολή (linear interpolation) σε όλες τις μεταβλητές, με συμπλήρωση και προς τις δύο κατευθύνσεις όπου χρειάζεται. Έτσι εξασφαλίζεται ένα πλήρες και συνεκτικό σύνολο περιβαλλοντικών χαρακτηριστικών για κάθε κατοικία.

Τέλος, τα επεξεργασμένα αρχεία αποθηκεύονται στη νέα δομή φακέλων, διατηρώντας τον αρχικό διαχωρισμό σε `Electric_data` και `Environmental_data`. Το script επομένως παράγει τη βάση πάνω στην οποία στηρίχθηκε η επόμενη φάση συγχρονισμού, ενοποίησης και μοντελοποίησης των δεδομένων.

Houses_Processing

Το συγκεκριμένο script αναλαμβάνει τη μετατροπή των προεπεξεργασμένων δεδομένων του PLEGMA σε ένα ενιαίο και συγχρονισμένο dataset ανά κατοικία, κατάλληλο για τα επόμενα στάδια ανάλυσης και μοντελοποίησης. Ως είσοδο χρησιμοποιεί τον φάκελο Clean_Dataset_Final, δηλαδή το dataset μετά τη συμπλήρωση των ελλিপών τιμών, και παράγει για κάθε κατοικία ένα ενιαίο αρχείο CSV στον φάκελο Processed_Houses_Final. Από τη διαδικασία εξαιρούνται οι κατοικίες House_06, House_08 και House_12.

Αρχικά, για κάθε σπίτι φορτώνονται όλα τα αρχεία των ηλεκτρικών δεδομένων, εξαιρουμένων των αρχείων metadata συσκευών. Στη συνέχεια, διατηρούνται μόνο οι έγκυρες τιμές της μεταβλητής P_agg και υπολογίζεται η καταναλωθείσα ενέργεια ανά 30λεπτο μέσω αναδειγματοληψίας (resampling). Η ενέργεια αντιστοιχίζεται στο τέλος κάθε 30λεπτου χρονικού διαστήματος, ώστε να υπάρχει χρονική συνέπεια με τη λογική που υιοθετήθηκε στην επεξεργασία του dataset.

Παράλληλα, φορτώνονται τα περιβαλλοντικά δεδομένα της ίδιας κατοικίας, εφαρμόζεται παρεμβολή όπου χρειάζεται και στη συνέχεια υπολογίζεται ο μέσος όρος κάθε περιβαλλοντικής μεταβλητής ανά 30λεπτο, επίσης με χρονική σφραγίδα στο τέλος του διαστήματος. Έτσι, τόσο τα ηλεκτρικά όσο και τα περιβαλλοντικά δεδομένα αποκτούν κοινή χρονική ανάλυση και κοινή χρονική αναφορά.

Στο τελικό στάδιο, τα δύο σύνολα δεδομένων συγχωνεύονται με βάση το κοινό timestamp, παράγοντας για κάθε κατοικία ένα ενιαίο dataset που περιλαμβάνει την κατανάλωση ενέργειας και τα αντίστοιχα περιβαλλοντικά χαρακτηριστικά για κάθε 30λεπτο. Το αποτέλεσμα αποθηκεύεται σε ξεχωριστό αρχείο ανά σπίτι.

PLEGMA_hourly_dataset

Το συγκεκριμένο script υλοποιεί τη μετατροπή των ενοποιημένων δεδομένων του PLEGMA από ημίωρη σε ωριαία ανάλυση και παράλληλα δημιουργεί ένα master dataset που θα επεξεργαστεί περαιτέρω και στο επόμενο script. Ως είσοδο λαμβάνει τα αρχεία του φακέλου Processed_Houses_Final, τα οποία περιέχουν ήδη συγχρονισμένα ηλεκτρικά και περιβαλλοντικά δεδομένα ανά 30 λεπτά, και παράγει για κάθε κατοικία ένα νέο ωριαίο αρχείο στον φάκελο Processed_Houses_Hourly, καθώς και ένα συνολικό αρχείο PLEGMA_hourly_master_dataset.csv.

Αρχικά, το script διατηρεί μόνο τις κατοικίες που έχουν επιλεγεί για την τελική ανάλυση και ελέγχει ότι σε κάθε αρχείο υπάρχουν οι αναμενόμενες μεταβλητές, όπως η κατανάλωση ενέργειας και τα βασικά περιβαλλοντικά χαρακτηριστικά. Στη συνέχεια εφαρμόζεται μετατροπή από ημίωρη σε ωριαία ανάλυση. Η διαδικασία αυτή βασίζεται σε μετατόπιση των χρονικών σφραγίδων κατά 30 λεπτά, ώστε κάθε ωριαία εγγραφή να αναφέρεται στο τέλος της ώρας και να προκύπτει από τη σωστή συνένωση των δύο αντίστοιχων ημίωρων διαστημάτων. Η κατανάλωση ενέργειας αθροίζεται σε ωριαίο επίπεδο, ενώ οι περιβαλλοντικές μεταβλητές υπολογίζονται ως μέσοι όροι.

Μετά τον χρονικό μετασχηματισμό, το script προσθέτει μια σειρά από χρονικά χαρακτηριστικά που είναι χρήσιμα για τα μοντέλα πρόβλεψης, όπως η ώρα, η ημέρα της εβδομάδας, η ένδειξη Σαββατοκύριακου, ο μήνας, η εποχή και η ένδειξη αργίας με βάση προκαθορισμένο σύνολο ελληνικών αργιών. Έτσι, κάθε ωριαία εγγραφή περιλαμβάνει όχι μόνο την κατανάλωση και τα περιβαλλοντικά δεδομένα, αλλά και εμπλουτισμένη χρονική πληροφορία.

Τέλος, για κάθε κατοικία αποθηκεύεται το αντίστοιχο ωριαίο αρχείο και παράλληλα όλα τα επιμέρους αρχεία ενώνονται σε ένα ενιαίο master dataset, ταξινομημένο ανά κατοικία και χρονική στιγμή.

PLEGMA_final_hourly_dataset

Το συγκεκριμένο script υλοποιεί την τελική φάση προεπεξεργασίας του PLEGMA dataset, συνδυάζοντας τα ωριαία δεδομένα κατανάλωσης και περιβαλλοντικών χαρακτηριστικών με τα δημογραφικά και κτηριακά μεταδεδομένα των κατοικιών. Ως είσοδο λαμβάνει το αρχείο PLEGMA_hourly_master_dataset.csv, το οποίο περιέχει τα πλήρως επεξεργασμένα ωριαία δεδομένα, και το αρχείο PLEGMA_metadata_filled.csv, το οποίο περιλαμβάνει τα χαρακτηριστικά των κατοικιών.

Αρχικά, τα δύο σύνολα δεδομένων συγχωνεύονται με βάση το home_id, έτσι ώστε κάθε ωριαία εγγραφή να εμπλουτιστεί με πληροφορίες όπως τύπος κατοικίας, σύστημα θέρμανσης, αριθμός ενοίκων και άλλα κοινωνικοδημογραφικά χαρακτηριστικά. Στη συνέχεια, πραγματοποιείται μετατροπή της ονομασίας των κατοικιών από τη μορφή House_XX στη μορφή homeXX, ώστε να υπάρχει συνέπεια με τη δομή που χρησιμοποιείται στο IDEAL dataset και στα μοντέλα.

Το τελικό dataset ταξινομείται χρονικά για κάθε κατοικία και αποθηκεύεται στο αρχείο PLEGMA_final_hourly_dataset.csv. Το αρχείο αυτό αποτελεί την ολοκληρωμένη αναπαράσταση του PLEGMA dataset, καθώς συνδυάζει κατανάλωση, περιβαλλοντικά δεδομένα, χρονικά χαρακτηριστικά και μεταδεδομένα σε ένα ενιαίο σύνολο.

house01_filler

Το συγκεκριμένο script εφαρμόζει στοχευμένη συμπλήρωση ελλিপών τιμών στα περιβαλλοντικά χαρακτηριστικά μιας συγκεκριμένης κατοικίας του PLEGMA dataset. Ως είσοδο λαμβάνει το ήδη επεξεργασμένο αρχείο μιας κατοικίας από τον φάκελο Processed_Houses_Final και επικεντρώνεται αποκλειστικά στις μεταβλητές θερμοκρασίας και υγρασίας.

Αρχικά, το script φορτώνει τα δεδομένα και εμφανίζει τον αριθμό των ελλিপών τιμών πριν από την επεξεργασία, ώστε να υπάρχει σαφής εικόνα της αρχικής κατάστασης. Στη συνέχεια, μετατρέπει τη χρονική μεταβλητή σε κατάλληλη μορφή και ταξινομεί τα δεδομένα χρονικά, κάτι που είναι απαραίτητο για τη σωστή εφαρμογή της παρεμβολής.

Η συμπλήρωση των NaN πραγματοποιείται με γραμμική παρεμβολή (linear interpolation) για τις περιβαλλοντικές μεταβλητές, εκμεταλλευόμενη τη χρονική συνέχεια των μετρήσεων. Επιπλέον, εφαρμόζεται συμπλήρωση προς τα εμπρός και προς τα πίσω (forward fill και backward fill) για την κάλυψη πιθανών κενών στην αρχή ή στο τέλος της χρονοσειράς. Με τον τρόπο αυτό εξασφαλίζεται πλήρης κάλυψη χωρίς ελλείψεις στις συγκεκριμένες μεταβλητές.

Τέλος, το script επανελέγχει την ύπαρξη NaN μετά την επεξεργασία και αποθηκεύει το ενημερωμένο αρχείο, είτε αντικαθιστώντας το αρχικό είτε δημιουργώντας εναλλακτικό αρχείο.

zero_consumption_filler

Το συγκεκριμένο script χρησιμοποιείται για τον εντοπισμό και τη διόρθωση συνεχόμενων χρονικών διαστημάτων στα οποία η μεταβλητή energy_Wh λαμβάνει μηδενικές τιμές, σε περιπτώσεις όπου αυτά τα μηδενικά θεωρούνται ύποπτα ή μη ρεαλιστικά. Η διαδικασία εφαρμόζεται μόνο σε συγκεκριμένες κατοικίες και στοχεύει σε blocks μηδενικών τιμών μεγαλύτερα από ένα προκαθορισμένο κατώφλι μήκους.

Αρχικά, το script φορτώνει τα αρχεία των επιλεγμένων κατοικιών, μετατρέπει το timestamp σε χρονική μορφή και ταξινομεί τις εγγραφές χρονολογικά. Στη συνέχεια εντοπίζει τα συνεχόμενα blocks όπου η τιμή της energy_Wh είναι μηδενική, ομαδοποιώντας διαδοχικές μηδενικές εγγραφές και υπολογίζοντας για κάθε block την αρχή, το τέλος και το πλήθος των χρονικών βημάτων που περιλαμβάνει.

Όταν ένα block υπερβαίνει το καθορισμένο όριο μήκους, οι αντίστοιχες τιμές επισημαίνονται προς διόρθωση. Οι μηδενικές τιμές αυτών των blocks αντικαθίστανται προσωρινά με κενές τιμές και στη συνέχεια συμπληρώνονται με γραμμική παρεμβολή, αξιοποιώντας τις γειτονικές μετρήσεις της χρονοσειράς. Για την κάλυψη πιθανών κενών στην αρχή ή στο τέλος της σειράς εφαρμόζεται επιπλέον forward fill και backward fill. Τέλος, αφαιρούνται οι βοηθητικές στήλες και το διορθωμένο αρχείο αποθηκεύεται ξανά.

EDA_PLEGMA

Το συγκεκριμένο script υλοποιεί μια αρχική διερευνητική ανάλυση του τελικού ωριαίου dataset του PLEGMA, με στόχο την κατανόηση των βασικών προτύπων κατανάλωσης και των σχέσεων τους με χρονικούς και περιβαλλοντικούς παράγοντες. Αρχικά φορτώνεται το αρχείο PLEGMA_final_hourly_dataset.csv, μετατρέπεται σωστά η χρονική μεταβλητή και πραγματοποιούνται βασικοί έλεγχοι για τη δομή του συνόλου δεδομένων, όπως το χρονικό εύρος κάλυψης, ο αριθμός των κατοικιών και οι διαθέσιμες στήλες.

Στη συνέχεια, το script εξετάζει τη μέση ωριαία κατανάλωση στο σύνολο του dataset, παράγοντας το γενικό ημερήσιο προφίλ κατανάλωσης. Έπειτα συγκρίνει τη συμπεριφορά μεταξύ καθημερινών και Σαββατοκύριακου, ώστε να αναδειχθούν πιθανές διαφοροποιήσεις στον τρόπο χρήσης της ενέργειας ανάλογα με τον τύπο της ημέρας.

Επιπλέον, αναλύεται η σχέση της κατανάλωσης με την εξωτερική θερμοκρασία. Για τον σκοπό αυτό, η θερμοκρασία ομαδοποιείται σε διαστήματα του 1°C και υπολογίζεται η μέση κατανάλωση για κάθε θερμοκρασιακό bin, επιτρέποντας την αποτύπωση της συνολικής τάσης μεταξύ καιρικών συνθηκών και ενεργειακής ζήτησης. Τέλος, μελετάται η κατανομή της κατανάλωσης μέσω ιστογράμματος, ώστε να αποτυπωθεί η διασπορά και η γενική μορφή των τιμών στο dataset.

Παράλληλα, το script αποθηκεύει τόσο τα παραγόμενα γραφήματα όσο και τους βασικούς συνοπτικούς πίνακες αποτελεσμάτων, όπως τα περιγραφικά στατιστικά της κατανάλωσης και τα συγκεντρωτικά αρχεία των επιμέρους αναλύσεων. Με αυτόν τον τρόπο, η EDA δεν λειτουργεί μόνο ως οπτική επιθεώρηση των δεδομένων, αλλά και ως οργανωμένη βάση τεκμηρίωσης για την ερμηνεία των χαρακτηριστικών του PLEGMA.

Baselines_PLEGMA

Το συγκεκριμένο script υλοποιεί μια συστηματική αξιολόγηση απλών baseline προσεγγίσεων πρόβλεψης πάνω στο τελικό ωριαίο dataset του PLEGMA. Στόχος του είναι να δημιουργηθεί ένα σαφές σημείο αναφοράς για την απόδοση των πιο σύνθετων μοντέλων μηχανικής μάθησης, δείχνοντας τι επίπεδο ακρίβειας μπορεί να επιτευχθεί με απλούς κανόνες βασισμένους σε ιστορικά ή μέσους όρους.

Αρχικά, φορτώνεται το PLEGMA_final_hourly_dataset.csv, γίνεται χρονική ταξινόμηση ανά κατοικία και δημιουργούνται δύο βασικές χρονικές υστερήσεις: η lag_1, που αντιστοιχεί στην κατανάλωση της αμέσως προηγούμενης ώρας, και η lag_24, που αντιστοιχεί στην κατανάλωση της ίδιας ώρας της προηγούμενης ημέρας. Στη συνέχεια, το dataset χωρίζεται χρονικά σε σύνολο εκπαίδευσης και σύνολο ελέγχου με αναλογία 80/20, ώστε η αξιολόγηση να ακολουθεί τη χρονική σειρά των δεδομένων.

Πάνω σε αυτόν τον διαχωρισμό υλοποιούνται πέντε διαφορετικά baselines. Το πρώτο βασίζεται στην τιμή της ίδιας ώρας της προηγούμενης ημέρας. Το δεύτερο χρησιμοποιεί τον συνολικό μέσο όρο κατανάλωσης ανά ώρα. Το τρίτο επεκτείνει αυτή τη λογική λαμβάνοντας υπόψη και τη διάκριση μεταξύ καθημερινών και Σαββατοκύριακου. Το τέταρτο είναι πιο εξειδικευμένο, καθώς προβλέπει με βάση τον μέσο όρο ανά ώρα για κάθε συγκεκριμένη κατοικία. Τέλος, το πέμπτο baseline εφαρμόζει persistence μίας ώρας, δηλαδή θεωρεί ότι η επόμενη τιμή θα είναι ίση με την αμέσως προηγούμενη.

Κάθε baseline αξιολογείται με τους δείκτες MAE, RMSE και MAPE, ενώ καταγράφεται και ο αριθμός των έγκυρων δειγμάτων που χρησιμοποιήθηκαν στην εκτίμηση. Τα αποτελέσματα αποθηκεύονται σε CSV και παράγονται επιπλέον γραφήματα σύγκρισης για κάθε μετρική, ώστε να είναι εύκολη η οπτική αποτίμηση της σχετικής απόδοσης των baseline μεθόδων.

permutation_importance

Το συγκεκριμένο script χρησιμοποιείται για την ερμηνεία της συμπεριφοράς των τελικών μοντέλων πρόβλεψης στο PLEGMA, μέσω της τεχνικής permutation importance. Στόχος του είναι να εκτιμηθεί η σχετική συμβολή κάθε χαρακτηριστικού στην απόδοση των μοντέλων, μετρώντας πόσο επιδεινώνεται το σφάλμα όταν οι τιμές ενός χαρακτηριστικού αναδιατάσσονται τυχαία.

Αρχικά, το script φορτώνει το τελικό dataset PLEGMA_final_hourly_dataset.csv, καθώς και τα ήδη εκπαιδευμένα μοντέλα Random Forest, XGBoost και LightGBM, μαζί με τις αντίστοιχες λίστες χαρακτηριστικών που χρησιμοποιήθηκαν κατά την εκπαίδευση. Η ανάλυση πραγματοποιείται ξεχωριστά για δύο τρόπους λειτουργίας: το no-history, όπου δεν χρησιμοποιούνται ιστορικά χαρακτηριστικά κατανάλωσης, και το with-history, όπου προστίθενται χρονικές υστερήσεις και κυλιόμενοι μέσοι όροι, όπως lag_1h, lag_24h, lag_168h, roll_mean_24h και roll_mean_168h.

Στη συνέχεια, το dataset προεπεξεργάζεται κατάλληλα, με one-hot encoding για τις κατηγορικές μεταβλητές, ενώ στο σενάριο with-history δημιουργούνται και τα απαραίτητα lag-based χαρακτηριστικά. Ακολουθεί χρονικός διαχωρισμός σε train και test, όπου το test set ορίζεται ως οι τελευταίες 30 ημέρες των διαθέσιμων δεδομένων. Πάνω στο test set υπολογίζεται η permutation importance κάθε χαρακτηριστικού με κριτήριο το negative mean absolute error, ώστε να αποτιμάται η πραγματική συμβολή του στην ακρίβεια πρόβλεψης.

Για κάθε μοντέλο και για κάθε σενάριο, τα αποτελέσματα αποθηκεύονται σε αρχεία CSV και παράγονται αντίστοιχα διαγράμματα με τα σημαντικότερα χαρακτηριστικά. Παράλληλα, δημιουργείται και ένα συγκεντρωτικό αρχείο με όλα τα αποτελέσματα, διευκολύνοντας τη συγκριτική ανάλυση μεταξύ μοντέλων και σεναρίων.

correlation

Το συγκεκριμένο σύνολο κώδικα διερευνά κατά πόσο η διάρκεια παραμονής των ενοίκων σε μια κατοικία σχετίζεται με την ηλεκτρική κατανάλωση στο PLEGMA dataset. Για τον σκοπό αυτό, αρχικά φορτώνεται το τελικό ωριαίο dataset και η κατηγορική μεταβλητή years_in_house μετατρέπεται σε δυαδικές στήλες μέσω one-hot encoding, με ιδιαίτερη έμφαση στην κατηγορία gt_5_years.

Στο πρώτο στάδιο πραγματοποιείται ανάλυση συσχέτισης μεταξύ της μεταβλητής στόχου consumption_Wh και ενός συνόλου βασικών αριθμητικών χαρακτηριστικών, όπως θερμοκρασία, υγρασία, αριθμός δωματίων και αριθμός ενοίκων, μαζί με τη δυαδική ένδειξη years_in_house_gt_5_years. Με αυτόν τον τρόπο ελέγχεται αν η μακροχρόνια παραμονή στην κατοικία παρουσιάζει κάποια γραμμική σχέση με την κατανάλωση.

Στη συνέχεια, η κατανάλωση αναλύεται συγκριτικά μεταξύ των δύο ομάδων — κατοικίες με παραμονή άνω των 5 ετών και κατοικίες με μικρότερη διάρκεια παραμονής — μέσω υπολογισμού περιγραφικών στατιστικών, όπως μέσος όρος, διάμεσος, τυπική απόκλιση και πλήθος παρατηρήσεων. Παράλληλα, παράγεται boxplot ώστε να αποτυπωθεί οπτικά η κατανομή της κατανάλωσης στις δύο κατηγορίες.

Τέλος, το script επεκτείνει την ανάλυση εξετάζοντας αν η κατηγορία years_in_house συνδέεται και με άλλα χαρακτηριστικά της κατοικίας ή του νοικοκυριού, όπως ο μέσος

αριθμός δωματίων, ο μέσος αριθμός ενοίκων και, εφόσον υπάρχει διαθέσιμη πληροφορία, η κατανομή του εισοδηματικού επιπέδου.

ML Scripts

RF_for_API3

Το συγκεκριμένο script χρησιμοποιείται για την εκπαίδευση των τελικών μοντέλων Random Forest που ενσωματώνονται στο περιβάλλον API/UI για το σύνολο δεδομένων PLEGMA. Εκπαιδεύονται δύο εκδοχές του μοντέλου: μία για το σενάριο με χρήση ιστορικών δεδομένων και μία για το σενάριο χωρίς χρήση ιστορικών δεδομένων.

Αρχικά, το script φορτώνει το τελικό ωριαίο σύνολο δεδομένων PLEGMA_final_hourly_dataset.csv, το οποίο περιλαμβάνει τις τιμές κατανάλωσης, τις περιβαλλοντικές μεταβλητές, τα χρονικά χαρακτηριστικά και τα στατικά χαρακτηριστικά των κατοικιών. Τα δεδομένα ταξινομούνται χρονικά ανά κατοικία, ενώ οι κατοικίες που είχαν ήδη αποκλειστεί από την ανάλυση λόγω εκτεταμένων ελλείψεων μπορούν να αφαιρεθούν αν υπάρχουν ακόμη στο τελικό αρχείο. Στη συνέχεια, η μεταβλητή στόχος consumption_Wh μετατρέπεται σε αριθμητική μορφή, αφαιρούνται οι μη έγκυρες τιμές και εφαρμόζεται κάτω φράγμα στο μηδέν, ώστε να αποφευχθούν αρνητικές τιμές κατανάλωσης. Το script προσθέτει χρονικά χαρακτηριστικά, όπως ώρα, ημέρα εβδομάδας, μήνα, ημέρα μήνα, εβδομάδα έτους, ένδειξη Σαββατοκύριακου και αργίας. Επιπλέον, δημιουργούνται κυκλικά χαρακτηριστικά για την ώρα, την ημέρα και τον μήνα, ώστε το μοντέλο να μπορεί να αποτυπώσει καλύτερα την περιοδικότητα των δεδομένων. Τα αριθμητικά χαρακτηριστικά συμπληρώνονται με διάμεσο, ενώ τα κατηγορικά χαρακτηριστικά συμπληρώνονται με την τιμή unknown και μετατρέπονται με one-hot encoding.

Στο σενάριο με χρήση ιστορικών δεδομένων, το script δημιουργεί χαρακτηριστικά υστέρησης και κυλιόμενων στατιστικών ανά κατοικία. Συγκεκριμένα, υπολογίζονται lags προηγούμενων ωρών, όπως 1, 2, 3, 6, 12, 24, 48, 72, 168 και 336 ώρες, καθώς και κυλιόμενοι μέσοι όροι, τυπικές αποκλίσεις, ελάχιστες και μέγιστες τιμές για επιλεγμένα χρονικά παράθυρα. Τα χαρακτηριστικά αυτά δημιουργούνται ανά κατοικία, ώστε να αποφεύγεται ανάμειξη ιστορικού μεταξύ διαφορετικών σπιτιών.

Παρότι το home_id χρησιμοποιείται εσωτερικά για τη δημιουργία των lag και rolling χαρακτηριστικών, δεν χρησιμοποιείται ως χαρακτηριστικό εισόδου στο μοντέλο. Το ίδιο ισχύει και για τα train-only home statistics και το consumption regime, τα οποία δεν ενσωματώνονται στην τελική έκδοση του Random Forest. Η επιλογή αυτή έγινε ώστε το μοντέλο να παραμένει γενικό και να μπορεί να χρησιμοποιηθεί στο UI/API χωρίς να απαιτείται από τον χρήστη να δηλώσει κάποιο γνωστό home_id.

Για την εκπαίδευση του μοντέλου με χρήση ιστορικών δεδομένων εφαρμόζεται χρονικός διαχωρισμός των δεδομένων σε train, validation, calibration και test set. Το τελικό test set αντιστοιχεί στις τελευταίες 30 ημέρες του διαθέσιμου χρονικού διαστήματος, ενώ πριν από αυτό διατηρούνται περίοδοι validation και calibration. Το Random Forest εκπαιδεύεται με επιλεγμένες υπερπαραμέτρους, όπως 500 δέντρα, μέγιστο βάθος 24, min_samples_leaf=10, min_samples_split=20, max_features=0.70 και χρήση bootstrap με max_samples=0.85.

Στο σενάριο χωρίς χρήση ιστορικών δεδομένων, το μοντέλο εκπαιδεύεται χωρίς lag ή rolling χαρακτηριστικά και χωρίς χρήση home_id. Χρησιμοποιούνται μόνο χρονικά, περιβαλλοντικά και στατικά χαρακτηριστικά, όπως θερμοκρασίες, υγρασίες, αριθμός δωματίων, αριθμός ενοίκων, ηλικιακή σύνθεση, ύπαρξη βασικών ηλεκτρικών συσκευών, τύπος κατοικίας, περίοδος κατασκευής, εισοδηματική κατηγορία, τύπος θέρμανσης και λοιπά δημογραφικά ή τεχνικά χαρακτηριστικά. Το τελικό no-history μοντέλο εκπαιδεύεται σε μεγαλύτερο τμήμα των δεδομένων, καθώς χρησιμοποιεί και τις περιόδους validation και calibration στο τελικό training set πριν από το test set.

Και στις δύο εκδοχές εφαρμόζεται λογαριθμικός μετασχηματισμός της μεταβλητής στόχου μέσω $\log_{1p}$, ώστε να μειωθεί η επίδραση των ακραίων τιμών κατανάλωσης. Μετά την πρόβλεψη, οι τιμές επανέρχονται στην αρχική κλίμακα Wh με χρήση της αντίστροφης μετατροπής. Στο σενάριο χωρίς χρήση ιστορικών δεδομένων εφαρμόζεται επιπλέον clipping στη μεταβλητή στόχου κατά την εκπαίδευση, ενώ στο σενάριο με χρήση ιστορικών δεδομένων το clipping δεν εφαρμόζεται στην τελική ρύθμιση.

Μετά την εκπαίδευση, το script υπολογίζει διαγνωστικές μετρικές στην περίοδο calibration και τελικές μετρικές στην περίοδο test. Οι μετρικές περιλαμβάνουν MAE, RMSE, MAPE, sMAPE, bias, μέσο ημερήσιο απόλυτο σφάλμα σε kWh και μέσο ημερήσιο ποσοστιαίο σφάλμα. Επιπλέον, υπολογίζονται μετρικές ανά κατοικία, ώστε να εξεταστεί η συμπεριφορά του μοντέλου σε κάθε επιμέρους σπίτι.

Τα εκπαιδευμένα μοντέλα αποθηκεύονται σε ξεχωριστούς φακέλους για το σενάριο με χρήση ιστορικών δεδομένων και για το σενάριο χωρίς χρήση ιστορικών δεδομένων. Για κάθε σενάριο αποθηκεύονται το αρχείο του μοντέλου model.joblib, ο προεπεξεργαστής preprocessor.pkl, το αρχείο feature_config.json, το αρχείο metadata.json, οι προβλέψεις του test set και οι μετρικές ανά κατοικία. Το feature_config.json καταγράφει τα χαρακτηριστικά εισόδου, τη χρήση λογαριθμικού στόχου, την πολιτική ως προς το home_id, καθώς και το αν το μοντέλο χρησιμοποιεί lag και rolling χαρακτηριστικά.

Τέλος, το script δημιουργεί συνολικό αρχείο training_summary.json, στο οποίο συγκεντρώνονται πληροφορίες για την εκπαίδευση και των δύο μοντέλων, οι τελικές μετρικές, οι φάκελοι αποθήκευσης και τα αρχεία που δημιουργήθηκαν. Με τον τρόπο αυτό, η εκπαίδευση των Random Forest μοντέλων για το PLEGMA API/UI γίνεται αναπαραγώγιμη, πλήρως τεκμηριωμένη και συμβατή με τα δύο βασικά σενάρια χρήσης του συστήματος.

XGB_for_API2

Το συγκεκριμένο script χρησιμοποιείται για την εκπαίδευση των τελικών μοντέλων XGBoost που ενσωματώνονται στο περιβάλλον API/UI για το σύνολο δεδομένων PLEGMA. Εκπαιδεύονται δύο εκδοχές του μοντέλου: μία για το σενάριο με χρήση ιστορικών δεδομένων και μία για το σενάριο χωρίς χρήση ιστορικών δεδομένων.

Αρχικά, το script φορτώνει το τελικό ωριαίο σύνολο δεδομένων PLEGMA_final_hourly_dataset.csv, το οποίο περιλαμβάνει τις ωριαίες τιμές κατανάλωσης, τις περιβαλλοντικές μεταβλητές, τα στατικά χαρακτηριστικά των κατοικιών και τα κοινωνικοδημογραφικά χαρακτηριστικά. Η μεταβλητή στόχος consumption_Wh μετατρέπεται σε αριθμητική μορφή, αφαιρούνται μη έγκυρες εγγραφές και οι αρνητικές τιμές κατανάλωσης περιορίζονται στο μηδέν. Στη συνέχεια προστίθενται χρονικά χαρακτηριστικά, όπως ώρα, ημέρα εβδομάδας, μήνας, ημέρα μήνα, εβδομάδα έτους, ένδειξη Σαββατοκύριακου και κυκλικές μεταβλητές για την ώρα, την ημέρα και τον μήνα. Η προεπεξεργασία των χαρακτηριστικών γίνεται με κοινή λογική και για τα δύο σενάρια. Τα αριθμητικά χαρακτηριστικά συμπληρώνονται με τη διάμεσο τιμή, ενώ τα κατηγορικά χαρακτηριστικά συμπληρώνονται με την τιμή unknown και μετατρέπονται με one-hot encoding. Με αυτόν τον τρόπο παράγονται οι τελικοί πίνακες εισόδου που χρησιμοποιούνται από τα XGBoost μοντέλα.

Στο σενάριο με χρήση ιστορικών δεδομένων, το script δημιουργεί lag και rolling χαρακτηριστικά κατανάλωσης ανά κατοικία. Τα lag χαρακτηριστικά περιλαμβάνουν προηγούμενες τιμές κατανάλωσης, όπως 1, 2, 3, 6, 12, 24, 48, 72 και 168 ώρες πριν. Επιπλέον, υπολογίζονται κυλιόμενοι μέσοι όροι, τυπικές αποκλίσεις, ελάχιστες και μέγιστες τιμές για διαφορετικά χρονικά παράθυρα. Η δημιουργία αυτών των χαρακτηριστικών

γίνεται ανά `home_id`, ώστε να διατηρείται η χρονική συνέχεια κάθε κατοικίας και να αποφεύγεται ανάμειξη δεδομένων μεταξύ διαφορετικών σπιτιών.

Παρότι το `home_id` χρησιμοποιείται εσωτερικά για τη δημιουργία των ιστορικών χαρακτηριστικών, δεν χρησιμοποιείται ως χαρακτηριστικό εισόδου στο μοντέλο. Επίσης, δεν χρησιμοποιούνται `train-only` στατιστικά ανά κατοικία, `consumption regimes`, `behavior profiles` ή `KNN profiles`. Η επιλογή αυτή διατηρεί το μοντέλο γενικό και κατάλληλο για χρήση στο UI/API, χωρίς να απαιτείται από τον χρήστη να αντιστοιχιστεί σε κάποια γνωστή κατοικία του αρχικού dataset.

Για το μοντέλο με χρήση ιστορικών δεδομένων εφαρμόζεται χρονικός διαχωρισμός των δεδομένων σε `train`, `early-stop validation`, `calibration validation` και `test set`. Το `test set` αντιστοιχεί στις τελευταίες 30 ημέρες του διαθέσιμου χρονικού διαστήματος. Το XGBoost εκπαιδεύεται με αντικειμενική συνάρτηση `reg:pseudohubererror`, η οποία αποτελεί πιο ανθεκτική επιλογή απέναντι σε ακραίες τιμές σε σχέση με το απλό τετραγωνικό σφάλμα. Η εκπαίδευση χρησιμοποιεί `early stopping`, ώστε να επιλεγεί κατάλληλος αριθμός δέντρων με βάση την απόδοση στην περίοδο `validation`.

Μετά την εκπαίδευση του μοντέλου με χρήση ιστορικών δεδομένων, το script υπολογίζει τις τελικές προβλέψεις στο `test set` και τις αξιολογεί με βασικές μετρικές, όπως MAE, RMSE, MAPE, sMAPE, `bias` και μέσο ημερήσιο απόλυτο σφάλμα σε kWh. Επιπλέον, δημιουργείται προαιρετική ημερήσια βαθμονόμηση μέσω ενός `global daily calibrator`, ο οποίος προσαρμόζει συνολικά το επίπεδο των προβλέψεων με βάση την περίοδο `calibration`. Η βασική πρόβλεψη παραμένει η `prediction_Wh`, ενώ η βαθμονομημένη εκδοχή αποθηκεύεται ως προαιρετική στήλη `prediction_daily_calibrated_Wh`.

Στο σενάριο χωρίς χρήση ιστορικών δεδομένων, το script εκπαιδεύει ένα απλούστερο XGBoost μοντέλο, το οποίο δεν χρησιμοποιεί `lag` ή `rolling` χαρακτηριστικά. Οι εισοδοί περιορίζονται σε χρονικά, περιβαλλοντικά, στατικά, τεχνικά και κοινωνικοδημογραφικά χαρακτηριστικά. Και σε αυτή την περίπτωση δεν χρησιμοποιείται `home_id`, ούτε `behavior profiles` ή `KNN profiles`, καθώς το PLEGMMA περιλαμβάνει μικρό αριθμό κατοικιών και τέτοιες τεχνικές δεν θεωρήθηκαν κατάλληλες για το τελικό γενικό UI/API μοντέλο.

Για το `no-history` μοντέλο εφαρμόζεται αρχικά φάση `tuning` με `early stopping`, χρησιμοποιώντας το αρχικό `train set` και την περίοδο `early-stop validation`. Από τη διαδικασία αυτή επιλέγεται ο τελικός αριθμός δέντρων. Στη συνέχεια το μοντέλο επανεκπαίδευεται στο σύνολο `train`, `early-stop validation` και `calibration validation`, πριν αξιολογηθεί στο ανεξάρτητο `test set`. Στο συγκεκριμένο σενάριο χρησιμοποιείται αντικειμενική συνάρτηση `reg:squarederror`, καθώς αυτή διατηρήθηκε ως η τελική ρύθμιση για το απλό `no-history` μοντέλο.

Και στις δύο εκδοχές εφαρμόζεται λογαριθμικός μετασχηματισμός της μεταβλητής στόχου μέσω `log1p`, με στόχο τη μείωση της επίδρασης πολύ υψηλών τιμών κατανάλωσης. Μετά την πρόβλεψη, οι τιμές επανέρχονται στην αρχική κλίμακα `Wh` μέσω της αντίστροφης μετατροπής. Στο `no-history` μοντέλο εφαρμόζεται επίσης `clipping` στη μεταβλητή στόχου κατά την εκπαίδευση, ενώ στο `with-history` μοντέλο το `clipping` δεν εφαρμόζεται.

Το script αποθηκεύει για κάθε σενάριο το εκπαιδευμένο μοντέλο `model.joblib`, τον προεπεξεργαστή `preprocessor.pkl`, το αρχείο ρυθμίσεων χαρακτηριστικών `feature_config.json`, το αρχείο μεταδεδομένων `metadata.json`, τις προβλέψεις του `test set` και τις μετρικές ανά κατοικία. Στο σενάριο με χρήση ιστορικών δεδομένων αποθηκεύεται επιπλέον το αρχείο `daily_calibrator.json`, το οποίο περιλαμβάνει τις παραμέτρους της προαιρετικής ημερήσιας βαθμονόμησης.

Τέλος, δημιουργείται συνολικό αρχείο `training_summary.json`, το οποίο συγκεντρώνει τις βασικές πληροφορίες της εκπαίδευσης, τις ρυθμίσεις των μοντέλων, τις τελικές μετρικές αξιολόγησης και τα παραγόμενα αρχεία. Με τον τρόπο αυτό, η εκπαίδευση των XGBoost

μοντέλων για το PLEGMA API/UI γίνεται πλήρως τεκμηριωμένη, αναπαραγώγιμη και συμβατή με τα δύο βασικά σενάρια χρήσης του συστήματος.

LGBM_for_API2

Το συγκεκριμένο script χρησιμοποιείται για την εκπαίδευση των τελικών μοντέλων LightGBM που ενσωματώνονται στο περιβάλλον API/UI για το σύνολο δεδομένων PLEGMA. Εκπαιδεύονται δύο εκδοχές του μοντέλου: μία για το σενάριο με χρήση ιστορικών δεδομένων και μία για το σενάριο χωρίς χρήση ιστορικών δεδομένων.

Αρχικά, το script φορτώνει το τελικό ωριαίο σύνολο δεδομένων PLEGMA_final_hourly_dataset.csv, το οποίο περιλαμβάνει τις ωριαίες τιμές κατανάλωσης, τις περιβαλλοντικές μεταβλητές, τα χρονικά χαρακτηριστικά, τα τεχνικά χαρακτηριστικά των κατοικιών και τα κοινωνικοδημογραφικά στοιχεία. Η μεταβλητή στόχος consumption_Wh μετατρέπεται σε αριθμητική μορφή, αφαιρούνται μη έγκυρες εγγραφές και οι αρνητικές τιμές κατανάλωσης περιορίζονται στο μηδέν.

Στη συνέχεια, το script δημιουργεί χρονικά χαρακτηριστικά, όπως ώρα, ημέρα εβδομάδας, μήνα, ημέρα μήνα, εβδομάδα έτους και ένδειξη Σαββατοκύριακου. Επιπλέον, προστίθενται κυκλικά χαρακτηριστικά για την ώρα, την ημέρα και τον μήνα, ώστε το μοντέλο να μπορεί να αναπαραστήσει καλύτερα την περιοδικότητα των δεδομένων. Δημιουργούνται επίσης απλές κατηγοριοποιήσεις στατικών χαρακτηριστικών, όπως ομάδες αριθμού ενοίκων και αριθμού δωματίων.

Η προεπεξεργασία των δεδομένων είναι κοινή και για τα δύο σενάρια. Τα αριθμητικά χαρακτηριστικά συμπληρώνονται με τη διάμεσο τιμή, ενώ τα κατηγορικά χαρακτηριστικά συμπληρώνονται με την τιμή unknown και μετατρέπονται με one-hot encoding. Με αυτόν τον τρόπο δημιουργούνται οι τελικοί πίνακες εισόδου που χρησιμοποιούνται από τα LightGBM μοντέλα.

Στο σενάριο με χρήση ιστορικών δεδομένων, το script δημιουργεί χαρακτηριστικά υστέρησης και κυλιόμενων στατιστικών με βάση την προηγούμενη κατανάλωση κάθε κατοικίας. Τα lag χαρακτηριστικά περιλαμβάνουν προηγούμενες τιμές κατανάλωσης, όπως 1, 2, 3, 6, 12, 24, 48, 72 και 168 ώρες πριν. Παράλληλα, υπολογίζονται κυλιόμενοι μέσοι όροι, τυπικές αποκλίσεις, ελάχιστες και μέγιστες τιμές για διαφορετικά χρονικά παράθυρα. Τα χαρακτηριστικά αυτά δημιουργούνται ανά κατοικία, ώστε να διατηρείται η χρονική συνέχεια και να αποφεύγεται ανάμειξη δεδομένων μεταξύ διαφορετικών σπιτιών.

Το home_id χρησιμοποιείται μόνο εσωτερικά για τη δημιουργία των ιστορικών χαρακτηριστικών και δεν εισάγεται ως χαρακτηριστικό στο μοντέλο. Επίσης, δεν χρησιμοποιούνται home-specific statistics, consumption regimes, behavior profiles ή KNN profiles. Η επιλογή αυτή διατηρεί το μοντέλο γενικό και κατάλληλο για χρήση στο UI/API, όπου ο χρήστης μπορεί να εισάγει χαρακτηριστικά και ιστορικό κατανάλωσης χωρίς να αντιστοιχεί απαραίτητα σε γνωστή κατοικία του αρχικού dataset.

Για το μοντέλο με χρήση ιστορικών δεδομένων εφαρμόζεται χρονικός διαχωρισμός των δεδομένων σε train, early-stop validation, calibration validation και test set. Το test set αντιστοιχεί στις τελευταίες 30 ημέρες του διαθέσιμου χρονικού διαστήματος. Το LightGBM εκπαιδεύεται με αντικειμενική συνάρτηση Huber, η οποία επιλέχθηκε ως πιο ανθεκτική επιλογή απέναντι σε ακραίες τιμές κατανάλωσης. Η εκπαίδευση γίνεται με early stopping, ώστε να επιλεγεί αυτόματα ο κατάλληλος αριθμός boosting iterations.

Μετά την εκπαίδευση του μοντέλου με χρήση ιστορικών δεδομένων, το script υπολογίζει τις προβλέψεις στο test set και τις αξιολογεί με μετρικές όπως MAE, RMSE, MAPE, sMAPE, bias, μέσο ημερήσιο απόλυτο σφάλμα σε kWh και μέσο ημερήσιο ποσοστιαίο σφάλμα. Επιπλέον, δημιουργείται προαιρετική ημερήσια βαθμονόμηση μέσω ενός global daily calibrator, ο οποίος προσαρμόζει το συνολικό επίπεδο των προβλέψεων με βάση την περίοδο calibration. Η βασική πρόβλεψη παραμένει η prediction_Wh, ενώ η

βαθμονομημένη εκδοχή αποθηκεύεται ως προαιρετική στήλη prediction_daily_calibrated_Wh.

Στο σενάριο χωρίς χρήση ιστορικών δεδομένων, το script εκπαιδεύει ένα απλούστερο LightGBM μοντέλο, το οποίο δεν χρησιμοποιεί lag ή rolling χαρακτηριστικά. Οι είσοδοι περιορίζονται σε χρονικά, περιβαλλοντικά, στατικά, τεχνικά και κοινωνικοδημογραφικά χαρακτηριστικά, όπως θερμοκρασίες, υγρασίες, αριθμός δωματίων, αριθμός ενοίκων, σύνθεση νοικοκυριού, ηλεκτρικές συσκευές, τύπος κατοικίας, περίοδος κατασκευής, εισοδηματική κατηγορία και τύπος θέρμανσης. Και σε αυτή την περίπτωση δεν χρησιμοποιούνται home_id, behavior profiles ή KNN profiles.

Για το no-history μοντέλο εφαρμόζεται αρχικά διαδικασία επιλογής του αριθμού των boosting iterations με χρήση early stopping. Στη συνέχεια, το μοντέλο επανεκπαιδεύεται στο σύνολο των διαθέσιμων δεδομένων πριν από το test set, δηλαδή στο train, early-stop validation και calibration validation. Με τον τρόπο αυτό, το τελικό μοντέλο αξιοποιεί περισσότερα δεδομένα εκπαίδευσης πριν αξιολογηθεί στις τελευταίες 30 ημέρες.

Και στις δύο εκδοχές εφαρμόζεται λογαριθμικός μετασχηματισμός της μεταβλητής στόχου μέσω log1p, ώστε να περιοριστεί η επίδραση πολύ υψηλών τιμών κατανάλωσης. Μετά την πρόβλεψη, οι τιμές επανέρχονται στην αρχική κλίμακα Wh μέσω της αντίστροφης μετατροπής. Στο σενάριο χωρίς χρήση ιστορικών δεδομένων εφαρμόζεται επιπλέον clipping στη μεταβλητή στόχου κατά την εκπαίδευση, ενώ στο σενάριο με χρήση ιστορικών δεδομένων το clipping δεν εφαρμόζεται στην τελική ρύθμιση.

Το script αποθηκεύει για κάθε σενάριο το εκπαιδευμένο μοντέλο model.joblib, τον προεπεξεργαστή preprocessor.pkl, το αρχείο χαρακτηριστικών feature_config.json, το αρχείο μεταδεδομένων metadata.json, τις προβλέψεις του test set και τις μετρικές ανά κατοικία. Στο σενάριο με χρήση ιστορικών δεδομένων αποθηκεύεται επιπλέον το αρχείο daily_calibrator.json, το οποίο περιλαμβάνει τις παραμέτρους της προαιρετικής ημερήσιας βαθμονόμησης.

Τέλος, δημιουργείται συνολικό αρχείο training_summary.json, στο οποίο συγκεντρώνονται οι βασικές πληροφορίες της εκπαίδευσης, οι ρυθμίσεις των μοντέλων, οι τελικές μετρικές αξιολόγησης και τα παραγόμενα αρχεία. Με τον τρόπο αυτό, η εκπαίδευση των LightGBM μοντέλων για το PLEGMA API/UI γίνεται πλήρως τεκμηριωμένη, αναπαραγώγιμη και συμβατή με τα δύο βασικά σενάρια χρήσης του συστήματος.

home03_as_history

Το συγκεκριμένο script δημιουργεί ένα πλήρες τεχνητό ιστορικό κατανάλωσης για μία συγκεκριμένη κατοικία του PLEGMA, μεταφέροντας πραγματικά δεδομένα προηγούμενων περιόδων σε μια νέα χρονική αναφορά, συγκεκριμένα στο έτος 2026. Στόχος του είναι να παραχθεί ένα συνεχές ετήσιο αρχείο ιστορικού (history store), το οποίο μπορεί να αξιοποιηθεί από το σύστημα πρόβλεψης σε σενάρια with-history.

Αρχικά, το script φορτώνει από το τελικό ωριαίο dataset μόνο τις απαραίτητες στήλες, δηλαδή το αναγνωριστικό της κατοικίας, το timestamp και την κατανάλωση. Στη συνέχεια φιλτράρει τα δεδομένα για τη συγκεκριμένη κατοικία και ελέγχει ότι υπάρχουν έγκυρες μετρήσεις διαθέσιμες.

Η βασική λογική του script είναι η δημιουργία δύο ξεχωριστών χρονικών blocks. Στο πρώτο block, το διάστημα από 1/1/2023 έως 30/9/2023 μεταφέρεται χρονικά στο διάστημα 1/1/2026 έως 30/9/2026. Στο δεύτερο block, το διάστημα από 1/10/2022 έως 31/12/2022 μεταφέρεται στο διάστημα 1/10/2026 έως 31/12/2026. Με αυτόν τον τρόπο επιτυγχάνεται πλήρης κάλυψη ολόκληρου του 2026 με δεδομένα που προέρχονται από πραγματικές ιστορικές μετρήσεις της ίδιας κατοικίας.

Για κάθε block, το script ανακατασκευάζει πρώτα ένα πλήρες ωριαίο χρονικό πλέγμα για την αρχική περίοδο, ελέγχει τη συνέχειά του και συμπληρώνει τυχόν ελλείψεις μέσω

forward fill και backward fill, ώστε το ιστορικό που θα παραχθεί να είναι ομαλό και συνεχές. Στη συνέχεια, οι τιμές κατανάλωσης αντιστοιχίζονται σε νέο target χρονικό πλέγμα ίδιου μήκους. Τα δύο blocks ενώνονται, ταξινομούνται χρονικά και ελέγχονται με safety checks ώστε να επιβεβαιωθεί ότι το τελικό αποτέλεσμα αποτελεί πλήρη και συνεχόμενη ωριαία χρονοσειρά για όλο το 2026.

Τέλος, το παραγόμενο αρχείο αποθηκεύεται και μπορεί να χρησιμοποιηθεί ως proxy history source στο πλαίσιο των σεναρίων with-history.

PLEGMA. API, UI

install_requirements.bat

Το συγκεκριμένο αρχείο batch υλοποιεί μια πλήρως αυτοματοποιημένη διαδικασία εγκατάστασης του απαραίτητου περιβάλλοντος εκτέλεσης για το demo πρόβλεψης κατανάλωσης του PLEGMA. Στόχος του είναι να εξασφαλίσει ότι το σύστημα μπορεί να στηθεί εύκολα και με ελεγχόμενο τρόπο σε έναν νέο υπολογιστή, χωρίς να απαιτούνται χειροκίνητες παρεμβάσεις από τον χρήστη.

Αρχικά, το script ελέγχει αν υπάρχει διαθέσιμη εγκατάσταση της Python 3.11 ή 3.12, καθώς αυτές θεωρούνται οι υποστηριζόμενες εκδόσεις για την εφαρμογή. Αν δεν εντοπιστεί κάποια από αυτές, η διαδικασία σταματά και εμφανίζεται αντίστοιχο μήνυμα σφάλματος. Στη συνέχεια, το script επιλέγει την κατάλληλη έκδοση Python και δημιουργεί νέο εικονικό περιβάλλον (venv), αφού πρώτα διαγράφει τυχόν προϋπάρχον περιβάλλον για να αποφευχθούν ασυμβατότητες ή κατάλοιπα προηγούμενων εγκαταστάσεων.

Έπειτα, αναβαθμίζεται το pip και εγκαθίστανται αυτόματα όλες οι απαιτούμενες βιβλιοθήκες από το αρχείο requirements.txt. Μετά την ολοκλήρωση της εγκατάστασης, εκτελείται ένας έλεγχος ορθής λειτουργίας (self-test), κατά τον οποίο γίνεται εισαγωγή όλων των βασικών βιβλιοθηκών που απαιτούνται από το έργο, όπως FastAPI, Uvicorn, pandas, numpy, scikit-learn, Gradio, requests, joblib, matplotlib, XGBoost και LightGBM. Αν οποιοδήποτε βήμα αποτύχει, το script διακόπτει τη διαδικασία και εμφανίζει κατάλληλο μήνυμα σφάλματος. Αν όλα ολοκληρωθούν επιτυχώς, εμφανίζεται μήνυμα επιβεβαίωσης ότι η εγκατάσταση ολοκληρώθηκε σωστά.

Σύντομη περιγραφή αρχείων py

history_store

Το συγκεκριμένο αρχείο έχει αντίστοιχη λειτουργία με το αντίστοιχο module του IDEAL και χρησιμοποιείται για τη διαχείριση του ιστορικού κατανάλωσης στο demo σύστημα του PLEGMA. Φορτώνει ένα αρχείο CSV μίας κατοικίας, ελέγχει και κανονικοποιεί τις βασικές στήλες timestamp και consumption_Wh, αφαιρεί μη έγκυρες εγγραφές, μετατρέπει τα δεδομένα σε ωριαία μορφή και επιστρέφει το απαιτούμενο συνεχόμενο ιστορικό παράθυρο πριν από την ημερομηνία πρόβλεψης. Από προεπιλογή χρησιμοποιούνται 168 ώρες ιστορικού, ενώ αν το ακριβές παράθυρο δεν είναι διαθέσιμο μπορεί να χρησιμοποιηθεί το πιο πρόσφατο συνεχόμενο διαθέσιμο ιστορικό. Με αυτόν τον τρόπο το API/UI του PLEGMA μπορεί να υποστηρίξει το σενάριο με χρήση ιστορικών δεδομένων χωρίς να απαιτείται βάση δεδομένων ή δήλωση home_id από τον χρήστη.

model_registry

Το συγκεκριμένο αρχείο έχει αντίστοιχη λειτουργία με το model_registry.py του IDEAL και χρησιμοποιείται για την καταγραφή και διαχείριση των διαθέσιμων μοντέλων του

PLEGMA API/UI. Ορίζει τα τελικά μοντέλα που μπορούν να χρησιμοποιηθούν στο σενάριο χωρίς χρήση ιστορικών δεδομένων και στο σενάριο με χρήση ιστορικών δεδομένων, καθώς και τις διαδρομές προς τους αντίστοιχους φακέλους αποθήκευσης των artifacts. Ως προεπιλεγμένο μοντέλο έχει οριστεί το LightGBM και για τα δύο σενάρια, ενώ τα Random Forest και XGBoost παραμένουν διαθέσιμα ως προαιρετικά μοντέλα σύγκρισης. Το αρχείο υποστηρίζει επίσης παλαιότερες ονομασίες, όπως coldstart και withhistory, ώστε να διατηρείται συμβατότητα με προηγούμενες εκδόσεις του API/UI. Παράλληλα, περιλαμβάνει βοηθητικές συναρτήσεις για την επιλογή του κατάλληλου μοντέλου, την κανονικοποίηση του σεναρίου πρόβλεψης, την εμφάνιση των διαθέσιμων επιλογών στο UI και τον έλεγχο ύπαρξης των απαραίτητων αρχείων κάθε μοντέλου.

inference

Το συγκεκριμένο αρχείο έχει αντίστοιχη λειτουργία με το inference.py του IDEAL και χρησιμοποιείται για την εκτέλεση προβλέψεων ημέρας στο PLEGMA API/UI. Φορτώνει τα κατάλληλα μοντέλα και τους αντίστοιχους προεπεξεργαστές μέσω του model_registry.py, δημιουργεί τα απαιτούμενα χαρακτηριστικά εισόδου και υποστηρίζει τόσο το σενάριο χωρίς χρήση ιστορικών δεδομένων όσο και το σενάριο με χρήση ιστορικών δεδομένων. Η βασική διαφοροποίηση σε σχέση με το IDEAL είναι ότι οι είσοδοι προσαρμόζονται στα διαθέσιμα χαρακτηριστικά του PLEGMA, όπως εσωτερική και εξωτερική θερμοκρασία, υγρασία, αριθμός δωματίων και ενοίκων, δημογραφικά στοιχεία, χαρακτηριστικά συσκευών, τύπος κατοικίας, θέρμανση και τύπος θερμοσίφωνα. Στο σενάριο με χρήση ιστορικών δεδομένων, η πρόβλεψη γίνεται αναδρομικά για το 24ωρο, καθώς κάθε προβλεπόμενη τιμή προστίθεται στο ιστορικό και χρησιμοποιείται για τον υπολογισμό των επόμενων lag και rolling χαρακτηριστικών. Επιπλέον, το αρχείο υποστηρίζει προαιρετική ημερήσια βαθμονόμηση και προσαρμοστική διόρθωση με βάση το πρόσφατο ιστορικό κατανάλωσης, ώστε οι τελικές προβλέψεις να μπορούν να αξιοποιούν καλύτερα το διαθέσιμο προφίλ χρήσης της κατοικίας.

api_app

Το συγκεκριμένο αρχείο έχει αντίστοιχη λειτουργία με το api_app.py του IDEAL και υλοποιεί το FastAPI περιβάλλον του demo συστήματος για το PLEGMA. Παρέχει τα βασικά endpoints /predict, /models, /health και /history/info, μέσω των οποίων γίνεται η εκτέλεση προβλέψεων, η εμφάνιση των διαθέσιμων μοντέλων, ο έλεγχος λειτουργίας του συστήματος και η πρόσβαση σε πληροφορίες για το αρχείο ιστορικού. Η βασική διαφοροποίηση σε σχέση με το IDEAL είναι ότι το request schema έχει προσαρμοστεί στα διαθέσιμα χαρακτηριστικά του PLEGMA, όπως θερμοκρασία, υγρασία, αριθμός δωματίων και ενοίκων, σύνθεση νοικοκυριού, ύπαρξη βασικών ηλεκτρικών συσκευών, τύπος κατοικίας, θέρμανση και τύπος θερμοσίφωνα. Το API υποστηρίζει τόσο το σενάριο χωρίς χρήση ιστορικών δεδομένων όσο και το σενάριο με χρήση ιστορικών δεδομένων, χωρίς να απαιτείται home_id από τον χρήστη. Επιπλέον, ενσωματώνει επιλογή μοντέλου μέσω του model_registry.py, δυνατότητα άντλησης καιρικών δεδομένων από Open-Meteo για επιλεγμένες ελληνικές πόλεις, fallback εισόδους για θερμοκρασία και υγρασία, χρήση ιστορικού από CSV ή από διάνυσμα τιμών, καθώς και προαιρετική προσαρμοστική διόρθωση των προβλέψεων με βάση το πρόσφατο ιστορικό κατανάλωσης.

ui_app

Το συγκεκριμένο αρχείο έχει αντίστοιχη λειτουργία με το ui_app.py του IDEAL και υλοποιεί τη γραφική διεπαφή χρήστη του demo συστήματος για το PLEGMA μέσω Gradio.

Η διεπαφή συνδέεται με το FastAPI backend του PLEGMA, επιτρέπει την επιλογή σεναρίου πρόβλεψης, μοντέλου και ημερομηνίας-στόχου, καθώς και την εισαγωγή των χαρακτηριστικών που είναι διαθέσιμα στο PLEGMA, όπως θερμοκρασία, υγρασία, αριθμός δωματίων και ενοίκων, σύνθεση νοικοκυριού, ηλεκτρικές συσκευές, τύπος κατοικίας, θέρμανση και τύπος θερμοσίφωνα. Υποστηρίζει τόσο το σενάριο χωρίς χρήση ιστορικών δεδομένων όσο και το σενάριο με χρήση ιστορικών δεδομένων, όπου το ιστορικό μπορεί να δοθεί είτε μέσω αρχείου CSV είτε χειροκίνητα ως λίστα τιμών. Επιπλέον, παρέχει λειτουργίες σύγκρισης μοντέλων, συνδυαστικής πρόβλεψης με βάρη ensemble, ανάλυσης σταθερότητας ιστορικού για επιλογή κατάλληλου `max_alpha`, προσαρμοστικής διόρθωσης με βάση το ιστορικό, καθώς και προαιρετικής συμπεριφορικής προσαρμογής συγκεκριμένων ωρών υψηλής ή χαμηλής κατανάλωσης. Τέλος, εμφανίζει τα αποτελέσματα σε μορφή πίνακα και γραφήματος και δίνει τη δυνατότητα αποθήκευσης των προβλέψεων σε CSV, διατηρώντας τις θύρες του PLEGMA ξεχωριστές από το αντίστοιχο περιβάλλον του IDEAL.

Evaluation Scripts (ipynb)

New_histories_creation

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το αντίστοιχο script του IDEAL και χρησιμοποιείται για τη δημιουργία αρχείων αξιολόγησης για επιλεγμένες κατοικίες του PLEGMA. Για κάθε κατοικία και ημερομηνία-στόχο εξάγει τις προηγούμενες 168 ώρες κατανάλωσης, οι οποίες αποθηκεύονται σε αρχείο `history_store.csv`, καθώς και τις πραγματικές τιμές της ημέρας πρόβλεψης, οι οποίες αποθηκεύονται σε αρχείο `actual_target_day.csv`. Παράλληλα, δημιουργούνται αρχεία μεταδεδομένων και συνοπτικό αρχείο `evaluation_cases_summary.csv`, ώστε οι περιπτώσεις αξιολόγησης να είναι οργανωμένες, πλήρεις και άμεσα συμβατές με το PLEGMA UI/API.

Ext_temp_for_UI

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το αντίστοιχο script του IDEAL (`Ext_temp_for_UI`) και χρησιμοποιείται για τη δημιουργία των περιβαλλοντικών εισόδων αξιολόγησης για επιλεγμένες κατοικίες του PLEGMA. Για κάθε κατοικία και ημερομηνία-στόχο εξάγει τις 24 ωριαίες τιμές εσωτερικής και εξωτερικής θερμοκρασίας, καθώς και εσωτερικής και εξωτερικής υγρασίας, σε μορφή κατάλληλη για χρήση στο UI/API. Παράλληλα, αποθηκεύει τα διαθέσιμα στατικά χαρακτηριστικά της κατοικίας, όπως τύπο κατοικίας, αριθμό ενοίκων, συσκευές, θέρμανση και λοιπά δημογραφικά στοιχεία. Τα αποτελέσματα αποθηκεύονται σε αρχεία `environmental_inputs.csv`, `environmental_inputs_for_ui.txt` και `environmental_inputs_metadata.json`, ενώ δημιουργείται και συνοπτικό αρχείο `environmental_inputs_summary.csv` για την οργάνωση των περιπτώσεων αξιολόγησης.

New_Evaluation

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το αντίστοιχο script αξιολόγησης του IDEAL (`New_Evaluation_June`) και χρησιμοποιείται για την αξιολόγηση των προβλέψεων του PLEGMA UI/API σε επιλεγμένες κατοικίες. Φορτώνει τις πραγματικές τιμές κατανάλωσης της ημέρας-στόχου και τα αρχεία προβλέψεων για το σενάριο χωρίς χρήση ιστορικών δεδομένων και το σενάριο με χρήση ιστορικών δεδομένων. Στη συνέχεια υπολογίζει βασικούς δείκτες αξιολόγησης, όπως MAE, RMSE, MAPE, sMAPE, ημερήσιο

σφάλμα κατανάλωσης και σφάλμα κορυφής, ενώ δημιουργεί αρχεία σύγκρισης, γραφήματα και συνοπτικά αποτελέσματα ανά κατοικία και συνολικά για όλα τα εξεταζόμενα σπίτια.

Create_Season_histories

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το προηγούμενο script δημιουργίας αρχείων ιστορικού (New_histories_creation), αλλά εφαρμόζεται στις εποχικές περιπτώσεις αξιολόγησης του PLEGMA. Για κάθε επιλεγμένη κατοικία και εποχική ημερομηνία-στόχο εξάγει τις προηγούμενες 168 ώρες κατανάλωσης και τις πραγματικές τιμές της ημέρας πρόβλεψης. Τα δεδομένα αποθηκεύονται σε αρχεία history_store.csv και actual_target_day.csv, ενώ δημιουργούνται επίσης αρχεία μεταδεδομένων και συνοπτικό αρχείο evaluation_cases_summary.csv. Με αυτόν τον τρόπο οργανώνονται οι απαραίτητες εισοδοί για την εποχική αξιολόγηση του PLEGMA UI/API στο σενάριο με χρήση ιστορικών δεδομένων.

Season_Env_Inputs_for_UI

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το προηγούμενο script περιβαλλοντικών εισόδων (Ext_temp_for_UI), αλλά εφαρμόζεται στις εποχικές περιπτώσεις αξιολόγησης του PLEGMA. Για κάθε επιλεγμένη κατοικία και εποχική ημερομηνία-στόχο εξάγει τις 24 ωριαίες τιμές εσωτερικής και εξωτερικής θερμοκρασίας, καθώς και εσωτερικής και εξωτερικής υγρασίας, μαζί με τα βασικά στατικά χαρακτηριστικά της κατοικίας. Τα δεδομένα αποθηκεύονται σε αρχεία environmental_inputs.csv, environmental_inputs_for_ui.txt και environmental_inputs_metadata.json, ενώ δημιουργείται και συνοπτικό αρχείο environmental_inputs_summary.csv για την οργάνωση των εποχικών περιπτώσεων αξιολόγησης.

Season_Evaluation

Το συγκεκριμένο script έχει αντίστοιχη λειτουργία με το προηγούμενο script αξιολόγησης, αλλά εφαρμόζεται στις εποχικές περιπτώσεις του PLEGMA (New_Evaluation). Φορτώνει τις πραγματικές τιμές κατανάλωσης και τις προβλέψεις του UI/API για το σενάριο χωρίς χρήση ιστορικών δεδομένων και το σενάριο με χρήση ιστορικών δεδομένων. Στη συνέχεια υπολογίζει δείκτες αξιολόγησης, όπως MAE, RMSE, MAPE, sMAPE, ημερήσιο σφάλμα και σφάλμα κορυφής, ενώ δημιουργεί αρχεία σύγκρισης, γραφήματα και συνοπτικά αποτελέσματα ανά κατοικία και συνολικά για όλες τις εποχικές περιπτώσεις.

Τεχνική περιγραφή υλοποίησης συστήματος πρόβλεψης IDEAL

Τεχνική ανάλυση κώδικα για την μηχανική μάθηση

Στο τεχνικό μέρος της εργασίας, βασικός στόχος ήταν η ανάπτυξη ενός ολοκληρωμένου συστήματος μηχανικής μάθησης για την πρόβλεψη της ηλεκτρικής κατανάλωσης κατοικιών. Η υλοποίηση δεν περιορίστηκε στην απλή εκπαίδευση και αξιολόγηση μοντέλων, αλλά οργανώθηκε με τρόπο που να επιτρέπει τη μελλοντική αξιοποίησή τους μέσα από μια λειτουργική εφαρμογή πρόβλεψης. Για τον λόγο αυτό, ο κώδικας καλύπτει όλα τα βασικά στάδια της διαδικασίας: φόρτωση και προεπεξεργασία δεδομένων, δημιουργία χαρακτηριστικών, χρονικό διαχωρισμό των συνόλων εκπαίδευσης και ελέγχου, εκπαίδευση μοντέλων, αξιολόγηση απόδοσης και αποθήκευση των απαραίτητων αρχείων για χρήση από το API και τη γραφική διεπαφή.

Κοινή βάση όλων των μοντέλων αποτέλεσε το τελικό ενοποιημένο σύνολο δεδομένων του IDEAL. Το σύνολο αυτό περιλαμβάνει ωριαίες τιμές ηλεκτρικής κατανάλωσης, περιβαλλοντικά δεδομένα, όπως εξωτερική και εσωτερική θερμοκρασία, καθώς και στατικά χαρακτηριστικά της κατοικίας, όπως ο τύπος κατοικίας, το εμβαδόν, ο αριθμός κατοίκων, η κατηγορία περιοχής και ο αριθμός ηλεκτρικών συσκευών. Πάνω σε αυτά τα δεδομένα δημιουργήθηκαν διαφορετικές εκδοχές μοντέλων, ώστε το τελικό σύστημα να μπορεί να λειτουργεί σε περισσότερα από ένα πρακτικά σενάρια χρήσης.

Η βασική διάκριση της τελικής αρχιτεκτονικής αφορά δύο σενάρια πρόβλεψης. Το πρώτο είναι το σενάριο χωρίς διαθέσιμο ιστορικό κατανάλωσης (`no_history`). Σε αυτή την περίπτωση, η πρόβλεψη βασίζεται μόνο σε πληροφορίες που μπορεί να εισαγάγει ο χρήστης, όπως τα χαρακτηριστικά της κατοικίας, η ημερομηνία στόχου και τα δεδομένα θερμοκρασίας. Το δεύτερο είναι το σενάριο με διαθέσιμο ιστορικό κατανάλωσης (`with_history`). Σε αυτό το σενάριο, εκτός από τα βασικά χαρακτηριστικά, αξιοποιούνται και πρόσφατες ωριαίες τιμές κατανάλωσης, από τις οποίες δημιουργούνται ιστορικά χαρακτηριστικά και κυλιόμενα στατιστικά.

Η διάκριση αυτή είναι ουσιαστική για τη λειτουργικότητα της εφαρμογής. Σε μια νέα κατοικία, όπου δεν υπάρχουν ακόμη διαθέσιμες μετρήσεις κατανάλωσης, το σύστημα μπορεί να παράγει πρόβλεψη με βάση μόνο τα διαθέσιμα στατικά και χρονικά στοιχεία. Αντίθετα, όταν ο χρήστης διαθέτει πρόσφατο ιστορικό κατανάλωσης, το σύστημα μπορεί να το αξιοποιήσει ώστε η πρόβλεψη να προσαρμοστεί καλύτερα στη συμπεριφορά της συγκεκριμένης κατοικίας.

Επειδή η πρόβλεψη ηλεκτρικής κατανάλωσης αποτελεί πρόβλημα χρονοσειρών, ο διαχωρισμός των δεδομένων έγινε με χρονική σειρά και όχι τυχαία. Ένας τυχαίος διαχωρισμός θα μπορούσε να οδηγήσει σε διαρροή πληροφορίας από μελλοντικές παρατηρήσεις προς το σύνολο εκπαίδευσης, με αποτέλεσμα υπερβολικά αισιόδοξη αξιολόγηση. Για τον λόγο αυτό, οι τελευταίες ημέρες του συνόλου δεδομένων χρησιμοποιήθηκαν ως σύνολο ελέγχου, ενώ τα προηγούμενα δεδομένα αξιοποιήθηκαν για την εκπαίδευση, την επικύρωση και, όπου απαιτήθηκε, τη βαθμονόμηση των μοντέλων. Με τον τρόπο αυτό, η αξιολόγηση προσεγγίζει πιο ρεαλιστικά τη διαδικασία πρόβλεψης μελλοντικής κατανάλωσης.

Ιδιαίτερη σημασία δόθηκε στη δημιουργία κατάλληλων χαρακτηριστικών εισόδου. Η χρονική σήμανση των μετρήσεων δεν χρησιμοποιήθηκε αυτούσια, αλλά μετατράπηκε σε

πιο χρήσιμες μεταβλητές, όπως η ώρα της ημέρας, η ημέρα της εβδομάδας, ο μήνας, η ένδειξη Σαββατοκύριακου και η ένδειξη αργίας. Σε αρκετές υλοποιήσεις δημιουργήθηκαν επίσης κυκλικά χρονικά χαρακτηριστικά, ώστε να αποτυπώνεται καλύτερα η περιοδικότητα της κατανάλωσης μέσα στο 24ωρο, στην εβδομάδα και στο έτος.

Στο σενάριο με ιστορικό κατανάλωσης, δημιουργήθηκαν επιπλέον χαρακτηριστικά που βασίζονται σε προηγούμενες τιμές της ίδιας χρονοσειράς. Τέτοια χαρακτηριστικά είναι οι προηγούμενες ωριαίες τιμές κατανάλωσης, οι κυλιόμενοι μέσοι όροι, η κυλιόμενη τυπική απόκλιση, καθώς και ελάχιστες και μέγιστες τιμές σε προηγούμενα χρονικά παράθυρα. Τα χαρακτηριστικά αυτά υπολογίστηκαν ξεχωριστά για κάθε κατοικία, ώστε να μη γίνεται ανάμειξη δεδομένων από διαφορετικές χρονοσειρές.

Η μεταβλητή στόχος της πρόβλεψης ήταν η ωριαία κατανάλωση ηλεκτρικής ενέργειας σε Wh. Επειδή οι τιμές κατανάλωσης μπορεί να εμφανίζουν ασυμμετρία, αιχμές και μεγάλες διακυμάνσεις, σε αρκετά μοντέλα εφαρμόστηκε λογαριθμικός μετασχηματισμός της μεταβλητής στόχου. Με τον τρόπο αυτό η εκπαίδευση πραγματοποιείται σε πιο σταθερή αριθμητική κλίμακα. Μετά την παραγωγή των προβλέψεων εφαρμόζεται ο αντίστροφος μετασχηματισμός, ώστε οι τελικές τιμές να επιστρέφουν σε Wh και να είναι άμεσα ερμηνεύσιμες.

Η προεπεξεργασία περιλαμβάνει επίσης χειρισμό ελλειπουσών τιμών και μετατροπή των κατηγορικών χαρακτηριστικών σε κατάλληλη αριθμητική μορφή. Οι αριθμητικές μεταβλητές συμπληρώνονται με κατάλληλες τιμές, ενώ οι κατηγορικές μεταβλητές, όπως ο τύπος κατοικίας και η κατηγορία περιοχής, κωδικοποιούνται ώστε να μπορούν να χρησιμοποιηθούν από τα μοντέλα. Η διαδικασία αυτή δεν εφαρμόζεται μόνο κατά την εκπαίδευση, αλλά αποθηκεύεται και επαναχρησιμοποιείται κατά τη φάση πρόβλεψης, ώστε να υπάρχει συνέπεια ανάμεσα στην εκπαίδευση και στη λειτουργική χρήση της εφαρμογής. Στο πλαίσιο της εργασίας χρησιμοποιήθηκαν τρεις βασικές οικογένειες μοντέλων: Random Forest, XGBoost και LightGBM. Το Random Forest βασίζεται στον συνδυασμό πολλών δέντρων απόφασης και προσφέρει σταθερότητα και ανθεκτικότητα σε διαφορετικά σύνολα δεδομένων. Τα XGBoost και LightGBM ανήκουν στις μεθόδους ενισχυτικής μάθησης, όπου τα δέντρα εκπαιδεύονται διαδοχικά με στόχο τη σταδιακή διόρθωση των σφαλμάτων των προηγούμενων βημάτων. Με αυτόν τον τρόπο μπορούν να αποτυπώσουν πιο σύνθετες μη γραμμικές σχέσεις, απαιτούν όμως πιο προσεκτική ρύθμιση.

Η αξιολόγηση των μοντέλων πραγματοποιήθηκε με κοινές μετρικές, ώστε τα αποτελέσματα να είναι συγκρίσιμα. Χρησιμοποιήθηκαν μετρικές ωριαίου σφάλματος, όπως το μέσο απόλυτο σφάλμα, η ρίζα του μέσου τετραγωνικού σφάλματος, το μέσο απόλυτο ποσοστιαίο σφάλμα και το συμμετρικό μέσο απόλυτο ποσοστιαίο σφάλμα. Επιπλέον, υπολογίστηκαν μετρικές που αφορούν το συνολικό ημερήσιο σφάλμα, ώστε να αξιολογείται όχι μόνο η ακρίβεια ανά ώρα, αλλά και η ικανότητα του μοντέλου να εκτιμά σωστά τη συνολική ημερήσια κατανάλωση.

Ένα κρίσιμο στοιχείο της τελικής υλοποίησης είναι η αποθήκευση των συνοδευτικών αρχείων κάθε μοντέλου. Για κάθε εκπαιδευμένο μοντέλο αποθηκεύεται το ίδιο το μοντέλο, ο μηχανισμός προεπεξεργασίας, το αρχείο ρυθμίσεων των χαρακτηριστικών, τα μεταδεδωμένα εκπαιδευσης και οι προβλέψεις του συνόλου ελέγχου. Τα αρχεία αυτά είναι απαραίτητα ώστε το API να μπορεί να φορτώσει το σωστό μοντέλο και να εφαρμόσει την ίδια ακριβώς διαδικασία προεπεξεργασίας που χρησιμοποιήθηκε κατά την εκπαίδευση.

Με αυτόν τον τρόπο, η εκπαίδευση και η πρόβλεψη δεν αντιμετωπίζονται ως δύο ανεξάρτητα στάδια, αλλά ως μέρη ενός ενιαίου συστήματος. Τα μοντέλα εκπαιδεύονται εκτός της εφαρμογής, αποθηκεύονται μαζί με όλα τα απαραίτητα στοιχεία τους και στη συνέχεια φορτώνονται από το API όταν ο χρήστης ζητήσει πρόβλεψη. Η γραφική διεπαφή λειτουργεί ως το ανώτερο επίπεδο του συστήματος, επιτρέποντας στον χρήστη να εισάγει

στοιχεία, να επιλέγει σενάριο πρόβλεψης, να συγκρίνει μοντέλα και να βλέπει τα αποτελέσματα σε πίνακες και διαγράμματα.

Συνολικά, η τεχνική υλοποίηση της μηχανικής μάθησης οργανώθηκε ως ολοκληρωμένο σύστημα πρόβλεψης και όχι ως μεμονωμένο πείραμα. Περιλαμβάνει καθαρή προετοιμασία δεδομένων, κατάλληλο χρονικό διαχωρισμό, δημιουργία χαρακτηριστικών, εκπαίδευση διαφορετικών μοντέλων, συγκρίσιμη αξιολόγηση, παραγωγή τελικών αρχείων και διασύνδεση με API και γραφική διεπαφή. Έτσι, το τελικό αποτέλεσμα μπορεί να χρησιμοποιηθεί πρακτικά για την παραγωγή προβλέψεων ηλεκτρικής κατανάλωσης σε διαφορετικά σενάρια διαθεσιμότητας δεδομένων.

Εκπαίδευση Μοντέλων και Παραγωγή Artifacts

Random Forest Regressor. Αρχείο: RF_for_API2.ipynb

Το παρόν script υλοποιεί την τελική διαδικασία εκπαίδευσης και αποθήκευσης των μοντέλων Random Forest που χρησιμοποιούνται στο σύστημα πρόβλεψης ηλεκτρικής κατανάλωσης. Σε αντίθεση με παλαιότερες πειραματικές εκδόσεις, η συγκεκριμένη υλοποίηση είναι προσανατολισμένη στην παραγωγή τελικών αρχείων μοντέλων και συνοδευτικών αρχείων, ώστε τα μοντέλα να μπορούν να αξιοποιηθούν από το API και τη γραφική διεπαφή της εφαρμογής.

Η βασική λογική του script είναι ότι εκπαιδεύονται δύο διαφορετικές εκδοχές Random Forest. Η πρώτη είναι το μοντέλο με ιστορικό κατανάλωσης (with_history), το οποίο χρησιμοποιεί εκτεταμένα ιστορικά χαρακτηριστικά, στατιστικά ανά κατοικία και το home_id ως κατηγορικό χαρακτηριστικό. Η δεύτερη είναι το μοντέλο χωρίς ιστορικό κατανάλωσης (no_history), το οποίο βασίζεται αποκλειστικά σε απλά χρονικά, στατικά και περιβαλλοντικά χαρακτηριστικά, χωρίς να χρησιμοποιεί home_id, ιστορικές μεταβλητές, KNN προφίλ ή προφίλ συμπεριφοράς.

Με αυτόν τον τρόπο, το script καλύπτει δύο διαφορετικές περιπτώσεις πρόβλεψης. Το with_history μοντέλο προορίζεται για περιπτώσεις όπου υπάρχει διαθέσιμο πρόσφατο ιστορικό κατανάλωσης και είναι δυνατή η αξιοποίηση της προηγούμενης συμπεριφοράς της κατοικίας. Το no_history μοντέλο, αντίθετα, προορίζεται για περιπτώσεις όπου δεν υπάρχει διαθέσιμο ιστορικό και η πρόβλεψη πρέπει να βασιστεί μόνο στα γενικά χαρακτηριστικά της κατοικίας, του χρόνου και της θερμοκρασίας.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import json
import math
import shutil
import time
import warnings
from pathlib import Path
from typing import Any, Dict, Tuple

import joblib
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
```

Η pandas χρησιμοποιείται για τη φόρτωση και επεξεργασία των δεδομένων σε μορφή πινάκων, ενώ η numpy χρησιμοποιείται για αριθμητικούς υπολογισμούς, μετασχηματισμούς και χειρισμό διανυσμάτων. Η RandomForestRegressor αποτελεί τον βασικό αλγόριθμο παλινδρόμησης που εκπαιδεύεται. Οι συναρτήσεις mean_absolute_error και mean_squared_error χρησιμοποιούνται για την αξιολόγηση της απόδοσης των μοντέλων.

Η joblib χρησιμοποιείται για την αποθήκευση των εκπαιδευμένων μοντέλων και των μηχανισμών προεπεξεργασίας, ενώ η json χρησιμοποιείται για την αποθήκευση μεταδεδομένων, ρυθμίσεων και πληροφοριών σχετικά με τα χαρακτηριστικά. Η χρήση των ColumnTransformer, Pipeline, SimpleImputer και OneHotEncoder δείχνει ότι η προεπεξεργασία έχει οργανωθεί με τρόπο συμβατό με το οικοσύστημα του scikit-learn και μπορεί να αποθηκευτεί και να επαναχρησιμοποιηθεί κατά τη φάση της πρόβλεψης.

Ρυθμίσεις παραμετροποίησης

Στη συνέχεια ορίζονται οι βασικές διαδρομές και οι σταθερές του script:

```
BASE_DIR = Path("C:/IDEAL_Programming")
DATA_PATH = BASE_DIR / "processed" / "final" /
"IDEAL_final_hourly_dataset.csv"
```

```
OUT_ROOT = BASE_DIR / "processed" / "models" / "final_api_models" / "RF"
WITH_HISTORY_DIR = OUT_ROOT / "with_history"
no_history_DIR = OUT_ROOT / "no_history"
```

```
TIME_COL = "timestamp"
HOME_COL = "home_id"
TARGET_COL = "consumption_Wh"
```

```
RANDOM_STATE = 42
```

Το BASE_DIR ορίζει τον βασικό φάκελο του έργου, ενώ το DATA_PATH δείχνει στο τελικό ενοποιημένο dataset του IDEAL. Ο φάκελος OUT_ROOT είναι ο βασικός φάκελος εξόδου για τα τελικά Random Forest μοντέλα, ενώ οι υποφάκελοι with_history και no_history χρησιμοποιούνται για την ξεχωριστή αποθήκευση των δύο εκδοχών του μοντέλου.

Οι σταθερές TIME_COL, HOME_COL και TARGET_COL δηλώνουν αντίστοιχα τη στήλη του χρόνου, τη στήλη της κατοικίας και τη στήλη της μεταβλητής στόχου. Η χρήση σταθερού RANDOM_STATE συμβάλλει στην αναπαραγωγικότητα των αποτελεσμάτων, αφού διασφαλίζει ότι διαδικασίες όπως η δειγματοληψία και η εκπαίδευση του Random Forest θα είναι επαναλήψιμες.

Χρονικός διαχωρισμός και περίοδοι αξιολόγησης

Το script ορίζει επίσης τη λογική του χρονικού διαχωρισμού:

```
TEST_DAYS = 30
VAL_DAYS_TOTAL = 30
CAL_DAYS = 15
```

Οι τελευταίες 30 ημέρες του dataset χρησιμοποιούνται ως σύνολο ελέγχου. Πριν από αυτές, ορίζεται μια περίοδος επικύρωσης συνολικής διάρκειας 30 ημερών, από την οποία οι τελευταίες 15 ημέρες χρησιμοποιούνται ως περίοδος βαθμονόμησης. Έτσι το σύνολο των δεδομένων χωρίζεται σε τέσσερα τμήματα: training, early-stop validation, calibration validation και test.

Παρότι το Random Forest δεν χρησιμοποιεί early stopping με τον ίδιο τρόπο που χρησιμοποιείται σε μοντέλα boosting, η διατήρηση αυτής της περιόδου βοηθά στη συνέπεια της συνολικής πειραματικής διαδικασίας και στη σύγκριση με άλλα μοντέλα. Η περίοδος

calibration χρησιμοποιείται ειδικά για τη δοκιμή ή εκτίμηση διορθωτικών μηχανισμών, όπως ο regime calibrator στο with_history μοντέλο.

Αφαίρεση προβληματικών κατοικιών

Στο script ορίζονται συγκεκριμένες κατοικίες που αφαιρούνται πριν από την εκπαίδευση:

```
BAD_HOMES = ["home171", "home219", "home228", "home271"]
```

```
REMOVE_BAD_HOMES = True
```

Η αφαίρεση αυτών των κατοικιών έγινε επειδή είχαν χαρακτηριστεί προβληματικές ως προς τη διαθεσιμότητα ή την αξιοπιστία των μεταδεδομένων τους. Η επιλογή αυτή συμβάλλει στη βελτίωση της ποιότητας του τελικού dataset που χρησιμοποιείται για την εκπαίδευση, καθώς αποφεύγεται η εισαγωγή ασυνεπών ή μη ρεαλιστικών πληροφοριών στο μοντέλο.

Μετασχηματισμός της μεταβλητής στόχου και clipping

Η μεταβλητή στόχος μετασχηματίζεται λογαριθμικά:

```
USE_LOG_TARGET = True
```

```
CLIP_TARGET_TRAIN_NOHISTORY = True
```

```
CLIP_TARGET_TRAIN_HISTORY = False
```

```
CLIP_Q_LOW = 0.005
```

```
CLIP_Q_HIGH = 0.995
```

Η επιλογή USE_LOG_TARGET = True σημαίνει ότι η κατανάλωση μετασχηματίζεται με log1p πριν από την εκπαίδευση. Αυτό βοηθά στη σταθεροποίηση της κατανομής της μεταβλητής στόχου, καθώς οι τιμές κατανάλωσης συχνά εμφανίζουν ασυμμετρία και αιχμές.

Στο no_history μοντέλο εφαρμόζεται clipping στο training target, με βάση τα ποσοστημόρια 0.5% και 99.5%. Αντίθετα, στο with_history μοντέλο το clipping έχει απενεργοποιηθεί. Η διαφοροποίηση αυτή δείχνει ότι τα δύο μοντέλα δεν αντιμετωπίζονται απολύτως όμοια, αλλά προσαρμόζονται στις ανάγκες του κάθε σεναρίου. Στο no_history μοντέλο, όπου δεν υπάρχουν ιστορικές μεταβλητές που να εξηγούν τις αιχμές, το clipping βοηθά στη σταθερότητα. Στο with_history μοντέλο, η ίδια η προηγούμενη κατανάλωση μπορεί να βοηθήσει στην ερμηνεία υψηλών τιμών, οπότε το target διατηρείται χωρίς clipping.

Ορισμός χαρακτηριστικών εισόδου

Το script οργανώνει τα χαρακτηριστικά εισόδου σε διαφορετικές ομάδες. Αρχικά ορίζονται τα στατικά αριθμητικά χαρακτηριστικά:

```
STATIC_NUMERIC = [  
    "external_temperature",  
    "internal_temperature",  
    "residents",  
    "total_floor_area_m2",  
    "num_electric_appliances",  
]
```

Τα χαρακτηριστικά αυτά περιγράφουν είτε περιβαλλοντικές συνθήκες είτε βασικά χαρακτηριστικά της κατοικίας και του νοικοκυριού. Η εξωτερική και εσωτερική θερμοκρασία μπορούν να επηρεάσουν την κατανάλωση, ενώ ο αριθμός κατοίκων, το εμβαδόν και ο αριθμός ηλεκτρικών συσκευών σχετίζονται με τη γενική ενεργειακή συμπεριφορά της κατοικίας.

Στη συνέχεια ορίζονται τα εκτεταμένα χρονικά χαρακτηριστικά:

```
TIME_NUMERIC_EXTENDED = [  
    "hour",  
    "day_of_week",  
    "month",  
    "day_of_month",  
    "week_of_year",  
    "is_weekend",  
    "is_holiday",  
    "hour_sin",  
]
```

```

        "hour_cos",
        "day_sin",
        "day_cos",
        "month_sin",
        "month_cos",
    ]

```

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να συλλάβει την περιοδικότητα της κατανάλωσης. Η ώρα της ημέρας, η ημέρα της εβδομάδας και ο μήνας είναι βασικά χρονικά στοιχεία. Τα κυκλικά χαρακτηριστικά, όπως `hour_sin` και `hour_cos`, βοηθούν στην καλύτερη αναπαράσταση περιοδικών μεταβλητών, επειδή αποφεύγουν το πρόβλημα της τεχνητής απόστασης μεταξύ διαδοχικών κυκλικών τιμών, όπως η ώρα 23:00 και η ώρα 00:00.

Για το `no_history` μοντέλο χρησιμοποιείται απλούστερο σύνολο χαρακτηριστικών:

```

NOHISTORY_NUMERIC = [
    "hour",
    "day_of_week",
    "is_weekend",
    "month",
    "external_temperature",
    "total_floor_area_m2",
    "residents",
]

```

```

NOHISTORY_CATEGORICAL = [
    "hometype",
    "urban_rural_class",
]

```

Το `no_history` μοντέλο διατηρεί τη λογική ενός απλού και γενικού μοντέλου. Δεν χρησιμοποιεί `home_id`, ούτε ιστορικά χαρακτηριστικά. Βασίζεται μόνο στη χρονική πληροφορία, στην εξωτερική θερμοκρασία και σε βασικά χαρακτηριστικά της κατοικίας.

Για το `with_history` μοντέλο ορίζονται επιπλέον κατηγορικά χαρακτηριστικά:

```

HISTORY_CATEGORICAL = [
    HOME_COL,
    "hometype",
    "urban_rural_class",
    "consumption_regime",
]

```

Εδώ το `home_id` χρησιμοποιείται ως κατηγορικό χαρακτηριστικό. Αυτό σημαίνει ότι η συγκεκριμένη έκδοσή του `with_history` Random Forest είναι πιο στενά συνδεδεμένη με τις κατοικίες του dataset. Επιπλέον, χρησιμοποιείται η μεταβλητή `consumption_regime`, η οποία εκφράζει την κατηγορία κατανάλωσης στην οποία ανήκει κάθε κατοικία.

Ιστορικά χαρακτηριστικά

Τα ιστορικά χαρακτηριστικά ορίζονται μέσω των λιστών:

```

LAG_HOURS = [1, 2, 3, 6, 12, 24, 48, 72, 168, 336]
ROLLING_WINDOWS = [3, 6, 12, 24, 48, 168]

```

```

REQUIRED_HISTORY_COLS = [
    "lag_1h",
    "lag_24h",
    "lag_168h",
    "lag_336h",
    "roll_mean_24h",
    "roll_mean_168h",
]

```

Η λίστα `LAG_HOURS` δηλώνει τις προηγούμενες χρονικές στιγμές που χρησιμοποιούνται ως χαρακτηριστικά. Για παράδειγμα, το `lag_1h` αντιστοιχεί στην κατανάλωση της

προηγούμενης ώρας, το lag_24h στην ίδια ώρα της προηγούμενης ημέρας, το lag_168h στην ίδια ώρα της προηγούμενης εβδομάδας και το lag_336h σε δύο εβδομάδες πριν.

Η λίστα ROLLING_WINDOWS δηλώνει τα χρονικά παράθυρα για τα οποία υπολογίζονται κυλιόμενοι μέσοι όροι. Τα REQUIRED_HISTORY_COLS δηλώνουν ποια ιστορικά χαρακτηριστικά θεωρούνται απαραίτητα ώστε μια γραμμή να μπορεί να χρησιμοποιηθεί στην εκπαίδευση του with_history μοντέλου. Αν κάποια από αυτά λείπει, η γραμμή αφαιρείται.

Παράμετροι Random Forest

Το script χρησιμοποιεί διαφορετικές παραμέτρους για τα δύο Random Forest μοντέλα:

```
RF_WITH_HISTORY_PARAMS = dict(  
    n_estimators=120,  
    max_depth=20,  
    min_samples_leaf=15,  
    min_samples_split=30,  
    max_features=0.55,  
    bootstrap=True,  
    max_samples=0.80,  
    n_jobs=-1,  
    random_state=RANDOM_STATE + 11,  
    verbose=1,  
)
```

Για το with_history μοντέλο χρησιμοποιούνται 120 δέντρα. Το μέγιστο βάθος περιορίζεται σε 20, ενώ τα min_samples_leaf και min_samples_split έχουν σχετικά αυξημένες τιμές, ώστε να περιοριστεί η υπερπροσαρμογή. Το max_features=0.55 σημαίνει ότι κάθε διάσπαση δέντρου εξετάζει μέρος των διαθέσιμων χαρακτηριστικών, ενώ το max_samples=0.80 σημαίνει ότι κάθε δέντρο εκπαιδεύεται σε υποσύνολο του διαθέσιμου δείγματος. Οι επιλογές αυτές βοηθούν στη γενίκευση και μειώνουν το υπολογιστικό κόστος.

Για το no_history μοντέλο χρησιμοποιείται διαφορετική παραμετροποίηση:

```
RF_no_history_PARAMS = dict(  
    n_estimators=200,  
    max_depth=20,  
    min_samples_leaf=10,  
    min_samples_split=20,  
    max_features=0.7,  
    bootstrap=True,  
    n_jobs=-1,  
    random_state=RANDOM_STATE,  
    verbose=1,  
)
```

Στο no_history μοντέλο χρησιμοποιούνται 200 δέντρα και ελαφρώς λιγότερο αυστηροί περιορισμοί στα φύλλα και στις διασπάσεις. Αυτό είναι λογικό, επειδή το μοντέλο έχει πολύ μικρότερο αριθμό χαρακτηριστικών και επομένως μικρότερο κίνδυνο υπερβολικής πολυπλοκότητας σε σχέση με το with_history μοντέλο.

Συναρτήσεις αξιολόγησης

Το script ορίζει συναρτήσεις για τον υπολογισμό βασικών μετρικών:

```
def compute_metrics(y_true, y_pred):  
    y_true = np.asarray(y_true, dtype=np.float64)  
    y_pred = np.asarray(y_pred, dtype=np.float64)  
  
    return {  
        "MAE": float(mean_absolute_error(y_true, y_pred)),  
        "RMSE": float(math.sqrt(mean_squared_error(y_true, y_pred))),  
        "MAPE_%": mape_safe(y_true, y_pred),  
        "sMAPE_%": smape_safe(y_true, y_pred),  
        "bias_Wh": float(np.mean(y_pred - y_true)),  
    }
```

Η συνάρτηση αυτή υπολογίζει το μέσο απόλυτο σφάλμα, τη ρίζα του μέσου τετραγωνικού σφάλματος, το μέσο απόλυτο ποσοστιαίο σφάλμα, το συμμετρικό μέσο απόλυτο ποσοστιαίο σφάλμα και τη μεροληψία σε Wh. Η μεροληψία δείχνει αν το μοντέλο τείνει να υπερεκτιμά ή να υποεκτιμά την κατανάλωση.

Επιπλέον, υπάρχει συνάρτηση που υπολογίζει και ημερήσιες μετρικές:

```
def evaluate_predictions(df_eval: pd.DataFrame, pred_col: str) -> Dict[str, float]:
    y_true = df_eval[TARGET_COL].values
    y_pred = df_eval[pred_col].values

    metrics = compute_metrics(y_true, y_pred)

    daily = (
        df_eval
        .assign(date=df_eval[TIME_COL].dt.date)
        .groupby([HOME_COL, "date"], as_index=False)
        .agg(
            actual_kWh=(TARGET_COL, lambda x: x.sum() / 1000),
            pred_kWh=(pred_col, lambda x: x.sum() / 1000),
        )
    )
```

Η συνάρτηση αυτή δεν περιορίζεται στην ωριαία αξιολόγηση, αλλά συγκεντρώνει τις προβλέψεις σε ημερήσιο επίπεδο ανά κατοικία. Έτσι μπορεί να υπολογιστεί το μέσο ημερήσιο απόλυτο σφάλμα σε kWh. Αυτό είναι σημαντικό, επειδή στην ενεργειακή πρόβλεψη ενδιαφέρει όχι μόνο η ακρίβεια ανά ώρα, αλλά και η σωστή εκτίμηση της συνολικής ημερήσιας κατανάλωσης.

Μετασχηματισμός και επαναφορά της μεταβλητής στόχου

Η λογική του λογαριθμικού μετασχηματισμού συγκεντρώνεται στις συναρτήσεις:

```
def transform_target(y):
    y = np.asarray(y, dtype=np.float64)
    if USE_LOG_TARGET:
        return np.log1p(np.maximum(y, 0.0))
    return y

def inverse_target(y_hat):
    y_hat = np.asarray(y_hat, dtype=np.float64)
    if USE_LOG_TARGET:
        return np.expml(y_hat)
    return y_hat
```

Η `transform_target` εφαρμόζει τον μετασχηματισμό `log1p`, ενώ η `inverse_target` επαναφέρει τις προβλέψεις στην αρχική κλίμακα με `expml`. Με αυτόν τον τρόπο το μοντέλο εκπαιδεύεται σε πιο σταθερή αριθμητική μορφή, αλλά τα τελικά αποτελέσματα επιστρέφονται σε Wh.

Μηχανισμός προεπεξεργασίας

Η προεπεξεργασία οργανώνεται με τη χρήση `ColumnTransformer`:

```
def build_preprocessor(numeric_cols, categorical_cols):
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
        ]
    )

    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant",
fill_value=UNKNOWN_LABEL)),
            ("onehot", make_one_hot_encoder()),
```

```

    ]
)

return ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_cols),
        ("cat", categorical_transformer, categorical_cols),
    ],
    remainder="drop",
    sparse_threshold=0.3,
)

```

Οι αριθμητικές μεταβλητές συμπληρώνονται με τη διάμεσο, ενώ οι κατηγορικές μεταβλητές συμπληρώνονται με την τιμή unknown και στη συνέχεια μετατρέπονται σε δυαδικές στήλες μέσω one-hot encoding. Το σημαντικό είναι ότι ο preprocessor αποθηκεύεται μαζί με το μοντέλο. Έτσι, κατά τη φάση inference, οι νέες εισοδοί μετατρέπονται με τον ίδιο τρόπο που μετατράπηκαν τα δεδομένα κατά την εκπαίδευση.

Προσθήκη χρονικών χαρακτηριστικών

Η συνάρτηση `add_time_features` δημιουργεί χρονικά χαρακτηριστικά από το timestamp:

```

def add_time_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df["hour"] = df[TIME_COL].dt.hour
    df["day_of_week"] = df[TIME_COL].dt.dayofweek
    df["month"] = df[TIME_COL].dt.month
    df["day_of_month"] = df[TIME_COL].dt.day
    df["week_of_year"] = df[TIME_COL].dt.isocalendar().week.astype(int)
    df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)

```

Το timestamp δεν χρησιμοποιείται απευθείας ως αριθμητική μεταβλητή. Αντίθετα, μετατρέπεται σε χαρακτηριστικά που έχουν σαφή ερμηνεία για το πρόβλημα της κατανάλωσης. Η ώρα και η ημέρα της εβδομάδας βοηθούν στην αποτύπωση καθημερινών μοτίβων, ενώ ο μήνας και η εβδομάδα του έτους βοηθούν στην αποτύπωση εποχικών διαφορών.

Στη συνέχεια δημιουργούνται κυκλικά χαρακτηριστικά:

```

df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
df["day_sin"] = np.sin(2 * np.pi * df["day_of_week"] / 7)
df["day_cos"] = np.cos(2 * np.pi * df["day_of_week"] / 7)
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

```

Τα κυκλικά χαρακτηριστικά είναι χρήσιμα επειδή αποτυπώνουν τη φυσική κυκλικότητα του χρόνου. Για παράδειγμα, η ώρα 23:00 και η ώρα 00:00 είναι χρονικά κοντά, κάτι που δεν εκφράζεται σωστά αν η ώρα δοθεί απλώς ως ακέραιος αριθμός.

Χρονικός διαχωρισμός δεδομένων

Η συνάρτηση `temporal_split` υλοποιεί τον χρονικό διαχωρισμό:

```

def temporal_split(df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame, pd.DataFrame]:
    max_time = df[TIME_COL].max()

    test_start = max_time - pd.Timedelta(days=TEST_DAYS) +
pd.Timedelta(hours=1)
    val_start = test_start - pd.Timedelta(days=VAL_DAYS_TOTAL)
    cal_start = test_start - pd.Timedelta(days=CAL_DAYS)

    train_df = df[df[TIME_COL] < val_start].copy()
    es_df = df[(df[TIME_COL] >= val_start) & (df[TIME_COL] <
cal_start)].copy()

```

```

cal_df = df[(df[TIME_COL] >= cal_start) & (df[TIME_COL] <
test_start)].copy()
test_df = df[df[TIME_COL] >= test_start].copy()

return train_df, es_df, cal_df, test_df

```

Η συνάρτηση αυτή εξασφαλίζει ότι το test set βρίσκεται χρονικά μετά από όλα τα υπόλοιπα δεδομένα. Αυτό είναι απαραίτητο για την αξιοπιστία της αξιολόγησης σε πρόβλημα χρονοσειράς. Το training set χρησιμοποιείται για την εκπαίδευση, το es_df διατηρείται ως ξεχωριστή περίοδος επικύρωσης, το cal_df χρησιμοποιείται για βαθμονόμηση και το test_df για την τελική αξιολόγηση.

Δημιουργία ιστορικών χαρακτηριστικών

Η συνάρτηση add_lag_and_rolling_features δημιουργεί τα ιστορικά χαρακτηριστικά για το with_history μοντέλο:

```

def add_lag_and_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.sort_values([HOME_COL, TIME_COL]).copy()
    g = df.groupby(HOME_COL)[TARGET_COL]

    for lag in LAG_HOURS:
        df[f"lag_{lag}h"] = g.shift(lag)

```

Η ομαδοποίηση με βάση το home_id είναι κρίσιμη. Εξασφαλίζει ότι οι προηγούμενες τιμές κατανάλωσης υπολογίζονται μόνο μέσα στην ίδια κατοικία και δεν αναμειγνύονται διαφορετικές χρονοσειρές.

Στη συνέχεια υπολογίζονται κυλιόμενοι μέσοι όροι:

```

shifted = g.shift(1)

for w in ROLLING_WINDOWS:
    df[f"roll_mean_{w}h"] = (
        shifted
        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .mean()
        .reset_index(level=0, drop=True)
    )

```

Η χρήση του shift(1) πριν από τον κυλιόμενο μέσο όρο είναι σημαντική, γιατί αποτρέπει τη χρήση της τρέχουσας τιμής κατανάλωσης για την πρόβλεψη της ίδιας χρονικής στιγμής. Με αυτόν τον τρόπο αποφεύγεται η διαρροή πληροφορίας.

Επιπλέον, το script υπολογίζει κυλιόμενη τυπική απόκλιση, ελάχιστες και μέγιστες τιμές:

```

for w in [24, 48, 168]:
    df[f"roll_std_{w}h"] = (
        shifted
        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .std()
        .reset_index(level=0, drop=True)
    )

```

Τα χαρακτηριστικά αυτά βοηθούν το μοντέλο να αντιληφθεί όχι μόνο το μέσο επίπεδο κατανάλωσης, αλλά και τη μεταβλητότητα, τις αιχμές και τα χαμηλά επίπεδα προηγούμενων χρονικών περιόδων.

Τέλος, δημιουργούνται πρόσθετα παράγωγα χαρακτηριστικά:

```

df["lag_1h_minus_24h"] = df["lag_1h"] - df["lag_24h"]
df["lag_24h_minus_168h"] = df["lag_24h"] - df["lag_168h"]
df["roll_24h_div_168h"] = df["roll_mean_24h"] / (df["roll_mean_168h"] +
1.0)

```

Τα χαρακτηριστικά αυτά εκφράζουν διαφορές και λόγους ανάμεσα σε πρόσφατη και παλαιότερη συμπεριφορά. Έτσι το μοντέλο μπορεί να αντιληφθεί αν η κατανάλωση της κατοικίας αυξάνεται, μειώνεται ή αποκλίνει από το συνηθισμένο εβδομαδιαίο μοτίβο.

Στατιστικά ανά κατοικία και κατηγοριοποίηση κατανάλωσης

Το with_history μοντέλο αξιοποιεί επίσης στατιστικά ανά κατοικία:

```
def build_train_only_home_stats(train_df: pd.DataFrame) ->
    Tuple[pd.DataFrame, Dict[str, float]]:
    tmp = train_df.copy()
    tmp["date"] = tmp[TIME_COL].dt.date

    hourly_stats = (
        tmp
        .groupby(HOME_COL)
        .agg(
            home_mean_Wh=(TARGET_COL, "mean"),
            home_median_Wh=(TARGET_COL, "median"),
            home_std_Wh=(TARGET_COL, "std"),
            home_min_Wh=(TARGET_COL, "min"),
            home_max_Wh=(TARGET_COL, "max"),
        )
        .reset_index()
    )
```

Τα στατιστικά αυτά υπολογίζονται αποκλειστικά από το training set. Αυτό είναι πολύ σημαντικό, επειδή αποφεύγεται η διαρροή πληροφορίας από τις μελλοντικές περιόδους validation, calibration και test. Τα στατιστικά ανά κατοικία βοηθούν το μοντέλο να γνωρίζει το γενικό επίπεδο κατανάλωσης κάθε σπιτιού.

Στη συνέχεια υπολογίζονται ημερήσια στατιστικά και μέσο ωριαίο προφίλ ανά κατοικία. Με βάση τη μέση ημερήσια κατανάλωση, κάθε κατοικία εντάσσεται σε κατηγορία κατανάλωσης:

```
q_low = home_stats["home_daily_mean_kWh"].quantile(0.33)
q_high = home_stats["home_daily_mean_kWh"].quantile(0.66)
q_very_high = home_stats["home_daily_mean_kWh"].quantile(0.90)
```

Η κατηγοριοποίηση αυτή δημιουργεί τη μεταβλητή consumption_regime, η οποία χρησιμοποιείται τόσο ως κατηγορικό χαρακτηριστικό όσο και για την προαιρετική βαθμονόμηση των προβλέψεων.

Regime calibrator

Το script περιλαμβάνει μηχανισμό βαθμονόμησης ανά κατηγορία κατανάλωσης:

```
def fit_regime_calibrator(cal_eval: pd.DataFrame, pred_col: str) ->
    Dict[str, Any]:
    params = {}

    for regime, g in cal_eval.groupby("consumption_regime"):
        if len(g) < 500:
            params[regime] = {"type": "none", "scale": 1.0}
            continue
```

Ο μηχανισμός αυτός χρησιμοποιεί την περίοδο calibration για να υπολογίσει αν οι προβλέψεις τείνουν να υποεκτιμούν ή να υπερεκτιμούν τη συνολική κατανάλωση σε συγκεκριμένες κατηγορίες κατοικιών. Αν υπάρχει αρκετό πλήθος παρατηρήσεων, υπολογίζεται ένας διορθωτικός συντελεστής κλίμακας.

```
raw_scale = float((np.sum(y_true) + 1e-6) / (np.sum(y_pred) + 1e-6))
raw_scale = float(np.clip(raw_scale, 0.95, 1.20))
```

Ο συντελεστής αυτός περιορίζεται σε συγκεκριμένα όρια, ώστε να μην εφαρμόζονται υπερβολικά μεγάλες διορθώσεις. Η διορθωμένη πρόβλεψη δεν αντικαθιστά την κύρια πρόβλεψη, αλλά αποθηκεύεται ως προαιρετική εκδοχή.

Φόρτωση και βασική προετοιμασία δεδομένων

Η φόρτωση των δεδομένων γίνεται μέσα από τη συνάρτηση load_and_prepare_base_data:

```
df = pd.read_csv(DATA_PATH, parse_dates=[TIME_COL], low_memory=False)
df = df.sort_values([HOME_COL, TIME_COL]).reset_index(drop=True)
```

Το dataset φορτώνεται με μετατροπή του timestamp σε τύπο ημερομηνίας και ώρας. Στη συνέχεια ταξινομείται ανά κατοικία και χρόνο, κάτι που είναι απαραίτητο για τον σωστό υπολογισμό ιστορικών χαρακτηριστικών.

Ακολουθεί η αφαίρεση των προβληματικών κατοικιών:

```
if REMOVE_BAD_HOMES:
    existing_bad = [h for h in BAD_HOMES if h in
set(df[HOME_COL].unique())]

    if existing_bad:
        print("Removing bad homes:", existing_bad)
        df = df[~df[HOME_COL].isin(existing_bad)].copy()
```

Έπειτα καθαρίζεται η μεταβλητή στόχος:

```
df[TARGET_COL] = pd.to_numeric(df[TARGET_COL], errors="coerce")
df = df.dropna(subset=[TARGET_COL]).copy()
df[TARGET_COL] = df[TARGET_COL].clip(lower=0)
```

Με αυτόν τον τρόπο αφαιρούνται γραμμές χωρίς έγκυρη κατανάλωση και αποτρέπεται η ύπαρξη αρνητικών τιμών, οι οποίες δεν έχουν φυσική ερμηνεία στο πλαίσιο της κατανάλωσης ηλεκτρικής ενέργειας.

Εκπαίδευση του μοντέλου με ιστορικό κατανάλωσης

Η εκπαίδευση του with_history μοντέλου ξεκινά με τη δημιουργία των ιστορικών χαρακτηριστικών:

```
df_hist = add_lag_and_rolling_features(df_base)
df_hist = df_hist.dropna(subset=REQUIRED_HISTORY_COLS).copy()
df_hist = df_hist.sort_values([HOME_COL,
TIME_COL]).reset_index(drop=True)
```

Οι γραμμές που δεν έχουν επαρκές ιστορικό αφαιρούνται. Αυτό είναι αναμενόμενο, καθώς για να υπολογιστούν χαρακτηριστικά όπως lag_336h ή roll_mean_168h, απαιτούνται αρκετές προηγούμενες ώρες διαθέσιμων μετρήσεων.

Στη συνέχεια εφαρμόζεται χρονικός διαχωρισμός:

```
hist_train, hist_es, hist_cal, hist_test = temporal_split(df_hist)
```

Ακολουθώς υπολογίζονται τα στατιστικά ανά κατοικία αποκλειστικά από το training set: home_stats, regime_thresholds = build_train_only_home_stats(hist_train)

```
home_stats.to_csv(WITH_HISTORY_DIR / "home_stats.csv", index=False)
save_json(regime_thresholds, WITH_HISTORY_DIR / "regime_thresholds.json")
```

Τα στατιστικά αυτά αποθηκεύονται σε αρχείο CSV, ενώ τα όρια των κατηγοριών κατανάλωσης αποθηκεύονται σε JSON. Στη συνέχεια, τα στατιστικά συγχωνεύονται με τα training, validation, calibration και test δεδομένα.

Το script δημιουργεί τις τελικές λίστες αριθμητικών και κατηγορικών χαρακτηριστικών:

```
lag_cols = [c for c in df_hist.columns if c.startswith("lag_")]
rolling_cols = [c for c in df_hist.columns if c.startswith("roll_")]

numeric_cols = [
    c for c in STATIC_NUMERIC + TIME_NUMERIC_EXTENDED + lag_cols +
rolling_cols + home_numeric_cols
    if c in hist_train.columns
]

categorical_cols = [
    c for c in HISTORY_CATEGORICAL
    if c in hist_train.columns
]
```

Το τελικό σύνολο χαρακτηριστικών του with_history μοντέλου είναι αρκετά πλούσιο. Περιλαμβάνει στατικά χαρακτηριστικά, χρονικά χαρακτηριστικά, ιστορικές μεταβλητές, κυλιόμενα στατιστικά και στατιστικά ανά κατοικία. Επιπλέον, περιλαμβάνει ως

κατηγορικές μεταβλητές το `home_id`, το `hometype`, το `urban_rural_class` και το `consumption_regime`.

Λόγω του μεγάλου όγκου δεδομένων, η εκπαίδευση του Random Forest με ιστορικά χαρακτηριστικά γίνεται σε δείγμα του training set:

```
hist_train_model = sample_train_for_rf(
    hist_train,
    max_rows=MAX_RF_TRAIN_ROWS_WITH_HISTORY,
    random_state=RANDOM_STATE + 101,
)
```

Το όριο έχει οριστεί στις 350.000 γραμμές. Η δειγματοληψία γίνεται ανά κατοικία, ώστε το δείγμα να παραμένει αντιπροσωπευτικό και να μη χαθεί πλήρως η πληροφορία από επιμέρους σπίτια.

Στη συνέχεια δημιουργείται και εφαρμόζεται ο `preprocessor`:

```
preprocessor = build_preprocessor(numeric_cols, categorical_cols)
```

```
preprocessor.fit(hist_train[numeric_cols + categorical_cols])
```

```
X_train = preprocessor.transform(hist_train_model[numeric_cols +
categorical_cols])
```

```
X_cal = preprocessor.transform(hist_cal[numeric_cols + categorical_cols])
```

```
X_test = preprocessor.transform(hist_test[numeric_cols +
categorical_cols])
```

Ο `preprocessor` εκπαιδεύεται στο πλήρες training set, ενώ το μοντέλο εκπαιδεύεται στο δείγμα. Αυτό επιτρέπει στον μηχανισμό προεπεξεργασίας να έχει δει το πλήρες εύρος κατηγοριών και αριθμητικών τιμών του training set, ενώ ταυτόχρονα περιορίζει το υπολογιστικό κόστος της εκπαίδευσης του Random Forest.

Η εκπαίδευση γίνεται ως εξής:

```
model = RandomForestRegressor(**RF_WITH_HISTORY_PARAMS)
```

```
start_train = time.perf_counter()
```

```
model.fit(X_train, y_train)
```

```
train_seconds = float(time.perf_counter() - start_train)
```

Μετά την εκπαίδευση, παράγονται προβλέψεις για την calibration και την test περίοδο:

```
pred_cal = inverse_target(model.predict(X_cal))
```

```
pred_test = inverse_target(model.predict(X_test))
```

```
pred_cal = np.clip(pred_cal, 0, None)
```

```
pred_test = np.clip(pred_test, 0, None)
```

Οι προβλέψεις επανέρχονται από τη λογαριθμική κλίμακα στην αρχική κλίμακα `Wh` και στη συνέχεια περιορίζονται ώστε να μην υπάρχουν αρνητικές προβλέψεις.

Το script υπολογίζει δύο εκδοχές μετρικών για το `with_history` μοντέλο:

```
metrics_balanced = evaluate_predictions(test_eval, "prediction_Wh")
```

```
metrics_peak_daily = evaluate_predictions(test_eval,
"prediction_peak_daily_Wh")
```

Η πρώτη αφορά τη βασική πρόβλεψη του μοντέλου, ενώ η δεύτερη αφορά την προαιρετική διορθωμένη πρόβλεψη με βάση το `consumption regime`. Η βασική πρόβλεψη είναι η κύρια εκδοχή, ενώ η διορθωμένη λειτουργεί ως εναλλακτική για διερεύνηση πιθανής βελτίωσης στο ημερήσιο επίπεδο.

Εκπαίδευση του μοντέλου χωρίς ιστορικό κατανάλωσης

Η δεύτερη βασική συνάρτηση του script είναι η `train_no_history_model`. Το μοντέλο αυτό ακολουθεί απλούστερη λογική, επειδή δεν χρησιμοποιεί προηγούμενες τιμές κατανάλωσης:

```
df_NOHISTORY =
```

```
df_base.sort_values(TIME_COL).reset_index(drop=True).copy()
```

```
train_df, es_df, cal_df, test_df = temporal_split(df_NOHISTORY)
```

Εδώ τα δεδομένα ταξινομούνται μόνο με βάση τον χρόνο, αφού δεν χρειάζεται να υπολογιστούν lags ανά κατοικία. Στη συνέχεια εφαρμόζεται ο ίδιος χρονικός διαχωρισμός. Για την τελική εκπαίδευση του no_history μοντέλου χρησιμοποιούνται τα training, early-stop validation και calibration δεδομένα μαζί:

```
final_train_df = (  
    pd.concat([train_df, es_df, cal_df], axis=0)  
    .sort_values(TIME_COL)  
    .reset_index(drop=True)  
)
```

Η επιλογή αυτή γίνεται επειδή το no_history μοντέλο προορίζεται ως τελικό μοντέλο λειτουργικής χρήσης. Εφόσον δεν γίνεται περαιτέρω επιλογή υπερπαραμέτρων σε αυτό το σημείο, αξιοποιούνται όλα τα διαθέσιμα δεδομένα πριν από το test set, ώστε το τελικό μοντέλο να εκπαιδευτεί σε όσο το δυνατόν μεγαλύτερο δείγμα.

Τα χαρακτηριστικά του no_history μοντέλου είναι περιορισμένα:

```
numeric_cols = [c for c in NOHISTORY_NUMERIC if c in  
final_train_df.columns]  
categorical_cols = [c for c in NOHISTORY_CATEGORICAL if c in  
final_train_df.columns]
```

Σύμφωνα με την εκτέλεση του script, το no_history μοντέλο χρησιμοποιεί 7 αριθμητικά και 2 κατηγορικά χαρακτηριστικά. Τα αριθμητικά είναι η ώρα, η ημέρα της εβδομάδας, η ένδειξη Σαββατοκύριακου, ο μήνας, η εξωτερική θερμοκρασία, το εμβαδόν και ο αριθμός κατοίκων. Τα κατηγορικά είναι ο τύπος κατοικίας και η κατηγορία περιοχής.

Το script εφαρμόζει την ίδια λογική προεπεξεργασίας:

```
preprocessor = build_preprocessor(numeric_cols, categorical_cols)
```

```
preprocessor.fit(final_train_df[numeric_cols + categorical_cols])
```

```
X_train = preprocessor.transform(train_df_model[numeric_cols +  
categorical_cols])
```

```
X_cal = preprocessor.transform(cal_df[numeric_cols + categorical_cols])
```

```
X_test = preprocessor.transform(test_df[numeric_cols + categorical_cols])
```

Στην περίπτωση αυτή, επειδή το MAX_RF_TRAIN_ROWS_no_history έχει οριστεί ως None, χρησιμοποιείται ολόκληρο το διαθέσιμο final training set. Αυτό σημαίνει ότι το μοντέλο εκπαιδεύεται σε όλα τα δεδομένα πριν από το test set.

Το target υφίσταται clipping και λογαριθμικό μετασχηματισμό:

```
y_train_raw = train_df_model[TARGET_COL].values
```

```
y_train_used, clip_bounds = maybe_clip_target(  
    y_train_raw,  
    enabled=CLIP_TARGET_TRAIN_NOHISTORY,  
)
```

```
y_train = transform_target(y_train_used)
```

Εδώ το clipping είναι ενεργό, σε αντίθεση με το with_history μοντέλο. Αυτό βοηθά το απλούστερο μοντέλο να μην επηρεάζεται υπερβολικά από πολύ ακραίες τιμές κατανάλωσης, τις οποίες δεν μπορεί να εξηγήσει μέσω ιστορικών χαρακτηριστικών.

Η εκπαίδευση πραγματοποιείται με τις παραμέτρους του RF_no_history_PARAMS. Μετά την εκπαίδευση, το script υπολογίζει διαγνωστικές μετρικές στην περίοδο calibration και τελικές μετρικές στο test set. Οι μετρικές της calibration περιόδου δεν θεωρούνται ανεξάρτητη τελική αξιολόγηση, επειδή η περίοδος αυτή έχει ήδη συμπεριληφθεί στο final training set. Γι' αυτό αντιμετωπίζονται ως διαγνωστικές.

Αποθήκευση artifacts

Και για τα δύο μοντέλα, το script αποθηκεύει το εκπαιδευμένο μοντέλο, τον preprocessor, το αρχείο χαρακτηριστικών, τα μεταδεδομένα και τις προβλέψεις του test set:

```
model_path = WITH_HISTORY_DIR / "model.joblib"
```

```
preprocessor_path = WITH_HISTORY_DIR / "preprocessor.pkl"
```

```
feature_config_path = WITH_HISTORY_DIR / "feature_config.json"
```

```
metadata_path = WITH_HISTORY_DIR / "metadata.json"
predictions_path = WITH_HISTORY_DIR / "test_predictions.csv"
```

```
joblib.dump(model, model_path)
joblib.dump(preprocessor, preprocessor_path)
```

Η αποθήκευση του preprocessor είναι εξίσου σημαντική με την αποθήκευση του μοντέλου. Αν στη φάση πρόβλεψης χρησιμοποιηθεί διαφορετική προεπεξεργασία, τότε οι είσοδοι του μοντέλου δεν θα έχουν την ίδια μορφή με αυτή που χρησιμοποιήθηκε κατά την εκπαίδευση. Το feature_config.json περιγράφει τη δομή των χαρακτηριστικών:

```
feature_config = {
    "mode": "with_history",
    "model_family": "RandomForestRegressor",
    "model_name": "rf_with_history_extended_sampled",
    "numeric_cols": numeric_cols,
    "categorical_cols": categorical_cols,
    "target_col": TARGET_COL,
    "time_col": TIME_COL,
    "home_col": HOME_COL,
    "use_log_target": USE_LOG_TARGET,
    "uses_home_id_as_feature": True,
    "uses_lag_features": True,
    "uses_rolling_features": True,
    "uses_home_stats": True,
}
```

Το αρχείο αυτό επιτρέπει στο σύστημα inference να γνωρίζει τι είδους χαρακτηριστικά απαιτεί το μοντέλο. Για παράδειγμα, στην περίπτωση του with_history Random Forest δηλώνεται ρητά ότι το μοντέλο χρησιμοποιεί home_id, lags, rolling features και στατιστικά κατοικίας. Αντίστοιχα, στο no_history μοντέλο δηλώνεται ότι δεν χρησιμοποιούνται ιστορικές μεταβλητές, KNN προφίλ ή behavioral profiles.

Το metadata.json αποθηκεύει πληροφορίες για την εκπαίδευση, όπως παραμέτρους, μετρικές, διάρκεια εκπαίδευσης, μέγεθος αρχείων και χρονικές περιόδους train, calibration και test. Με αυτόν τον τρόπο, κάθε εκπαιδευμένο μοντέλο συνοδεύεται από πλήρη τεκμηρίωση.

Συγκεντρωτικό αρχείο εκπαίδευσης

Στο τέλος του script δημιουργείται το συνολικό αρχείο training_summary.json:

```
summary = {
    "data_path": str(DATA_PATH),
    "target": TARGET_COL,
    "time_col": TIME_COL,
    "home_col": HOME_COL,
    "test_days": TEST_DAYS,
    "val_days_total": VAL_DAYS_TOTAL,
    "cal_days": CAL_DAYS,
    "bad_homes_removed": existing_bad,
    "use_log_target": USE_LOG_TARGET,
    "clip_q_low": CLIP_Q_LOW,
    "clip_q_high": CLIP_Q_HIGH,
    "output_root": str(OUT_ROOT),
}
```

Το αρχείο αυτό συγκεντρώνει τις βασικές πληροφορίες και για τα δύο Random Forest μοντέλα. Περιλαμβάνει τη διαδρομή του dataset, τη μεταβλητή στόχο, τις περιόδους διαχωρισμού, τις κατοικίες που αφαιρέθηκαν, τις ρυθμίσεις του target και τις διαδρομές εξόδου.

Η εκτέλεση του script οργανώνεται από τη συνάρτηση main:

```
if TRAIN_WITH_HISTORY:
    summary["with_history"] = train_with_history_model(df_base)
else:
```

```
summary["with_history"] = None

if TRAIN_no_history:
    summary["no_history"] = train_no_history_model(df_base)
else:
    summary["no_history"] = None

save_json(summary, OUT_ROOT / "training_summary.json")
```

Με αυτή τη δομή, είναι δυνατή η ενεργοποίηση ή απενεργοποίηση κάθε σεναρίου εκπαίδευσης ξεχωριστά. Το script μπορεί να εκπαιδεύσει και τα δύο μοντέλα ή μόνο ένα από αυτά, ανάλογα με τις αντίστοιχες παραμέτρους.

XGBoost. Αρχείο: XGB_for_API2.ipynb

XGBoost Regressor. Αρχείο: train_xgb_api_generic_models.py

Το παρόν script υλοποιεί την εκπαίδευση και αποθήκευση των τελικών μοντέλων XGBoost που χρησιμοποιούνται ως συμβατά μοντέλα σύγκρισης στο API και στη γραφική διεπαφή της εφαρμογής. Η βασική ιδιαιτερότητα της συγκεκριμένης υλοποίησης είναι ότι τα μοντέλα έχουν σχεδιαστεί για τη γενική έκδοση της εφαρμογής, όπου ο τελικός χρήστης δεν παρέχει home_id.

Αυτό σημαίνει ότι το home_id χρησιμοποιείται μόνο εσωτερικά κατά την προετοιμασία και αξιολόγηση των δεδομένων, κυρίως για να υπολογιστούν σωστά τα ιστορικά χαρακτηριστικά ανά κατοικία και οι μετρικές ανά κατοικία και ημέρα. Δεν χρησιμοποιείται όμως ως χαρακτηριστικό εισόδου στο τελικό μοντέλο. Με αυτόν τον τρόπο, τα μοντέλα XGBoost που παράγονται από το script μπορούν να χρησιμοποιηθούν σε νέα δεδομένα χρήστη, χωρίς ο χρήστης να πρέπει να αντιστοιχεί σε κάποια γνωστή κατοικία του αρχικού dataset.

Το script εκπαιδεύει δύο εκδοχές XGBoost. Η πρώτη είναι το γενικό μοντέλο με ιστορικό κατανάλωσης (with_history_generic), το οποίο χρησιμοποιεί lag και rolling χαρακτηριστικά, χρονικά χαρακτηριστικά, στατικά χαρακτηριστικά και περιβαλλοντικές μεταβλητές, χωρίς όμως να χρησιμοποιεί home_id, στατιστικά γνωστής κατοικίας ή consumption_regime. Η δεύτερη είναι το προαιρετικό μοντέλο χωρίς ιστορικό (no_history_optional), το οποίο χρησιμοποιεί στατικά, χρονικά και περιβαλλοντικά χαρακτηριστικά, καθώς και πρόσθετα προφίλ συμπεριφοράς και KNN προφίλ παρόμοιων κατοικιών. Το δεύτερο μοντέλο δεν αποτελεί την τελική προεπιλογή του συστήματος για το no-history σενάριο, αλλά παραμένει διαθέσιμο ως προαιρετικό μοντέλο σύγκρισης.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import json
import math
import time
import warnings
from pathlib import Path
from typing import Any, Dict, Tuple

import joblib
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.neighbors import NearestNeighbors
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
from xgboost import XGBRegressor
```

Η pandas χρησιμοποιείται για τη φόρτωση, τον καθαρισμό και την επεξεργασία των δεδομένων, ενώ η numpy χρησιμοποιείται για αριθμητικούς υπολογισμούς, μετασχηματισμούς και δημιουργία χαρακτηριστικών. Η XGBRegressor είναι ο βασικός αλγόριθμος παλινδρόμησης που εκπαιδεύεται. Οι συναρτήσεις mean_absolute_error και mean_squared_error χρησιμοποιούνται για την αξιολόγηση των προβλέψεων.

Η joblib χρησιμοποιείται για την αποθήκευση του εκπαιδευμένου μοντέλου, του μηχανισμού προεπεξεργασίας, των προφίλ συμπεριφοράς και του KNN bundle. Οι κλάσεις ColumnTransformer, Pipeline, SimpleImputer και OneHotEncoder οργανώνουν τη διαδικασία προεπεξεργασίας των αριθμητικών και κατηγορικών μεταβλητών. Επιπλέον, ο StandardScaler και ο NearestNeighbors χρησιμοποιούνται στο no-history σενάριο για τη δημιουργία προφίλ παρόμοιων κατοικιών μέσω KNN.

Ρυθμίσεις παραμετροποίησης

Στη συνέχεια ορίζονται οι βασικές διαδρομές του έργου και οι φάκελοι εξόδου:

```
BASE_DIR = Path("C:/IDEAL_Programming")
DATA_PATH = BASE_DIR / "processed" / "final" /
"IDEAL_final_hourly_dataset.csv"
```

```
OUT_ROOT = BASE_DIR / "processed" / "models" / "final_api_models" / "XGB2"
WITH_HISTORY_DIR = OUT_ROOT / "with_history_generic"
no_history_DIR = OUT_ROOT / "no_history_optional"
```

Το DATA_PATH δείχνει στο τελικό ενοποιημένο dataset του IDEAL, ενώ το OUT_ROOT είναι ο βασικός φάκελος στον οποίο αποθηκεύονται τα τελικά XGBoost artifacts. Το μοντέλο με ιστορικό αποθηκεύεται στον φάκελο with_history_generic, ενώ το προαιρετικό μοντέλο χωρίς ιστορικό αποθηκεύεται στον φάκελο no_history_optional.

Ακολουθούν οι βασικές στήλες του dataset:

```
TIME_COL = "timestamp"
HOME_COL = "home_id"
TARGET_COL = "consumption_Wh"
```

```
RANDOM_STATE = 42
```

Η στήλη timestamp περιέχει τη χρονική στιγμή της μέτρησης, η στήλη home_id χρησιμοποιείται μόνο εσωτερικά για ομαδοποίηση και αξιολόγηση, ενώ η στήλη consumption_Wh είναι η μεταβλητή στόχος. Το RANDOM_STATE εξασφαλίζει αναπαραγωγικότητα στα σημεία του κώδικα όπου υπάρχει τυχαιότητα.

Χρονικός διαχωρισμός δεδομένων

Το script χρησιμοποιεί χρονικό διαχωρισμό των δεδομένων:

```
TEST_DAYS = 30
VAL_DAYS_TOTAL = 30
CAL_DAYS = 15
```

Οι τελευταίες 30 ημέρες του dataset χρησιμοποιούνται ως test set. Πριν από αυτές ορίζεται περίοδος επικύρωσης συνολικής διάρκειας 30 ημερών, από την οποία οι τελευταίες 15 ημέρες χρησιμοποιούνται ως calibration period. Έτσι τα δεδομένα χωρίζονται σε training set, early-stop validation set, calibration validation set και test set.

Η επιλογή χρονικού διαχωρισμού είναι απαραίτητη, επειδή η πρόβλεψη κατανάλωσης είναι πρόβλημα χρονοσειρών. Αν χρησιμοποιούνταν τυχαίος διαχωρισμός, θα υπήρχε κίνδυνος να περάσει πληροφορία από μελλοντικές χρονικές στιγμές στο training set, οδηγώντας σε μη ρεαλιστικά αισιόδοξη αξιολόγηση.

Αφαίρεση προβληματικών κατοικιών

Στην αρχή της εκπαίδευσης αφαιρούνται οι κατοικίες που έχουν χαρακτηριστεί προβληματικές:

```
BAD_HOMES = ["home171", "home219", "home228", "home271"]
```

Η αφαίρεση αυτών των κατοικιών γίνεται ώστε να μη χρησιμοποιηθούν παρατηρήσεις με μη αξιόπιστα ή μη ρεαλιστικά μεταδεδομένα. Στην εκτέλεση του script, μετά την αφαίρεση των συγκεκριμένων κατοικιών, το dataset μειώθηκε από 254 σε 250 κατοικίες.

Μετασχηματισμός μεταβλητής στόχου

Η μεταβλητή στόχος μετασχηματίζεται λογαριθμικά:

```
USE_LOG_TARGET = True
CLIP_TARGET_TRAIN = False
CLIP_Q_LOW = 0.005
CLIP_Q_HIGH = 0.995
```

Η επιλογή `USE_LOG_TARGET = True` σημαίνει ότι η κατανάλωση μετασχηματίζεται με `log1p` πριν από την εκπαίδευση. Ο μετασχηματισμός αυτός μειώνει την επίδραση μεγάλων τιμών και βοηθά τη διαδικασία μάθησης να γίνει σε πιο σταθερό αριθμητικό χώρο. Σε αντίθεση με ορισμένες προηγούμενες εκδοχές, εδώ το clipping του target είναι απενεργοποιημένο, καθώς το `CLIP_TARGET_TRAIN` έχει οριστεί σε `False`.

Οι αντίστοιχες συναρτήσεις είναι:

```
def transform_target(y):
    y = np.asarray(y, dtype=np.float64)
    if USE_LOG_TARGET:
        return np.log1p(np.maximum(y, 0.0))
    return y

def inverse_target(y_hat):
    y_hat = np.asarray(y_hat, dtype=np.float64)
    if USE_LOG_TARGET:
        return np.expm1(y_hat)
    return y_hat
```

Η πρώτη συνάρτηση μετατρέπει την κατανάλωση πριν από την εκπαίδευση, ενώ η δεύτερη επαναφέρει τις προβλέψεις στην αρχική κλίμακα `Wh`. Έτσι το μοντέλο εκπαιδεύεται σε μετασχηματισμένη κλίμακα, αλλά η τελική αξιολόγηση γίνεται σε πραγματικές μονάδες κατανάλωσης.

Βασικά χαρακτηριστικά εισόδου

Τα χαρακτηριστικά που μπορούν να χρησιμοποιηθούν από το API και τη γραφική διεπαφή οργανώνονται σε ομάδες. Αρχικά ορίζονται τα στατικά και περιβαλλοντικά αριθμητικά χαρακτηριστικά:

```
STATIC_NUMERIC = [
    "external_temperature",
    "internal_temperature",
    "residents",
    "total_floor_area_m2",
    "num_electric_appliances",
]
```

Τα χαρακτηριστικά αυτά περιγράφουν τόσο το περιβάλλον όσο και την ίδια την κατοικία. Η εξωτερική και εσωτερική θερμοκρασία μπορούν να επηρεάσουν τη ζήτηση ηλεκτρικής ενέργειας, ενώ ο αριθμός κατοίκων, το εμβαδόν και ο αριθμός ηλεκτρικών συσκευών συνδέονται με το γενικό επίπεδο κατανάλωσης.

Στη συνέχεια ορίζονται τα χρονικά χαρακτηριστικά:

```
TIME_NUMERIC = [
    "hour",
    "day_of_week",
    "month",
    "day_of_month",
    "week_of_year",
    "is_weekend",
    "is_holiday",
]
```

```

        "hour_sin",
        "hour_cos",
        "day_sin",
        "day_cos",
        "month_sin",
        "month_cos",
    ]

```

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να αξιοποιήσει την περιοδικότητα της κατανάλωσης. Η ώρα, η ημέρα της εβδομάδας και ο μήνας αποτυπώνουν καθημερινές, εβδομαδιαίες και εποχικές διαφοροποιήσεις. Τα κυκλικά χαρακτηριστικά βοηθούν ώστε οι περιοδικές μεταβλητές να αναπαρίστανται πιο σωστά.

Τα βασικά κατηγορικά χαρακτηριστικά είναι:

```

BASE_CATEGORICAL = [
    "hometype",
    "urban_rural_class",
]

```

Το `hometype` δηλώνει τον τύπο κατοικίας, ενώ το `urban_rural_class` περιγράφει την κατηγορία περιοχής. Αυτές οι μεταβλητές μετατρέπονται αργότερα σε αριθμητική μορφή μέσω `one-hot encoding`.

Παράμετροι XGBoost

Το `script` ορίζει διαφορετικές παραμέτρους για το μοντέλο με ιστορικό και για το μοντέλο χωρίς ιστορικό. Για το `with_history_generic` μοντέλο χρησιμοποιούνται οι εξής παράμετροι:

```

XGB_WITH_HISTORY_PARAMS = make_xgb_params(dict(
    n_estimators=5000,
    learning_rate=0.015,
    max_depth=8,
    min_child_weight=20,
    subsample=0.90,
    colsample_bytree=0.90,
    reg_alpha=0.05,
    reg_lambda=0.25,
    objective="reg:squarederror",
    eval_metric="rmse",
    tree_method="hist",
    max_bin=256,
    random_state=RANDOM_STATE + 21,
    n_jobs=-1,
    verbosity=1,
))

```

Το `n_estimators=5000` δηλώνει τον μέγιστο αριθμό `boosting iterations`. Ο χαμηλός ρυθμός μάθησης `learning_rate=0.015` επιτρέπει στο μοντέλο να μαθαίνει σταδιακά, μειώνοντας τον κίνδυνο υπερπροσαρμογής. Το `max_depth=8` ελέγχει το βάθος των δέντρων, ενώ οι παράμετροι `subsample` και `colsample_bytree` εφαρμόζουν υποδειγματοληψία γραμμών και χαρακτηριστικών. Οι `reg_alpha` και `reg_lambda` προσθέτουν κανονικοποίηση, βοηθώντας το μοντέλο να γενικεύει καλύτερα.

Για το προαιρετικό `no_history` μοντέλο χρησιμοποιούνται παρόμοιες παράμετροι, αλλά με ελαφρώς διαφορετικό βάθος και αριθμό επαναλήψεων:

```

XGB_no_history_PARAMS = make_xgb_params(dict(
    n_estimators=5500,
    learning_rate=0.015,
    max_depth=7,
    min_child_weight=25,
    subsample=0.90,
    colsample_bytree=0.90,
    reg_alpha=0.05,
    reg_lambda=0.25,
))

```

```

objective="reg:squarederror",
eval_metric="rmse",
tree_method="hist",
max_bin=256,
random_state=RANDOM_STATE + 22,
n_jobs=-1,
verbosity=1,
))

```

Η επιλογή μεγαλύτερου αριθμού επαναλήψεων και ελαφρώς μικρότερου βάθους στο no-history μοντέλο αντανακλά την ανάγκη για σταδιακή μάθηση σε ένα σενάριο όπου δεν υπάρχουν άμεσες ιστορικές τιμές κατανάλωσης. Το μοντέλο πρέπει να αντλήσει πληροφορία από στατικά, χρονικά, περιβαλλοντικά και έμμεσα συμπεριφορικά χαρακτηριστικά.

Early stopping

Το XGBoost υποστηρίζει early stopping, δηλαδή πρόωρη διακοπή της εκπαίδευσης όταν η απόδοση στο validation set δεν βελτιώνεται περαιτέρω:

```
EARLY_STOPPING_ROUNDS_WITH_HISTORY = 150
```

```
EARLY_STOPPING_ROUNDS_no_history = 150
```

Η αντίστοιχη βοηθητική συνάρτηση είναι:

```

def fit_xgb_with_early_stopping(
    model: XGBRegressor,
    X_train,
    y_train,
    X_val,
    y_val,
    early_stopping_rounds: int,
):
    try:
        model.fit(
            X_train,
            y_train,
            eval_set=[(X_val, y_val)],
            verbose=100,
            early_stopping_rounds=early_stopping_rounds,
        )
    except TypeError:
        model.set_params(early_stopping_rounds=early_stopping_rounds)
        model.fit(
            X_train,
            y_train,
            eval_set=[(X_val, y_val)],
            verbose=100,
        )

    return model

```

Η συνάρτηση αυτή έχει σχεδιαστεί ώστε να είναι συμβατή με διαφορετικές εκδόσεις της βιβλιοθήκης XGBoost. Αν η έκδοση υποστηρίζει το όρισμα early_stopping_rounds απευθείας στη fit, χρησιμοποιείται η πρώτη εκδοχή. Αν όχι, το early stopping περνάει ως παράμετρος του μοντέλου. Με αυτόν τον τρόπο το script παραμένει πιο ανθεκτικό σε διαφορές εκδόσεων.

Μετρικές αξιολόγησης

Οι βασικές μετρικές υπολογίζονται από τη συνάρτηση compute_metrics:

```

def compute_metrics(y_true, y_pred) -> Dict[str, float]:
    y_true = np.asarray(y_true, dtype=np.float64)
    y_pred = np.asarray(y_pred, dtype=np.float64)

    return {
        "MAE": float(mean_absolute_error(y_true, y_pred)),
    }

```

```

    "RMSE": float(math.sqrt(mean_squared_error(y_true, y_pred))),
    "MAPE_%": mape_safe(y_true, y_pred),
    "sMAPE_%": smape_safe(y_true, y_pred),
    "bias_Wh": float(np.mean(y_pred - y_true)),
}

```

Το MAE εκφράζει τη μέση απόλυτη απόκλιση σε Wh, ενώ το RMSE τιμωρεί περισσότερο τα μεγάλα σφάλματα. Το MAPE και το sMAPE δίνουν ποσοστιαία εικόνα του σφάλματος, ενώ το bias_Wh δείχνει αν το μοντέλο τείνει να υποεκτιμά ή να υπερεκτιμά την κατανάλωση.

Η συνολική αξιολόγηση γίνεται με τη συνάρτηση `evaluate_predictions`:

```

def evaluate_predictions(df_eval: pd.DataFrame, pred_col: str) -> Dict[str, float]:
    y_true = df_eval[TARGET_COL].values
    y_pred = df_eval[pred_col].values

    metrics = compute_metrics(y_true, y_pred)

    daily = (
        df_eval
        .assign(date=df_eval[TIME_COL].dt.date)
        .groupby([HOME_COL, "date"], as_index=False)
        .agg(
            actual_kWh=(TARGET_COL, lambda x: x.sum() / 1000),
            pred_kWh=(pred_col, lambda x: x.sum() / 1000),
        )
    )

```

Η συνάρτηση αυτή υπολογίζει πρώτα τις ωριαίες μετρικές και στη συνέχεια συγκεντρώνει τις προβλέψεις σε ημερήσιο επίπεδο ανά κατοικία. Έτσι υπολογίζεται το ημερήσιο απόλυτο σφάλμα σε kWh, το οποίο είναι ιδιαίτερα χρήσιμο για την αξιολόγηση της συνολικής ημερήσιας ενεργειακής πρόβλεψης.

Μηχανισμός προεπεξεργασίας

Η προεπεξεργασία οργανώνεται με `ColumnTransformer`:

```

def build_preprocessor(numeric_cols, categorical_cols):
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
        ]
    )

    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant",
fill_value="unknown")),
            ("onehot", make_one_hot_encoder()),
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_cols),
            ("cat", categorical_transformer, categorical_cols),
        ],
        remainder="drop",
        sparse_threshold=0.3,
    )

```

Οι αριθμητικές μεταβλητές συμπληρώνονται με τη διάμεσο, ενώ οι κατηγορικές μεταβλητές συμπληρώνονται με την τιμή `unknown` και στη συνέχεια μετατρέπονται σε `one-`

hot encoded στήλες. Ο preprocessor αποθηκεύεται μαζί με το μοντέλο, ώστε κατά τη φάση inference να εφαρμόζεται ακριβώς η ίδια προεπεξεργασία με αυτή της εκπαίδευσης.

Προσθήκη χρονικών χαρακτηριστικών

Η συνάρτηση `add_time_features` δημιουργεί τα χρονικά χαρακτηριστικά:

```
def add_time_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df[TIME_COL] = pd.to_datetime(df[TIME_COL])
    df["hour"] = df[TIME_COL].dt.hour
    df["day_of_week"] = df[TIME_COL].dt.dayofweek
    df["month"] = df[TIME_COL].dt.month
    df["day_of_month"] = df[TIME_COL].dt.day
    df["week_of_year"] = df[TIME_COL].dt.isocalendar().week.astype(int)
    df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)
```

Το αρχικό timestamp δεν εισάγεται απευθείας στο μοντέλο, αλλά μετατρέπεται σε χαρακτηριστικά με σαφή ερμηνεία. Η ώρα της ημέρας, η ημέρα της εβδομάδας και ο μήνας είναι βασικοί παράγοντες για την πρόβλεψη της ηλεκτρικής κατανάλωσης, καθώς η κατανάλωση παρουσιάζει έντονα ημερήσια, εβδομαδιαία και εποχικά μοτίβα.

Στη συνέχεια δημιουργούνται κυκλικά χαρακτηριστικά:

```
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
df["day_sin"] = np.sin(2 * np.pi * df["day_of_week"] / 7)
df["day_cos"] = np.cos(2 * np.pi * df["day_of_week"] / 7)
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
```

Τα κυκλικά χαρακτηριστικά βοηθούν το μοντέλο να αντιληφθεί ότι οι χρονικές μεταβλητές είναι περιοδικές. Για παράδειγμα, η ώρα 23:00 και η ώρα 00:00 είναι κοντινές χρονικά, παρότι αριθμητικά έχουν μεγάλη διαφορά αν δοθούν απλώς ως 23 και 0.

Στατικές κατηγορίες για το no-history σενάριο

Για το no-history μοντέλο δημιουργούνται επιπλέον κατηγορίες από στατικά χαρακτηριστικά:

```
def add_static_bins(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    r = pd.to_numeric(df["residents"], errors="coerce")
    df["residents_bin"] = np.select(
        [r.isna(), r <= 0, r == 1, r == 2, r == 3, r == 4, r >= 5],
        ["unknown", "unknown", "1", "2", "3", "4", "5plus"],
        default="unknown",
    )
```

Η συνάρτηση αυτή δημιουργεί κατηγορίες για τον αριθμό κατοίκων, το εμβαδόν και τον αριθμό ηλεκτρικών συσκευών. Αυτές οι κατηγορίες χρησιμοποιούνται για τη δημιουργία προφίλ συμπεριφοράς. Έτσι, κατοικίες με παρόμοια στατικά χαρακτηριστικά μπορούν να ομαδοποιηθούν και να εξαχθεί ενδεικτική αναμενόμενη κατανάλωση.

Η τελική κλείδα στατικού προφίλ δημιουργείται ως εξής:

```
df["static_profile_key"] = (
    df["hometype"].astype(str) + "|" +
    df["urban_rural_class"].astype(str) + "|" +
    df["residents_bin"].astype(str) + "|" +
    df["area_bin"].astype(str) + "|" +
    df["appliances_bin"].astype(str)
)
```

Το `static_profile_key` συνδυάζει τα βασικά στατικά χαρακτηριστικά της κατοικίας και χρησιμοποιείται ως βάση για την αναζήτηση συμπεριφορικών μοτίβων.

Χρονικός διαχωρισμός

Ο χρονικός διαχωρισμός πραγματοποιείται από τη συνάρτηση `temporal_split`:

```

def temporal_split(df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame, pd.DataFrame]:
    max_time = df[TIME_COL].max()

    test_start = max_time - pd.Timedelta(days=TEST_DAYS) +
pd.Timedelta(hours=1)
    val_start = test_start - pd.Timedelta(days=VAL_DAYS_TOTAL)
    cal_start = test_start - pd.Timedelta(days=CAL_DAYS)

    train_df = df[df[TIME_COL] < val_start].copy()
    es_df = df[(df[TIME_COL] >= val_start) & (df[TIME_COL] <
cal_start)].copy()
    cal_df = df[(df[TIME_COL] >= cal_start) & (df[TIME_COL] <
test_start)].copy()
    test_df = df[df[TIME_COL] >= test_start].copy()

    return train_df, es_df, cal_df, test_df

```

Η συνάρτηση επιστρέφει τέσσερα διακριτά χρονικά τμήματα. Το training set χρησιμοποιείται για την εκπαίδευση, το es_df για early stopping, το cal_df για βαθμονόμηση και το test_df για τελική αξιολόγηση. Αυτή η διάκριση είναι ιδιαίτερα σημαντική στο XGBoost, επειδή το μοντέλο χρησιμοποιεί early stopping με βάση την απόδοση στο validation set.

Ιστορικά χαρακτηριστικά για το with-history generic μοντέλο

Το μοντέλο with_history_generic χρησιμοποιεί ιστορικά χαρακτηριστικά, αλλά δεν χρησιμοποιεί home_id ως είσοδο. Η δημιουργία των ιστορικών χαρακτηριστικών γίνεται με τη συνάρτηση:

```

def add_lag_and_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.sort_values([HOME_COL, TIME_COL]).copy()
    g = df.groupby(HOME_COL)[TARGET_COL]

    for lag in LAG_HOURS:
        df[f"lag_{lag}h"] = g.shift(lag)

```

Το home_id χρησιμοποιείται εδώ μόνο για ομαδοποίηση των μετρήσεων, ώστε τα lags να υπολογίζονται μέσα στην ίδια κατοικία. Δεν εισάγεται όμως στο τελικό feature matrix του μοντέλου.

Τα rolling χαρακτηριστικά δημιουργούνται με βάση τις προηγούμενες τιμές κατανάλωσης:

```

shifted = g.shift(1)

for w in ROLLING_WINDOWS:
    df[f"roll_mean_{w}h"] = (
        shifted
        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .mean()
        .reset_index(level=0, drop=True)
    )

```

Η χρήση του shift(1) πριν από τον κυλιόμενο υπολογισμό είναι κρίσιμη, γιατί αποτρέπει τη χρήση της τρέχουσας τιμής κατανάλωσης για την πρόβλεψη της ίδιας ώρας. Έτσι αποφεύγεται η διαρροή πληροφορίας.

Επιπλέον, δημιουργούνται χαρακτηριστικά μεταβλητότητας και ακραίων τιμών:

```

for w in ROLLING_EXTREME_WINDOWS:
    df[f"roll_std_{w}h"] = (
        shifted
        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .std()
        .reset_index(level=0, drop=True)
    )

```

)

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να αξιοποιήσει όχι μόνο το επίπεδο της προηγούμενης κατανάλωσης, αλλά και τη μεταβλητότητα ή τις αιχμές προηγούμενων περιόδων.

Στο τέλος δημιουργούνται και παράγωγα χαρακτηριστικά:

```
df["lag_1h_minus_24h"] = df["lag_1h"] - df["lag_24h"]
df["lag_24h_minus_168h"] = df["lag_24h"] - df["lag_168h"]
df["roll_24h_div_168h"] = df["roll_mean_24h"] / (df["roll_mean_168h"] + 1.0)
```

Τα χαρακτηριστικά αυτά συγκρίνουν πρόσφατη και παλαιότερη κατανάλωση. Για παράδειγμα, η διαφορά μεταξύ lag_1h και lag_24h δείχνει αν η τρέχουσα συμπεριφορά αποκλίνει από την αντίστοιχη ώρα της προηγούμενης ημέρας.

Global daily calibrator

Το script περιλαμβάνει προαιρετικό μηχανισμό ημερήσιας βαθμονόμησης:

```
def fit_global_daily_calibrator(cal_eval: pd.DataFrame, pred_col: str) -> Dict[str, Any]:
    y_true_sum = float(cal_eval[TARGET_COL].sum())
    y_pred_sum = float(cal_eval[pred_col].sum())

    raw_scale = (y_true_sum + 1e-6) / (y_pred_sum + 1e-6)
    raw_scale = float(np.clip(raw_scale, 0.90, 1.15))

    alpha = 0.50
    scale = float(1.0 + alpha * (raw_scale - 1.0))
```

Ο μηχανισμός αυτός υπολογίζει έναν ενιαίο διορθωτικό συντελεστή με βάση την περίοδο calibration. Στόχος του είναι να μειώσει πιθανή συστηματική υποεκτίμηση ή υπερεκτίμηση της συνολικής κατανάλωσης. Η βασική πρόβλεψη παραμένει η κύρια έξοδος του μοντέλου, ενώ η ημερήσια βαθμονομημένη πρόβλεψη αποθηκεύεται ως προαιρετική εκδοχή.

Behavioral profiles για το no-history μοντέλο

Το no-history XGBoost δεν έχει πρόσβαση σε πραγματικό ιστορικό κατανάλωσης του χρήστη. Για τον λόγο αυτό δημιουργεί προφίλ συμπεριφοράς από το training set:

```
def build_behavior_profiles(train_df: pd.DataFrame) -> Dict[str, Any]:
    tmp = train_df.copy()

    static_dow_hour = (
        tmp
        .groupby(["static_profile_key", "day_of_week", "hour"])
        .agg(
            expected_static_dow_hour_Wh=(TARGET_COL, "mean"),
            count_static_dow_hour=(TARGET_COL, "size"),
        )
        .reset_index()
    )
```

Το προφίλ static_dow_hour υπολογίζει τη μέση κατανάλωση για κάθε συνδυασμό στατικού προφίλ, ημέρας εβδομάδας και ώρας. Αν δεν υπάρχουν αρκετές παρατηρήσεις σε αυτό το επίπεδο, εφαρμόζεται fallback σε γενικότερα επίπεδα, όπως static_hour, static_overall, global_dow_hour ή global_hour.

Η λογική fallback εφαρμόζεται στη συνάρτηση add_behavior_profile:

```
fallback = [
    ("expected_static_dow_hour_Wh", "static_dow_hour"),
    ("expected_static_hour_Wh", "static_hour"),
    ("expected_static_overall_Wh", "static_overall"),
    ("expected_global_dow_hour_Wh", "global_dow_hour"),
    ("expected_global_hour_Wh", "global_hour"),
]
```

Με αυτόν τον τρόπο, το μοντέλο προσπαθεί πρώτα να χρησιμοποιήσει το πιο εξειδικευμένο διαθέσιμο προφίλ και, αν αυτό δεν είναι αξιόπιστο, καταφεύγει σε γενικότερο προφίλ. Αυτό βοηθά το no-history μοντέλο να αποκτήσει έμμεση πληροφορία συμπεριφοράς, χωρίς να απαιτεί πραγματικό ιστορικό κατανάλωσης από τον χρήστη.

KNN προφίλ παρόμοιων κατοικιών

Εκτός από τα behavioral profiles, το script δημιουργεί και KNN προφίλ παρόμοιων κατοικιών. Αρχικά κατασκευάζεται πίνακας στατικών χαρακτηριστικών ανά κατοικία:

```
def build_home_static_table(train_df: pd.DataFrame) -> pd.DataFrame:
    cols = [
        HOME_COL,
        "residents",
        "total_floor_area_m2",
        "num_electric_appliances",
        "hometype",
        "urban_rural_class",
    ]

    available_cols = [c for c in cols if c in train_df.columns]
    home_static = train_df[available_cols].drop_duplicates(subset=[HOME_COL]).copy()

    scaler = StandardScaler()
    X_static = scaler.fit_transform(home_static[available_cols])

    knn = NearestNeighbors(n_neighbors=n_neighbors, metric="euclidean")
    knn.fit(X_static)
```

Με αυτόν τον τρόπο, για κάθε κατοικία μπορούν να εντοπιστούν οι πιο παρόμοιες κατοικίες του training set ως προς τα στατικά τους χαρακτηριστικά. Από αυτές τις παρόμοιες κατοικίες υπολογίζονται ενδεικτικά προφίλ κατανάλωσης ανά ώρα, ανά ημέρα και ώρα, καθώς και συνολικά.

Η συνάρτηση add_knn_behavior_profile προσθέτει αυτά τα KNN-based χαρακτηριστικά στα δεδομένα. Έτσι το no-history XGBoost δεν βασίζεται μόνο στα άμεσα χαρακτηριστικά της κατοικίας, αλλά και σε μοτίβα κατανάλωσης κατοικιών που είναι στατικά παρόμοιες.

Φόρτωση και προετοιμασία δεδομένων

Η βασική φόρτωση των δεδομένων γίνεται ως εξής:

```
def load_and_prepare_base_data() -> Tuple[pd.DataFrame, list]:
    df = pd.read_csv(DATA_PATH, parse_dates=[TIME_COL], low_memory=False)
    df = df.sort_values([HOME_COL, TIME_COL]).reset_index(drop=True)
```

Το dataset φορτώνεται με μετατροπή του timestamp σε ημερομηνία και ώρα και ταξινομείται ανά κατοικία και χρόνο. Η ταξινόμηση αυτή είναι απαραίτητη για τη σωστή δημιουργία των ιστορικών χαρακτηριστικών.

Ακολουθεί η αφαίρεση των προβληματικών κατοικιών και ο καθαρισμός της μεταβλητής στόχου:

```
existing_bad = [h for h in BAD_HOMES if h in set(df[HOME_COL].unique())]
df = df[~df[HOME_COL].isin(existing_bad)].copy()

df[TARGET_COL] = pd.to_numeric(df[TARGET_COL], errors="coerce")
df = df.dropna(subset=[TARGET_COL]).copy()
df[TARGET_COL] = df[TARGET_COL].clip(lower=0)
```

Στη συνέχεια δημιουργούνται τα χρονικά χαρακτηριστικά και συμπληρώνονται τυχόν ελλείπουσες στήλες, ώστε το dataset να έχει ενιαία μορφή πριν περάσει στις διαδικασίες εκπαίδευσης.

Εκπαίδευση του with-history generic XGBoost

Η εκπαίδευση του with_history_generic μοντέλου ξεκινά με τη δημιουργία lag και rolling χαρακτηριστικών:

```
df_hist = add_lag_and_rolling_features(df_base)
df_hist = df_hist.dropna(subset=REQUIRED_HISTORY_COLS).copy()
df_hist = df_hist.sort_values([HOME_COL, TIME_COL]).reset_index(drop=True)
```

Οι γραμμές που δεν έχουν επαρκές ιστορικό αφαιρούνται. Το ελάχιστο απαιτούμενο ιστορικό είναι 168 ώρες, δηλαδή μία εβδομάδα ωριαίων τιμών, ώστε το μοντέλο να μπορεί να χρησιμοποιηθεί πρακτικά από τη γραφική διεπαφή.

Στη συνέχεια εφαρμόζεται χρονικός διαχωρισμός:

```
hist_train, hist_es, hist_cal, hist_test = temporal_split(df_hist)
```

Το τελικό σύνολο χαρακτηριστικών του with-history μοντέλου περιλαμβάνει στατικά, χρονικά, lag και rolling χαρακτηριστικά, αλλά όχι home_id και όχι στατιστικά γνωστής κατοικίας. Οι κατηγορικές μεταβλητές είναι μόνο:

```
categorical_cols = [
    "hometype",
    "urban_rural_class",
]
```

Αυτό αποτελεί βασική διαφορά από παλαιότερα known-home μοντέλα. Το μοντέλο δεν μαθαίνει ταυτότητα κατοικίας, αλλά γενικούς κανόνες που μπορούν να εφαρμοστούν σε νέο χρήστη που παρέχει πρόσφατο ιστορικό κατανάλωσης.

Η εκπαίδευση γίνεται με full training set, καθώς η σχετική ρύθμιση δειγματοληψίας είναι None:

```
WITH_HISTORY_XGB_MAX_TRAIN_ROWS = None
```

Το μοντέλο εκπαιδεύεται με early stopping πάνω στο validation set:

```
model = XGBRegressor(**XGB_WITH_HISTORY_PARAMS)
```

```
model = fit_xgb_with_early_stopping(
    model,
    X_train,
    y_train,
    X_es,
    y_es,
    early_stopping_rounds=EARLY_STOPPING_ROUNDS_WITH_HISTORY,
)
```

Μετά την εκπαίδευση, οι προβλέψεις γίνονται χρησιμοποιώντας την καλύτερη διαθέσιμη επανάληψη του XGBoost:

```
pred_test = inverse_target(xgb_predict_best(model, X_test))
pred_test = np.clip(pred_test, 0, None)
```

Στη συνέχεια υπολογίζεται τόσο η βασική πρόβλεψη όσο και η προαιρετική ημερήσια βαθμονομημένη πρόβλεψη:

```
metrics_balanced = evaluate_predictions(test_eval, "prediction_Wh")
metrics_daily_calibrated = evaluate_predictions(test_eval,
"prediction_daily_calibrated_Wh")
```

Η βασική πρόβλεψη παραμένει η κύρια εκδοχή, ενώ η daily-calibrated εκδοχή χρησιμοποιείται μόνο προαιρετικά.

Εκπαίδευση του no-history optional XGBoost

Η δεύτερη εκδοχή του script αφορά το προαιρετικό no-history XGBoost. Σε αυτή την περίπτωση δεν υπάρχουν πραγματικές ιστορικές τιμές κατανάλωσης. Για τον λόγο αυτό, το script δημιουργεί έμμεσα χαρακτηριστικά συμπεριφοράς και παρόμοιων κατοικιών.

Αρχικά εφαρμόζεται ο χρονικός διαχωρισμός:

```
train_df, es_df, cal_df, test_df = temporal_split(df_nohist)
```

Στη συνέχεια δημιουργούνται train-only behavioral profiles:

```
behavior_profiles = build_behavior_profiles(train_df)
```

Η έννοια train-only είναι πολύ σημαντική. Τα προφίλ συμπεριφοράς υπολογίζονται μόνο από το training set, ώστε να μη χρησιμοποιηθεί πληροφορία από τις μελλοντικές περιόδους validation, calibration ή test.

Στη συνέχεια δημιουργείται το KNN bundle:

```
knn_bundle = fit_knn_static_model(train_df, n_neighbors=N_KNN_NEIGHBORS)
```

Το KNN bundle περιλαμβάνει τον scaler, το μοντέλο κοντινότερων γειτόνων, τα στατικά χαρακτηριστικά των κατοικιών και τα προφίλ κατανάλωσης των γειτονικών κατοικιών. Αυτά αποθηκεύονται ώστε να μπορούν να χρησιμοποιηθούν αργότερα και από το API.

Το τελικό no-history XGBoost μοντέλο χρησιμοποιεί αριθμητικά χαρακτηριστικά όπως τα βασικά στατικά και χρονικά χαρακτηριστικά, τα expected behavior χαρακτηριστικά και τα KNN χαρακτηριστικά. Χρησιμοποιεί επίσης κατηγορικά χαρακτηριστικά όπως:

```
NOHISTORY_CATEGORICAL = [  
    "hometype",  
    "urban_rural_class",  
    "residents_bin",  
    "area_bin",  
    "appliances_bin",  
    "behavior_profile_source",  
]
```

Το behavior_profile_source δείχνει από ποιο επίπεδο προφίλ προήλθε η αναμενόμενη κατανάλωση. Για παράδειγμα, μπορεί να προήλθε από εξειδικευμένο προφίλ static_dow_hour ή από γενικότερο fallback προφίλ.

Πριν από την τελική εκπαίδευση, το script υπολογίζει και δύο baseline προφίλ: το behavior profile only και το KNN profile only. Αυτά δεν είναι τα τελικά μοντέλα, αλλά λειτουργούν ως διαγνωστικά σημεία αναφοράς, ώστε να φανεί αν τα έμμεσα προφίλ έχουν κάποια προγνωστική αξία πριν χρησιμοποιηθούν ως χαρακτηριστικά στο XGBoost.

Η εκπαίδευση του no-history XGBoost γίνεται επίσης με early stopping:

```
model = XGBRegressor(**XGB_no_history_PARAMS)
```

```
model = fit_xgb_with_early_stopping(  
    model,  
    X_train,  
    y_train,  
    X_es,  
    y_es,  
    early_stopping_rounds=EARLY_STOPPING_ROUNDS_no_history,  
)
```

Με αυτόν τον τρόπο περιορίζεται ο κίνδυνος υπερπροσαρμογής, καθώς η εκπαίδευση παρακολουθεί την απόδοση στο validation set.

Αποθήκευση artifacts

Και για τις δύο εκδοχές του XGBoost αποθηκεύονται τα βασικά αρχεία που απαιτούνται από το API:

```
model_path = WITH_HISTORY_DIR / "model.joblib"  
preprocessor_path = WITH_HISTORY_DIR / "preprocessor.pkl"  
feature_config_path = WITH_HISTORY_DIR / "feature_config.json"  
metadata_path = WITH_HISTORY_DIR / "metadata.json"  
predictions_path = WITH_HISTORY_DIR / "test_predictions.csv"
```

Το model.joblib περιέχει το εκπαιδευμένο XGBoost μοντέλο. Το preprocessor.pkl περιέχει τη διαδικασία μετατροπής των αριθμητικών και κατηγορικών χαρακτηριστικών. Το feature_config.json περιγράφει ποια χαρακτηριστικά αναμένει το μοντέλο και ποιες λειτουργίες χρησιμοποιεί. Το metadata.json αποθηκεύει τις παραμέτρους εκπαίδευσης, τις μετρικές, τη διάρκεια εκπαίδευσης, το καλύτερο iteration και τις διαδρομές των παραγόμενων αρχείων.

Στο with-history μοντέλο αποθηκεύεται επιπλέον ο global daily calibrator:

```
save_json(global_daily_calibrator, WITH_HISTORY_DIR /  
    "global_daily_calibrator.json")
```

Στο no-history μοντέλο αποθηκεύονται επίσης τα προφίλ συμπεριφοράς και το KNN bundle:

```
joblib.dump(behavior_profiles, no_history_DIR /
"behavior_profiles.joblib")
joblib.dump(knn_bundle, no_history_DIR / "knn_bundle.joblib")
```

Αυτά τα αρχεία είναι απαραίτητα ώστε το API να μπορεί να ανακατασκευάσει τα ίδια behavior και KNN χαρακτηριστικά όταν ο χρήστης ζητήσει πρόβλεψη χωρίς ιστορικό κατανάλωσης.

Συγκεντρωτικό αρχείο εκπαίδευσης

Στο τέλος του script δημιουργείται το συνολικό αρχείο training_summary.json:

```
summary = {
    "script": "train_xgb_api_generic_models.py",
    "data_path": str(DATA_PATH),
    "target": TARGET_COL,
    "time_col": TIME_COL,
    "home_col": HOME_COL,
    "test_days": TEST_DAYS,
    "val_days_total": VAL_DAYS_TOTAL,
    "cal_days": CAL_DAYS,
    "bad_homes_removed": existing_bad,
    "use_log_target": USE_LOG_TARGET,
    "clip_target_train": CLIP_TARGET_TRAIN,
    "clip_q_low": CLIP_Q_LOW,
    "clip_q_high": CLIP_Q_HIGH,
    "use_gpu": USE_GPU,
    "output_root": str(OUT_ROOT),
}
```

Το αρχείο αυτό συγκεντρώνει τις βασικές πληροφορίες εκτέλεσης, τις ρυθμίσεις του script, τα μοντέλα που εκπαιδεύτηκαν και τις διαδρομές των παραγόμενων αρχείων. Περιλαμβάνει επίσης σημείωση ότι τα XGBoost μοντέλα που παράγονται εδώ είναι API-compatible optional comparison models και δεν απαιτούν home_id από τον χρήστη.

Η εκτέλεση του script οργανώνεται από τη συνάρτηση main:

```
if TRAIN_WITH_HISTORY_GENERIC:
    summary["with_history_generic"] =
train_with_history_generic_model(df_base)
else:
    summary["with_history_generic"] = None

if TRAIN_no_history_OPTIONAL:
    summary["no_history_optional"] =
train_no_history_optional_model(df_base)
else:
    summary["no_history_optional"] = None
```

```
save_json(summary, OUT_ROOT / "training_summary.json")
```

Με αυτή τη δομή, το script μπορεί να εκπαιδεύσει και τις δύο εκδοχές μοντέλων ή μόνο μία από αυτές, ανάλογα με τις αντίστοιχες ρυθμίσεις.

LightGBM. Αρχείο: LGBM_for_API2.ipynb

LightGBM Regressor. Αρχείο: train_lgbm_api_generic_models.py

Το παρόν script υλοποιεί την εκπαίδευση και αποθήκευση των τελικών μοντέλων LightGBM που χρησιμοποιούνται από το API και τη γραφική διεπαφή της εφαρμογής. Η βασική λογική του είναι αντίστοιχη με το XGBoost script, καθώς και εδώ τα μοντέλα έχουν σχεδιαστεί για τη γενική έκδοση της εφαρμογής, όπου ο τελικός χρήστης δεν παρέχει home_id.

Το `home_id` χρησιμοποιείται μόνο εσωτερικά κατά την προετοιμασία των δεδομένων, ώστε να υπολογίζονται σωστά τα ιστορικά χαρακτηριστικά ανά κατοικία και να αξιολογούνται οι προβλέψεις ανά κατοικία και ημέρα. Δεν χρησιμοποιείται όμως ως χαρακτηριστικό εισόδου στο τελικό μοντέλο. Αυτό είναι σημαντικό, διότι επιτρέπει στα μοντέλα να χρησιμοποιηθούν από τη γραφική διεπαφή για νέους χρήστες, χωρίς αυτοί να πρέπει να ταυτιστούν με κάποια κατοικία του αρχικού συνόλου δεδομένων.

Το script εκπαιδεύει δύο εκδοχές LightGBM. Η πρώτη είναι το γενικό μοντέλο με ιστορικό κατανάλωσης (`with_history_generic`), το οποίο χρησιμοποιεί `lag` και `rolling` χαρακτηριστικά, χρονικά χαρακτηριστικά, περιβαλλοντικές μεταβλητές και στατικά χαρακτηριστικά της κατοικίας. Η δεύτερη είναι το προαιρετικό μοντέλο χωρίς ιστορικό (`no_history_optional`), το οποίο χρησιμοποιεί βασικά χαρακτηριστικά, προφίλ συμπεριφοράς και KNN προφίλ παρόμοιων κατοικιών. Το δεύτερο μοντέλο παραμένει διαθέσιμο ως μοντέλο σύγκρισης, ενώ η τελική προεπιλογή του no-history σεναρίου μπορεί να παραμείνει το Random Forest.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import json
import math
import warnings
from pathlib import Path
from typing import Any, Dict, Tuple

import joblib
import lightgbm as lgb
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.neighbors import NearestNeighbors
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

Η `pandas` χρησιμοποιείται για τη φόρτωση και επεξεργασία των δεδομένων, ενώ η `numpy` για αριθμητικές πράξεις, μετασχηματισμούς και δημιουργία χαρακτηριστικών. Η βιβλιοθήκη `lightgbm` χρησιμοποιείται για την εκπαίδευση των μοντέλων LightGBM. Οι συναρτήσεις `mean_absolute_error` και `mean_squared_error` χρησιμοποιούνται για την αξιολόγηση των προβλέψεων.

Η `joblib` χρησιμοποιείται για την αποθήκευση των εκπαιδευμένων μοντέλων, των μηχανισμών προεπεξεργασίας και των πρόσθετων αντικειμένων που απαιτούνται από το API. Οι κλάσεις `ColumnTransformer`, `Pipeline`, `SimpleImputer` και `OneHotEncoder` οργανώνουν την προεπεξεργασία αριθμητικών και κατηγορικών χαρακτηριστικών. Επιπλέον, οι `NearestNeighbors` και `StandardScaler` χρησιμοποιούνται στο no-history μοντέλο για την κατασκευή προφίλ παρόμοιων κατοικιών.

Ρυθμίσεις παραμετροποίησης

Στη συνέχεια ορίζονται οι βασικές διαδρομές του έργου:

```
BASE_DIR = Path("C:/IDEAL Programming")
DATA_PATH = BASE_DIR / "processed" / "final" /
"IDEAL_final_hourly_dataset.csv"

OUT_ROOT = BASE_DIR / "processed" / "models" / "final_api_models" / "LGBM2"
WITH_HISTORY_DIR = OUT_ROOT / "with_history_generic"
no_history_DIR = OUT_ROOT / "no_history_optional"
```

Το DATA_PATH δείχνει στο τελικό ενοποιημένο σύνολο δεδομένων του IDEAL. Το OUT_ROOT ορίζει τον βασικό φάκελο εξόδου για τα LightGBM μοντέλα. Το μοντέλο με ιστορικό αποθηκεύεται στον φάκελο with_history_generic, ενώ το προαιρετικό μοντέλο χωρίς ιστορικό αποθηκεύεται στον φάκελο no_history_optional.

Οι βασικές στήλες του dataset ορίζονται ως εξής:

```
TIME_COL = "timestamp"  
HOME_COL = "home_id"  
TARGET_COL = "consumption_Wh"
```

```
RANDOM_STATE = 42
```

Η στήλη timestamp δηλώνει τη χρονική στιγμή της μέτρησης, η στήλη home_id χρησιμοποιείται μόνο για εσωτερική ομαδοποίηση και αξιολόγηση, ενώ η στήλη consumption_Wh αποτελεί τη μεταβλητή στόχο. Το RANDOM_STATE χρησιμοποιείται για την αναπαραγωγικότητα των αποτελεσμάτων.

Χρονικός διαχωρισμός δεδομένων

Ο χρονικός διαχωρισμός των δεδομένων ορίζεται με τις παρακάτω παραμέτρους:

```
TEST_DAYS = 30  
VAL_DAYS_TOTAL = 30  
CAL_DAYS = 15
```

Οι τελευταίες 30 ημέρες του συνόλου δεδομένων χρησιμοποιούνται ως test set. Η περίοδος πριν από το test set χωρίζεται σε δύο μέρη: early-stop validation και calibration validation. Η πρώτη χρησιμοποιείται για την παρακολούθηση της εκπαίδευσης, ενώ η δεύτερη για την προαιρετική ημερήσια βαθμονόμηση των προβλέψεων.

Η χρήση χρονικού διαχωρισμού είναι απαραίτητη, επειδή η πρόβλεψη ηλεκτρικής κατανάλωσης είναι πρόβλημα χρονοσειρών. Αν γινόταν τυχαίος διαχωρισμός, θα υπήρχε κίνδυνος διαρροής πληροφορίας από μελλοντικές χρονικές στιγμές προς το training set.

Αφαίρεση προβληματικών κατοικιών

Στην αρχή της εκτέλεσης αφαιρούνται συγκεκριμένες κατοικίες που είχαν χαρακτηριστεί προβληματικές:

```
BAD_HOMES = ["home171", "home219", "home228", "home271"]
```

Μετά την αφαίρεσή τους, το dataset μειώνεται από 254 σε 250 κατοικίες. Η επιλογή αυτή βοηθά ώστε η εκπαίδευση να βασιστεί σε πιο αξιόπιστα δεδομένα και να μη χρησιμοποιηθούν κατοικίες με προβληματικά ή μη ρεαλιστικά μεταδεδομένα.

Μετασχηματισμός μεταβλητής στόχου

Η μεταβλητή στόχος μετασχηματίζεται λογαριθμικά:

```
USE_LOG_TARGET = True  
CLIP_TARGET_TRAIN = False
```

Η επιλογή USE_LOG_TARGET = True σημαίνει ότι η κατανάλωση μετατρέπεται με τη συνάρτηση log1p πριν από την εκπαίδευση. Αυτό βοηθά στη σταθεροποίηση της αριθμητικής κλίμακας, καθώς οι τιμές κατανάλωσης μπορεί να έχουν έντονες αιχμές και ασυμμετρία. Το clipping του target είναι απενεργοποιημένο, καθώς η παράμετρος CLIP_TARGET_TRAIN έχει τιμή False.

Οι συναρτήσεις που εφαρμόζουν τον μετασχηματισμό είναι:

```
def transform_target(y):  
    y = np.asarray(y, dtype=np.float64)  
    if USE_LOG_TARGET:  
        return np.log1p(np.maximum(y, 0.0))  
    return y  
  
def inverse_target(y_hat):  
    y_hat = np.asarray(y_hat, dtype=np.float64)  
    if USE_LOG_TARGET:  
        return np.expm1(y_hat)  
    return y_hat
```

Η πρώτη συνάρτηση χρησιμοποιείται πριν από την εκπαίδευση, ενώ η δεύτερη επαναφέρει τις προβλέψεις στην αρχική κλίμακα Wh. Έτσι, η εκπαίδευση γίνεται σε πιο σταθερή κλίμακα, αλλά η τελική αξιολόγηση γίνεται σε πραγματικές μονάδες ηλεκτρικής κατανάλωσης.

Βασικά χαρακτηριστικά εισόδου

Τα αριθμητικά χαρακτηριστικά που μπορούν να δοθούν ή να συμπληρωθούν από το API και τη γραφική διεπαφή ορίζονται ως εξής:

```
STATIC_NUMERIC = [  
    "external_temperature",  
    "internal_temperature",  
    "residents",  
    "total_floor_area_m2",  
    "num_electric_appliances",  
]
```

Τα χαρακτηριστικά αυτά περιλαμβάνουν περιβαλλοντικές και στατικές πληροφορίες. Η εξωτερική και η εσωτερική θερμοκρασία μπορούν να σχετίζονται με την ενεργειακή ζήτηση, ενώ ο αριθμός κατοίκων, το εμβαδόν και ο αριθμός ηλεκτρικών συσκευών σχετίζονται με τη συνολική ενεργειακή συμπεριφορά της κατοικίας.

Τα χρονικά χαρακτηριστικά ορίζονται ως εξής:

```
TIME_NUMERIC = [  
    "hour",  
    "day_of_week",  
    "month",  
    "day_of_month",  
    "week_of_year",  
    "is_weekend",  
    "is_holiday",  
    "hour_sin",  
    "hour_cos",  
    "day_sin",  
    "day_cos",  
    "month_sin",  
    "month_cos",  
]
```

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να αποτυπώσει ημερήσια, εβδομαδιαία και εποχικά μοτίβα κατανάλωσης. Τα κυκλικά χαρακτηριστικά, όπως hour_sin, hour_cos, day_sin, day_cos, month_sin και month_cos, βοηθούν ώστε οι χρονικές μεταβλητές να εκφράζονται ως περιοδικές και όχι ως απλοί ακέραιοι αριθμοί.

Τα βασικά κατηγορικά χαρακτηριστικά είναι:

```
BASE_CATEGORICAL = [  
    "hometype",  
    "urban_rural_class",  
]
```

Το hometype περιγράφει τον τύπο κατοικίας, ενώ το urban_rural_class περιγράφει την κατηγορία περιοχής. Οι μεταβλητές αυτές μετατρέπονται αργότερα σε αριθμητική μορφή μέσω one-hot encoding.

Χαρακτηριστικά ιστορικού για το with-history μοντέλο

Για το γενικό μοντέλο με ιστορικό κατανάλωσης ορίζονται οι χρονικές υστερήσεις και τα κυλίσματα παράθυρα:

```
LAG_HOURS = [1, 2, 3, 6, 12, 24, 48, 72, 168]  
ROLLING_WINDOWS = [3, 6, 12, 24, 48, 168]  
ROLLING_EXTREME_WINDOWS = [24, 48, 168]
```

Η λίστα LAG_HOURS δηλώνει προηγούμενες χρονικές στιγμές κατανάλωσης που χρησιμοποιούνται ως χαρακτηριστικά. Για παράδειγμα, το lag_1h αντιστοιχεί στην

προηγούμενη ώρα, το lag_24h στην ίδια ώρα της προηγούμενης ημέρας και το lag_168h στην ίδια ώρα της προηγούμενης εβδομάδας.

Τα ROLLING_WINDOWS χρησιμοποιούνται για τον υπολογισμό κυλιόμενων μέσων όρων, ενώ τα ROLLING_EXTREME_WINDOWS χρησιμοποιούνται για κυλιόμενη τυπική απόκλιση, ελάχιστες και μέγιστες τιμές.

Τα απαραίτητα ιστορικά χαρακτηριστικά είναι:

```
REQUIRED_HISTORY_COLS = [  
    "lag_1h",  
    "lag_24h",  
    "lag_168h",  
    "roll_mean_24h",  
    "roll_mean_168h",  
]
```

```
MIN_REQUIRED_HISTORY_HOURS = 168
```

Η τιμή MIN_REQUIRED_HISTORY_HOURS = 168 σημαίνει ότι το μοντέλο απαιτεί τουλάχιστον μία εβδομάδα ωριαίου ιστορικού. Αυτό είναι σημαντικό για την πρακτική χρήση στη γραφική διεπαφή, καθώς ο χρήστης πρέπει να παρέχει επαρκές πρόσφατο ιστορικό ώστε να μπορούν να υπολογιστούν τα βασικά ιστορικά χαρακτηριστικά.

Παράμετροι LightGBM για το with-history μοντέλο

Οι παράμετροι του LightGBM με ιστορικό ορίζονται ως εξής:

```
HISTORY_LGBM_PARAMS = dict(  
    n_estimators=9000,  
    learning_rate=0.012,  
    num_leaves=127,  
    max_depth=-1,  
    min_child_samples=80,  
    subsample=0.9,  
    colsample_bytree=0.9,  
    reg_alpha=0.05,  
    reg_lambda=0.20,  
    objective="regression",  
    random_state=RANDOM_STATE + 5,  
    n_jobs=-1,  
    force_row_wise=True,  
    verbosity=-1,  
)
```

Το n_estimators=9000 δηλώνει μεγάλο αριθμό πιθανών επαναλήψεων boosting. Ο χαμηλός ρυθμός μάθησης learning_rate=0.012 επιτρέπει στο μοντέλο να μαθαίνει σταδιακά. Το num_leaves=127 καθορίζει την πολυπλοκότητα των δέντρων, ενώ το min_child_samples=80 λειτουργεί ως μηχανισμός περιορισμού της υπερπροσαρμογής.

Οι παράμετροι subsample και colsample_bytree εφαρμόζουν υποδειγματοληψία γραμμών και χαρακτηριστικών, ενώ οι reg_alpha και reg_lambda προσθέτουν κανονικοποίηση. Με αυτόν τον τρόπο επιδιώκεται ισορροπία ανάμεσα στην ικανότητα του μοντέλου να αποτυπώνει σύνθετες σχέσεις και στην ανάγκη για καλή γενίκευση.

Παράμετροι LightGBM για το no-history μοντέλο

Για το προαιρετικό no-history μοντέλο χρησιμοποιούνται διαφορετικές παράμετροι:

```
NOHISTORY_LGBM_PARAMS = dict(  
    n_estimators=7000,  
    learning_rate=0.015,  
    num_leaves=63,  
    max_depth=-1,  
    min_child_samples=80,  
    subsample=0.9,  
    colsample_bytree=0.9,  
    reg_alpha=0.05,
```

```

reg_lambda=0.20,
objective="regression",
random_state=RANDOM_STATE,
n_jobs=-1,
force_row_wise=True,
verbosity=-1,
)

```

Το no-history μοντέλο έχει μικρότερο αριθμό φύλλων (num_leaves=63) σε σχέση με το with-history μοντέλο. Αυτό είναι λογικό, επειδή δεν διαθέτει πραγματικές ιστορικές τιμές κατανάλωσης και βασίζεται περισσότερο σε γενικά χαρακτηριστικά και έμμεσα προφίλ. Με τις συγκεκριμένες ρυθμίσεις επιδιώκεται η αποφυγή υπερβολικά σύνθετων δέντρων.

Μετρικές αξιολόγησης

Το script χρησιμοποιεί κοινές μετρικές αξιολόγησης με τα υπόλοιπα μοντέλα:

```

def compute_metrics(y_true, y_pred) -> Dict[str, float]:
    y_true = np.asarray(y_true, dtype=np.float64)
    y_pred = np.asarray(y_pred, dtype=np.float64)

    return {
        "MAE": float(mean_absolute_error(y_true, y_pred)),
        "RMSE": float(math.sqrt(mean_squared_error(y_true, y_pred))),
        "MAPE_%": mape_safe(y_true, y_pred),
        "sMAPE_%": smape_safe(y_true, y_pred),
        "bias_Wh": float(np.mean(y_pred - y_true)),
    }

```

Το MAE εκφράζει τη μέση απόλυτη απόκλιση σε Wh, ενώ το RMSE δίνει μεγαλύτερη βαρύτητα στα μεγάλα σφάλματα. Το MAPE και το sMAPE δίνουν ποσοστιαία εικόνα του σφάλματος, ενώ το bias_Wh δείχνει αν το μοντέλο τείνει να υποεκτιμά ή να υπερεκτιμά την κατανάλωση.

Η συνάρτηση evaluate_predictions υπολογίζει επιπλέον ημερήσιες μετρικές:

```

def evaluate_predictions(df_eval: pd.DataFrame, pred_col: str) -> Dict[str, float]:
    y_true = df_eval[TARGET_COL].values
    y_pred = df_eval[pred_col].values

    metrics = compute_metrics(y_true, y_pred)

    daily = (
        df_eval
        .assign(date=df_eval[TIME_COL].dt.date)
        .groupby([HOME_COL, "date"], as_index=False)
        .agg(
            actual_kWh=(TARGET_COL, lambda x: x.sum() / 1000),
            pred_kWh=(pred_col, lambda x: x.sum() / 1000),
        )
    )

```

Με αυτή τη συνάρτηση οι ωριαίες προβλέψεις συγκεντρώνονται σε ημερήσιο επίπεδο ανά κατοικία. Έτσι υπολογίζεται το μέσο ημερήσιο απόλυτο σφάλμα σε kWh, το οποίο είναι ιδιαίτερα χρήσιμο για την αξιολόγηση της συνολικής ημερήσιας κατανάλωσης.

Μηχανισμός προεπεξεργασίας

Η προεπεξεργασία των χαρακτηριστικών οργανώνεται με ColumnTransformer:

```

def build_preprocessor(numeric_cols, categorical_cols):
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
        ]
    )

    categorical_transformer = Pipeline(

```

```

        steps=[
            ("imputer", SimpleImputer(strategy="constant",
fill_value="unknown")),
            ("onehot", make_one_hot_encoder()),
        ]
    )

    return ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_cols),
            ("cat", categorical_transformer, categorical_cols),
        ],
        remainder="drop",
        sparse_threshold=0.3,
    )

```

Οι αριθμητικές μεταβλητές συμπληρώνονται με τη διάμεσο, ενώ οι κατηγορικές μεταβλητές συμπληρώνονται με την τιμή unknown και μετατρέπονται σε one-hot encoded στήλες. Ο preprocessor αποθηκεύεται μαζί με το μοντέλο, ώστε κατά τη φάση πρόβλεψης να εφαρμοστεί ακριβώς η ίδια προεπεξεργασία.

Προσθήκη χρονικών χαρακτηριστικών

Η δημιουργία χρονικών χαρακτηριστικών γίνεται με τη συνάρτηση `add_time_features`:

```

def add_time_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    df[TIME_COL] = pd.to_datetime(df[TIME_COL])
    df["hour"] = df[TIME_COL].dt.hour
    df["day_of_week"] = df[TIME_COL].dt.dayofweek
    df["month"] = df[TIME_COL].dt.month
    df["day_of_month"] = df[TIME_COL].dt.day
    df["week_of_year"] = df[TIME_COL].dt.isocalendar().week.astype(int)
    df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)

```

Το αρχικό timestamp δεν χρησιμοποιείται αυτούσιο, αλλά μετατρέπεται σε χαρακτηριστικά με σαφή ερμηνεία. Η ώρα, η ημέρα της εβδομάδας και ο μήνας βοηθούν στην αποτύπωση καθημερινών, εβδομαδιαίων και εποχικών μοτίβων.

Στη συνέχεια δημιουργούνται κυκλικά χαρακτηριστικά:

```

df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
df["day_sin"] = np.sin(2 * np.pi * df["day_of_week"] / 7)
df["day_cos"] = np.cos(2 * np.pi * df["day_of_week"] / 7)
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

```

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να κατανοήσει καλύτερα την κυκλική φύση του χρόνου. Για παράδειγμα, η ώρα 23:00 και η ώρα 00:00 είναι χρονικά κοντά, κάτι που αποτυπώνεται καλύτερα μέσω των ημιτονοειδών και συνημιτονοειδών μετασχηματισμών.

Στατικές κατηγορίες για το no-history σενάριο

Για το no-history μοντέλο δημιουργούνται επιπλέον στατικές κατηγορίες:

```

def add_static_bins(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()

    r = pd.to_numeric(df["residents"], errors="coerce")
    df["residents_bin"] = np.select(
        [r.isna(), r <= 0, r == 1, r == 2, r == 3, r == 4, r >= 5],
        ["unknown", "unknown", "1", "2", "3", "4", "5plus"],
        default="unknown",
    )

```

Η συνάρτηση αυτή δημιουργεί κατηγορίες για τον αριθμό κατοίκων, το εμβαδόν και τον αριθμό ηλεκτρικών συσκευών. Οι κατηγορίες αυτές χρησιμοποιούνται για την κατασκευή προφίλ συμπεριφοράς από ομάδες κατοικιών με παρόμοια στατικά χαρακτηριστικά.

Η τελική κλείδα στατικού προφίλ δημιουργείται ως εξής:

```
df["static_profile_key"] = (
    df["hometype"].astype(str) + "|" +
    df["urban_rural_class"].astype(str) + "|" +
    df["residents_bin"].astype(str) + "|" +
    df["area_bin"].astype(str) + "|" +
    df["appliances_bin"].astype(str)
)
```

Το `static_profile_key` συνδυάζει βασικά στατικά χαρακτηριστικά και χρησιμοποιείται για την αναζήτηση αναμενόμενων προφίλ κατανάλωσης.

Χρονικός διαχωρισμός δεδομένων

Ο χρονικός διαχωρισμός πραγματοποιείται με τη συνάρτηση `temporal_split`:

```
def temporal_split(df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame, pd.DataFrame]:
    max_time = df[TIME_COL].max()

    test_start = max_time - pd.Timedelta(days=TEST_DAYS) +
pd.Timedelta(hours=1)
    val_start = test_start - pd.Timedelta(days=VAL_DAYS_TOTAL)
    cal_start = test_start - pd.Timedelta(days=CAL_DAYS)

    train_df = df[df[TIME_COL] < val_start].copy()
    es_df = df[(df[TIME_COL] >= val_start) & (df[TIME_COL] <
cal_start)].copy()
    cal_df = df[(df[TIME_COL] >= cal_start) & (df[TIME_COL] <
test_start)].copy()
    test_df = df[df[TIME_COL] >= test_start].copy()

    return train_df, es_df, cal_df, test_df
```

Η συνάρτηση αυτή εξασφαλίζει ότι το test set βρίσκεται χρονικά μετά από όλα τα υπόλοιπα σύνολα. Το training set χρησιμοποιείται για την εκπαίδευση, το `es_df` για την παρακολούθηση της εκπαίδευσης, το `cal_df` για βαθμονόμηση και το `test_df` για την τελική αξιολόγηση.

Δημιουργία ιστορικών χαρακτηριστικών

Το `with-history generic` μοντέλο χρησιμοποιεί lag και rolling χαρακτηριστικά. Η δημιουργία τους γίνεται με τη συνάρτηση `add_lag_and_rolling_features`:

```
def add_lag_and_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.sort_values([HOME_COL, TIME_COL]).copy()

    g = df.groupby(HOME_COL)[TARGET_COL]

    for lag in LAG_HOURS:
        df[f"lag_{lag}h"] = g.shift(lag)
```

Το `home_id` χρησιμοποιείται εδώ μόνο για ομαδοποίηση των μετρήσεων κατά την προετοιμασία των δεδομένων. Δεν εισάγεται όμως ως χαρακτηριστικό στο τελικό μοντέλο. Η ομαδοποίηση είναι απαραίτητη, ώστε οι προηγούμενες τιμές κατανάλωσης να υπολογίζονται μέσα στην ίδια κατοικία και να μη γίνεται ανάμειξη διαφορετικών χρονοσειρών.

Οι κυλιόμενοι μέσοι όροι υπολογίζονται με βάση τις προηγούμενες τιμές:

```
shifted = g.shift(1)

for w in ROLLING_WINDOWS:
    df[f"roll_mean_{w}h"] = (
        shifted
```

```

        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .mean()
        .reset_index(level=0, drop=True)
    )

```

Η χρήση του `shift(1)` είναι σημαντική, διότι αποτρέπει τη χρήση της τρέχουσας τιμής κατανάλωσης για την πρόβλεψη της ίδιας χρονικής στιγμής. Έτσι αποφεύγεται η διαρροή πληροφορίας.

Επιπλέον, υπολογίζονται κυλιόμενη τυπική απόκλιση, ελάχιστες και μέγιστες τιμές:

```

for w in ROLLING_EXTREME_WINDOWS:
    df[f"roll_std_{w}h"] = (
        shifted
        .groupby(df[HOME_COL])
        .rolling(w, min_periods=max(2, w // 4))
        .std()
        .reset_index(level=0, drop=True)
    )

```

Αυτά τα χαρακτηριστικά βοηθούν το μοντέλο να αξιοποιήσει όχι μόνο το μέσο επίπεδο προηγούμενης κατανάλωσης, αλλά και τη μεταβλητότητα ή τις αιχμές προηγούμενων χρονικών παραθύρων.

Στο τέλος δημιουργούνται παράγωγα χαρακτηριστικά:

```

df["lag_1h_minus_24h"] = df["lag_1h"] - df["lag_24h"]
df["lag_24h_minus_168h"] = df["lag_24h"] - df["lag_168h"]
df["roll_24h_div_168h"] = df["roll_mean_24h"] / (df["roll_mean_168h"] + 1.0)

```

Τα χαρακτηριστικά αυτά επιτρέπουν στο μοντέλο να συγκρίνει πρόσφατη και παλαιότερη συμπεριφορά. Για παράδειγμα, η διαφορά μεταξύ `lag_24h` και `lag_168h` δείχνει αν η κατανάλωση της ίδιας ώρας διαφοροποιείται από το αντίστοιχο εβδομαδιαίο μοτίβο.

Global daily calibrator

Το script περιλαμβάνει προαιρετικό μηχανισμό ημερήσιας βαθμονόμησης:

```

def fit_global_daily_calibrator(cal_eval: pd.DataFrame, pred_col: str) -> Dict[str, Any]:
    y_true_sum = float(cal_eval[TARGET_COL].sum())
    y_pred_sum = float(cal_eval[pred_col].sum())

    raw_scale = (y_true_sum + 1e-6) / (y_pred_sum + 1e-6)
    raw_scale = float(np.clip(raw_scale, 0.90, 1.15))

    alpha = 0.50
    scale = float(1.0 + alpha * (raw_scale - 1.0))

```

Ο μηχανισμός αυτός υπολογίζει έναν ενιαίο διορθωτικό συντελεστή από την περίοδο calibration. Σκοπός του είναι να μειώσει πιθανή συστηματική υποεκτίμηση ή υπερεκτίμηση της συνολικής κατανάλωσης. Η βασική πρόβλεψη παραμένει η κύρια εκδοχή, ενώ η daily-calibrated πρόβλεψη αποθηκεύεται ως προαιρετική εναλλακτική.

Behavioral profiles για το no-history μοντέλο

Το no-history LightGBM δεν έχει πρόσβαση σε πραγματικό ιστορικό κατανάλωσης του χρήστη. Για τον λόγο αυτό δημιουργεί προφίλ συμπεριφοράς από το training set:

```

def build_behavior_profiles(train_df: pd.DataFrame) -> Dict[str, Any]:
    tmp = train_df.copy()

    static_dow_hour = (
        tmp
        .groupby(["static_profile_key", "day_of_week", "hour"])
        .agg(
            expected_static_dow_hour_Wh=(TARGET_COL, "mean"),
            count_static_dow_hour=(TARGET_COL, "size"),
        )
    )

```

```
        .reset_index()
    )
```

Το προφίλ `static_dow_hour` υπολογίζει τη μέση κατανάλωση για κάθε συνδυασμό στατικού προφίλ, ημέρας εβδομάδας και ώρας. Αν δεν υπάρχουν αρκετές παρατηρήσεις, το σύστημα χρησιμοποιεί γενικότερα επίπεδα προφίλ.

Η σειρά `fallback` ορίζεται ως εξής:

```
fallback = [
    ("expected_static_dow_hour_Wh", "static_dow_hour"),
    ("expected_static_hour_Wh", "static_hour"),
    ("expected_static_overall_Wh", "static_overall"),
    ("expected_global_dow_hour_Wh", "global_dow_hour"),
    ("expected_global_hour_Wh", "global_hour"),
]
```

Με αυτή τη λογική, το σύστημα προσπαθεί πρώτα να χρησιμοποιήσει το πιο εξειδικευμένο διαθέσιμο προφίλ. Αν αυτό δεν είναι διαθέσιμο ή δεν έχει αρκετές παρατηρήσεις, χρησιμοποιεί πιο γενικό προφίλ. Έτσι το no-history μοντέλο αποκτά έμμεση πληροφορία συμπεριφοράς, χωρίς να απαιτεί πραγματικές προηγούμενες μετρήσεις από τον χρήστη.

KNN προφίλ παρόμοιων κατοικιών

Εκτός από τα behavioral profiles, το script δημιουργεί και KNN προφίλ παρόμοιων κατοικιών. Αρχικά κατασκευάζεται πίνακας στατικών χαρακτηριστικών ανά κατοικία:

```
def build_home_static_table(train_df: pd.DataFrame) -> pd.DataFrame:
    cols = [
        HOME_COL,
        "residents",
        "total_floor_area_m2",
        "num_electric_appliances",
        "hometype",
        "urban_rural_class",
    ]

    available_cols = [c for c in cols if c in train_df.columns]
    home_static = train_df[available_cols].drop_duplicates(subset=[HOME_COL]).copy()
```

Στη συνέχεια τα στατικά χαρακτηριστικά κωδικοποιούνται και τυποποιούνται:

```
scaler = StandardScaler()
X_static = scaler.fit_transform(home_static[available_cols].values)

knn = NearestNeighbors(
    n_neighbors=n_neighbors,
    metric="euclidean",
)
knn.fit(X_static)
```

Με αυτόν τον τρόπο εντοπίζονται κατοικίες με παρόμοια στατικά χαρακτηριστικά. Από τις κατοικίες αυτές υπολογίζονται ενδεικτικά προφίλ κατανάλωσης, τα οποία στη συνέχεια χρησιμοποιούνται ως πρόσθετα χαρακτηριστικά στο no-history μοντέλο.

Φόρτωση και προετοιμασία δεδομένων

Η φόρτωση των δεδομένων γίνεται μέσα από τη συνάρτηση `load_and_prepare_base_data`:

```
df = pd.read_csv(DATA_PATH, parse_dates=[TIME_COL], low_memory=False)
df = df.sort_values([HOME_COL, TIME_COL]).reset_index(drop=True)
```

Το dataset φορτώνεται με το timestamp σε μορφή ημερομηνίας και ώρας και ταξινομείται ανά κατοικία και χρόνο. Η ταξινόμηση αυτή είναι απαραίτητη για τον σωστό υπολογισμό των ιστορικών χαρακτηριστικών.

Στη συνέχεια αφαιρούνται οι προβληματικές κατοικίες και καθαρίζεται η μεταβλητή στόχος:

```
existing_bad = [h for h in BAD_HOMES if h in set(df[HOME_COL].unique())]
```

```

if existing_bad:
    print("Removing bad homes:", existing_bad)
    df = df[~df[HOME_COL].isin(existing_bad)].copy()

df[TARGET_COL] = pd.to_numeric(df[TARGET_COL], errors="coerce")
df = df.dropna(subset=[TARGET_COL]).copy()
df[TARGET_COL] = df[TARGET_COL].clip(lower=0)

```

Με αυτόν τον τρόπο αφαιρούνται γραμμές χωρίς έγκυρη τιμή κατανάλωσης και περιορίζονται τυχόν αρνητικές τιμές στο μηδέν, καθώς δεν έχουν φυσική ερμηνεία στο συγκεκριμένο πρόβλημα.

Εκπαίδευση του with-history generic LightGBM

Η εκπαίδευση του μοντέλου με ιστορικό ξεκινά με τη δημιουργία των lag και rolling χαρακτηριστικών:

```

df_hist = add_lag_and_rolling_features(df_base)
df_hist = df_hist.dropna(subset=REQUIRED_HISTORY_COLS).copy()
df_hist = df_hist.sort_values([HOME_COL, TIME_COL]).reset_index(drop=True)

```

Οι γραμμές που δεν διαθέτουν το απαιτούμενο ιστορικό αφαιρούνται. Στην εκτέλεση του script, μετά τη δημιουργία των ιστορικών χαρακτηριστικών παρέμειναν 1.471.767 γραμμές από 250 κατοικίες.

Ακολουθεί ο χρονικός διαχωρισμός:

```
hist_train, hist_es, hist_cal, hist_test = temporal_split(df_hist)
```

Το τελικό μοντέλο με ιστορικό χρησιμοποιεί 47 αριθμητικά χαρακτηριστικά και 2 κατηγορικά χαρακτηριστικά. Τα κατηγορικά χαρακτηριστικά είναι μόνο το hometype και το urban_rural_class. Δεν χρησιμοποιείται home_id ως χαρακτηριστικό και δεν χρησιμοποιούνται στατιστικά γνωστής κατοικίας. Αυτό είναι βασικό στοιχείο της generic UI εκδοχής.

Η εκπαίδευση του LightGBM γίνεται με παρακολούθηση της απόδοσης στο validation set. Στην εκτέλεση του script, το μοντέλο έφτασε μέχρι τις 9000 επαναλήψεις και δεν ενεργοποιήθηκε early stopping, καθώς η απόδοση συνέχισε να βελτιώνεται μέχρι το τέλος. Η καλύτερη τιμή validation RMSE στη λογαριθμική κλίμακα ήταν 0,528988 στο iteration 9000.

Μετά την εκπαίδευση, παράγονται προβλέψεις για το calibration και το test set. Οι προβλέψεις επανέρχονται από τη λογαριθμική κλίμακα στην αρχική κλίμακα Wh και περιορίζονται ώστε να μην υπάρχουν αρνητικές τιμές.

Εκπαίδευση του no-history optional LightGBM

Το προαιρετικό no-history LightGBM ακολουθεί διαφορετική λογική. Επειδή δεν υπάρχουν πραγματικές προηγούμενες τιμές κατανάλωσης του χρήστη, το script δημιουργεί train-only behavioral profiles και KNN profiles. Αρχικά εφαρμόζεται χρονικός διαχωρισμός:

```
train_df, es_df, cal_df, test_df = temporal_split(df_nohist)
```

Στη συνέχεια δημιουργούνται τα προφίλ συμπεριφοράς:

```
behavior_profiles = build_behavior_profiles(train_df)
```

Η έννοια train-only είναι σημαντική, διότι τα προφίλ συμπεριφοράς υπολογίζονται μόνο από το training set και όχι από τις μελλοντικές περιόδους. Έτσι αποφεύγεται η διαρροή πληροφορίας.

Έπειτα δημιουργείται το KNN bundle:

```
knn_bundle = fit_knn_static_model(train_df, n_neighbors=N_KNN_NEIGHBORS)
```

Το KNN bundle περιλαμβάνει το μοντέλο κοντινότερων γειτόνων, τον scaler, τα στατικά χαρακτηριστικά των κατοικιών και τα προφίλ κατανάλωσης των παρόμοιων κατοικιών. Αυτά τα στοιχεία αποθηκεύονται ώστε να μπορούν να χρησιμοποιηθούν και κατά τη φάση inference.

Το no-history μοντέλο χρησιμοποιεί 26 αριθμητικά και 6 κατηγορικά χαρακτηριστικά. Τα κατηγορικά χαρακτηριστικά είναι:

```
NOHISTORY_CATEGORICAL = [
    "hometype",
    "urban_rural_class",
    "residents_bin",
    "area_bin",
    "appliances_bin",
    "behavior_profile_source",
]
```

Τα πρόσθετα αριθμητικά χαρακτηριστικά περιλαμβάνουν την αναμενόμενη κατανάλωση από τα behavioral profiles, τα αντίστοιχα λογαριθμικά χαρακτηριστικά, τις αναμενόμενες τιμές από τα KNN profiles και τη διαφορά ανάμεσα στο KNN και στο behavioral profile.

Αποθήκευση artifacts

Και για τις δύο εκδοχές του LightGBM αποθηκεύονται τα απαραίτητα αρχεία για χρήση από το API:

```
model_path = WITH_HISTORY_DIR / "model.joblib"
preprocessor_path = WITH_HISTORY_DIR / "preprocessor.pkl"
feature_config_path = WITH_HISTORY_DIR / "feature_config.json"
metadata_path = WITH_HISTORY_DIR / "metadata.json"
predictions_path = WITH_HISTORY_DIR / "test_predictions.csv"
```

Το model.joblib περιέχει το εκπαιδευμένο LightGBM μοντέλο. Το preprocessor.pkl περιέχει τον μηχανισμό προεπεξεργασίας. Το feature_config.json περιγράφει τα χαρακτηριστικά που αναμένει το μοντέλο, ενώ το metadata.json περιέχει πληροφορίες για τις παραμέτρους εκπαίδευσης, τις μετρικές και τις χρονικές περιόδους που χρησιμοποιήθηκαν.

Στο with-history μοντέλο αποθηκεύεται επίσης ο global daily calibrator, ενώ στο no-history μοντέλο αποθηκεύονται τα behavioral profiles και το KNN bundle:

```
joblib.dump(behavior_profiles, no_history_DIR /
    "behavior_profiles.joblib")
joblib.dump(knn_bundle, no_history_DIR / "knn_bundle.joblib")
```

Τα αρχεία αυτά είναι απαραίτητα ώστε το API να μπορεί να δημιουργήσει τα ίδια πρόσθετα χαρακτηριστικά όταν ο χρήστης ζητήσει πρόβλεψη χωρίς ιστορικό κατανάλωσης.

Συγκεντρωτικό αρχείο εκπαίδευσης

Στο τέλος της εκτέλεσης δημιουργείται το αρχείο training_summary.json, το οποίο συγκεντρώνει τις βασικές πληροφορίες του training run:

```
summary = {
    "script": "train_lgbm_api_generic_models.py",
    "data_path": str(DATA_PATH),
    "target": TARGET_COL,
    "time_col": TIME_COL,
    "home_col": HOME_COL,
    "test_days": TEST_DAYS,
    "val_days_total": VAL_DAYS_TOTAL,
    "cal_days": CAL_DAYS,
    "bad_homes_removed": existing_bad,
    "use_log_target": USE_LOG_TARGET,
    "clip_target_train": CLIP_TARGET_TRAIN,
    "output_root": str(OUT_ROOT),
}
```

Το αρχείο αυτό περιλαμβάνει πληροφορίες για το dataset, τις χρονικές περιόδους, τα μοντέλα που εκπαιδεύτηκαν, τις κατοικίες που αφαιρέθηκαν, τις ρυθμίσεις του target και τις διαδρομές των αποθηκευμένων αρχείων.

Τεχνική ανάλυση του API και του Gradio UI

Το δεύτερο μεγάλο τεχνικό κομμάτι της εργασίας ήταν η μετατροπή των εκπαιδευμένων μοντέλων σε λειτουργική εφαρμογή πρόβλεψης. Ο στόχος δεν ήταν απλώς να αποθηκευτούν τα μοντέλα στον δίσκο, αλλά να δημιουργηθεί ένα ολοκληρωμένο σύστημα στο οποίο ο χρήστης μπορεί να δίνει στοιχεία εισόδου, το backend να επιλέγει το κατάλληλο μοντέλο, να δημιουργεί τα απαιτούμενα χαρακτηριστικά και να επιστρέφει την πρόβλεψη σε εύχρηστη και ερμηνεύσιμη μορφή. Η υλοποίηση αυτή βασίστηκε σε έναν συνδυασμό βοηθητικών modules, εφαρμογής FastAPI και γραφικής διεπαφής Gradio.

Η τελική αρχιτεκτονική της εφαρμογής οργανώθηκε γύρω από δύο βασικά σενάρια πρόβλεψης. Το πρώτο είναι το `no_history`, το οποίο αφορά περιπτώσεις όπου ο χρήστης δεν διαθέτει πρόσφατο ιστορικό κατανάλωσης. Σε αυτό το σενάριο, η πρόβλεψη βασίζεται αποκλειστικά σε στατικά χαρακτηριστικά της κατοικίας, χρονικά χαρακτηριστικά και μετεωρολογικές πληροφορίες. Το δεύτερο είναι το `with_history`, το οποίο αφορά περιπτώσεις όπου ο χρήστης διαθέτει πρόσφατες ωριαίες μετρήσεις κατανάλωσης. Σε αυτή την περίπτωση, η εφαρμογή μπορεί να αξιοποιήσει το ιστορικό φορτίου για να δημιουργήσει lag και rolling χαρακτηριστικά και να προσαρμόσει την πρόβλεψη στη συμπεριφορά της συγκεκριμένης κατοικίας.

Για λόγους συμβατότητας με παλαιότερες εκδόσεις, το σύστημα εξακολουθεί να αναγνωρίζει ορισμένες παλιές ονομασίες, όπως `coldstart`, `cold_start` και `withhistory`. Ωστόσο, η τελική ορολογία της εφαρμογής βασίζεται στα δύο καθαρά σενάρια `no_history` και `with_history`. Αυτή η διάκριση κάνει πιο σαφή τη λειτουργία του συστήματος, καθώς δηλώνει άμεσα αν η πρόβλεψη βασίζεται ή όχι σε διαθέσιμο ιστορικό κατανάλωσης.

Ένα σημαντικό βοηθητικό κομμάτι της αρχιτεκτονικής είναι το `history_store.py`, το οποίο διαχειρίζεται την ανάκτηση ιστορικών δεδομένων κατανάλωσης από αρχείο CSV. Στην τελική μορφή της εφαρμογής, το ιστορικό αρχείο θεωρείται ότι αφορά μία μόνο κατοικία και δεν απαιτείται να περιέχει `home_id`. Οι βασικές στήλες που απαιτούνται είναι η χρονική στήλη `timestamp` και η στήλη κατανάλωσης `consumption_Wh`. Με αυτόν τον τρόπο, ο χρήστης μπορεί να επιλέξει ή να ανεβάσει ένα απλό αρχείο ιστορικού, χωρίς να χρειάζεται να γνωρίζει την εσωτερική δομή του αρχικού dataset.

Η βασική λειτουργία του `history_store.py` είναι να επιστρέφει ένα συνεχόμενο παράθυρο προηγούμενων ωριαίων τιμών πριν από την ημερομηνία στόχου. Από προεπιλογή, το παράθυρο αυτό έχει διάρκεια 168 ωρών, δηλαδή μίας εβδομάδας. Η επιλογή αυτή είναι σημαντική επειδή τα generic `with_history` μοντέλα απαιτούν πρόσφατο ιστορικό ώστε να μπορούν να δημιουργήσουν χαρακτηριστικά όπως προηγούμενες τιμές κατανάλωσης, κυλιόμενους μέσους όρους και εβδομαδιαία μοτίβα.

Όταν ζητείται πρόβλεψη για συγκεκριμένη ημερομηνία, το σύστημα προσπαθεί αρχικά να εντοπίσει το ακριβές ιστορικό παράθυρο που προηγείται της ημέρας στόχου. Αν το παράθυρο αυτό δεν είναι πλήρες και έχει ενεργοποιηθεί η εναλλακτική λειτουργία, μπορεί να χρησιμοποιηθεί το πιο πρόσφατο διαθέσιμο συνεχόμενο τμήμα ιστορικού από το αρχείο. Με αυτόν τον τρόπο, η εφαρμογή μπορεί να λειτουργήσει ακόμη και όταν το ιστορικό του χρήστη δεν είναι απόλυτα ευθυγραμμισμένο με την ημερομηνία πρόβλεψης, αρκεί να υπάρχει επαρκές συνεχόμενο ιστορικό.

Σε παλαιότερες εκδόσεις της εφαρμογής, το `history_store.csv` είχε χρησιμοποιηθεί ως σταθερή πηγή ιστορικού για λόγους επίδειξης. Στην τελική όμως λογική του συστήματος, η βασική προσέγγιση είναι πιο γενική: ο χρήστης μπορεί να επιλέξει διαφορετικό αρχείο ιστορικού για διαφορετική κατοικία. Το προεπιλεγμένο `history_store.csv` διατηρείται κυρίως για λόγους συμβατότητας ή εναλλακτικής λειτουργίας, αλλά η κύρια τελική χρήση βασίζεται σε επιλεγόμενο CSV ιστορικού μίας κατοικίας.

Ένα ακόμη βασικό στοιχείο του συστήματος είναι το `model_registry.py`, το οποίο λειτουργεί ως κεντρικό μητρώο των διαθέσιμων μοντέλων. Αντί το API ή η διεπαφή να περιέχουν διάσπαρτες διαδρομές αρχείων και κανόνες επιλογής μοντέλων, όλες οι σχετικές

πληροφορίες συγκεντρώνονται στο registry. Για κάθε μοντέλο καταγράφεται το αναγνωριστικό του, η φιλική ονομασία, η οικογένεια αλγορίθμου, ο φάκελος των artifacts, τα υποστηριζόμενα σενάρια πρόβλεψης και το αν το μοντέλο είναι προεπιλεγμένο ή προαιρετικό.

Το `model_registry.py` είναι ιδιαίτερα σημαντικό επειδή προστατεύει την εφαρμογή από λανθασμένες επιλογές μοντέλων. Στη γενική έκδοση του UI, ο χρήστης δεν παρέχει `home_id`, επομένως δεν πρέπει να επιλέγονται μοντέλα που απαιτούν γνωστή κατοικία του αρχικού dataset. Το registry οργανώνει τα διαθέσιμα μοντέλα με βάση αυτόν τον περιορισμό και επιτρέπει στο API να επιλέγει μόνο μοντέλα που είναι συμβατά με το αντίστοιχο σενάριο πρόβλεψης.

Στο τελικό σύστημα, το προεπιλεγμένο μοντέλο για το `no_history` σενάριο είναι το Random Forest, καθώς αποτελεί σταθερή και πιο απλή επιλογή για περιπτώσεις χωρίς ιστορικό κατανάλωσης. Για το `with_history` σενάριο, το προεπιλεγμένο μοντέλο είναι το LightGBM generic with-history, το οποίο αξιοποιεί πρόσφατες τιμές κατανάλωσης χωρίς να απαιτεί `home_id`. Παράλληλα, το registry επιτρέπει την ύπαρξη προαιρετικών μοντέλων σύγκρισης, όπως XGBoost και Random Forest για το σενάριο με ιστορικό, καθώς και LightGBM ή XGBoost για το σενάριο χωρίς ιστορικό.

Το επόμενο βασικό επίπεδο είναι το `inference.py`, το οποίο αποτελεί τον πυρήνα της διαδικασίας πρόβλεψης. Το αρχείο αυτό αναλαμβάνει τη φόρτωση των αποθηκευμένων μοντέλων και των συνοδευτικών αρχείων τους, τη δημιουργία των κατάλληλων χαρακτηριστικών εισόδου και την παραγωγή της τελικής 24ωρης πρόβλεψης. Με άλλα λόγια, το `inference.py` είναι το σημείο όπου τα εκπαιδευμένα μοντέλα μετατρέπονται σε λειτουργική υπηρεσία πρόβλεψης.

Κατά τη φάση inference, το σύστημα φορτώνει όχι μόνο το εκπαιδευμένο μοντέλο, αλλά και τον αντίστοιχο μηχανισμό προεπεξεργασίας, το αρχείο ρυθμίσεων χαρακτηριστικών και τα μεταδεδομένα. Αυτό είναι κρίσιμο, επειδή διασφαλίζει ότι τα δεδομένα που δίνει ο χρήστης μετατρέπονται με τον ίδιο τρόπο με τον οποίο είχαν μετατραπεί και τα δεδομένα κατά την εκπαίδευση. Έτσι αποφεύγονται ασυνέπειες ανάμεσα στο training και στο serving. Στο `no_history` σενάριο, η διαδικασία πρόβλεψης είναι σχετικά απλή. Για κάθε μία από τις 24 ώρες της ημέρας στόχου δημιουργείται μια γραμμή χαρακτηριστικών, η οποία περιλαμβάνει τα στατικά χαρακτηριστικά της κατοικίας, τα χρονικά χαρακτηριστικά και τις θερμοκρασιακές τιμές. Οι γραμμές αυτές περνούν από τον αποθηκευμένο preprocessor και στη συνέχεια από το επιλεγμένο μοντέλο, ώστε να παραχθούν οι 24 ωριαίες προβλέψεις κατανάλωσης.

Στο `with_history` σενάριο, η διαδικασία είναι πιο σύνθετη, επειδή απαιτείται πρόσφατο ιστορικό κατανάλωσης. Το ιστορικό μπορεί να δοθεί είτε ως λίστα ωριαίων τιμών είτε ως αρχείο CSV είτε ως dataframe που έχει ήδη προετοιμαστεί από το API. Το `inference.py` ελέγχει αν υπάρχει επαρκές ιστορικό, το μετατρέπει σε κανονική ωριαία μορφή και δημιουργεί τα απαραίτητα ιστορικά χαρακτηριστικά. Σε αυτά περιλαμβάνονται lag features, rolling means, rolling standard deviations και πρόσθετες μεταβλητές που συγκρίνουν πρόσφατη και παλαιότερη συμπεριφορά.

Η πρόβλεψη στο `with_history` σενάριο γίνεται αναδρομικά για τις 24 ώρες της ημέρας στόχου. Για την πρώτη ώρα χρησιμοποιείται το πραγματικό διαθέσιμο ιστορικό. Μόλις προβλεφθεί η πρώτη ώρα, η πρόβλεψη προστίθεται στο ιστορικό και χρησιμοποιείται για τον υπολογισμό των χαρακτηριστικών της επόμενης ώρας. Η διαδικασία αυτή επαναλαμβάνεται μέχρι να δημιουργηθούν και οι 24 ωριαίες προβλέψεις. Με αυτόν τον τρόπο, το σύστημα μπορεί να παράγει day-ahead πρόβλεψη ακόμη και όταν οι μελλοντικές πραγματικές τιμές δεν είναι διαθέσιμες.

Μία σημαντική λειτουργία της τελικής έκδοσης του `inference.py` είναι η προαιρετική adaptive history-aware correction. Η λειτουργία αυτή εφαρμόζεται μόνο στο `with_history`

σενάριο και μόνο μετά την αρχική πρόβλεψη του μοντέλου. Η βασική ιδέα είναι ότι το πρόσφατο ιστορικό της κατοικίας μπορεί να δώσει χρήσιμη πληροφορία για το ωριαίο σχήμα της κατανάλωσης. Για τον λόγο αυτό, το σύστημα δημιουργεί ένα ωριαίο προφίλ από τις πρόσφατες ημέρες ιστορικού και το χρησιμοποιεί για να εφαρμόσει ήπια προσαρμογή στο σχήμα της πρόβλεψης.

Η προσαρμογή αυτή δεν αντικαθιστά το μοντέλο και δεν αποτελεί νέα εκπαίδευση. Πρόκειται για ελεγχόμενο βήμα μετα-επεξεργασίας, το οποίο περιορίζεται από μέγιστο συντελεστή διόρθωσης. Έτσι, το μοντέλο παραμένει η κύρια πηγή της πρόβλεψης, ενώ το πρόσφατο ιστορικό χρησιμοποιείται για να βελτιώσει προσεκτικά την κατανομή της κατανάλωσης μέσα στο 24ωρο. Η λειτουργία αυτή είναι ιδιαίτερα χρήσιμη όταν το πρόσφατο ιστορικό είναι επαρκές και σχετικά σταθερό.

Το κεντρικό σημείο ενοποίησης της αρχιτεκτονικής είναι το `api_app.py`, δηλαδή η εφαρμογή FastAPI. Το API συνδέει το `model registry`, το `inference layer`, τη διαχείριση ιστορικού, τη διαχείριση θερμοκρασιών και τη γραφική διεπαφή. Τα βασικά endpoints της εφαρμογής είναι το `/health`, το `/models`, το `/history/info` και το `/predict`. Το endpoint `/predict` είναι το κύριο σημείο παραγωγής πρόβλεψης.

Μέσω του `/predict`, ο χρήστης ή η γραφική διεπαφή μπορεί να στείλει τα απαιτούμενα στοιχεία για την πρόβλεψη. Αυτά περιλαμβάνουν το σενάριο πρόβλεψης, το μοντέλο ή την επιλογή `auto`, την ημερομηνία στόχου, την πόλη, τα χαρακτηριστικά της κατοικίας, τις θερμοκρασίες και, εφόσον πρόκειται για `with_history` πρόβλεψη, το διαθέσιμο ιστορικό κατανάλωσης. Το API ελέγχει τις εισόδους, επιλέγει το σωστό μοντέλο, φορτώνει τα απαραίτητα artifacts και καλεί την κατάλληλη διαδικασία πρόβλεψης από το `inference.py`. Η επιλογή μοντέλου μπορεί να γίνει είτε ρητά είτε αυτόματα. Αν ο χρήστης επιλέξει `auto`, τότε το API αξιοποιεί το `model_registry.py` για να επιλέξει το προεπιλεγμένο μοντέλο του αντίστοιχου σεναρίου. Έτσι, για το `no_history` σενάριο επιλέγεται το προεπιλεγμένο Random Forest, ενώ για το `with_history` σενάριο επιλέγεται το προεπιλεγμένο LightGBM. Παράλληλα, αν ο χρήστης επιλέξει προαιρετικό μοντέλο, το API ελέγχει αν αυτό είναι συμβατό με το επιλεγμένο `mode`.

Η διαχείριση των θερμοκρασιακών δεδομένων γίνεται επίσης στο επίπεδο του API. Αν ο χρήστης δώσει πλήρες ωριαίο προφίλ εξωτερικής θερμοκρασίας 24 τιμών, το API το χρησιμοποιεί απευθείας. Αν δεν δοθεί τέτοιο προφίλ, το σύστημα μπορεί να επιχειρήσει ανάκτηση πρόγνωσης από την υπηρεσία Open-Meteo, με βάση την πόλη και την ημερομηνία στόχου. Αν αυτό δεν είναι διαθέσιμο, μπορεί να δημιουργήσει ένα απλό συνθετικό ωριαίο θερμοκρασιακό προφίλ με βάση την ελάχιστη και τη μέγιστη θερμοκρασία που παρέχει ο χρήστης. Με αυτόν τον τρόπο, η εφαρμογή μπορεί να λειτουργεί και με αυτόματα και με χειροκίνητη παροχή θερμοκρασιακών δεδομένων.

Στο `with_history` σενάριο, το API ακολουθεί συγκεκριμένη σειρά προτεραιότητας για την πηγή ιστορικού. Αν ο χρήστης δώσει άμεσα λίστα ωριαίων τιμών, αυτή χρησιμοποιείται πρώτη. Αν δεν δοθεί λίστα αλλά υπάρχει διαδρομή προς επιλεγμένο CSV ιστορικού, τότε χρησιμοποιείται το συγκεκριμένο αρχείο. Αν δεν δοθεί τίποτα από τα δύο και είναι ενεργοποιημένη η σχετική επιλογή, μπορεί να χρησιμοποιηθεί το προεπιλεγμένο `legacy history store`. Αυτή η σειρά προτεραιότητας επιτρέπει στο σύστημα να υποστηρίζει τόσο χειροκίνητα δεδομένα όσο και επιλεγμένα αρχεία ιστορικού.

Το endpoint `/history/info` παρέχει πληροφορίες για τη μορφή του ιστορικού που μπορεί να χρησιμοποιηθεί από την εφαρμογή. Είναι χρήσιμο τόσο για τον έλεγχο της κατάστασης του προεπιλεγμένου `history store` όσο και για την ενημέρωση του χρήστη σχετικά με το αναμενόμενο σχήμα του CSV. Στην τελική λογική της εφαρμογής, το ιστορικό αρχείο πρέπει να περιέχει τουλάχιστον τις στήλες `timestamp` και `consumption_Wh`.

Η απόκριση του API δεν περιορίζεται στις 24 προβλεπόμενες τιμές κατανάλωσης. Περιλαμβάνει επίσης πληροφορίες για το μοντέλο που χρησιμοποιήθηκε, το `mode`, την

ημερομηνία στόχου, τη συνολική ημερήσια πρόβλεψη σε kWh, την πηγή των θερμοκρασιακών δεδομένων, την πηγή του ιστορικού και τα βασικά flags της ημέρας, όπως αν είναι Σαββατοκύριακο ή αργία. Όταν εφαρμόζεται προσαρμογή με βάση το ιστορικό, επιστρέφονται και επιπλέον διαγνωστικές πληροφορίες σχετικά με τη διόρθωση.

Στο ανώτερο επίπεδο της αρχιτεκτονικής βρίσκεται το `ui_app.py`, το οποίο υλοποιεί τη γραφική διεπαφή χρήστη με τη βιβλιοθήκη Gradio. Η διεπαφή λειτουργεί ως επίπεδο παρουσίασης. Δεν εκπαιδεύει μοντέλα και δεν εκτελεί απευθείας τη βασική λογική πρόβλεψης. Αντίθετα, συλλέγει τα στοιχεία που εισάγει ο χρήστης, τα οργανώνει σε κατάλληλο αίτημα και τα αποστέλλει στο FastAPI backend.

Η διεπαφή επιτρέπει στον χρήστη να επιλέξει σενάριο πρόβλεψης, δηλαδή `no_history` ή `with_history`, να επιλέξει μοντέλο ή αυτόματη επιλογή, να εισαγάγει τα χαρακτηριστικά της κατοικίας, να ορίσει ημερομηνία στόχου και να δώσει θερμοκρασιακά δεδομένα. Για το `with_history` σενάριο, παρέχει τη δυνατότητα επιλογής ή ανεβάσματος αρχείου CSV ιστορικού. Το επιλεγμένο αρχείο αντιγράφεται σε σταθερό φάκελο του έργου, ώστε να μπορεί να χρησιμοποιηθεί με αξιόπιστο τρόπο από το API.

Η δυνατότητα επιλογής αρχείου ιστορικού είναι σημαντική, επειδή επιτρέπει στον χρήστη να δοκιμάσει διαφορετικές κατοικίες ή διαφορετικά ιστορικά παράθυρα χωρίς να αλλάξει τον κώδικα. Το UI δεν απαιτεί από τον χρήστη να δώσει `home_id`, αλλά μόνο ένα αρχείο με χρονικές σημάνσεις και τιμές κατανάλωσης. Έτσι, η εφαρμογή παραμένει πιο κοντά σε ένα ρεαλιστικό σενάριο χρήσης από νέο χρήστη.

Η γραφική διεπαφή περιλαμβάνει επίσης λειτουργία ανάλυσης ιστορικού. Μέσα από αυτήν μπορεί να ελεγχθεί η κάλυψη του διαθέσιμου ιστορικού, η μέση ημερήσια κατανάλωση, η σταθερότητα του ημερήσιου προφίλ και άλλα στοιχεία που βοηθούν στην ερμηνεία της πρόβλεψης. Η ανάλυση αυτή είναι χρήσιμη ιδιαίτερα για το `with_history` σενάριο, επειδή η ποιότητα και η σταθερότητα του ιστορικού επηρεάζουν την αξιοπιστία της τελικής πρόβλεψης.

Στο UI υπάρχει επίσης δυνατότητα ελέγχου ή ρύθμισης της `adaptive history-aware correction`. Ο χρήστης μπορεί να αφήσει την προεπιλεγμένη ή προτεινόμενη τιμή του μέγιστου συντελεστή διόρθωσης ή να τη μεταβάλει από τις προχωρημένες ρυθμίσεις. Με αυτόν τον τρόπο, η εφαρμογή διατηρεί την κύρια πρόβλεψη του μοντέλου, αλλά επιτρέπει ελεγχόμενη προσαρμογή όταν υπάρχει επαρκές πρόσφατο ιστορικό.

Μία ακόμη λειτουργία της διεπαφής είναι η σύγκριση μοντέλων. Σε αυτή την περίπτωση, το UI καλεί το API για περισσότερα από ένα μοντέλα και παρουσιάζει τις αντίστοιχες προβλέψεις σε συγκριτικό πίνακα και γράφημα. Έτσι ο χρήστης μπορεί να δει πώς διαφοροποιούνται οι προβλέψεις μεταξύ Random Forest, XGBoost και LightGBM, ανάλογα με το επιλεγμένο σενάριο πρόβλεψης.

Επιπλέον, η διεπαφή υποστηρίζει συνδυαστική πρόβλεψη μέσω σταθμισμένου μέσου όρου. Η λειτουργία αυτή δεν αποτελεί νέο εκπαιδευμένο μοντέλο. Αντίθετα, αξιοποιεί τις 24ωρες προβλέψεις των επιμέρους μοντέλων και υπολογίζει μια τελική πρόβλεψη με βάση προκαθορισμένα ή προσαρμοσμένα βάρη. Έτσι η συνδυαστική πρόβλεψη λειτουργεί ως επίπεδο `post-processing` πάνω στις εξόδους των μοντέλων.

Η γραφική διεπαφή περιλαμβάνει επίσης δυνατότητα συμπεριφορικής προσαρμογής της πρόβλεψης. Ο χρήστης μπορεί να δηλώσει συγκεκριμένες ώρες ή χρονικά διαστήματα στα οποία αναμένει διαφορετική κατανάλωση και να εφαρμόσει έναν συντελεστή μεταβολής. Η λειτουργία αυτή υλοποιείται αποκλειστικά ως `post-processing` στη διεπαφή και δεν επηρεάζει τα εκπαιδευμένα μοντέλα. Αποτελεί διαδραστικό εργαλείο σεναριακής ανάλυσης, όταν ο χρήστης γνωρίζει ότι η ημέρα στόχος θα έχει ιδιαίτερη συμπεριφορά.

Το UI εμφανίζει τα αποτελέσματα σε μορφή πίνακα, σύνοψης και γραφήματος. Ο χρήστης μπορεί να δει τις 24 ωριαίες προβλέψεις, τη συνολική ημερήσια κατανάλωση, την πηγή των δεδομένων θερμοκρασίας, την πηγή του ιστορικού και τις βασικές πληροφορίες του

μοντέλου που χρησιμοποιήθηκε. Επιπλέον, υπάρχει δυνατότητα αποθήκευσης των αποτελεσμάτων σε CSV, ώστε οι προβλέψεις να μπορούν να χρησιμοποιηθούν για μελλοντική τεκμηρίωση ή σύγκριση.

Ιδιαίτερη πρόβλεψη υπάρχει και για παρελθοντικές ημερομηνίες. Η εφαρμογή επιτρέπει την επιλογή ημερομηνίας που βρίσκεται στο παρελθόν, ώστε να μπορεί να χρησιμοποιηθεί για backtesting και αξιολόγηση. Σε αυτή την περίπτωση, η διεπαφή πρέπει να καθιστά σαφές ότι δεν πρόκειται για πραγματική μελλοντική πρόβλεψη, αλλά για ιστορική δοκιμή της συμπεριφοράς του συστήματος. Ειδικά στο with_history σενάριο, το επιλεγμένο ιστορικό πρέπει να προηγείται χρονικά της ημερομηνίας στόχου.

Συνολικά, η τεχνική αρχιτεκτονική του API και του Gradio UI μετατρέπει τα εκπαιδευμένα μοντέλα σε λειτουργική εφαρμογή πρόβλεψης. Τα training scripts παράγουν τα απαραίτητα artifacts, το model_registry.py οργανώνει και ελέγχει τα διαθέσιμα μοντέλα, το inference.py αναλαμβάνει τη δημιουργία χαρακτηριστικών και την παραγωγή των προβλέψεων, το api_app.py ενοποιεί τις λειτουργίες σε ένα backend και το ui_app.py προσφέρει ένα διαδραστικό περιβάλλον χρήστη. Με αυτόν τον τρόπο, η εργασία δεν περιορίζεται στην αξιολόγηση μοντέλων, αλλά καταλήγει σε ένα ολοκληρωμένο σύστημα πρόβλεψης που μπορεί να λειτουργήσει τόσο χωρίς ιστορικό κατανάλωσης όσο και με πρόσφατες ωριαίες μετρήσεις.

Υλοποίηση API- UI

Δημιουργία History Store για το API και το UI. Αρχείο: home100_as_history.ipynb

Το παρόν script χρησιμοποιείται για τη δημιουργία ενός βοηθητικού αρχείου ιστορικών μετρήσεων κατανάλωσης (history_store.csv), το οποίο αξιοποιείται από το API και το UI ως σταθερή πηγή ιστορικού φορτίου. Ο βασικός στόχος του είναι να επιλέξει ένα σπίτι από το IDEAL dataset με επαρκή χρονική κάλυψη, να εξαγάγει τις ωριαίες μετρήσεις ενός πλήρους έτους και να τις αντιστοιχίσει σε ένα μελλοντικό έτος-στόχο, ώστε να υπάρχει διαθέσιμο, συνεπές και πλήρες ιστορικό για τις λειτουργίες της εφαρμογής.

Στην αρχή του script ορίζονται τα βασικά paths και οι παράμετροι λειτουργίας:

```
BASE_DIR = Path("C:/IDEAL_Programming")
DATA_PATH = BASE_DIR / "processed" / "final" / "IDEAL_final_hourly_dataset.csv"
OUT_PATH = BASE_DIR / "processed" / "stores" / "history_store.csv"
```

Εδώ το script διαβάζει το τελικό ενοποιημένο dataset και δημιουργεί, αν δεν υπάρχει ήδη, τον φάκελο όπου θα αποθηκευτεί το history store. Στη συνέχεια ορίζονται τρεις κρίσιμες παράμετροι:

```
HOME_ID = "home100"
SOURCE_YEAR = 2017
TARGET_YEAR = 2026
```

Η μεταβλητή HOME_ID δηλώνει ποια κατοικία θα χρησιμοποιηθεί ως πηγή ιστορικών δεδομένων, ενώ οι SOURCE_YEAR και TARGET_YEAR καθορίζουν αντίστοιχα το έτος από το οποίο αντλούνται οι πραγματικές μετρήσεις και το έτος στο οποίο θα μεταφερθούν χρονικά. Με άλλα λόγια, το script παίρνει το ιστορικό φορτίο ενός πραγματικού σπιτιού από το 2017 και το “χαρτογραφεί” στο 2026, διατηρώντας τον ίδιο μήνα, ημέρα και ώρα.

Στη συνέχεια διαβάζονται μόνο οι απολύτως απαραίτητες στήλες:

```
usecols = ["home_id", "timestamp", "consumption_Wh"]  
df = pd.read_csv(DATA_PATH, usecols=usecols, parse_dates=["timestamp"],  
low_memory=False)
```

Αυτό είναι σημαντικό γιατί το script δεν χρειάζεται όλα τα χαρακτηριστικά του dataset, αλλά μόνο:

- το αναγνωριστικό κατοικίας,
- τη χρονική σήμανση,
- και την κατανάλωση.

Αμέσως μετά αφαιρούνται οι γραμμές με ελλείψεις:

```
df = df.dropna(subset=["home_id", "timestamp", "consumption_Wh"]).copy()  
df["home_id"] = df["home_id"].astype(str)
```

Με τον τρόπο αυτό διασφαλίζεται ότι το input του history store είναι καθαρό και ότι το home_id έχει ενιαία μορφή string, κάτι χρήσιμο για ασφαλές filtering.

Το επόμενο βήμα είναι η επιλογή των εγγραφών που αντιστοιχούν στο επιλεγμένο σπίτι και στο συγκεκριμένο έτος πηγής:

```
src = df[(df["home_id"] == HOME_ID) & (df["timestamp"].dt.year ==  
SOURCE_YEAR)].copy()  
src = src.sort_values("timestamp")
```

Εδώ το script περιορίζεται αποκλειστικά στο home100 και στο έτος 2017, ενώ ταυτόχρονα ταξινομεί χρονικά τις εγγραφές. Στόχος είναι να δημιουργηθεί μια καθαρή, μονοετής, ωριαία χρονοσειρά κατανάλωσης.

Στη συνέχεια εφαρμόζεται επιπλέον φίλτρο ώστε να ληφθούν μόνο οι χρονικές στιγμές που ανήκουν ακριβώς στο ημερολογιακό διάστημα του έτους:

```
start = pd.Timestamp(f"{SOURCE_YEAR}-01-01 00:00:00")  
end = pd.Timestamp(f"{SOURCE_YEAR+1}-01-01 00:00:00")  
src = src[(src["timestamp"] >= start) & (src["timestamp"] < end)].copy()
```

Το βήμα αυτό είναι σημαντικό, γιατί εξασφαλίζει ότι το script δουλεύει πάνω στο πλήρες διάστημα από 1 Ιανουαρίου έως 31 Δεκεμβρίου, χωρίς να περιλαμβάνονται εγγραφές εκτός του επιθυμητού ορίου.

Έπειτα γίνεται ένα πολύ ουσιαστικό βήμα: η ανακατασκευή πλήρους ωριαίου index για όλο το έτος.

```
full_idx = pd.date_range(start=start, end=end, freq="h", inclusive="left")  
src = src.set_index("timestamp").reindex(full_idx)  
src.index.name = "timestamp"
```

Εδώ δημιουργείται ένα πλήρες ωριαίο ημερολόγιο για ολόκληρο το έτος, δηλαδή μία εγγραφή για κάθε ώρα. Αν από τα πραγματικά δεδομένα λείπουν ώρες, το reindex εισάγει κενές γραμμές στα αντίστοιχα σημεία. Αυτή η επιλογή είναι ιδιαίτερα σημαντική για το API και το UI, γιατί τους επιτρέπει να βασίζονται σε ένα συνεπές και πλήρες ωριαίο ιστορικό, χωρίς κενά στη χρονοσειρά.

Στη συνέχεια η κατανάλωση μετατρέπεται αριθμητικά και τα κενά που προέκυψαν συμπληρώνονται:

```
src["consumption_Wh"] = pd.to_numeric(src["consumption_Wh"],
errors="coerce")

src["consumption_Wh"] = src["consumption_Wh"].ffill().bfill()
```

Το `ffill()` κάνει forward fill, δηλαδή συμπληρώνει μια ελλείπουσα τιμή με την προηγούμενη διαθέσιμη, ενώ το `bfill()` χρησιμοποιείται στη συνέχεια για να καλύψει τυχόν κενά στην αρχή του έτους όπου δεν υπάρχει προηγούμενη τιμή. Με αυτόν τον τρόπο το script παράγει μια πλήρως συμπληρωμένη ωριαία σειρά 8760 ωρών, κάτι που είναι πολύ χρήσιμο σε demo εφαρμογές ή σε API που απαιτούν συνεχή ιστορικά δεδομένα για τον υπολογισμό lag features.

Το πιο ιδιαίτερο βήμα του script είναι ο χρονικός αναχρονισμός των timestamps:

```
ts = src.index

mapped = [pd.Timestamp(year=TARGET_YEAR, month=t.month, day=t.day,
hour=t.hour) for t in ts]
```

Εδώ κάθε χρονική στιγμή του έτους πηγής μεταφέρεται στο έτος-στόχο, κρατώντας σταθερά τα στοιχεία μήνα, ημέρας και ώρας. Έτσι, για παράδειγμα, η μέτρηση της 15ης Μαρτίου 2017 στις 14:00 μετατρέπεται σε 15 Μαρτίου 2026 στις 14:00. Η λογική αυτή δεν παράγει νέες μετρήσεις, αλλά επαναχρησιμοποιεί ένα υπαρκτό, πραγματικό προφίλ κατανάλωσης σε ένα διαφορετικό ημερολογιακό πλαίσιο, κατάλληλο για τις ανάγκες του συστήματος.

Στη συνέχεια δημιουργείται το τελικό dataframe εξόδου:

```
out = pd.DataFrame({
    "timestamp": mapped,
    "consumption_Wh": src["consumption_Wh"].astype(float).values
}).sort_values("timestamp")
```

Το αποτέλεσμα περιέχει δύο μόνο στήλες:

- timestamp
- consumption_Wh

και είναι ταξινομημένο χρονικά. Ακολουθεί ένας τελικός έλεγχος ώστε να διατηρηθούν μόνο οι εγγραφές που ανήκουν πράγματι στο 2026:

```
out = out[(out["timestamp"] >= pd.Timestamp("2026-01-01")) &
(out["timestamp"] < pd.Timestamp("2027-01-01"))]
```

Το βήμα αυτό λειτουργεί ως sanity check και εξασφαλίζει ότι το τελικό store δεν περιέχει timestamps εκτός του έτους-στόχου.

Τέλος, το dataframe αποθηκεύεται στο αρχείο:

```
out.to_csv(OUT_PATH, index=False, encoding="utf-8")
```

Το παραγόμενο `history_store.csv` αποτελεί ένα βοηθητικό αποθετήριο ιστορικής κατανάλωσης, το οποίο μπορεί να χρησιμοποιηθεί από το API και το UI για την υποστήριξη σεναρίων όπου απαιτείται διαθέσιμο ιστορικό φορτίου. Στην πράξη, το script επιτρέπει στο σύστημα να λειτουργεί πάνω σε ένα πλήρες και καθαρό ωριαίο ιστορικό, χωρίς να εξαρτάται κάθε φορά από δυναμική αναζήτηση ή περίπλοκη αναδόμηση δεδομένων.

Συνολικά, το συγκεκριμένο script αποτελεί κρίσιμο βοηθητικό κομμάτι του application layer, καθώς γεφυρώνει το ερευνητικό dataset με τις λειτουργικές ανάγκες της εφαρμογής. Ενώ τα training scripts παράγουν τα μοντέλα, το history store script προετοιμάζει την

ιστορική πληροφορία που χρειάζονται το API και το UI για να υποστηρίξουν ρεαλιστικές προβλέψεις και συνεκτική επίδειξη του συστήματος.

Υλοποίηση History Store για το API. Αρχείο: history_store.py

Το αρχείο history_store.py υλοποιεί έναν απλό αλλά κρίσιμο μηχανισμό διαχείρισης ιστορικών δεδομένων κατανάλωσης για το API και τη γραφική διεπαφή της εφαρμογής. Ο ρόλος του είναι να παρέχει στο σύστημα τις προηγούμενες ωριαίες τιμές κατανάλωσης που απαιτούνται από τα μοντέλα with_history. Σε αντίθεση με παλαιότερες λογικές που βασίζονταν σε συγκεκριμένες κατοικίες του dataset, το συγκεκριμένο module έχει σχεδιαστεί ώστε να λειτουργεί με ένα απλό αρχείο CSV που αφορά μία μόνο κατοικία και δεν απαιτεί home_id.

Η βασική ιδέα είναι ότι τα generic with_history μοντέλα χρειάζονται ένα πρόσφατο ιστορικό ωριαίων τιμών κατανάλωσης, συνήθως 168 ωρών, δηλαδή μίας εβδομάδας. Το history_store.py αναλαμβάνει να εντοπίσει και να επιστρέψει αυτό το ιστορικό παράθυρο πριν από την ημερομηνία στόχου. Αν το ακριβές χρονικό παράθυρο δεν είναι διαθέσιμο ή δεν είναι πλήρες, μπορεί προαιρετικά να επιστρέψει το πιο πρόσφατο συνεχόμενο ιστορικό παράθυρο που υπάρχει στο CSV. Με αυτόν τον τρόπο, η εφαρμογή γίνεται πιο ανθεκτική σε ελλείψεις ή μικρές ασυνέχειες των δεδομένων.

Βασικός σκοπός του script

Στην αρχή του αρχείου περιγράφεται καθαρά ο σκοπός του module. Το αρχείο λειτουργεί ως CSV-only history store για το IDEAL API/UI. Το προεπιλεγμένο αρχείο ιστορικού βρίσκεται στη διαδρομή:

C:/IDEAL_Programming/processed/stores/history_store.csv

Το ελάχιστο αναμενόμενο σχήμα του CSV περιλαμβάνει δύο στήλες:

timestamp
consumption_Wh

Η στήλη timestamp πρέπει να περιέχει χρονικές σημάνσεις που μπορούν να μετατραπούν σε datetime, ενώ η στήλη consumption_Wh πρέπει να περιέχει την ωριαία κατανάλωση σε Wh. Η απλότητα αυτής της δομής είναι σημαντική, επειδή επιτρέπει στον χρήστη ή στο UI να χρησιμοποιεί αρχεία ιστορικού χωρίς να χρειάζεται να ακολουθεί την πλήρη δομή του αρχικού IDEAL dataset.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import os
from pathlib import Path
from typing import Dict, List, Tuple

import pandas as pd
```

Η βιβλιοθήκη os χρησιμοποιείται για την ανάγνωση μεταβλητών περιβάλλοντος, ώστε οι βασικές διαδρομές να μπορούν να αλλάξουν χωρίς τροποποίηση του κώδικα. Η Path χρησιμοποιείται για τη διαχείριση διαδρομών αρχείων με πιο καθαρό και ασφαλή τρόπο. Οι τύποι Dict, List και Tuple βοηθούν στην τεκμηρίωση των επιστρεφόμενων τιμών των συναρτήσεων. Η pandas χρησιμοποιείται για τη φόρτωση, τον καθαρισμό και τη χρονική επεξεργασία του CSV ιστορικού.

Ορισμός διαδρομών και βασικών παραμέτρων

Στη συνέχεια ορίζονται οι βασικές διαδρομές και οι σταθερές του module:

```
BASE_DIR = Path(os.getenv("IDEAL_BASE_DIR", "C:/IDEAL_Programming"))
STORES_DIR = Path(os.getenv("IDEAL_STORES_DIR", str(BASE_DIR / "processed"
/ "stores")))
```

```
HISTORY_CSV_PATH = Path(os.getenv("IDEAL_HISTORY_CSV", str(STORES_DIR /
"history_store.csv")))
```

```
DEFAULT_HISTORY_HOURS = int(os.getenv("IDEAL_DEFAULT_HISTORY_HOURS",
"168"))
```

Η μεταβλητή `BASE_DIR` ορίζει τον βασικό φάκελο του έργου. Η `STORES_DIR` δείχνει στον φάκελο όπου αποθηκεύονται τα αρχεία ιστορικού, ενώ η `HISTORY_CSV_PATH` δείχνει στο προεπιλεγμένο αρχείο `history_store.csv`. Η χρήση μεταβλητών περιβάλλοντος επιτρέπει μεγαλύτερη ευελιξία, επειδή το ίδιο module μπορεί να χρησιμοποιηθεί με διαφορετικές διαδρομές χωρίς να αλλάξει ο πηγαίος κώδικας.

Η παράμετρος `DEFAULT_HISTORY_HOURS` ορίζεται στις 168 ώρες. Η τιμή αυτή αντιστοιχεί σε μία εβδομάδα ωριαίου ιστορικού και συνδέεται με τις απαιτήσεις των generic with_history μοντέλων, τα οποία χρειάζονται επαρκές πρόσφατο ιστορικό για να υπολογίσουν lag και rolling χαρακτηριστικά.

Ορισμός στηλών και εναλλακτικών ονομάτων

Το script ορίζει τις βασικές στήλες του CSV:

```
TIMESTAMP_COL = "timestamp"
```

```
CONSUMPTION_COL = "consumption_Wh"
```

Παράλληλα, υποστηρίζει και εναλλακτικά ονόματα στηλών:

```
TIMESTAMP_ALIASES = ["time", "datetime", "date_time", "date", "ds"]
```

```
CONSUMPTION_ALIASES = ["consumption", "energy_Wh", "Wh", "value", "y"]
```

Αυτό σημαίνει ότι το CSV δεν είναι απαραίτητο να έχει αυστηρά τα ονόματα `timestamp` και `consumption_Wh`. Αν, για παράδειγμα, η χρονική στήλη ονομάζεται `datetime` ή η κατανάλωση ονομάζεται `value`, το module μπορεί να τις αναγνωρίσει και να τις μετονομάσει εσωτερικά. Αυτή η επιλογή κάνει το σύστημα πιο φιλικό προς διαφορετικές μορφές αρχείων που μπορεί να εισαγάγει ο χρήστης.

Κανονικοποίηση ονομάτων στηλών

Η συνάρτηση `_normalize_columns` ελέγχει και κανονικοποιεί τα ονόματα των στηλών:

```
def _normalize_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Normalize expected timestamp and consumption column names."""
    df = df.copy()
```

```
    if TIMESTAMP_COL not in df.columns:
        for alt in TIMESTAMP_ALIASES:
            if alt in df.columns:
                df = df.rename(columns={alt: TIMESTAMP_COL})
                break
```

Αρχικά ελέγχεται αν υπάρχει η στήλη `timestamp`. Αν δεν υπάρχει, η συνάρτηση ψάχνει στα εναλλακτικά ονόματα, όπως `time`, `datetime` ή `date`. Αν βρεθεί κάποιο από αυτά, μετονομάζεται σε `timestamp`.

Αν δεν βρεθεί καμία αποδεκτή χρονική στήλη, η συνάρτηση επιστρέφει σαφές σφάλμα:

```
if TIMESTAMP_COL not in df.columns:
    raise KeyError(
        "history_store.csv must contain a 'timestamp' column "
        "or one of these alternatives: " + ", ".join(TIMESTAMP_ALIASES)
    )
```

Η ίδια λογική εφαρμόζεται και για τη στήλη κατανάλωσης:

```
if CONSUMPTION_COL not in df.columns:
    for alt in CONSUMPTION_ALIASES:
        if alt in df.columns:
            df = df.rename(columns={alt: CONSUMPTION_COL})
            break
```

Με αυτόν τον τρόπο, το module διασφαλίζει ότι μετά την κανονικοποίηση το dataframe θα έχει πάντα τις δύο εσωτερικές στήλες `timestamp` και `consumption_Wh`.

Φόρτωση και καθαρισμός του CSV ιστορικού

Η συνάρτηση `_load_history_csv` είναι υπεύθυνη για τη φόρτωση και τον καθαρισμό του αρχείου ιστορικού:

```
def _load_history_csv() -> pd.DataFrame:
    """Load, normalize and clean the history CSV."""
    if not HISTORY_CSV_PATH.exists():
        raise FileNotFoundError(
            f"History CSV not found: {HISTORY_CSV_PATH}\n"
            f"Put a file named 'history_store.csv' under: {STORES_DIR}"
        )
```

Αρχικά ελέγχεται αν το αρχείο υπάρχει στη διαδρομή που έχει οριστεί. Αν δεν υπάρχει, εμφανίζεται σαφές μήνυμα σφάλματος που ενημερώνει τον χρήστη σε ποιον φάκελο πρέπει να τοποθετήσει το αρχείο.

Στη συνέχεια, το CSV φορτώνεται και οι στήλες του κανονικοποιούνται:

```
df = pd.read_csv(HISTORY_CSV_PATH)
df = _normalize_columns(df)
```

Ακολουθεί η μετατροπή των τύπων δεδομένων:

```
df[TIMESTAMP_COL] = pd.to_datetime(df[TIMESTAMP_COL], errors="coerce")
df[CONSUMPTION_COL] = pd.to_numeric(df[CONSUMPTION_COL], errors="coerce")
```

Η χρονική στήλη μετατρέπεται σε `datetime`, ενώ η στήλη κατανάλωσης μετατρέπεται σε αριθμητική. Αν κάποια τιμή δεν μπορεί να μετατραπεί, γίνεται NaN. Στη συνέχεια αφαιρούνται οι γραμμές στις οποίες λείπει είτε το `timestamp` είτε η κατανάλωση:

```
df = df.dropna(subset=[TIMESTAMP_COL, CONSUMPTION_COL]).copy()
```

Ένα σημαντικό σημείο είναι η κανονικοποίηση των `timestamps` σε ωριαία βάση:

```
df[TIMESTAMP_COL] = df[TIMESTAMP_COL].dt.floor("h")
```

Αν οι χρονικές σημάνσεις είναι ήδη ωριαίες, δεν αλλάζουν. Αν όμως υπάρχουν μη ωριαίες χρονικές σημάνσεις, αυτές στρογγυλοποιούνται προς τα κάτω στην πλησιέστερη ώρα. Αυτό βοηθά ώστε το ιστορικό να αποκτήσει ενιαία ωριαία μορφή.

Στη συνέχεια το `dataframe` ταξινομείται και αφαιρούνται διπλότυπα `timestamps`:

```
df = df.sort_values(TIMESTAMP_COL)
df = df.drop_duplicates(subset=[TIMESTAMP_COL], keep="last")
```

Αν υπάρχουν περισσότερες από μία τιμές για την ίδια ώρα, διατηρείται η τελευταία. Τέλος, οι αρνητικές τιμές κατανάλωσης περιορίζονται στο μηδέν:

```
df[CONSUMPTION_COL] = df[CONSUMPTION_COL].clip(lower=0)
```

Η κατανάλωση ηλεκτρικής ενέργειας δεν μπορεί να είναι αρνητική στο πλαίσιο της συγκεκριμένης εφαρμογής, οπότε αυτή η διόρθωση διασφαλίζει ότι το ιστορικό που θα χρησιμοποιηθεί από το μοντέλο είναι φυσικά έγκυρο.

Δημιουργία αναμενόμενου ωριαίου δείκτη

Η συνάρτηση `_expected_hourly_index` δημιουργεί το αναμενόμενο ωριαίο χρονικό πλέγμα για ένα παράθυρο:

```
def _expected_hourly_index(start: pd.Timestamp, end: pd.Timestamp) -> pd.DatetimeIndex:
    """Return hourly timestamps in [start, end)."""
    return pd.date_range(start=start, end=end - pd.Timedelta(hours=1),
                          freq="h")
```

Η συνάρτηση επιστρέφει όλες τις ωριαίες χρονικές στιγμές από την αρχή του παραθύρου μέχρι μία ώρα πριν από το τέλος. Η λογική αυτή αντιστοιχεί στο διάστημα `[start, end)`, δηλαδή περιλαμβάνει την αρχή αλλά όχι το τέλος. Για παράδειγμα, αν το παράθυρο λήγει στις 00:00 της ημέρας στόχου, η τελευταία ιστορική ώρα είναι η 23:00 της προηγούμενης ημέρας.

Έλεγχος πληρότητας ιστορικού παραθύρου

Η συνάρτηση `_is_complete_hourly_window` ελέγχει αν ένα υποσύνολο δεδομένων καλύπτει πλήρως το ζητούμενο χρονικό παράθυρο:

```
def _is_complete_hourly_window(df_window: pd.DataFrame, start: pd.Timestamp, end: pd.Timestamp) -> bool:
```

```

expected = _expected_hourly_index(start, end)

if len(df_window) != len(expected):
    return False

actual =
pd.DatetimeIndex(df_window[TIMESTAMP_COL].sort_values().values)
return actual.equals(expected)

```

Αρχικά δημιουργείται το αναμενόμενο ωριαίο index. Αν ο αριθμός γραμμών του διαθέσιμου παραθύρου δεν είναι ίσος με τον αριθμό αναμενόμενων ωρών, τότε το παράθυρο θεωρείται ελλιπές. Αν ο αριθμός γραμμών είναι σωστός, ελέγχεται αν οι πραγματικές χρονικές σημάνσεις ταυτίζονται ακριβώς με τις αναμενόμενες.

Αυτός ο έλεγχος είναι ιδιαίτερα σημαντικός, επειδή ένα ιστορικό αρχείο μπορεί να έχει τον σωστό αριθμό γραμμών αλλά να λείπει κάποια ενδιάμεση ώρα και να υπάρχει κάποια άλλη ώρα εκτός παραθύρου. Η συνάρτηση διασφαλίζει ότι το παράθυρο είναι πραγματικά συνεχόμενο και πλήρες.

Εντοπισμός ελλειπουσών ωρών

Αν ένα παράθυρο δεν είναι πλήρες, η συνάρτηση `_format_missing_hours` μπορεί να επιστρέψει ενδεικτικές ελλείπουσες ώρες:

```

def _format_missing_hours(df_window: pd.DataFrame, start: pd.Timestamp,
end: pd.Timestamp, max_show: int = 10) -> List[str]:
    expected = set(_expected_hourly_index(start, end))
    actual = set(pd.to_datetime(df_window[TIMESTAMP_COL]).tolist())
    missing = sorted(expected - actual)
    return [pd.Timestamp(x).isoformat() for x in missing[:max_show]]

```

Η συνάρτηση αυτή δεν χρησιμοποιείται για να συμπληρώσει τα δεδομένα, αλλά για να δημιουργήσει πιο κατανοητό μήνυμα σφάλματος. Αν λείπουν ώρες από το ζητούμενο παράθυρο, μπορεί να εμφανιστούν οι πρώτες ελλείψεις ώστε ο χρήστης ή ο προγραμματιστής να γνωρίζει πού βρίσκεται το πρόβλημα.

Εύρεση του πιο πρόσφατου συνεχόμενου παραθύρου

Μία από τις σημαντικότερες βοηθητικές συναρτήσεις είναι η `_latest_contiguous_window`:

```

def _latest_contiguous_window(df: pd.DataFrame, history_hours: int) ->
pd.DataFrame:

```

```

    """Find the latest contiguous hourly block of length history_hours."""
    if len(df) < history_hours:
        raise ValueError(
            f"CSV history too short: only {len(df)} rows, need >=
{history_hours}."
        )

```

Αρχικά ελέγχεται αν το CSV έχει αρκετές γραμμές για να καλύψει το απαιτούμενο πλήθος ωρών. Αν οι γραμμές είναι λιγότερες από το `history_hours`, επιστρέφεται σφάλμα.

Στη συνέχεια το dataframe ταξινομείται και εντοπίζονται οι διακοπές στη χρονοσειρά:

```

df = df.sort_values(TIMESTAMP_COL).reset_index(drop=True).copy()

```

```

diffs = df[TIMESTAMP_COL].diff()
new_group = diffs.ne(pd.Timedelta(hours=1)).fillna(True)
df["_contiguous_group"] = new_group.cumsum()

```

Η λογική είναι ότι κάθε φορά που η διαφορά ανάμεσα σε δύο διαδοχικά timestamps δεν είναι ακριβώς μία ώρα, ξεκινά νέα ομάδα. Έτσι το script εντοπίζει συνεχόμενα τμήματα της χρονοσειράς.

Στη συνέχεια εξετάζονται όλες οι συνεχόμενες ομάδες:

```

valid_groups = []
for _, g in df.groupby("_contiguous_group", sort=False):
    if len(g) >= history_hours:
        valid_groups.append(g.tail(history_hours).copy())

```

Αν κάποια ομάδα έχει τουλάχιστον όσες γραμμές απαιτούνται, κρατούνται οι τελευταίες `history_hours` γραμμές της. Αν υπάρχουν περισσότερες από μία έγκυρες ομάδες, επιστρέφεται η πιο πρόσφατη:

```
latest = valid_groups[-1].drop(columns=["_contiguous_group"])
return latest.reset_index(drop=True)
```

Αυτός ο μηχανισμός λειτουργεί ως fallback. Αν το ακριβές ιστορικό παράθυρο πριν από την ημερομηνία στόχου δεν είναι διαθέσιμο, η εφαρμογή μπορεί να χρησιμοποιήσει το πιο πρόσφατο πλήρες συνεχόμενο ιστορικό που υπάρχει στο CSV.

Μετατροπή παραθύρου σε tuple ιστορικού

Η συνάρτηση `_history_tuple_from_window` μετατρέπει το τελικό dataframe ιστορικού στη μορφή που χρειάζεται το API:

```
def _history_tuple_from_window(df_window: pd.DataFrame, source: str) ->
    Tuple[List[float], str, str, str]:
    df_window = df_window.sort_values(TIMESTAMP_COL).reset_index(drop=True)

    ws = pd.Timestamp(df_window[TIMESTAMP_COL].iloc[0])
    we = pd.Timestamp(df_window[TIMESTAMP_COL].iloc[-1]) +
    pd.Timedelta(hours=1)
```

Αρχικά ταξινομεί το παράθυρο ιστορικού. Έπειτα υπολογίζει την αρχή και το τέλος του παραθύρου. Το τέλος ορίζεται ως η τελευταία χρονική στιγμή συν μία ώρα, ώστε να εκφράζεται πάλι το διάστημα `[start, end)`.

Η συνάρτηση επιστρέφει:

```
return (
    df_window[CONSUMPTION_COL].astype(float).tolist(),
    source,
    ws.isoformat(),
    we.isoformat(),
)
```

Δηλαδή επιστρέφει τη λίστα των τιμών κατανάλωσης, την πηγή του ιστορικού, την αρχή του παραθύρου και το τέλος του παραθύρου. Αυτή η μορφή είναι χρήσιμη για το API, επειδή δεν χρειάζεται μόνο τις τιμές, αλλά και μεταδεδομένα που εξηγούν από πού προήλθε το ιστορικό.

Κύρια συνάρτηση ανάκτησης ιστορικού

Η βασική δημόσια συνάρτηση του module είναι η `get_history_from_csv`:

```
def get_history_from_csv(
    target_date: str,
    history_hours: int = DEFAULT_HISTORY_HOURS,
    fallback_to_latest: bool = True,
) -> Tuple[List[float], str, str, str]:
```

Η συνάρτηση δέχεται την ημερομηνία στόχου, τον αριθμό προηγούμενων ωρών που πρέπει να επιστραφούν και το αν επιτρέπεται fallback στο πιο πρόσφατο συνεχόμενο ιστορικό. Η ημερομηνία στόχου αντιμετωπίζεται ως η αρχή της ημέρας πρόβλεψης, δηλαδή ως `target_date 00:00`.

Αρχικά ελέγχεται ότι το `history_hours` είναι θετικό:

```
history_hours = int(history_hours)
if history_hours <= 0:
    raise ValueError("history_hours must be > 0")
```

Στη συνέχεια φορτώνεται το CSV:

```
df = _load_history_csv()
```

Υπολογίζεται το ζητούμενο ιστορικό παράθυρο:

```
window_end = pd.Timestamp(target_date).normalize()
window_start = window_end - pd.Timedelta(hours=history_hours)
```

Αν, για παράδειγμα, η ημερομηνία στόχου είναι 2018-06-30 και το `history_hours` είναι 168, τότε το σύστημα αναζητά τις 168 ώρες που προηγούνται της 2018-06-30 00:00.

Στη συνέχεια απομονώνεται το αντίστοιχο τμήμα του CSV:

```
w = df[(df[TIMESTAMP_COL] >= window_start) & (df[TIMESTAMP_COL] <
window_end)].copy()
w = w.sort_values(TIMESTAMP_COL).reset_index(drop=True)
```

Αν το παράθυρο είναι πλήρες, επιστρέφεται ως csv_window:

```
if _is_complete_hourly_window(w, window_start, window_end):
    return _history_tuple_from_window(w, "csv_window")
```

Αν δεν είναι πλήρες και το fallback είναι ενεργό, επιστρέφεται το πιο πρόσφατο συνεχόμενο παράθυρο:

```
if fallback_to_latest:
    latest = _latest_contiguous_window(df, history_hours)
    return _history_tuple_from_window(latest,
"csv_fallback_latest_contiguous")
```

Αν ούτε το ακριβές παράθυρο είναι διαθέσιμο ούτε επιτρέπεται fallback, τότε εμφανίζεται σαφές σφάλμα:

```
missing_examples = _format_missing_hours(w, window_start, window_end)
raise ValueError(
    f"Not enough contiguous history for requested window [{window_start},
{window_end}]. "
    f"Expected {history_hours} hourly rows, found {len(w)}. "
    f"Missing examples: {missing_examples}"
)
```

Με αυτόν τον τρόπο, η συνάρτηση χειρίζεται τόσο την ιδανική περίπτωση όσο και περιπτώσεις ελλιπούς ιστορικού, χωρίς να αποκρύπτει το πρόβλημα.

Επιστροφή ιστορικού σε μορφή DataFrame

Η δεύτερη δημόσια συνάρτηση είναι η get_history_dataframe_from_csv:

```
def get_history_dataframe_from_csv(
    target_date: str,
    history_hours: int = DEFAULT_HISTORY_HOURS,
    fallback_to_latest: bool = True,
) -> Tuple[pd.DataFrame, Dict[str, str]]:
```

Η συνάρτηση αυτή χρησιμοποιεί εσωτερικά την get_history_from_csv, αλλά επιστρέφει το ιστορικό σε μορφή dataframe μαζί με metadata. Αυτό είναι χρήσιμο για το API και το inference layer, επειδή εκεί συχνά χρειάζονται τόσο οι τιμές κατανάλωσης όσο και οι αντίστοιχες χρονικές σημάνσεις.

Αρχικά καλείται η βασική συνάρτηση:

```
history, source, window_start, window_end = get_history_from_csv(
    target_date=target_date,
    history_hours=history_hours,
    fallback_to_latest=fallback_to_latest,
)
```

Στη συνέχεια ανακατασκευάζονται τα timestamps του ιστορικού:

```
start = pd.Timestamp(window_start)
timestamps = pd.date_range(start=start, periods=len(history), freq="h")
```

Έπειτα δημιουργείται dataframe:

```
df = pd.DataFrame({
    TIMESTAMP_COL: timestamps,
    CONSUMPTION_COL: history,
})
```

Παράλληλα δημιουργείται dictionary μεταδεδομένων:

```
meta = {
    "history_source": source,
    "window_start_iso": window_start,
    "window_end_iso": window_end,
    "history_hours": str(history_hours),
}
```

Η συνάρτηση επιστρέφει τόσο το dataframe όσο και τα metadata. Αυτή η μορφή είναι πιο πλούσια από μια απλή λίστα τιμών, επειδή διατηρεί και την πληροφορία του χρονικού παραθύρου.

Πληροφορίες κατάστασης history store

Η τρίτη δημόσια συνάρτηση είναι η `history_store_info`:

```
def history_store_info() -> dict:
    """Return file/status information for UI/debug/API health
    endpoints."""
    info = {
        "history_csv_path": str(HISTORY_CSV_PATH),
        "stores_dir": str(STORES_DIR),
        "default_history_hours": DEFAULT_HISTORY_HOURS,
        "exists": HISTORY_CSV_PATH.exists(),
    }
```

Η συνάρτηση αυτή επιστρέφει πληροφορίες για την κατάσταση του history store. Περιλαμβάνει τη διαδρομή του αρχείου, τον φάκελο αποθήκευσης, το προεπιλεγμένο πλήθος ωρών ιστορικού και το αν το αρχείο υπάρχει.

Αν το αρχείο υπάρχει, επιχειρείται φόρτωση και καθαρισμός του:

```
df = _load_history_csv()
info.update({
    "rows": int(len(df)),
    "min_timestamp": str(df[TIMESTAMP_COL].min()) if len(df) else None,
    "max_timestamp": str(df[TIMESTAMP_COL].max()) if len(df) else None,
    "can_supply_default_history": bool(len(df) >= DEFAULT_HISTORY_HOURS),
})
```

Έτσι το API ή το UI μπορούν να εμφανίσουν βασικές πληροφορίες για το διαθέσιμο ιστορικό, όπως το πλήθος γραμμών, το χρονικό εύρος και το αν μπορεί να καλυφθεί το προεπιλεγμένο παράθυρο των 168 ωρών.

Επιπλέον, αν υπάρχουν τουλάχιστον δύο γραμμές, υπολογίζονται πληροφορίες για τη χρονική συνέχεια:

```
diffs = df[TIMESTAMP_COL].diff().dropna()
info.update({
    "median_step_hours": float(diffs.median() / pd.Timedelta(hours=1)),
    "non_hourly_gaps": int((diffs != pd.Timedelta(hours=1)).sum()),
})
```

Το `median_step_hours` δείχνει το τυπικό χρονικό βήμα του αρχείου, ενώ το `non_hourly_gaps` δείχνει πόσες φορές η διαφορά ανάμεσα σε δύο διαδοχικές εγγραφές δεν είναι μία ώρα. Αυτή η πληροφορία είναι χρήσιμη για τον εντοπισμό κενών στο ιστορικό.

Model Registry για το API. Αρχείο: `model_registry.py`

Model Registry. Αρχείο: `model_registry.py`

Το αρχείο `model_registry.py` αποτελεί ένα από τα βασικά βοηθητικά αρχεία της τελικής εφαρμογής, καθώς λειτουργεί ως κεντρικό μητρώο των διαθέσιμων μοντέλων μηχανικής μάθησης. Ο ρόλος του είναι να συγκεντρώνει σε ένα σημείο όλες τις πληροφορίες που χρειάζεται το API και η γραφική διεπαφή για να γνωρίζουν ποια μοντέλα είναι διαθέσιμα, ποιο μοντέλο πρέπει να χρησιμοποιηθεί ως προεπιλογή, ποια μοντέλα είναι προαιρετικά και ποια σενάρια πρόβλεψης υποστηρίζει κάθε μοντέλο.

Η ύπαρξη ενός ξεχωριστού registry είναι σημαντική για τη συντηρησιμότητα της εφαρμογής. Αν οι διαδρομές των μοντέλων, τα ονόματα, τα modes και οι περιορισμοί χρήσης ήταν διάσπαρτα μέσα στο API ή στο UI, κάθε αλλαγή θα απαιτούσε επεμβάσεις σε πολλά σημεία του κώδικα. Αντίθετα, με το `model_registry.py`, η πληροφορία για τα μοντέλα συγκεντρώνεται σε ένα ενιαίο αρχείο. Έτσι, η προσθήκη, αφαίρεση ή τροποποίηση ενός μοντέλου μπορεί να γίνει πιο ελεγχόμενα.

Η βασική παραδοχή της συγκεκριμένης έκδοσης είναι ότι η τελική γραφική διεπαφή είναι generic. Δηλαδή, ο τελικός χρήστης δεν παρέχει home_id. Για τον λόγο αυτό, το registry αποφεύγει μοντέλα που απαιτούν γνωστή κατοικία από το αρχικό dataset και επιτρέπει μόνο μοντέλα που μπορούν να λειτουργήσουν χωρίς να ζητούν από τον χρήστη αναγνωριστικό κατοικίας.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import os
from pathlib import Path
from typing import Any, Dict, List, Optional
```

Η βιβλιοθήκη os χρησιμοποιείται για την ανάγνωση μεταβλητών περιβάλλοντος, ώστε η βασική διαδρομή του έργου να μπορεί να αλλάξει χωρίς τροποποίηση του κώδικα. Η Path χρησιμοποιείται για τη διαχείριση διαδρομών αρχείων με πιο ασφαλή και καθαρό τρόπο. Οι τύποι Any, Dict, List και Optional χρησιμοποιούνται για την τεκμηρίωση των δομών δεδομένων και των συναρτήσεων του registry.

Ορισμός βασικών διαδρομών

Στη συνέχεια ορίζονται οι βασικές διαδρομές των μοντέλων:

```
BASE_DIR = Path(os.getenv("IDEAL_BASE_DIR", "C:/IDEAL_Programming"))
MODELS_ROOT = BASE_DIR / "processed" / "models" / "final_api_models"
```

```
RF_ROOT = MODELS_ROOT / "RF2"
LGBM_ROOT = MODELS_ROOT / "LGBM2"
XGB_ROOT = MODELS_ROOT / "XGB2"
```

Το BASE_DIR ορίζει τον βασικό φάκελο του έργου. Η χρήση της μεταβλητής περιβάλλοντος IDEAL_BASE_DIR επιτρέπει την αλλαγή της διαδρομής εγκατάστασης χωρίς να χρειάζεται αλλαγή του πηγαιού κώδικα. Το MODELS_ROOT δείχνει στον φάκελο όπου αποθηκεύονται τα τελικά μοντέλα της εφαρμογής.

Οι μεταβλητές RF_ROOT, LGBM_ROOT και XGB_ROOT δείχνουν αντίστοιχα στους φακέλους των μοντέλων Random Forest, LightGBM και XGBoost. Με αυτόν τον τρόπο, το registry γνωρίζει πού πρέπει να αναζητήσει τα αποθηκευμένα artifacts κάθε οικογένειας μοντέλων.

Κανονικοποίηση των modes πρόβλεψης

Το script ορίζει δύο βασικά modes πρόβλεψης:

```
MODE_NO_HISTORY = "no_history"
MODE_WITH_HISTORY = "with_history"
```

Το no_history αντιστοιχεί στο σενάριο όπου ο χρήστης δεν διαθέτει πρόσφατο ιστορικό κατανάλωσης. Σε αυτό το mode, η πρόβλεψη βασίζεται σε χρονικά, στατικά και περιβαλλοντικά χαρακτηριστικά. Το with_history αντιστοιχεί στο σενάριο όπου ο χρήστης παρέχει πρόσφατο ιστορικό ωριαίας κατανάλωσης, ώστε το μοντέλο να μπορεί να υπολογίσει lag και rolling χαρακτηριστικά.

Για λόγους συμβατότητας με παλαιότερες εκδόσεις, το script διατηρεί και παλιές ονομασίες ως aliases:

```
MODE_COLDSTART = MODE_NO_HISTORY
MODE_WITHHISTORY = MODE_WITH_HISTORY
```

Οι παλιές ονομασίες coldstart και withhistory δεν προτείνονται πλέον ως κύριες ετικέτες στο UI, αλλά παραμένουν ώστε να μη σπάσουν παλαιότερες κλήσεις του API ή παλαιότερα τμήματα κώδικα που τις χρησιμοποιούσαν.

Ορισμός επιλογών βελτιστοποίησης

Το registry ορίζει επίσης δύο επιλογές βελτιστοποίησης:

```
OPT_BALANCED = "balanced"
OPT_DAILY = "daily"
```

```
DEFAULT_MODE = MODE_NO_HISTORY
DEFAULT_OPTIMIZATION = OPT_BALANCED
```

Η επιλογή `balanced` αντιστοιχεί στη βασική, ισορροπημένη πρόβλεψη. Η επιλογή `daily` μπορεί να χρησιμοποιηθεί όταν η εφαρμογή θέλει να δώσει έμφαση στη συνολική ημερήσια κατανάλωση. Παρότι η `daily` επιλογή δεν οδηγεί απαραίτητα σε διαφορετικό εκπαιδευμένο μοντέλο, διατηρείται ως λογική επιλογή του API και του UI, ώστε να μπορεί να υποστηριχθεί διαφορετική έξοδος ή `post-processing`.

Κεντρικό μητρώο μοντέλων

Το βασικότερο αντικείμενο του αρχείου είναι το `MODEL_REGISTRY`:

```
MODEL_REGISTRY: Dict[str, Dict[str, Any]] = {
    ...
}
```

Πρόκειται για `dictionary` στο οποίο κάθε κλειδί είναι ένα μοναδικό `model_id` και κάθε τιμή είναι ένα δεύτερο `dictionary` με τα μεταδεδομένα του αντίστοιχου μοντέλου. Τα μεταδεδομένα αυτά περιλαμβάνουν το όνομα, τη σύντομη ονομασία, την οικογένεια μοντέλου, τον τύπο, το `mode` που υποστηρίζεται, τη διαδρομή των `artifacts`, αν το μοντέλο είναι `default` ή `optional` και αν απαιτεί ιστορικό κατανάλωσης.

Η επιλογή αυτή κάνει το `registry` πολύ ευέλικτο. Το API δεν χρειάζεται να γνωρίζει χειροκίνητα πού βρίσκεται κάθε μοντέλο ή ποια χαρακτηριστικά χρησιμοποιεί. Αντίθετα, μπορεί να ζητήσει από το `registry` τις πληροφορίες του μοντέλου με βάση το `model_id`.

Προεπιλεγμένο `no-history` μοντέλο

Το προεπιλεγμένο μοντέλο για το `no_history` σενάριο είναι το `Random Forest`:

```
"rf_coldstart_default": {
    "name": "Random Forest - no-history default",
    "short_name": "RF no-history default",
    "model_family": "Random Forest",
    "type": "sklearn_rf",
    "mode": MODE_NO_HISTORY,
    "optimization": OPT_BALANCED,
    "artifact_dir": str(RF_ROOT / "cold_start_default"),
    "is_default": True,
    "is_optional": False,
    "ui_visible": True,
    "supports": [MODE_NO_HISTORY],
    "requires_history": False,
    "requires_user_home_id": False,
}
```

Το μοντέλο αυτό έχει οριστεί ως `default` για την περίπτωση όπου ο χρήστης δεν παρέχει ιστορικό. Δεν απαιτεί `home_id`, δεν χρησιμοποιεί `lag` ή `rolling` χαρακτηριστικά και δεν χρησιμοποιεί στατιστικά συγκεκριμένης κατοικίας. Η περιγραφή του δηλώνει ότι βασίζεται σε απλά χρονικά, μετεωρολογικά και στατικά χαρακτηριστικά.

Αυτή η επιλογή είναι συνεπής με τη λογική της τελικής εφαρμογής, όπου το `no_history` σενάριο πρέπει να είναι απλό, σταθερό και εφαρμόσιμο σε νέο χρήστη που δεν διαθέτει προηγούμενες μετρήσεις κατανάλωσης.

Προεπιλεγμένο `with-history` μοντέλο

Το προεπιλεγμένο μοντέλο για το `with_history` σενάριο είναι το `LightGBM`:

```
"lgbm_withhistory_default": {
    "name": "LightGBM - with-history default",
    "short_name": "LGBM with-history default",
    "model_family": "LightGBM",
    "type": "lightgbm",
    "mode": MODE_WITH_HISTORY,
    "optimization": OPT_BALANCED,
    "artifact_dir": str(LGBM_ROOT / "with_history_generic"),
}
```

```

    "is_default": True,
    "is_optional": False,
    "ui_visible": True,
    "supports": [MODE_WITH_HISTORY],
    "requires_history": True,
    "min_history_hours": 168,
    "requires_user_home_id": False,
    "uses_home_id_as_feature": False,
    "uses_lag_features": True,
    "uses_rolling_features": True,
}

```

Το μοντέλο αυτό απαιτεί πρόσφατο ιστορικό κατανάλωσης, με ελάχιστο μήκος 168 ωρών. Χρησιμοποιεί lag και rolling χαρακτηριστικά, αλλά δεν απαιτεί home_id και δεν το χρησιμοποιεί ως χαρακτηριστικό. Αυτό είναι σημαντικό, διότι το καθιστά κατάλληλο για τη generic γραφική διεπαφή.

Η επιλογή του LightGBM ως προεπιλεγμένου with_history μοντέλου συνδέεται με την τελική λογική της εφαρμογής, σύμφωνα με την οποία το ιστορικό κατανάλωσης μπορεί να αξιοποιηθεί χωρίς να χρειάζεται ο χρήστης να ταυτίζεται με κάποια γνωστή κατοικία του dataset.

Προαιρετικά with-history μοντέλα σύγκρισης

Το registry περιλαμβάνει επίσης προαιρετικά μοντέλα για το with_history mode. Το πρώτο είναι το XGBoost:

```

"xgb_withhistory_optional": {
    "name": "XGBoost - with-history optional",
    "short_name": "XGB with-history optional",
    "model_family": "XGBoost",
    "type": "xgboost",
    "mode": MODE_WITH_HISTORY,
    "artifact_dir": str(XGB_ROOT / "with_history_generic"),
    "is_default": False,
    "is_optional": True,
    "supports": [MODE_WITH_HISTORY],
    "requires_history": True,
    "min_history_hours": 168,
    "requires_user_home_id": False,
}

```

Το μοντέλο αυτό λειτουργεί ως εναλλακτικό generic μοντέλο με ιστορικό. Δεν είναι η προεπιλογή, αλλά είναι διαθέσιμο για σύγκριση μέσα από το UI ή το API.

Το δεύτερο προαιρετικό with-history μοντέλο είναι το Random Forest:

```

"rf_withhistory_optional": {
    "name": "Random Forest - with-history optional",
    "short_name": "RF with-history optional",
    "model_family": "Random Forest",
    "type": "sklearn_rf",
    "mode": MODE_WITH_HISTORY,
    "artifact_dir": str(RF_ROOT / "with_history_generic"),
    "is_default": False,
    "is_optional": True,
    "supports": [MODE_WITH_HISTORY],
    "requires_history": True,
    "min_history_hours": 168,
    "requires_user_home_id": False,
}

```

Η ύπαρξη των optional μοντέλων επιτρέπει στη γραφική διεπαφή να προσφέρει σύγκριση μεταξύ διαφορετικών αλγορίθμων. Ο χρήστης μπορεί να δει πώς διαφοροποιείται η πρόβλεψη ανάμεσα σε LightGBM, XGBoost και Random Forest, χωρίς το API να αλλάζει τη βασική του λογική.

Προαιρετικά no-history μοντέλα σύγκρισης

Το registry περιλαμβάνει και προαιρετικά μοντέλα για το no_history σενάριο. Το LightGBM no-history optional ορίζεται ως εξής:

```
"lgbm_coldstart_optional": {
    "name": "LightGBM - no-history optional",
    "short_name": "LGBM no-history optional",
    "model_family": "LightGBM",
    "type": "lightgbm",
    "mode": MODE_NO_HISTORY,
    "artifact_dir": str(LGBM_ROOT / "cold_start_optional"),
    "is_default": False,
    "is_optional": True,
    "supports": [MODE_NO_HISTORY],
    "requires_history": False,
    "requires_user_home_id": False,
    "uses_behavior_profiles": True,
    "uses_knn_profiles": True,
}
```

Το μοντέλο αυτό δεν απαιτεί ιστορικό από τον χρήστη, αλλά χρησιμοποιεί behavior και KNN profiles που έχουν δημιουργηθεί από το training set. Επειδή ο χρήστης δεν παρέχει home_id, το API μπορεί εσωτερικά να χρησιμοποιεί ένα pseudo home id για βοηθητική λογική KNN, χωρίς αυτό να εμφανίζεται ως απαίτηση προς τον χρήστη.

Αντίστοιχα, υπάρχει και το XGBoost no-history optional:

```
"xgb_coldstart_optional": {
    "name": "XGBoost - no-history optional",
    "short_name": "XGB no-history optional",
    "model_family": "XGBoost",
    "type": "xgboost",
    "mode": MODE_NO_HISTORY,
    "artifact_dir": str(XGB_ROOT / "cold_start_optional"),
    "is_default": False,
    "is_optional": True,
    "supports": [MODE_NO_HISTORY],
    "requires_history": False,
    "uses_behavior_profiles": True,
    "uses_knn_profiles": True,
}
```

Και αυτό το μοντέλο λειτουργεί ως προαιρετικό μοντέλο σύγκρισης. Δεν είναι το default no-history μοντέλο, αλλά μπορεί να χρησιμοποιηθεί στο UI για να συγκριθεί η πρόβλεψή του με εκείνη του Random Forest.

Προεπιλεγμένα μοντέλα ανά mode

Το registry ορίζει ξεκάθαρα ποιο μοντέλο χρησιμοποιείται ως προεπιλογή σε κάθε mode:

```
DEFAULT_MODEL_BY_MODE: Dict[str, str] = {
    MODE_NO_HISTORY: "rf_coldstart_default",
    MODE_WITH_HISTORY: "lgbm_withhistory_default",
}
```

Αυτό σημαίνει ότι αν ο χρήστης ή το UI επιλέξει auto, τότε το API θα χρησιμοποιήσει Random Forest για το no_history σενάριο και LightGBM για το with_history σενάριο.

Υπάρχει επίσης πιο αναλυτικός ορισμός ανά mode και optimization:

```
DEFAULT_MODEL_BY_MODE_AND_OPTIMIZATION: Dict[tuple[str, str], str] = {
    (MODE_NO_HISTORY, OPT_BALANCED): "rf_coldstart_default",
    (MODE_NO_HISTORY, OPT_DAILY): "rf_coldstart_default",
    (MODE_WITH_HISTORY, OPT_BALANCED): "lgbm_withhistory_default",
    (MODE_WITH_HISTORY, OPT_DAILY): "lgbm_withhistory_default",
}
```

Ακόμη και όταν ζητείται daily optimization, το registry οδηγεί στο ίδιο βασικό μοντέλο, εκτός αν το API ή το UI εφαρμόσει ξεχωριστή βαθμονομημένη έξοδο. Με αυτόν τον τρόπο,

το registry παραμένει απλό, ενώ η επιπλέον λογική daily βελτιστοποίησης μπορεί να εφαρμοστεί σε μεταγενέστερο στάδιο.

Legacy aliases για παλαιότερα model IDs

Το αρχείο υποστηρίζει και παλαιότερα ονόματα μοντέλων, όπως rf, lgbm και xgb:

```
LEGACY_MODEL_ALIASES: Dict[str, Dict[str, str]] = {
    "rf": {
        MODE_NO_HISTORY: "rf_coldstart_default",
        MODE_WITH_HISTORY: "rf_withhistory_optional",
    },
    "lgbm": {
        MODE_NO_HISTORY: "lgbm_coldstart_optional",
        MODE_WITH_HISTORY: "lgbm_withhistory_default",
    },
    "xgb": {
        MODE_NO_HISTORY: "xgb_coldstart_optional",
        MODE_WITH_HISTORY: "xgb_withhistory_optional",
    },
}
```

Αυτή η δομή είναι σημαντική επειδή τα παλιά model IDs δεν αντιστοιχούν σε ένα μόνο τελικό μοντέλο. Για παράδειγμα, το rf μπορεί να σημαίνει διαφορετικό μοντέλο ανάλογα με το mode. Αν το mode είναι no_history, το rf αντιστοιχεί στο προεπιλεγμένο Random Forest χωρίς ιστορικό. Αν το mode είναι with_history, αντιστοιχεί στο προαιρετικό Random Forest με ιστορικό.

Με αυτή τη λογική, το registry διατηρεί συμβατότητα με παλαιότερες κλήσεις, αλλά ταυτόχρονα τις μεταφράζει σε σαφή και συγκεκριμένα τελικά model IDs.

Κανονικοποίηση mode

Η συνάρτηση normalize_mode μετατρέπει διαφορετικές πιθανές ονομασίες mode στις δύο τελικές κανονικές τιμές:

```
def normalize_mode(mode: Optional[str]) -> str:
    if not mode:
        return DEFAULT_MODE

    raw = str(mode).strip().lower()
    value = raw.replace("-", "_").replace(" ", "_")
    compact = value.replace("_", "")
```

Αρχικά, η είσοδος μετατρέπεται σε πεζά γράμματα και κανονικοποιούνται παύλες και κενά.

Στη συνέχεια γίνεται αντιστοίχιση σε λεξικό aliases:

```
aliases = {
    "no_history": MODE_NO_HISTORY,
    "nohistory": MODE_NO_HISTORY,
    "coldstart": MODE_NO_HISTORY,
    "cold_start": MODE_NO_HISTORY,
    "cold": MODE_NO_HISTORY,
    "with_history": MODE_WITH_HISTORY,
    "withhistory": MODE_WITH_HISTORY,
    "history": MODE_WITH_HISTORY,
    "recent_history": MODE_WITH_HISTORY,
}
```

Έτσι, είτε ο χρήστης δώσει no_history, είτε coldstart, είτε withhistory, το API καταλήγει σε μία από τις δύο κανονικές τιμές. Αν δοθεί μη υποστηριζόμενο mode, η συνάρτηση επιστρέφει σαφές σφάλμα.

Κανονικοποίηση optimization

Αντίστοιχα, η συνάρτηση normalize_optimization κανονικοποιεί την επιλογή optimization:

```
def normalize_optimization(optimization: Optional[str]) -> str:
    if not optimization:
        return DEFAULT_OPTIMIZATION
```

```

value = str(optimization).strip().lower().replace("_", "-")

if value in {"balanced", "default", "pointwise"}:
    return OPT_BALANCED

if value in {"daily", "daily-optimized", "dailyoptimized", "total",
"daily-total"}:
    return OPT_DAILY

```

Με αυτόν τον τρόπο, διαφορετικές διατυπώσεις όπως default, pointwise, daily-optimized ή daily-total μετατρέπονται σε μία από τις δύο εσωτερικές τιμές: balanced ή daily.

Επιλογή προεπιλεγμένου μοντέλου

Η συνάρτηση `get_default_model_id` επιστρέφει το προεπιλεγμένο μοντέλο για ένα mode και μία επιλογή optimization:

```

def get_default_model_id(mode: Optional[str] = None, optimization:
Optional[str] = None) -> str:
    normalized_mode = normalize_mode(mode)
    normalized_optimization = normalize_optimization(optimization)

    key = (normalized_mode, normalized_optimization)

    if key in DEFAULT_MODEL_BY_MODE_AND_OPTIMIZATION:
        return DEFAULT_MODEL_BY_MODE_AND_OPTIMIZATION[key]

    return DEFAULT_MODEL_BY_MODE[normalized_mode]

```

Η συνάρτηση αυτή χρησιμοποιείται όταν ο χρήστης επιλέγει auto ή δεν δηλώνει συγκεκριμένο μοντέλο. Έτσι, το API δεν χρειάζεται να περιέχει δική του λογική για το ποιο μοντέλο είναι default. Το registry αναλαμβάνει αυτή την απόφαση.

Επίλυση model_id

Η συνάρτηση `resolve_model_id` είναι μία από τις σημαντικότερες συναρτήσεις του αρχείου:

```

def resolve_model_id(model_id: Optional[str], mode: Optional[str] = None,
optimization: Optional[str] = None) -> str:
    """Resolve explicit, auto/default and legacy model IDs to a concrete
registry ID."""
    normalized_mode = normalize_mode(mode)

```

Η συνάρτηση αυτή δέχεται το `model_id` που έρχεται από το API ή το UI και το μετατρέπει σε συγκεκριμένο, τελικό registry ID. Αν ο χρήστης δεν δώσει `model_id` ή δώσει auto ή default, επιστρέφεται το προεπιλεγμένο μοντέλο:

```

if model_id is None or str(model_id).strip().lower() in {"", "auto",
"default"}:
    return get_default_model_id(normalized_mode, optimization)

```

Αν δοθεί ήδη έγκυρο `model_id`, επιστρέφεται άμεσα. Αν δοθεί παλιό alias, όπως rf, xgb ή lgbm, τότε το registry το επιλύει με βάση το mode:

```

if key in LEGACY_MODEL_ALIASES:
    legacy_by_mode = LEGACY_MODEL_ALIASES[key]
    if normalized_mode not in legacy_by_mode:
        raise ValueError(f"Legacy model_id '{model_id}' is not available
for mode '{normalized_mode}'.")
    return legacy_by_mode[normalized_mode]

```

Αυτό είναι ιδιαίτερα χρήσιμο γιατί επιτρέπει στο API να δέχεται απλούστερα ή παλαιότερα model IDs, αλλά να καταλήγει πάντα σε συγκεκριμένη και σωστή επιλογή μοντέλου.

Ανάκτηση configuration μοντέλου

Η συνάρτηση `get_model_config` επιστρέφει το πλήρες configuration ενός μοντέλου:

```

def get_model_config(model_id: str, mode: Optional[str] = None,
optimization: Optional[str] = None) -> Dict[str, Any]:
    resolved_id = resolve_model_id(model_id, mode=mode,
optimization=optimization)
    cfg = dict(MODEL_REGISTRY[resolved_id])

```

```

cfg["model_id"] = resolved_id
cfg["resolved_model_id"] = resolved_id
return cfg

```

Αρχικά επιλύει το `model_id` και στη συνέχεια επιστρέφει αντίγραφο των μεταδεδομένων του μοντέλου. Προσθέτει επίσης τα πεδία `model_id` και `resolved_model_id`, ώστε το API να γνωρίζει ποιο συγκεκριμένο μοντέλο επιλέχθηκε τελικά.

Έλεγχος συμβατότητας μοντέλου και mode

Η συνάρτηση `validate_model_for_mode` ελέγχει αν ένα μοντέλο μπορεί να χρησιμοποιηθεί στο ζητούμενο `mode`:

```

def validate_model_for_mode(model_id: str, mode: Optional[str]) -> Dict[str, Any]:
    normalized_mode = normalize_mode(mode)
    cfg = get_model_config(model_id, mode=normalized_mode)

    if normalized_mode not in cfg.get("supports", []):
        raise ValueError(
            f"Model '{cfg['model_id']}' does not support mode '{normalized_mode}'. "
            f"Supported modes: {cfg.get('supports', [])}"
        )

```

Αν το μοντέλο δεν υποστηρίζει το συγκεκριμένο `mode`, επιστρέφεται σφάλμα. Για παράδειγμα, ένα `no_history` μοντέλο δεν πρέπει να χρησιμοποιηθεί σε `with_history` πρόβλεψη.

Η ίδια συνάρτηση ελέγχει και αν το μοντέλο απαιτεί `home_id` από τον χρήστη:

```

if cfg.get("requires_user_home_id", False):
    raise ValueError(
        f"Model '{cfg['model_id']}' requires user home_id and is not allowed in the generic UI."
    )

```

Αυτός ο έλεγχος είναι κρίσιμος για την generic εφαρμογή. Το UI δεν ζητά `home_id` από τον χρήστη, άρα δεν πρέπει να επιτραπεί μοντέλο που το απαιτεί. Με αυτόν τον τρόπο, το registry λειτουργεί και ως μηχανισμός ασφάλειας της αρχιτεκτονικής.

Λίστα διαθέσιμων μοντέλων

Η συνάρτηση `list_models` επιστρέφει τα διαθέσιμα μοντέλα, με δυνατότητα φιλτραρίσματος ανά `mode`:

```

def list_models(
    mode: Optional[str] = None,
    ui_visible_only: bool = True,
    include_optional: bool = True,
) -> Dict[str, Dict[str, Any]]:

```

Αν δοθεί `mode`, επιστρέφονται μόνο τα μοντέλα που το υποστηρίζουν. Αν το `ui_visible_only` είναι ενεργό, επιστρέφονται μόνο τα μοντέλα που πρέπει να εμφανίζονται στη γραφική διεπαφή. Αν το `include_optional` είναι `False`, αποκλείονται τα προαιρετικά μοντέλα.

Αυτή η συνάρτηση χρησιμοποιείται από το API και το UI για να εμφανίζουν στον χρήστη τις διαθέσιμες επιλογές μοντέλων χωρίς να χρειάζεται να γνωρίζουν τη δομή του registry.

Επιλογές για τη γραφική διεπαφή

Η συνάρτηση `list_model_options_for_ui` επιστρέφει compact επιλογές κατάλληλες για dropdown:

```

def list_model_options_for_ui(mode: Optional[str] = None) -> List[Dict[str, Any]]:
    """Return compact dropdown-friendly model options."""
    models = list_models(mode=mode, ui_visible_only=True, include_optional=True)

```

Η πρώτη επιλογή που προστίθεται είναι το Auto default:

```

options: List[Dict[str, Any]] = [

```

```

{
    "label": "Auto default",
    "value": "auto",
    "is_default": True,
    "is_optional": False,
    "description": "Automatically select the default model for the
selected scenario.",
}
]

```

Στη συνέχεια προστίθενται τα μοντέλα που είναι διαθέσιμα για το επιλεγμένο mode. Κάθε επιλογή περιλαμβάνει label, value, mode, αν είναι default, αν είναι optional και περιγραφή. Με αυτόν τον τρόπο, το UI μπορεί να δημιουργήσει δυναμικά τις επιλογές μοντέλων, χωρίς να έχει σκληροκωδικοποιημένα ονόματα.

Έλεγχος ύπαρξης artifacts

Η συνάρτηση `check_artifact_paths` ελέγχει αν υπάρχουν οι αναμενόμενοι φάκελοι και τα βασικά αρχεία κάθε μοντέλου:

```

def check_artifact_paths() -> Dict[str, Dict[str, Any]]:
    """Check whether expected artifact folders and core files exist."""
    result: Dict[str, Dict[str, Any]] = {}

    for model_id, cfg in MODEL_REGISTRY.items():
        artifact_dir = Path(cfg["artifact_dir"])
        model_path = artifact_dir / "model.joblib"
        preprocessor_path = artifact_dir / "preprocessor.pkl"
        feature_config_path = artifact_dir / "feature_config.json"
        metadata_path = artifact_dir / "metadata.json"

```

Για κάθε μοντέλο ελέγχεται αν υπάρχει ο φάκελος artifacts, το αρχείο του μοντέλου, ο preprocessor, το feature configuration και τα metadata. Το αποτέλεσμα περιλαμβάνει και ένα πεδίο ready:

```

"ready": all([
    artifact_dir.exists(),
    model_path.exists(),
    preprocessor_path.exists(),
    feature_config_path.exists(),
])

```

Το πεδίο αυτό δείχνει αν το μοντέλο είναι πρακτικά έτοιμο για χρήση από το API. Η συνάρτηση αυτή είναι χρήσιμη για health checks, debugging και έλεγχο εγκατάστασης της εφαρμογής.

Inference Utilities για το API. Αρχείο: inference.py

Inference Layer. Αρχείο: inference.py

Το αρχείο `inference.py` αποτελεί τον πυρήνα της διαδικασίας πρόβλεψης της τελικής εφαρμογής. Ενώ τα training scripts είναι υπεύθυνα για την εκπαίδευση και αποθήκευση των μοντέλων, το `inference.py` είναι υπεύθυνο για τη χρήση αυτών των μοντέλων σε πραγματικά αιτήματα πρόβλεψης. Με άλλα λόγια, το συγκεκριμένο αρχείο μετατρέπει τα αποθηκευμένα artifacts σε λειτουργική υπηρεσία day-ahead πρόβλεψης.

Η βασική παραδοχή της τελικής εφαρμογής είναι ότι ο χρήστης δεν παρέχει `home_id`. Τα μοντέλα επιλέγονται μέσω του `model_registry.py`, ενώ κάθε φάκελος artifacts περιλαμβάνει το εκπαιδευμένο μοντέλο, τον preprocessor, το αρχείο χαρακτηριστικών και τα μεταδεδομένα. Το `inference.py` χρησιμοποιεί αυτά τα αρχεία για να δημιουργήσει τα κατάλληλα χαρακτηριστικά εισόδου και να επιστρέψει τις 24 ωριαίες προβλέψεις κατανάλωσης.

Το αρχείο υποστηρίζει δύο βασικά σενάρια πρόβλεψης: `no_history` και `with_history`. Το πρώτο χρησιμοποιείται όταν ο χρήστης δεν διαθέτει πρόσφατο ιστορικό κατανάλωσης, ενώ

το δεύτερο χρησιμοποιείται όταν υπάρχει διαθέσιμο πρόσφατο ωριαίο ιστορικό. Για λόγους συμβατότητας, το script εξακολουθεί να υποστηρίζει παλαιότερες ονομασίες όπως coldstart, cold_start και withhistory, όμως η τελική ορολογία της εφαρμογής βασίζεται στα canonical modes no_history και with_history.

Εισαγωγή βιβλιοθηκών και σύνδεση με το model registry

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import json
from collections import deque
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple

import joblib
import numpy as np
import pandas as pd
```

Η json χρησιμοποιείται για τη φόρτωση αρχείων ρυθμίσεων και μεταδεδομένων. Η deque χρησιμοποιείται στη recursive πρόβλεψη με ιστορικό, επειδή επιτρέπει την αποδοτική ενημέρωση της σειράς ιστορικών τιμών καθώς προστίθενται οι προβλεπόμενες ώρες. Η Path χρησιμοποιείται για διαχείριση διαδρομών, ενώ οι τύποι Any, Dict, Iterable, List, Optional και Tuple βοηθούν στην τεκμηρίωση των συναρτήσεων.

Η joblib χρησιμοποιείται για τη φόρτωση των αποθηκευμένων μοντέλων και preprocessors. Η numpy και η pandas χρησιμοποιούνται για αριθμητικούς υπολογισμούς, δημιουργία χαρακτηριστικών και διαχείριση χρονοσειρών.

Στη συνέχεια το script επιχειρεί να εισαγάγει βασικές συναρτήσεις και σταθερές από το model_registry.py:

```
from src.model_registry import (
    DEFAULT_MODE,
    MODE_NO_HISTORY,
    MODE_WITH_HISTORY,
    normalize_mode,
    validate_model_for_mode,
)
```

Αν η εισαγωγή από το src.model_registry δεν λειτουργήσει, υπάρχει fallback ώστε να γίνει εισαγωγή από τοπικό model_registry. Επιπλέον, υπάρχει και δεύτερο fallback για παλαιότερες εκδόσεις του registry που χρησιμοποιούσαν τις ονομασίες MODE_COLDSTART και MODE_WITHHISTORY. Αυτή η δομή κάνει το inference.py πιο ανθεκτικό σε αλλαγές του project structure και διατηρεί συμβατότητα με προηγούμενες εκδόσεις.

Σταθερές και βασικές διαδρομές

Στη συνέχεια ορίζονται οι βασικές διαδρομές και σταθερές:

```
DEFAULT_BASE_DIR = Path("C:/IDEAL_Programming")
DEFAULT_OUT_DIR = DEFAULT_BASE_DIR / "processed" / "predictions" /
"api_like"
DEFAULT_HOLIDAYS_CSV = DEFAULT_BASE_DIR / "metadata" / "holidays.csv"
```

```
TIMESTAMP_COL = "timestamp"
TARGET_COL = "consumption_Wh"
PRED_COL = "pred_consumption_Wh"
PSEUDO_HOME_ID = "ui_user"
UNKNOWN_LABEL = "unknown"
```

Το DEFAULT_BASE_DIR δείχνει στον βασικό φάκελο του έργου. Το DEFAULT_OUT_DIR χρησιμοποιείται όταν οι προβλέψεις πρέπει να αποθηκευτούν σε CSV. Το DEFAULT_HOLIDAYS_CSV δείχνει στο αρχείο αργιών, το οποίο χρησιμοποιείται για τη δημιουργία του χαρακτηριστικού is_holiday.

Οι σταθερές `TIMESTAMP_COL`, `TARGET_COL` και `PRED_COL` ορίζουν τα βασικά ονόματα στηλών που χρησιμοποιούνται στο inference. Το `PSEUDO_HOME_ID` χρησιμοποιείται εσωτερικά μόνο για βοηθητικές λειτουργίες των optional no-history μοντέλων με KNN ή behavioral profiles. Δεν αντιστοιχεί σε πραγματικό `home_id` και δεν ζητείται από τον χρήστη.

Παράμετροι **adaptive history correction**

Το script ορίζει επίσης παραμέτρους για την προαιρετική διόρθωση με βάση το πρόσφατο ιστορικό:

```
DEFAULT_HISTORY_DAYS_FOR_CORRECTION = 7
DEFAULT_HISTORY_CORRECTION_MAX_ALPHA = 0.20
DEFAULT_HISTORY_CORRECTION_MIN_HOURLY_OBS = 3
DEFAULT_HISTORY_CORRECTION_MIN_TOTAL_ROWS = 120
DEFAULT_HISTORY_COVERAGE_THRESHOLD = 0.85
```

Οι παράμετροι αυτές χρησιμοποιούνται στο `with_history` σενάριο. Η διόρθωση βασίζεται σε πρόσφατο ιστορικό επτά ημερών και εφαρμόζεται με μέγιστο συντελεστή α ίσο με 0.20. Αυτό σημαίνει ότι η διόρθωση είναι ελεγχόμενη και δεν αντικαθιστά πλήρως την πρόβλεψη του μοντέλου. Η απαίτηση ελάχιστων παρατηρήσεων και ελάχιστης κάλυψης αποτρέπει την εφαρμογή της διόρθωσης όταν το ιστορικό είναι πολύ ελλιπές ή μη αξιόπιστο.

Φόρτωση αργιών

Η συνάρτηση `load_holidays_set` φορτώνει τις αργίες από αρχείο CSV:

```
def _load_holidays_set(holidays_csv: Path = DEFAULT_HOLIDAYS_CSV) -> set:
    holidays_csv = Path(holidays_csv)
    if not holidays_csv.exists():
        return set()

    hdf = pd.read_csv(holidays_csv)
    if "date" not in hdf.columns:
        return set()

    dates = pd.to_datetime(hdf["date"], errors="coerce").dt.date.dropna()
    return set(dates.tolist())
```

Αν το αρχείο αργιών δεν υπάρχει ή δεν περιέχει στήλη `date`, η συνάρτηση επιστρέφει κενό σύνολο. Αν υπάρχει, μετατρέπει τις ημερομηνίες σε `date` και τις αποθηκεύει σε `set` για γρήγορο έλεγχο. Το αποτέλεσμα αποθηκεύεται στη μεταβλητή `_HOLIDAYS_SET` και χρησιμοποιείται κατά τη δημιουργία των χρονικών χαρακτηριστικών.

Φόρτωση **artifacts** μοντέλου

Μία από τις βασικές συναρτήσεις του αρχείου είναι η `load_model_artifacts`:

```
def load_model_artifacts(
    model_id: Optional[str] = "auto",
    mode: Optional[str] = DEFAULT_MODE,
    optimization: Optional[str] = "balanced",
) -> Dict[str, Any]:
```

Η συνάρτηση αυτή φορτώνει όλα τα αρχεία που απαιτούνται για να γίνει πρόβλεψη με ένα συγκεκριμένο μοντέλο. Αρχικά χρησιμοποιεί το `validate_model_for_mode` από το `model_registry.py`, ώστε να ελεγχθεί αν το ζητούμενο μοντέλο είναι συμβατό με το επιλεγμένο `mode`:

```
cfg = validate_model_for_mode(model_id or "auto", mode)
artifact_dir = Path(cfg["artifact_dir"])
```

Στη συνέχεια ορίζονται οι βασικές διαδρομές των `artifacts`:

```
model_path = artifact_dir / "model.joblib"
preprocessor_path = artifact_dir / "preprocessor.pkl"
feature_config_path = artifact_dir / "feature_config.json"
metadata_path = artifact_dir / "metadata.json"
```

Το script ελέγχει ότι υπάρχουν τα βασικά αρχεία:

```

if not model_path.exists():
    raise FileNotFoundError(f"Missing model file: {model_path}")
if not preprocessor_path.exists():
    raise FileNotFoundError(f"Missing preprocessor file: {preprocessor_path}")
if not feature_config_path.exists():
    raise FileNotFoundError(f"Missing feature_config file: {feature_config_path}")

```

Τα τρία υποχρεωτικά αρχεία είναι το model.joblib, το preprocessor.pkl και το feature_config.json. Το metadata.json είναι προαιρετικό αλλά χρήσιμο, καθώς περιέχει πληροφορίες για την εκπαίδευση και την αξιολόγηση.

Η φόρτωση των βασικών αντικειμένων γίνεται ως εξής:

```

model = joblib.load(model_path)
preprocessor = joblib.load(preprocessor_path)
feature_config = _load_json_if_exists(feature_config_path, {})
metadata = _load_json_if_exists(metadata_path, {})

```

Επιπλέον, αν υπάρχουν, φορτώνονται και προαιρετικά αρχεία:

```

global_daily_calibrator = _load_json_if_exists(
    artifact_dir / "global_daily_calibrator.json",
    None,
)

```

```

behavior_profiles = _safe_joblib_load(artifact_dir /
    "behavior_profiles.joblib", default=None)
knn_bundle = _safe_joblib_load(artifact_dir / "knn_bundle.joblib",
    default=None)

```

Το global_daily_calibrator.json χρησιμοποιείται για προαιρετική ημερήσια βαθμονόμηση. Τα behavior_profiles.joblib και knn_bundle.joblib χρησιμοποιούνται από τα optional no-history μοντέλα LightGBM και XGBoost, τα οποία αξιοποιούν συμπεριφορικά και KNN profiles.

Η συνάρτηση επιστρέφει ένα ενιαίο dictionary που περιλαμβάνει το μοντέλο, τον preprocessor, το configuration χαρακτηριστικών, τα metadata και τη registry πληροφορία:

```

return {
    "model": model,
    "preprocessor": preprocessor,
    "feature_config": feature_config,
    "metadata": metadata,
    "registry_config": cfg,
    "model_id": cfg["model_id"],
    "mode": cfg["mode"],
    "model_type": cfg.get("type"),
    "artifact_dir": str(artifact_dir),
    "global_daily_calibrator": global_daily_calibrator,
    "behavior_profiles": behavior_profiles,
    "knn_bundle": knn_bundle,
}

```

Με αυτόν τον τρόπο, οι επόμενες συναρτήσεις δεν χρειάζεται να γνωρίζουν πού βρίσκονται τα αρχεία του μοντέλου. Λαμβάνουν ένα έτοιμο artifact bundle με όλα τα απαραίτητα στοιχεία.

Δημιουργία χρονικών χαρακτηριστικών

Η συνάρτηση time_features δημιουργεί τα χρονικά χαρακτηριστικά για κάθε timestamp:

```

def time_features(ts: pd.Timestamp) -> Dict[str, Any]:
    ts = pd.Timestamp(ts)
    dow = ts.dayofweek

    return {
        "hour": int(ts.hour),
        "day_of_week": int(dow),
    }

```

```

"month": int(ts.month),
"day_of_month": int(ts.day),
"week_of_year": int(ts.isocalendar().week),
"is_weekend": int(dow >= 5),
"is_holiday": int(ts.date() in _HOLIDAYS_SET),
"hour_sin": float(np.sin(2 * np.pi * ts.hour / 24)),
"hour_cos": float(np.cos(2 * np.pi * ts.hour / 24)),
"day_sin": float(np.sin(2 * np.pi * dow / 7)),
"day_cos": float(np.cos(2 * np.pi * dow / 7)),
"month_sin": float(np.sin(2 * np.pi * ts.month / 12)),
"month_cos": float(np.cos(2 * np.pi * ts.month / 12)),
}

```

Η συνάρτηση επιστρέφει απλά χρονικά χαρακτηριστικά, όπως ώρα, ημέρα εβδομάδας, μήνα, ημέρα μήνα, εβδομάδα έτους, ένδειξη Σαββατοκύριακου και ένδειξη αργίας. Επιπλέον, δημιουργεί κυκλικά χαρακτηριστικά για την ώρα, την ημέρα και τον μήνα. Τα κυκλικά χαρακτηριστικά βοηθούν το μοντέλο να αντιληφθεί τη φυσική περιοδικότητα του χρόνου.

Η δημιουργία των 24 χρονικών στιγμών της ημέρας στόχου γίνεται από τη συνάρτηση `_build_24h_timestamps`:

```

def _build_24h_timestamps(target_date: str) -> pd.DatetimeIndex:
    start = pd.Timestamp(target_date).normalize()
    return pd.date_range(start=start, periods=24, freq="h")

```

Η συνάρτηση αυτή δημιουργεί τις 24 ωριαίες χρονικές στιγμές από τις 00:00 έως τις 23:00 της ημερομηνίας στόχου.

Έλεγχος και κανονικοποίηση διανυσμάτων εισόδου

Το inference layer χρειάζεται συχνά διανύσματα 24 τιμών, όπως 24 τιμές εξωτερικής ή εσωτερικής θερμοκρασίας. Για τον σκοπό αυτό χρησιμοποιούνται οι συναρτήσεις `_ensure_len` και `_normalize_24h_numeric`:

```

def _ensure_len(arr: Iterable[Any], n: int, name: str) -> List[Any]:
    values = list(arr)
    if len(values) != n:
        raise ValueError(f"{name} must have length {n}, but has {len(values)}.")
    return values

```

Η συνάρτηση αυτή ελέγχει ότι το διάνυσμα έχει το σωστό μήκος. Αν, για παράδειγμα, δοθούν λιγότερες ή περισσότερες από 24 θερμοκρασίες, επιστρέφεται σφάλμα.

Η κανονικοποίηση αριθμητικών 24ωρων διανυσμάτων γίνεται ως εξής:

```

def _normalize_24h_numeric(values, name: str, default: float = np.nan) -> List[float]:
    if values is None:
        return [float(default)] * 24
    values = _ensure_len(values, 24, name)
    return [float(x) if pd.notna(x) else float(default) for x in values]

```

Αν δεν δοθούν τιμές, επιστρέφεται λίστα 24 τιμών με default τιμή. Αν δοθούν τιμές, ελέγχεται το μήκος και μετατρέπονται σε float. Αυτή η λογική διασφαλίζει ότι το μοντέλο λαμβάνει πάντα είσοδο με σωστή 24ωρη μορφή.

Στατικά χαρακτηριστικά κατοικίας

Η συνάρτηση `_base_static_features` δημιουργεί τα βασικά στατικά χαρακτηριστικά της κατοικίας:

```

def _base_static_features(
    total_floor_area_m2: float,
    residents: float,
    hometype: str,
    urban_rural_class: str,
    num_electric_appliances: Optional[float] = None,
) -> Dict[str, Any]:

```

```

    return {
        "total_floor_area_m2": _as_float_or_nan(total_floor_area_m2),
        "residents": _as_float_or_nan(residents),
        "num_electric_appliances":
            _as_float_or_nan(num_electric_appliances),
        "hometype": str(hometype) if hometype is not None else
            UNKNOWN_LABEL,
        "urban_rural_class": str(urban_rural_class) if urban_rural_class
            is not None else UNKNOWN_LABEL,
    }

```

Η συνάρτηση επιστρέφει dictionary με το εμβαδόν, τον αριθμό κατοίκων, τον αριθμό ηλεκτρικών συσκευών, τον τύπο κατοικίας και την κατηγορία περιοχής. Οι αριθμητικές τιμές μετατρέπονται σε float ή NaN, ενώ οι κατηγορικές τιμές μετατρέπονται σε string. Αν κάποια κατηγορική τιμή λείπει, χρησιμοποιείται η τιμή unknown.

Επαναφορά προβλέψεων στην αρχική κλίμακα

Τα μοντέλα έχουν εκπαιδευτεί συχνά σε λογαριθμικά μετασχηματισμένη μεταβλητή στόχο. Για τον λόγο αυτό, το inference layer πρέπει να επαναφέρει τις προβλέψεις στην αρχική κλίμακα Wh. Αυτό γίνεται με τη συνάρτηση `_inverse_target_pred`:

```

def _inverse_target_pred(y_pred, feature_config: Dict[str, Any]) ->
    np.ndarray:
    y = np.asarray(y_pred, dtype=np.float64)

    if feature_config.get("use_log_target", False):
        y = np.expml(y)

    y = np.maximum(y, 0.0)
    return y.astype("float32")

```

Αν στο `feature_config` δηλώνεται ότι χρησιμοποιήθηκε λογαριθμικός μετασχηματισμός, εφαρμόζεται `expml`. Στη συνέχεια οι αρνητικές προβλέψεις περιορίζονται στο μηδέν, επειδή η κατανάλωση δεν μπορεί να είναι αρνητική. Το αποτέλεσμα επιστρέφεται σε Wh.

Πρόβλεψη με σεβασμό στο best iteration του XGBoost

Η συνάρτηση `_predict_raw_model` εκτελεί την πρόβλεψη του μοντέλου:

```

def _predict_raw_model(model, X, model_type: Optional[str] = None) ->
    np.ndarray:
    """Predict while respecting XGBoost best_iteration when available."""
    if model_type == "xgboost":
        best_iteration = getattr(model, "best_iteration", None)
        if best_iteration is not None:
            try:
                return model.predict(X, iteration_range=(0,
                    int(best_iteration) + 1))
            except TypeError:
                pass

    return model.predict(X)

```

Για τα XGBoost μοντέλα, αν υπάρχει διαθέσιμο `best_iteration`, χρησιμοποιείται μόνο το βέλτιστο εύρος επαναλήψεων. Αυτό είναι σημαντικό όταν το μοντέλο έχει εκπαιδευτεί με early stopping. Για τα υπόλοιπα μοντέλα, η συνάρτηση καλεί απλώς τη μέθοδο `predict`.

Προαιρετική ημερήσια βαθμονόμηση

Η συνάρτηση `_apply_daily_calibrator_if_requested` εφαρμόζει προαιρετική ημερήσια διόρθωση:

```

def _apply_daily_calibrator_if_requested(
    preds: np.ndarray,
    artifacts: Dict[str, Any],
    use_daily_calibrated: bool = False,
) -> np.ndarray:

```

Αν η επιλογή `use_daily_calibrated` είναι απενεργοποιημένη, οι προβλέψεις επιστρέφονται χωρίς αλλαγή, αφού πρώτα περιοριστούν στο μηδέν. Αν είναι ενεργοποιημένη και υπάρχει διαθέσιμος `global daily calibrator`, εφαρμόζεται ένας συντελεστής κλίμακας:

```
calibrator = artifacts.get("global_daily_calibrator")
if not calibrator or calibrator.get("type") != "global_scale":
    return np.maximum(preds, 0.0).astype("float32")
```

```
scale = float(calibrator.get("scale", 1.0))
return np.maximum(preds * scale, 0.0).astype("float32")
```

Η λειτουργία αυτή είναι προαιρετική και αφορά κυρίως `generic with-history` μοντέλα που έχουν αποθηκευμένο `global_daily_calibrator.json`.

Προετοιμασία χαρακτηριστικών για τον preprocessor

Το script πρέπει να εξασφαλίζει ότι τα χαρακτηριστικά που δίνονται στον `preprocessor` έχουν την ίδια μορφή με αυτή που είχε χρησιμοποιηθεί κατά την εκπαίδευση. Για τον σκοπό αυτό χρησιμοποιείται η συνάρτηση `_prepare_feature_frame_for_preprocessor`:

```
def _prepare_feature_frame_for_preprocessor(
    df_feat: pd.DataFrame,
    feature_config: Dict[str, Any],
) -> pd.DataFrame:
    """Ensure all expected feature columns exist before
    preprocessor.transform."""
    df = df_feat.copy()
    numeric_cols, categorical_cols =
    _feature_columns_from_config(feature_config)
```

Η συνάρτηση διαβάζει από το `feature_config` ποιες αριθμητικές και κατηγορικές στήλες αναμένει το μοντέλο. Για κάθε αριθμητική στήλη που λείπει, δημιουργείται με `NaN`:

```
for c in numeric_cols:
    if c not in df.columns:
        df[c] = np.nan
    df[c] = pd.to_numeric(df[c], errors="coerce")
```

Για κάθε κατηγορική στήλη που λείπει, δημιουργείται με την τιμή `unknown`:

```
for c in categorical_cols:
    if c not in df.columns:
        df[c] = UNKNOWN_LABEL
    df[c] = df[c].fillna(UNKNOWN_LABEL).astype(str).replace({"nan":
UNKNOWN_LABEL, "None": UNKNOWN_LABEL})
```

Στο τέλος επιστρέφονται οι στήλες με τη σωστή σειρά:

```
return df[numeric_cols + categorical_cols]
```

Αυτό είναι κρίσιμο για την αξιοπιστία του `inference`. Το μοντέλο πρέπει να λάβει τα χαρακτηριστικά στην ίδια δομή, με τα ίδια ονόματα και την ίδια σειρά που είχε κατά την εκπαίδευση.

Η πραγματική μετατροπή γίνεται με τη συνάρτηση `_transform_features`:

```
def _transform_features(artifacts: Dict[str, Any], df_feat: pd.DataFrame):
    feature_config = artifacts["feature_config"]
    preprocessor = artifacts["preprocessor"]
    X_df = _prepare_feature_frame_for_preprocessor(df_feat, feature_config)
    return preprocessor.transform(X_df)
```

Εδώ χρησιμοποιείται ο αποθηκευμένος `preprocessor` του μοντέλου, διασφαλίζοντας ότι η προεπεξεργασία στο `inference` είναι ίδια με την προεπεξεργασία στο `training`.

Behavior και KNN profiles για optional no-history μοντέλα

Τα προαιρετικά `no-history` μοντέλα `LightGBM` και `XGBoost` μπορούν να χρησιμοποιούν `behavior` και `KNN profiles`. Για τον λόγο αυτό, το `inference.py` περιλαμβάνει βοηθητικές συναρτήσεις που ανακατασκευάζουν αυτά τα χαρακτηριστικά.

Αρχικά δημιουργούνται στατικές κατηγορίες:

```
def _add_static_bins(df: pd.DataFrame) -> pd.DataFrame:
```

```

df = df.copy()

r = pd.to_numeric(df.get("residents"), errors="coerce")
df["residents_bin"] = np.select(
    [r.isna(), r <= 0, r == 1, r == 2, r == 3, r == 4, r >= 5],
    [UNKNOWN_LABEL, UNKNOWN_LABEL, "1", "2", "3", "4", "5plus"],
    default=UNKNOWN_LABEL,
)

```

Η συνάρτηση αυτή δημιουργεί κατηγορίες για τον αριθμό κατοίκων, το εμβαδόν και τον αριθμό ηλεκτρικών συσκευών. Έπειτα δημιουργεί το `static_profile_key`, το οποίο χρησιμοποιείται για την αντιστοίχιση με τα `behavior profiles` που είχαν αποθηκευτεί κατά την εκπαίδευση.

Η συνάρτηση `_add_behavior_profile` προσθέτει την αναμενόμενη κατανάλωση από τα `behavioral profiles`:

```

def _add_behavior_profile(df_part: pd.DataFrame, profiles:
Optional[Dict[str, Any]]) -> pd.DataFrame:
    out = df_part.copy()

    if not profiles:
        out["expected_behavior_Wh"] = np.nan
        out["expected_behavior_loglp"] = np.nan
        out["behavior_profile_source"] = "missing"
        return out

```

Αν δεν υπάρχουν διαθέσιμα `profiles`, δημιουργούνται κενές τιμές. Αν υπάρχουν, το `dataframe` συγχωνεύεται με διαφορετικά επίπεδα προφίλ, όπως `static_dow_hour`, `static_hour`, `static_overall`, `global_dow_hour` και `global_hour`. Έπειτα εφαρμόζεται `fallback` λογική ώστε να χρησιμοποιηθεί το πιο εξειδικευμένο διαθέσιμο προφίλ.

Η συνάρτηση `_add_knn_behavior_profile` προσθέτει τα KNN-based χαρακτηριστικά:

```

def _add_knn_behavior_profile(df_part: pd.DataFrame, knn_bundle:
Optional[Dict[str, Any]]) -> pd.DataFrame:
    out = df_part.copy()

    if not knn_bundle:
        out["knn_expected_dow_hour_Wh"] = np.nan
        out["knn_expected_hour_Wh"] = np.nan
        out["knn_expected_overall_Wh"] = np.nan
        out["knn_expected_Wh"] = out.get("expected_behavior_Wh", np.nan)
        out["knn_expected_loglp"] =
np.loglp(out["knn_expected_Wh"].clip(lower=0))
        out["knn_minus_behavior_Wh"] = out["knn_expected_Wh"] -
out.get("expected_behavior_Wh", 0.0)
        return out

```

Αν υπάρχει διαθέσιμο KNN bundle, το script βρίσκει τις στατικά παρόμοιες κατοικίες και χρησιμοποιεί τα αντίστοιχα προφίλ κατανάλωσης. Η λειτουργία αυτή εφαρμόζεται μόνο εσωτερικά. Ο χρήστης δεν παρέχει πραγματικό `home_id`, αλλά το σύστημα μπορεί να χρησιμοποιήσει το `PSEUDO_HOME_ID` ώστε να επαναχρησιμοποιήσει τη λογική των KNN helpers.

Η συνάρτηση `_maybe_add_cold_optional_profiles` ελέγχει αν το επιλεγμένο μοντέλο χρειάζεται `behavior` ή KNN `profiles`:

```

def _maybe_add_cold_optional_profiles(df_feat: pd.DataFrame, artifacts:
Dict[str, Any]) -> pd.DataFrame:
    cfg = artifacts.get("feature_config", {})

    if not cfg.get("uses_behavior_profiles", False) and not
cfg.get("uses_knn_profiles", False):
        return df_feat

```

Αν το μοντέλο δεν τα χρειάζεται, επιστρέφει τα χαρακτηριστικά χωρίς αλλαγή. Αν τα χρειάζεται, προσθέτει το pseudo home_id, τα static bins, τα behavioral profiles και τα KNN profiles.

Δημιουργία ιστορικών χαρακτηριστικών

Για το with_history mode, το inference layer πρέπει να δημιουργεί δυναμικά lag και rolling χαρακτηριστικά. Η συνάρτηση _history_value επιστρέφει τιμή από το ιστορικό με συγκεκριμένο lag:

```
def _history_value(hist_deque: deque, lag: int) -> float:
    if len(hist_deque) >= lag:
        return float(hist_deque[-lag])
    if len(hist_deque) > 0:
        return float(hist_deque[-1])
    return 0.0
```

Αν υπάρχει αρκετό ιστορικό, επιστρέφει την τιμή που αντιστοιχεί στο ζητούμενο lag. Αν όχι, επιστρέφει την πιο πρόσφατη διαθέσιμη τιμή. Αυτό λειτουργεί ως ασφαλές fallback.

Η συνάρτηση _add_recursive_history_features δημιουργεί όλα τα ιστορικά χαρακτηριστικά για μία ώρα πρόβλεψης:

```
def _add_recursive_history_features(row: Dict[str, Any], hist_deque: deque, feature_config: Dict[str, Any]) -> Dict[str, Any]:
    lag_hours = feature_config.get("lag_hours", [1, 24, 168])
    rolling_windows = feature_config.get("rolling_windows", [24, 168])
    rolling_extreme_windows = feature_config.get("rolling_extreme_windows", [24, 48, 168])
```

Η συνάρτηση διαβάζει από το feature_config ποια lags και ποια rolling windows πρέπει να δημιουργηθούν. Στη συνέχεια προσθέτει στο row τιμές όπως lag_1h, lag_24h, lag_168h, roll_mean_24h, roll_mean_168h, roll_std_24h, roll_min_24h και roll_max_24h.

Επιπλέον, δημιουργεί και παράγωγα χαρακτηριστικά:

```
row["lag_1h_minus_24h"] = float(row.get("lag_1h",
_history_value(hist_deque, 1))) - float(row.get("lag_24h",
_history_value(hist_deque, 24)))
row["lag_24h_minus_168h"] = float(row.get("lag_24h",
_history_value(hist_deque, 24))) - float(row.get("lag_168h",
_history_value(hist_deque, 168)))
row["roll_24h_div_168h"] = float(row.get("roll_mean_24h",
np.mean(_history_window_values(hist_deque, 24)))) /
(float(row.get("roll_mean_168h",
np.mean(_history_window_values(hist_deque, 168)))) + 1.0)
```

Τα χαρακτηριστικά αυτά πρέπει να συμφωνούν με τα χαρακτηριστικά που χρησιμοποιήθηκαν στην εκπαίδευση των generic with-history μοντέλων. Με αυτόν τον τρόπο το inference layer αναπαράγει τη λογική feature engineering που είχε εφαρμοστεί στο training.

Φόρτωση και προετοιμασία single-home history CSV

Η συνάρτηση load_single_home_history_csv φορτώνει ιστορικό από CSV μίας κατοικίας:

```
def load_single_home_history_csv(history_csv_path: Any) -> pd.DataFrame:
    """Load a single-home history CSV.
```

```
    Expected columns:
        timestamp
        consumption_Wh
```

```
    The file must refer to one household only. No home_id filtering is
    performed.
    """
```

Η συνάρτηση δέχεται είτε απλή διαδρομή αρχείου είτε αντικείμενο που μπορεί να προέρχεται από το Gradio upload component. Για αυτόν τον λόγο υποστηρίζει και περιπτώσεις όπου το input είναι dictionary ή αντικείμενο με attribute .name:

```

raw_path = history_csv_path
if isinstance(raw_path, dict):
    raw_path = raw_path.get("name") or raw_path.get("path") or
raw_path.get("orig_name")
elif hasattr(raw_path, "name"):
    raw_path = raw_path.name

```

Αν το αρχείο δεν υπάρχει, επιστρέφεται σφάλμα. Αν υπάρχει, φορτώνεται με pandas και περνά από τη συνάρτηση `_prepare_history_dataframe`.

Η `_prepare_history_dataframe` ελέγχει ότι υπάρχουν οι στήλες `timestamp` και `consumption_Wh`:

```

if TIMESTAMP_COL not in df.columns:
    raise ValueError(f"History CSV must contain column '{TIMESTAMP_COL}'.")

```

```

if TARGET_COL not in df.columns:
    raise ValueError(f"History CSV must contain column '{TARGET_COL}'.")

```

Στη συνέχεια μετατρέπει τις στήλες στους σωστούς τύπους, αφαιρεί μη έγκυρες γραμμές, περιορίζει αρνητικές τιμές στο μηδέν και, αν υπάρχουν διπλά timestamps, τα ομαδοποιεί κρατώντας τον μέσο όρο κατανάλωσης:

```

df = (
    df
    .groupby(TIMESTAMP_COL, as_index=False)
    .agg(**{TARGET_COL: (TARGET_COL, "mean")})
    .sort_values(TIMESTAMP_COL)
    .reset_index(drop=True)
)

```

Αυτό διασφαλίζει ότι το ιστορικό έχει καθαρή και συνεπή μορφή πριν χρησιμοποιηθεί για την πρόβλεψη.

Προετοιμασία ιστορικού για πρόβλεψη

Η συνάρτηση `prepare_history_for_prediction_from_dataframe` μετατρέπει το ιστορικό σε κανονικό ωριαίο παράθυρο πριν από την ημερομηνία στόχου:

```

def prepare_history_for_prediction_from_dataframe(
    history_df: pd.DataFrame,
    target_date: str,
    min_history_hours: int = 168,
    coverage_threshold: float = DEFAULT_HISTORY_COVERAGE_THRESHOLD,
) -> Tuple[List[float], pd.DataFrame, Dict[str, Any]]:

```

Αρχικά καθαρίζει το dataframe και κρατά μόνο τις γραμμές πριν από την ημερομηνία στόχου:

```

target_start = pd.Timestamp(target_date).normalize()
history_start = target_start - pd.Timedelta(hours=int(min_history_hours))

```

```

df = df[df[TIMESTAMP_COL] < target_start].copy()

```

Έπειτα μετατρέπει τις χρονικές σημάνσεις σε ωριαία βάση και δημιουργεί το απαιτούμενο ωριαίο index:

```

required_index = pd.date_range(
    start=history_start,
    end=target_start - pd.Timedelta(hours=1),
    freq="h",
)

```

Το ιστορικό επαναδεικτοδοτείται πάνω σε αυτό το ωριαίο index:

```

hourly = (
    hourly
    .set_index(TIMESTAMP_COL)
    .reindex(required_index)
)

```

Στη συνέχεια υπολογίζεται η κάλυψη του ιστορικού:

```
observed_count = int(hourly[TARGET_COL].notna().sum())
required_count = int(len(required_index))
coverage = observed_count / max(required_count, 1)
```

Αν η κάλυψη είναι χαμηλότερη από το προκαθορισμένο όριο, επιστρέφεται σφάλμα. Αυτό προστατεύει το with_history mode από χρήση πολύ ελλιπούς ιστορικού.

Αν η κάλυψη είναι επαρκής, τα κενά συμπληρώνονται με interpolation και forward/backward fill:

```
hourly[TARGET_COL] = (
    hourly[TARGET_COL]
    .astype(float)
    .interpolate(method="time", limit_direction="both")
    .ffill()
    .bfill()
    .clip(lower=0)
)
```

Η συνάρτηση επιστρέφει τη λίστα των τιμών ιστορικού, το regularized dataframe και διαγνωστικές πληροφορίες όπως η κάλυψη, οι ώρες που συμπληρώθηκαν και η συνολική κατανάλωση του ιστορικού.

Adaptive history-aware correction

Η λειτουργία adaptive history-aware correction υλοποιείται από τη συνάρτηση apply_adaptive_history_correction:

```
def apply_adaptive_history_correction(
    prediction_df: pd.DataFrame,
    history_df: pd.DataFrame,
    target_date: str,
    pred_col: str = PRED_COL,
    history_days: int = DEFAULT_HISTORY_DAYS_FOR_CORRECTION,
    max_alpha: float = DEFAULT_HISTORY_CORRECTION_MAX_ALPHA,
    min_hourly_obs: int = DEFAULT_HISTORY_CORRECTION_MIN_HOURLY_OBS,
) -> Tuple[pd.DataFrame, Dict[str, Any]]:
```

Η συνάρτηση αυτή εφαρμόζεται μόνο σε 24ωρη πρόβλεψη και μόνο όταν υπάρχει διαθέσιμο ιστορικό. Σκοπός της είναι να προσαρμόσει ήπια το ωριαίο σχήμα της πρόβλεψης με βάση το πρόσφατο ιστορικό της κατοικίας.

Αρχικά υπολογίζεται το άθροισμα της αρχικής πρόβλεψης:

```
base_pred = pd.to_numeric(out[pred_col],
errors="coerce").fillna(0).clip(lower=0).to_numpy(dtype=float)
pred_total = float(base_pred.sum())
```

Αν το σύνολο της πρόβλεψης είναι μηδενικό, η διόρθωση δεν εφαρμόζεται. Διαφορετικά, δημιουργείται πρόσφατο ωριαίο προφίλ ιστορικού από τις τελευταίες ημέρες:

```
profile, _, profile_info = _build_recent_history_profile(
    history_df=history_df,
    target_date=target_date,
    history_days=history_days,
)
```

Το ιστορικό προφίλ βασίζεται στη διάμεσο κατανάλωση ανά ώρα από το πρόσφατο ιστορικό. Στη συνέχεια το σχήμα αυτό κανονικοποιείται και κλιμακώνεται στο συνολικό ημερήσιο επίπεδο της αρχικής πρόβλεψης:

```
history_shape = median_profile / float(median_profile.sum())
history_shape_scaled = history_shape * pred_total
```

Αυτό είναι σημαντικό, επειδή η διόρθωση δεν εφαρμόζει νέο daily-level scaling. Η συνολική ημερήσια κατανάλωση παραμένει ουσιαστικά στο επίπεδο που εκτίμησε το μοντέλο, ενώ αλλάζει μόνο ήπια η κατανομή της κατανάλωσης μέσα στο 24ωρο.

Ο συντελεστής διόρθωσης alpha υπολογίζεται δυναμικά με βάση τη σταθερότητα του πρόσφατου ιστορικού:

```
cv = profile["history_std_7d_Wh"].to_numpy(dtype=float) /
(np.abs(median_profile) + 1.0)
```

```
stability = 1.0 - np.clip(cv / 1.0, 0.0, 1.0)
```

```
alpha = float(max_alpha) * stability
```

Αν μία ώρα εμφανίζει σταθερή συμπεριφορά στο πρόσφατο ιστορικό, τότε το alpha μπορεί να είναι μεγαλύτερο. Αν η συμπεριφορά είναι ασταθής, το alpha μειώνεται. Επίσης, αν δεν υπάρχουν αρκετές παρατηρήσεις για κάποια ώρα, το alpha μηδενίζεται.

Η τελική διορθωμένη πρόβλεψη υπολογίζεται ως σταθμισμένος συνδυασμός της αρχικής πρόβλεψης και του ιστορικού σχήματος:

```
corrected = (1.0 - alpha) * base_pred + alpha * history_shape_scaled  
corrected = np.maximum(corrected, 0.0)
```

Η συνάρτηση προσθέτει στο αποτέλεσμα επιπλέον στήλες, όπως pred_model_raw_Wh, history_median_7d_Wh, history_shape_scaled_Wh και adaptive_alpha. Έτσι το API και το UI μπορούν να εμφανίσουν όχι μόνο την τελική πρόβλεψη, αλλά και πληροφορίες για το αν εφαρμόστηκε διόρθωση.

Δημιουργία feature frame για no-history πρόβλεψη

Η συνάρτηση build_coldstart_feature_frame δημιουργεί τα 24 rows χαρακτηριστικών για το no_history mode:

```
def build_coldstart_feature_frame(  
    target_date: str,  
    external_temperature_24h,  
    total_floor_area_m2: float,  
    residents: float,  
    hometype: str,  
    urban_rural_class: str,  
    internal_temperature_24h=None,  
    num_electric_appliances: Optional[float] = None,  
) -> pd.DataFrame:
```

Παρότι η συνάρτηση διατηρεί το όνομα coldstart για λόγους συμβατότητας, στη νέα ορολογία αντιστοιχεί στο no_history mode. Η συνάρτηση δημιουργεί τις 24 χρονικές στιγμές της ημέρας στόχου, κανονικοποιεί τις θερμοκρασίες, δημιουργεί τα στατικά χαρακτηριστικά και προσθέτει τα χρονικά χαρακτηριστικά.

Για κάθε ώρα δημιουργείται ένα row:

```
rows.append({  
    TIMESTAMP_COL: ts,  
    "external_temperature": ext[i],  
    "internal_temperature": internal[i],  
    **static,  
    **_time_features(ts),  
})
```

Το αποτέλεσμα είναι dataframe 24 γραμμών, το οποίο μπορεί να περάσει από τον preprocessor και στη συνέχεια από το μοντέλο.

Recursive feature rows για with-history πρόβλεψη

Η συνάρτηση build_withhistory_feature_rows_recursive δημιουργεί τη βασική δομή για recursive πρόβλεψη με ιστορικό:

```
def build_withhistory_feature_rows_recursive(  
    feature_config: Dict[str, Any],  
    target_date: str,  
    external_temperature_24h,  
    history_consumption_Wh,  
    total_floor_area_m2: float,  
    residents: float,  
    hometype: str,  
    urban_rural_class: str,  
    internal_temperature_24h=None,  
    num_electric_appliances: Optional[float] = None,  
    min_history_hours: Optional[int] = None,
```

```
) -> Tuple[pd.DatetimeIndex, deque, List[Dict[str, Any]]]:
```

Η συνάρτηση αυτή ελέγχει ότι υπάρχει επαρκές ιστορικό και δημιουργεί deque με τις προηγούμενες τιμές κατανάλωσης. Παρότι δημιουργεί rows με ιστορικά χαρακτηριστικά, η πραγματική πρόβλεψη γίνεται μέσα στη συνάρτηση predict_withhistory_dayahead, όπου κάθε προβλεπόμενη τιμή προστίθεται στο ιστορικό πριν υπολογιστεί η επόμενη ώρα.

Πρόβλεψη no-history

Η δημόσια συνάρτηση predict_coldstart_dayahead υλοποιεί τη day-ahead πρόβλεψη χωρίς ιστορικό:

```
def predict_coldstart_dayahead(
    artifacts: Optional[Dict[str, Any]] = None,
    target_date: str = None,
    external_temperature_24h=None,
    total_floor_area_m2: float = None,
    residents: float = None,
    hometype: str = "unknown",
    urban_rural_class: str = "unknown",
    internal_temperature_24h=None,
    num_electric_appliances: Optional[float] = None,
    model_id: Optional[str] = "auto",
    optimization: Optional[str] = "balanced",
    save_csv: bool = False,
    out_dir: Path = DEFAULT_OUT_DIR,
    out_name: str = "pred_no_history_dayahead.csv",
) -> pd.DataFrame:
```

Η συνάρτηση αυτή διατηρεί το όνομα coldstart για συμβατότητα, αλλά ουσιαστικά αντιστοιχεί στο no_history σενάριο. Αν δεν δοθούν έτοιμα artifacts, φορτώνει αυτόματα το κατάλληλο μοντέλο:

```
if artifacts is None:
    artifacts = load_model_artifacts(model_id=model_id,
    mode=MODE_NO_HISTORY, optimization=optimization)
```

Στη συνέχεια ελέγχει ότι το artifact bundle αντιστοιχεί πράγματι σε no_history mode. Έπειτα δημιουργεί το feature frame, προσθέτει αν χρειάζεται optional behavior/KNN profiles, εφαρμόζει τον preprocessor και παράγει τις προβλέψεις:

```
X = _transform_features(artifacts, df_feat)
y_hat_tr = _predict_raw_model(artifacts["model"], X,
artifacts.get("model_type"))
y_hat = _inverse_target_pred(y_hat_tr, artifacts["feature_config"])
```

Το αποτέλεσμα επιστρέφεται ως dataframe με timestamp, πρόβλεψη, model_id, mode και prediction variant.

Πρόβλεψη with-history

Η συνάρτηση predict_withhistory_dayahead υλοποιεί τη day-ahead πρόβλεψη με ιστορικό:

```
def predict_withhistory_dayahead(
    artifacts: Optional[Dict[str, Any]] = None,
    target_date: str = None,
    external_temperature_24h=None,
    history_consumption_Wh=None,
    history_csv_path: Optional[Any] = None,
    history_df: Optional[pd.DataFrame] = None,
    ...
) -> pd.DataFrame:
```

Η συνάρτηση μπορεί να λάβει ιστορικό με τρεις τρόπους: ως dataframe, ως CSV path ή ως απλή λίστα τιμών κατανάλωσης. Η σειρά προτεραιότητας είναι:

history_df

→ history_csv_path

→ history_consumption_Wh

Αν δοθεί dataframe, αυτό κανονικοποιείται και μετατρέπεται σε regularized ωριαίο ιστορικό. Αν δοθεί CSV path, φορτώνεται και προετοιμάζεται το CSV. Αν δοθεί απλή λίστα, θεωρείται ότι η τελευταία τιμή αντιστοιχεί στην ώρα πριν από την ημερομηνία στόχου.

Η συνάρτηση ελέγχει ότι υπάρχει επαρκές ιστορικό:

```
if len(hist) < int(min_history_hours):
    raise ValueError(
        f"history must have at least {min_history_hours} hours. Received:
        {len(hist)}"
    )
```

Στη συνέχεια δημιουργείται deque με το ιστορικό:

```
hist_deque = deque([float(x) for x in hist], maxlen=200000)
```

Η πρόβλεψη γίνεται επαναληπτικά για κάθε μία από τις 24 ώρες. Για κάθε ώρα δημιουργείται row χαρακτηριστικών, προστίθενται τα ιστορικά χαρακτηριστικά, εφαρμόζεται ο preprocessor και παράγεται η πρόβλεψη:

```
for h, ts in enumerate(ts24):
    row = {
        TIMESTAMP_COL: ts,
        "external_temperature": ext[h],
        "internal_temperature": internal[h],
        **static,
        **_time_features(ts),
    }
    row = _add_recursive_history_features(row, hist_deque,
        feature_config)

    df_row = pd.DataFrame([row])
    X = _transform_features(artifacts, df_row)

    y_hat_tr = _predict_raw_model(artifacts["model"], X,
        artifacts.get("model_type"))
    y_hat = float(_inverse_target_pred(y_hat_tr, feature_config)[0])

    preds.append(y_hat)
    rows_used.append(row)
    hist_deque.append(y_hat)
```

Το πιο σημαντικό σημείο είναι το `hist_deque.append(y_hat)`. Κάθε προβλεπόμενη τιμή προστίθεται στο ιστορικό και χρησιμοποιείται για τον υπολογισμό των lag και rolling χαρακτηριστικών των επόμενων ωρών. Αυτό κάνει την πρόβλεψη autoregressive ή recursive.

Μετά τη δημιουργία των 24 προβλέψεων, μπορεί να εφαρμοστεί ημερήσια βαθμονόμηση και, αν είναι ενεργοποιημένη, adaptive history correction:

```
if apply_history_correction:
    out, correction_info = apply_adaptive_history_correction(
        prediction_df=out,
        history_df=history_regularized_df,
        target_date=target_date,
        pred_col=PRED_COL,
        history_days=int(history_correction_days),
        max_alpha=float(history_correction_max_alpha),
    )
```

Αν η διόρθωση εφαρμοστεί, το `prediction_variant` ενημερώνεται ώστε να φαίνεται ότι η τελική πρόβλεψη είναι adaptive history corrected. Αν δεν εφαρμοστεί, αποθηκεύεται ο λόγος μη εφαρμογής.

Η συνάρτηση προσθέτει επίσης μεταδεδομένα στο dataframe:

```
out["history_source"] = history_input_info.get("history_source",
    "unknown")
```

```

out["history_rows_used"] =
int(history_input_info.get("required_history_hours", len(hist)))
out["history_coverage"] = float(history_input_info.get("coverage", 1.0))
out["history_correction_reason"] =
correction_info.get("history_correction_reason", "")

```

Επιπλέον, αποθηκεύει compact diagnostics στα attrs του dataframe:

```

out.attrs["history_input_info"] = history_input_info
out.attrs["history_correction_info"] = correction_info

```

Τα στοιχεία αυτά μπορούν να αξιοποιηθούν από το API και το UI για να εμφανιστούν πληροφορίες σχετικά με την πηγή ιστορικού, την κάλυψη και τη διόρθωση.

Ενιαίο σημείο πρόβλεψης

Η συνάρτηση `predict_dayahead` αποτελεί το ενιαίο public entry point του inference layer:

```

def predict_dayahead(
    mode: str = MODE_NO_HISTORY,
    model_id: Optional[str] = "auto",
    optimization: Optional[str] = "balanced",
    target_date: str = None,
    external_temperature_24h=None,
    history_consumption_Wh=None,
    history_csv_path: Optional[Any] = None,
    history_df: Optional[pd.DataFrame] = None,
    ...
) -> pd.DataFrame:

```

Η συνάρτηση δέχεται το `mode`, το `model_id`, την ημερομηνία στόχου, τις θερμοκρασίες, τα στατικά χαρακτηριστικά και, αν χρειάζεται, το ιστορικό. Αρχικά κανονικοποιεί το `mode`:

```
normalized_mode = normalize_mode(mode)
```

Αν το `mode` είναι `no_history`, καλεί την `predict_coldstart_dayahead`:

```

if normalized_mode == MODE_NO_HISTORY:
    return predict_coldstart_dayahead(
        artifacts=None,
        target_date=target_date,
        external_temperature_24h=external_temperature_24h,
        ...
    )

```

Αν το `mode` είναι `with_history`, καλεί την `predict_withhistory_dayahead`:

```

if normalized_mode == MODE_WITH_HISTORY:
    return predict_withhistory_dayahead(
        artifacts=None,
        target_date=target_date,
        external_temperature_24h=external_temperature_24h,
        history_consumption_Wh=history_consumption_Wh,
        history_csv_path=history_csv_path,
        history_df=history_df,
        ...
    )

```

Με αυτόν τον τρόπο, το API μπορεί να χρησιμοποιεί μία ενιαία συνάρτηση για όλα τα σενάρια πρόβλεψης. Η εσωτερική λογική αποφασίζει ποια επιμέρους διαδικασία θα εκτελεστεί.

Στο τέλος υπάρχει και φιλικό `alias`:

```
predict_no_history_dayahead = predict_coldstart_dayahead
```

Αυτό επιτρέπει στον νέο κώδικα να χρησιμοποιεί την καθαρή ονομασία `predict_no_history_dayahead`, ενώ ταυτόχρονα διατηρείται η παλαιότερη συνάρτηση `predict_coldstart_dayahead` για συμβατότητα.

FastAPI Application. Αρχείο: `api_app.py`

Το αρχείο `api_app.py` υλοποιεί το backend της εφαρμογής πρόβλεψης ηλεκτρικής κατανάλωσης με χρήση FastAPI. Αποτελεί το κεντρικό σημείο επικοινωνίας ανάμεσα στη γραφική διεπαφή Gradio και στο inference layer. Ο ρόλος του δεν είναι να εκπαιδεύει μοντέλα, αλλά να δέχεται αιτήματα πρόβλεψης, να ελέγχει τις εισόδους, να επιλύει το κατάλληλο μοντέλο, να διαχειρίζεται τα μετεωρολογικά και ιστορικά δεδομένα και τελικά να επιστρέφει τη 24ωρη πρόβλεψη σε δομημένη μορφή.

Η τελική έκδοση του API έχει σχεδιαστεί για τη γενική έκδοση της εφαρμογής, όπου ο χρήστης δεν παρέχει `home_id`. Η επιλογή μοντέλου γίνεται μέσω του `model_registry.py`, ενώ η πραγματική παραγωγή πρόβλεψης γίνεται από το `inference.py`. Με αυτόν τον τρόπο, το `api_app.py` λειτουργεί ως επίπεδο ενοποίησης, συνδέοντας το UI, τα διαθέσιμα μοντέλα, τα ιστορικά δεδομένα και τις εξωτερικές θερμοκρασιακές εισόδους.

Το API υποστηρίζει δύο βασικά σενάρια πρόβλεψης: `no_history` και `with_history`. Στο `no_history` σενάριο, η πρόβλεψη γίνεται χωρίς πρόσφατες τιμές κατανάλωσης. Στο `with_history` σενάριο, η πρόβλεψη αξιοποιεί πρόσφατο ιστορικό ωριαίας κατανάλωσης, το οποίο μπορεί να δοθεί είτε ως λίστα τιμών είτε ως CSV μίας κατοικίας είτε, εναλλακτικά, από το legacy default history store.

Εισαγωγή βιβλιοθηκών

Το script ξεκινά με την εισαγωγή των απαραίτητων βιβλιοθηκών:

```
from __future__ import annotations

import math
import os
from pathlib import Path
from typing import Any, Dict, List, Literal, Optional, Tuple

import pandas as pd
import requests
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
```

Η βιβλιοθήκη `math` χρησιμοποιείται για τη δημιουργία συνθετικού ωριαίου θερμοκρασιακού προφίλ όταν δίνονται μόνο ελάχιστη και μέγιστη θερμοκρασία. Η `os` επιτρέπει την ανάγνωση μεταβλητών περιβάλλοντος, ενώ η `Path` χρησιμοποιείται για τη διαχείριση διαδρομών αρχείων. Η `pandas` χρησιμοποιείται για χειρισμό ημερομηνιών και αρχείων CSV, ενώ η `requests` χρησιμοποιείται για την ανάκτηση δεδομένων από εξωτερικές υπηρεσίες, όπως το Open-Meteo.

Η `FastAPI` χρησιμοποιείται για τη δημιουργία της εφαρμογής API, ενώ η `HTTPException` χρησιμοποιείται για την επιστροφή κατάλληλων σφαλμάτων HTTP. Η `BaseModel` και το `Field` της `Pydantic` χρησιμοποιούνται για τον ορισμό και την επικύρωση της δομής των αιτημάτων πρόβλεψης.

Σύνδεση με τα υπόλοιπα modules

Στη συνέχεια, το script εισάγει συναρτήσεις από τα υπόλοιπα βασικά αρχεία του συστήματος:

```
from src.history_store import get_history_from_csv, history_store_info
from src.inference import load_model_artifacts, predict_dayahead
from src.model_registry import (
    DEFAULT_MODE,
    DEFAULT_MODEL_ID,
    MODEL_REGISTRY,
    check_artifact_paths,
    get_default_model_id,
    list_model_options_for_ui,
    list_models as registry_list_models,
    normalize_mode,
    normalize_optimization,
```

```

        resolve_model_id,
        validate_model_for_mode,
    )

```

Το `history_store.py` χρησιμοποιείται για την ανάκτηση ιστορικού από το legacy default CSV. Το `inference.py` παρέχει τις συναρτήσεις φόρτωσης artifacts και παραγωγής πρόβλεψης. Το `model_registry.py` παρέχει τη λογική επιλογής και επικύρωσης μοντέλων. Υπάρχει επίσης fallback import χωρίς το πρόθεμα `src`, ώστε το API να μπορεί να εκτελεστεί και σε περιβάλλον όπου τα αρχεία βρίσκονται απευθείας στον ίδιο φάκελο:

```

except ModuleNotFoundError:
    from history_store import get_history_from_csv, history_store_info
    from inference import load_model_artifacts, predict_dayahead
    from model_registry import (...)

```

Αυτή η λογική αυξάνει τη συμβατότητα του script με διαφορετικούς τρόπους εκτέλεσης.

Βασικές διαδρομές και ρυθμίσεις

Στη συνέχεια ορίζονται οι βασικές διαδρομές και οι αρχικές ρυθμίσεις:

```

BASE_DIR = Path(os.getenv("IDEAL_BASE_DIR", "C:/IDEAL_Programming"))

```

```

OUT_DIR = Path(
    os.getenv(
        "IDEAL_API_OUT_DIR",
        str(BASE_DIR / "processed" / "predictions" / "api_like"),
    )
)

```

```

OUT_DIR.mkdir(parents=True, exist_ok=True)

```

Το `BASE_DIR` δείχνει στον βασικό φάκελο του έργου. Η χρήση της μεταβλητής περιβάλλοντος `IDEAL_BASE_DIR` επιτρέπει την αλλαγή της διαδρομής χωρίς τροποποίηση του κώδικα. Το `OUT_DIR` είναι ο φάκελος στον οποίο μπορούν να αποθηκευτούν οι προβλέψεις όταν ζητηθεί αποθήκευση σε CSV.

Οι υπόλοιπες βασικές ρυθμίσεις είναι:

```

DEFAULT_CITY = os.getenv("IDEAL_DEFAULT_CITY", "Edinburgh")
DEFAULT_MIN_HISTORY_HOURS = int(os.getenv("IDEAL_DEFAULT_HISTORY_HOURS",
"168"))
OPENMETEO_TIMEOUT_SECONDS = int(os.getenv("OPENMETEO_TIMEOUT_SECONDS",
"30"))

```

Η προεπιλεγμένη πόλη είναι το Edinburgh, ενώ το προεπιλεγμένο ελάχιστο ιστορικό είναι 168 ώρες. Το timeout του Open-Meteo ορίζεται σε 30 δευτερόλεπτα, ώστε το API να μην καθυστερεί υπερβολικά σε περίπτωση αποτυχίας της εξωτερικής υπηρεσίας.

Φόρτωση αργιών

Το API υποστηρίζει τη χρήση αργιών μέσω αρχείου CSV. Αρχικά ορίζονται πιθανές διαδρομές για το αρχείο αργιών:

```

HOLIDAYS_CANDIDATES = [
    Path(os.getenv("HOLIDAYS_PATH", "")) if os.getenv("HOLIDAYS_PATH")
else None,
    BASE_DIR / "holidays.csv",
    BASE_DIR / "metadata" / "holidays.csv",
    BASE_DIR / "processed" / "metadata" / "holidays.csv",
    BASE_DIR / "processed" / "holidays.csv",
]

```

Η συνάρτηση `_load_holidays_set` προσπαθεί να εντοπίσει το πρώτο διαθέσιμο αρχείο και να διαβάσει τις ημερομηνίες:

```

def _load_holidays_set() -> set:
    for p in HOLIDAYS_CANDIDATES:
        try:
            if p.exists():
                df = pd.read_csv(p)
                if df.shape[1] == 0:
                    continue

```

```

        if "date" in df.columns:
            s = pd.to_datetime(df["date"],
errors="coerce").dt.date.dropna()
        else:
            s = pd.to_datetime(df.iloc[:, 0],
errors="coerce").dt.date.dropna()

        return set(s.tolist())
    except Exception:
        continue

return set()

```

Αν βρεθεί στήλη date, χρησιμοποιείται αυτή. Διαφορετικά, χρησιμοποιείται η πρώτη στήλη του αρχείου. Αν δεν βρεθεί κανένα έγκυρο αρχείο, επιστρέφεται κενό set. Η συνάρτηση is_holiday_date αξιοποιεί αυτό το set για να ελέγξει αν μία ημερομηνία είναι αργία.

Ανάκτηση θερμοκρασίας από Open-Meteo

Το API μπορεί να ανακτήσει αυτόματα εξωτερικές θερμοκρασίες από την υπηρεσία Open-Meteo. Πρώτα χρησιμοποιείται η συνάρτηση _geocode_city, η οποία μετατρέπει το όνομα πόλης σε γεωγραφικές συντεταγμένες:

```

def _geocode_city(city: str) -> Optional[Dict[str, float]]:
    url = "https://geocoding-api.open-meteo.com/v1/search"
    params = {"name": city, "count": 1, "language": "en", "format": "json"}

```

Αν η υπηρεσία επιστρέψει αποτέλεσμα, η συνάρτηση επιστρέφει latitude και longitude. Αν αποτύχει, επιστρέφει None.

Η συνάρτηση fetch_openmeteo_hourly_temp χρησιμοποιεί τις συντεταγμένες για να ανακτήσει 24 ωριαίες τιμές θερμοκρασίας:

```

def fetch_openmeteo_hourly_temp(city: str, target_date: str) ->
Optional[List[float]]:
    geo = _geocode_city(city)
    if geo is None:
        return None

    target_day = str(pd.Timestamp(target_date).date())

    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": geo["lat"],
        "longitude": geo["lon"],
        "hourly": "temperature_2m",
        "timezone": "Europe/London",
        "start_date": target_day,
        "end_date": target_day,
    }

```

Η συνάρτηση χρησιμοποιεί το forecast endpoint του Open-Meteo. Για ιστορικές ημερομηνίες, όπως οι ημερομηνίες του test set του IDEAL, η ανάκτηση μπορεί να αποτύχει, επειδή το forecast endpoint αφορά κυρίως τρέχουσες ή μελλοντικές περιόδους. Σε αυτή την περίπτωση, το API δεν σταματά απαραίτητα, αλλά μπορεί να χρησιμοποιήσει εναλλακτικά τις τιμές t_min και t_max ή 24ωρο προφίλ θερμοκρασίας που δίνει ο χρήστης.

Δημιουργία συνθετικού θερμοκρασιακού προφίλ

Όταν δεν είναι διαθέσιμη αυτόματη θερμοκρασία και ο χρήστης παρέχει ελάχιστη και μέγιστη θερμοκρασία, το API δημιουργεί συνθετικό ωριαίο προφίλ:

```

def build_hourly_temp_from_minmax(tmin: float, tmax: float) ->
List[float]:
    tmin = float(tmin)
    tmax = float(tmax)

```

```
if tmax < tmin:
    tmin, tmax = tmax, tmin
```

```
mean = 0.5 * (tmin + tmax)
amp = 0.5 * (tmax - tmin)
```

Η συνάρτηση υπολογίζει μέση τιμή και πλάτος και στη συνέχεια δημιουργεί 24 τιμές με συνημιτονοειδή καμπύλη:

```
for h in range(24):
    val = mean + amp * math.cos(2 * math.pi * (h - 15) / 24.0)
    temps.append(float(val))
```

Η μέγιστη θερμοκρασία τοποθετείται περίπου στις 15:00, ενώ οι χαμηλότερες τιμές εμφανίζονται τις νυχτερινές ώρες. Το αποτέλεσμα δεν είναι πραγματική πρόγνωση καιρού, αλλά ένα πρακτικό fallback ώστε το API να μπορεί να λειτουργήσει όταν δεν υπάρχει διαθέσιμο πλήρες θερμοκρασιακό προφίλ.

Επίλυση εξωτερικής θερμοκρασίας

Η συνάρτηση `resolve_external_temperature_24h` καθορίζει την τελική πηγή της εξωτερικής θερμοκρασίας:

```
def resolve_external_temperature_24h(
    city: str,
    target_date: str,
    external_temperature_24h: Optional[List[float]],
    tmin: Optional[float],
    tmax: Optional[float],
) -> Tuple[List[float], str, Optional[float], Optional[float]]:
```

Η σειρά προτεραιότητας είναι η εξής:

1. user-provided `external_temperature_24h`
2. Open-Meteo forecast
3. `t_min / t_max` fallback

Αν ο χρήστης δώσει λίστα 24 τιμών, αυτή χρησιμοποιείται απευθείας:

```
if external_temperature_24h is not None:
    if len(external_temperature_24h) != 24:
        raise HTTPException(status_code=422,
            detail="external_temperature_24h must have length 24.")
    return [float(x) for x in external_temperature_24h], "user_24h", None,
    None
```

Αν δεν δοθεί 24ωρο προφίλ, το API επιχειρεί να ανακτήσει θερμοκρασία από το Open-Meteo. Αν και αυτό αποτύχει, χρησιμοποιεί το fallback από `t_min` και `t_max`. Αν καμία πηγή δεν είναι διαθέσιμη, επιστρέφει σφάλμα 422 και ζητά από τον χρήστη να δώσει είτε `t_min/t_max` είτε 24ωρο προφίλ θερμοκρασίας.

Δημιουργία εφαρμογής FastAPI και cache artifacts

Η εφαρμογή δημιουργείται ως εξής:

```
app = FastAPI(title="IDEAL Load Forecasting API", version="0.6.0-generic-ui")
```

Η έκδοση 0.6.0-generic-ui δηλώνει ότι πρόκειται για έκδοση προσαρμοσμένη στη γενική γραφική διεπαφή χωρίς απαίτηση `home_id`.

Το script ορίζει επίσης cache για τα φορτωμένα artifacts:

```
ARTIFACT_CACHE: Dict[str, Dict[str, Any]] = {}
```

Η cache αποτρέπει την επαναφόρτωση του ίδιου μοντέλου από τον δίσκο σε κάθε αίτημα.

Η συνάρτηση `_get_artifacts_for_model` αναλαμβάνει να επιλύσει το `model_id`, να ελέγξει τη συμβατότητα με το `mode` και να φορτώσει τα artifacts μόνο αν δεν υπάρχουν ήδη στην cache:

```
cache_key = f"{resolved_id}::{mode}::{optimization}"
if cache_key in ARTIFACT_CACHE:
    return ARTIFACT_CACHE[cache_key]
```

Αν τα artifacts δεν έχουν φορτωθεί, καλείται η `load_model_artifacts` από το `inference.py`. Αν η φόρτωση αποτύχει, το API επιστρέφει σφάλμα 500.

Βοηθητικές συναρτήσεις απόκρισης

Το API περιλαμβάνει βοηθητικές συναρτήσεις για τη διαμόρφωση της απόκρισης. Η `_total_kwh_from_preds` υπολογίζει τη συνολική ημερήσια πρόβλεψη:

```
def _total_kwh_from_preds(pred_df: pd.DataFrame) -> float:
    return float(pred_df["pred_consumption_Wh"].sum() / 1000.0)

Η _derived_flags_for_day επιστρέφει πληροφορίες για την ημέρα στόχου:
def _derived_flags_for_day(target_date: str) -> Dict[str, int]:
    d0 = pd.Timestamp(target_date)
    return {
        "is_weekend": int(d0.dayofweek >= 5),
        "is_holiday": int(is_holiday_date(d0)),
    }
```

Η `_normalize_user_history` ελέγχει και κανονικοποιεί τη λίστα ιστορικών τιμών που μπορεί να δώσει ο χρήστης:

```
def _normalize_user_history(history: List[float], min_hours: int) -> List[float]:
    try:
        hist = [float(x) for x in history]
    except Exception:
        raise HTTPException(status_code=422,
                             detail="history_consumption_Wh must be a numeric list.")
```

Αν το ιστορικό έχει λιγότερες τιμές από το απαιτούμενο ελάχιστο, επιστρέφεται σφάλμα.

Αν έχει περισσότερες, κρατούνται οι τελευταίες `min_history_hours` τιμές.

Η `_prediction_rows` μετατρέπει το dataframe πρόβλεψης σε λίστα dictionaries κατάλληλη για JSON απόκριση:

```
def _prediction_rows(pred: pd.DataFrame) -> List[Dict[str, Any]]:
    optional_cols = [
        "pred_model_raw_Wh",
        "history_median_7d_Wh",
        "history_shape_scaled_Wh",
        "adaptive_alpha",
        "history_correction_applied",
        "history_correction_reason",
    ]
```

Εκτός από τις βασικές προβλέψεις, η συνάρτηση συμπεριλαμβάνει προαιρετικές στήλες που σχετίζονται με την `adaptive history-aware correction`, εφόσον αυτές υπάρχουν στο αποτέλεσμα.

Endpoint /health

Το endpoint `/health` επιστρέφει βασικές πληροφορίες κατάστασης του API:

```
@app.get("/health")
def health() -> Dict[str, Any]:
    artifact_status = check_artifact_paths()
    ready_models = [model_id for model_id, info in artifact_status.items()
                    if info.get("ready")]
```

Η συνάρτηση ελέγχει ποια μοντέλα έχουν έτοιμα artifacts και επιστρέφει πληροφορίες όπως:

```
return {
    "status": "ok",
    "api_version": API_VERSION,
    "base_dir": str(BASE_DIR),
    "out_dir": str(OUT_DIR),
    "default_mode": normalize_mode(DEFAULT_MODE),
    "default_model_id": DEFAULT_MODEL_ID,
    "default_no_history_model":
get_default_model_id(CANONICAL_NO_HISTORY),
```

```

        "default_with_history_model":
get_default_model_id(CANONICAL_WITH_HISTORY),
        "ready_models": ready_models,
        "num_ready_models": len(ready_models),
        "history_schema": _history_schema_info(),
        "legacy_default_history_store": _get_default_history_store_info(),
    }

```

Το endpoint αυτό είναι χρήσιμο για έλεγχο λειτουργίας της εφαρμογής, debugging και επιβεβαίωση ότι τα μοντέλα και το history store είναι διαθέσιμα.

Endpoint /models

Το endpoint /models επιστρέφει τα διαθέσιμα μοντέλα:

```

@app.get("/models")
def list_models(
    mode: Optional[str] = None,
) -> Dict[str, Any]:

```

Αν δοθεί mode, αυτό κανονικοποιείται μέσω του normalize_mode. Στη συνέχεια καλείται το registry για να επιστραφούν τα μοντέλα που είναι διαθέσιμα για το συγκεκριμένο mode: models = registry_list_models(mode=normalized_mode, ui_visible_only=True, include_optional=True) artifact_status = check_artifact_paths()

Η απόκριση περιλαμβάνει τα default μοντέλα ανά mode, τις επιλογές για το UI και αναλυτικές πληροφορίες για κάθε μοντέλο, όπως model_id, όνομα, οικογένεια, τύπο, mode, αν είναι default ή optional, αν απαιτεί ιστορικό και αν υπάρχουν τα απαραίτητα artifacts.

Με αυτόν τον τρόπο, το Gradio UI μπορεί να δημιουργεί δυναμικά dropdown επιλογές μοντέλων χωρίς να έχει σκληροκωδικοποιημένες τις διαθέσιμες επιλογές.

Endpoint /history/info

Το endpoint /history/info παρέχει πληροφορίες για τη μορφή του ιστορικού:

```

@app.get("/history/info")
def history_info() -> Dict[str, Any]:
    return {
        "history_schema": _history_schema_info(),
        "legacy_default_history_store":
_get_default_history_store_info(),
        "note": (
            "For the updated UI, prefer passing a user-selected single-
home CSV "
            "through history_csv_path. The legacy default
history_store.csv remains "
            "available only as a fallback."
        ),
    }

```

Η απόκριση ξεκαθαρίζει ότι η νέα προτεινόμενη λογική είναι η χρήση επιλεγμένου single-home CSV μέσω history_csv_path. Το legacy history_store.csv παραμένει διαθέσιμο μόνο ως fallback. Το schema του ιστορικού δηλώνει ότι απαιτούνται οι στήλες:

timestamp

consumption_Wh

και ότι δεν απαιτείται στήλη home_id.

Δομή αιτήματος πρόβλεψης

Το request body του /predict ορίζεται με την κλάση PredictRequest:

```

class PredictRequest(BaseModel):
    model_id: str = Field(default="auto", example="auto")
    mode: str = Field(default=DEFAULT_MODE, example="no_history")
    optimization: str = Field(default="balanced", example="balanced")

```

Ο χρήστης μπορεί να επιλέξει συγκεκριμένο μοντέλο ή να αφήσει το model_id="auto", ώστε το API να επιλέξει το default μοντέλο για το αντίστοιχο mode. Το mode μπορεί να είναι no_history ή with_history, ενώ το optimization μπορεί να είναι balanced ή daily.

Στη συνέχεια ορίζονται οι βασικές πληροφορίες του αιτήματος:

```
city: str = Field(default=DEFAULT_CITY, example="Edinburgh")
target_date: str = Field(..., example="2026-02-21")
```

```
total_floor_area_m2: float = Field(..., example=85, ge=0)
residents: float = Field(..., example=2, ge=0)
hometype: str = Field(..., example="flat")
urban_rural_class: str = Field(..., example="1")
num_electric_appliances: Optional[float] = Field(default=None,
example=12, ge=0)
```

Τα πεδία αυτά περιγράφουν την πόλη, την ημερομηνία στόχου και τα βασικά χαρακτηριστικά της κατοικίας.

Τα θερμοκρασιακά inputs ορίζονται ως εξής:

```
external_temperature_24h: Optional[List[float]] = None
internal_temperature_24h: Optional[List[float]] = None
t_min: Optional[float] = None
t_max: Optional[float] = None
```

Ο χρήστης μπορεί είτε να δώσει πλήρες 24ωρο προφίλ θερμοκρασίας είτε να δώσει ελάχιστη και μέγιστη θερμοκρασία είτε να αφήσει το API να επιχειρήσει αυτόματη ανάκτηση από Open-Meteo.

Τα inputs ιστορικού για το with_history mode είναι:

```
history_consumption_Wh: Optional[List[float]] = None
history_csv_path: Optional[str] = Field(default=None,
example="C:/IDEAL_Programming/processed/stores/history_store.csv")
min_history_hours: int = Field(default=DEFAULT_MIN_HISTORY_HOURS, ge=1)
```

Το ιστορικό μπορεί να δοθεί είτε ως λίστα τιμών είτε ως διαδρομή προς CSV. Το ελάχιστο ιστορικό είναι 168 ώρες από προεπιλογή.

Υπάρχει επίσης επιλογή για χρήση του legacy default history store:

```
use_default_history_store: Optional[bool] = None
use_proxy_history: bool = True
```

Το use_proxy_history παραμένει για συμβατότητα με παλαιότερες εκδόσεις, ενώ το use_default_history_store είναι η νεότερη και πιο σαφής ονομασία.

Τέλος, ορίζονται οι παράμετροι adaptive correction:

```
apply_history_correction: bool = True
history_correction_days: int = Field(default=7, ge=1, le=30)
history_correction_max_alpha: float = Field(default=0.20, ge=0.0, le=1.0)
```

Με αυτές τις παραμέτρους το API μπορεί να ενεργοποιεί ή να απενεργοποιεί την προσαρμογή με βάση το πρόσφατο ιστορικό.

Κύριο endpoint /predict

Το σημαντικότερο endpoint του API είναι το /predict:

```
@app.post("/predict")
def predict(req: PredictRequest) -> Dict[str, Any]:
```

Αρχικά κανονικοποιούνται το mode και το optimization:

```
try:
    mode = normalize_mode(req.mode)
    optimization = normalize_optimization(req.optimization)
except Exception as exc:
    raise HTTPException(status_code=422, detail=str(exc))
```

Αν ο χρήστης δώσει παλιά ονομασία όπως coldstart ή withhistory, αυτή μετατρέπεται στις canonical τιμές no_history ή with_history.

Στη συνέχεια επιλύεται το μοντέλο:

```
resolved_model_id = resolve_model_id(req.model_id, mode=mode,
optimization=optimization)
model_cfg = validate_model_for_mode(resolved_model_id, mode)
```

Αν ο χρήστης έχει επιλέξει auto, το registry επιλέγει το default μοντέλο για το mode. Αν έχει επιλέξει συγκεκριμένο μοντέλο, το API ελέγχει ότι το μοντέλο αυτό υποστηρίζει το ζητούμενο mode και δεν απαιτεί home_id.

Έπειτα φορτώνονται τα artifacts:

```
_ = _get_artifacts_for_model(
    model_id=resolved_model_id,
    mode=mode,
    optimization=optimization,
)
```

Ακόμη και αν τα artifacts δεν χρησιμοποιούνται άμεσα στη μεταβλητή _, η κλήση αυτή ελέγχει ότι το μοντέλο μπορεί πράγματι να φορτωθεί και ενημερώνει την cache.

Επίλυση θερμοκρασιών στο /predict

Το API επιλύει την εξωτερική θερμοκρασία ως εξής:

```
temps24, weather_source, tmin_used, tmax_used =
resolve_external_temperature_24h(
    city=req.city,
    target_date=req.target_date,
    external_temperature_24h=req.external_temperature_24h,
    tmin=req.t_min,
    tmax=req.t_max,
)
```

Το αποτέλεσμα είναι η λίστα των 24 εξωτερικών θερμοκρασιών, η πηγή της θερμοκρασίας και, αν χρησιμοποιήθηκε fallback, οι τιμές t_min και t_max. Αν δοθεί εσωτερική θερμοκρασία, ελέγχεται επίσης ότι έχει μήκος 24:

```
internal24 =
resolve_internal_temperature_24h(req.internal_temperature_24h)
```

Με αυτόν τον τρόπο το API εξασφαλίζει ότι το inference layer θα λάβει θερμοκρασιακά δεδομένα με σωστή μορφή.

Διαχείριση ιστορικού στο with-history mode

Αν το mode είναι with_history, το API πρέπει να εξασφαλίσει ότι υπάρχει επαρκές ιστορικό:

```
if mode == CANONICAL_WITH_HISTORY:
    min_history_hours = int(
        max(
            req.min_history_hours,
            int(model_cfg.get("min_history_hours",
                DEFAULT_MIN_HISTORY_HOURS)),
        )
    )
```

Το API λαμβάνει ως ελάχιστο ιστορικό το μεγαλύτερο μεταξύ του min_history_hours που ζητήθηκε και του ελάχιστου που απαιτεί το μοντέλο.

Η σειρά προτεραιότητας για την πηγή ιστορικού είναι η εξής:

1. history_consumption_Wh
2. history_csv_path
3. legacy default history_store.csv

Αν ο χρήστης δώσει λίστα τιμών, αυτή χρησιμοποιείται πρώτη:

```
if req.history_consumption_Wh is not None:
    history = _normalize_user_history(req.history_consumption_Wh,
    min_history_hours)
    history_source = "user_vector"
```

Αν δεν δοθεί λίστα αλλά δοθεί CSV path, τότε το path περνά στο inference layer:

```
elif req.history_csv_path:
    history_csv_path = str(req.history_csv_path)
    history_source = "user_selected_csv"
    history_store_path = history_csv_path
```

Αν δεν δοθεί ούτε λίστα ούτε CSV, το API εξετάζει αν επιτρέπεται η χρήση του legacy default store:

```
else:
    use_default_store = req.use_proxy_history if
req.use_default_history_store is None else
bool(req.use_default_history_store)
```

Αν δεν επιτρέπεται, επιστρέφεται σφάλμα. Αν επιτρέπεται, το API καλεί το get_history_from_csv:

```
history, hs, ws, we = get_history_from_csv(
    target_date=req.target_date,
    history_hours=min_history_hours,
    fallback_to_latest=True,
)
```

Έτσι το API μπορεί να λειτουργήσει και με παλιότερο default history store, αλλά η νεότερη προτεινόμενη λογική είναι η χρήση επιλεγμένου CSV μέσω history_csv_path.

Κλήση του inference layer

Αφού επιλυθούν μοντέλο, θερμοκρασίες και ιστορικό, το API καλεί τη συνάρτηση predict_dayahead:

```
pred = predict_dayahead(
    mode=mode,
    model_id=resolved_model_id,
    optimization=optimization,
    target_date=req.target_date,
    external_temperature_24h=temps24,
    history_consumption_Wh=history,
    history_csv_path=history_csv_path,
    total_floor_area_m2=req.total_floor_area_m2,
    residents=req.residents,
    hometype=req.hometype,
    urban_rural_class=req.urban_rural_class,
    internal_temperature_24h=internal24,
    num_electric_appliances=req.num_electric_appliances,
    min_history_hours=min_history_hours,
    apply_history_correction=req.apply_history_correction,
    history_correction_days=req.history_correction_days,
    history_correction_max_alpha=req.history_correction_max_alpha,
    save_csv=req.save_csv,
    out_dir=OUT_DIR,

    out_name=f"pred_{resolved_model_id}_{mode}_{req.city}_{req.target_date}.
csv".replace(" ", "_"),
)
```

Η predict_dayahead αποφασίζει εσωτερικά αν θα εκτελέσει no_history ή with_history πρόβλεψη. Αν το mode είναι with_history, μπορεί να εφαρμοστεί και adaptive history-aware correction, ανάλογα με τις παραμέτρους του αιτήματος.

Δομή της απόκρισης

Μετά την παραγωγή της πρόβλεψης, το API ανακτά διαγνωστικές πληροφορίες από τα attributes του dataframe:

```
history_input_info = pred.attrs.get("history_input_info", {}) if
hasattr(pred, "attrs") else {}
history_correction_info = pred.attrs.get("history_correction_info", {})
if hasattr(pred, "attrs") else {}
```

Οι πληροφορίες αυτές μπορεί να περιλαμβάνουν την κάλυψη του ιστορικού, το χρονικό παράθυρο που χρησιμοποιήθηκε και το αν εφαρμόστηκε adaptive correction.

Η τελική απόκριση περιλαμβάνει πολλές πληροφορίες:

```
return {
    "requested_model_id": req.model_id,
    "model_id": resolved_model_id,
```

```

"model_name": model_cfg.get("name"),
"model_type": model_cfg.get("type"),
"mode": mode,
"optimization": optimization,
"prediction_variant": pred["prediction_variant"].iloc[0] if
"prediction_variant" in pred.columns else "balanced",

"city": req.city,
"target_date": req.target_date,
"total_kWh_day": _total_kwh_from_preds(pred),

"weather_source": weather_source,
"t_min_used": tmin_used,
"t_max_used": tmax_used,
"external_temperature_24h": temps24,
"internal_temperature_24h": internal24,
}

```

Η απόκριση δεν επιστρέφει μόνο τις 24 προβλέψεις. Επιστρέφει επίσης ποιο μοντέλο χρησιμοποιήθηκε, ποια ήταν η πηγή της θερμοκρασίας, ποια ήταν η συνολική ημερήσια κατανάλωση και ποιο prediction variant παρήχθη.

Για το ιστορικό επιστρέφονται επίσης:

```

"history_source": history_source,
"history_csv_path": history_csv_path,
"history_store_path": history_store_path,
"history_window_start": history_window_start,
"history_window_end": history_window_end,
"history_hours_used": (...),
"history_input_info": history_input_info,
"history_correction_info": history_correction_info,
"history_correction_applied":
bool(history_correction_info.get("history_correction_applied", False)),
"history_correction_reason":
history_correction_info.get("history_correction_reason"),

```

Αυτά τα στοιχεία κάνουν την πρόβλεψη πιο διαφανή, επειδή ο χρήστης μπορεί να γνωρίζει από πού προήλθε το ιστορικό, πόσες ώρες χρησιμοποιήθηκαν και αν εφαρμόστηκε διόρθωση με βάση το πρόσφατο ιστορικό.

Τέλος, επιστρέφονται τα flags της ημέρας και οι αναλυτικές προβλέψεις:

```

"derived_flags": _derived_flags_for_day(req.target_date),

"predictions": _prediction_rows(pred),

```

Οι προβλέψεις περιλαμβάνουν timestamp και pred_consumption_Wh για κάθε ώρα. Αν έχει εφαρμοστεί adaptive correction, μπορεί να περιλαμβάνουν και πρόσθετα πεδία όπως την αρχική πρόβλεψη του μοντέλου, το ιστορικό ωριαίο προφίλ και τον συντελεστή adaptive_alpha.

Gradio UI Αρχείο: ui_app.py

Gradio User Interface. Αρχείο: ui_app.py

Το αρχείο ui_app.py υλοποιεί τη γραφική διεπαφή χρήστη της τελικής εφαρμογής πρόβλεψης ηλεκτρικής κατανάλωσης. Η διεπαφή έχει αναπτυχθεί με τη βιβλιοθήκη Gradio και λειτουργεί ως frontend πάνω από το FastAPI backend. Ο ρόλος της δεν είναι να εκπαιδεύει μοντέλα ή να εκτελεί απευθείας τη βασική διαδικασία inference, αλλά να συλλέγει τις εισόδους του χρήστη, να τις οργανώνει σε κατάλληλο αίτημα προς το API, να λαμβάνει την πρόβλεψη και να την παρουσιάζει σε μορφή σύνοψης, πίνακα και γραφήματος.

Η τελική έκδοση της διεπαφής ακολουθεί τη γενική λογική της εφαρμογής, όπου ο χρήστης δεν παρέχει `home_id`. Αντίθετα, επιλέγει αν θέλει πρόβλεψη χωρίς ιστορικό (`no_history`) ή με χρήση πρόσφατου ιστορικού (`with_history`). Στο δεύτερο σενάριο, το ιστορικό μπορεί να δοθεί με τρεις τρόπους: μέσω επιλεγμένου CSV μίας κατοικίας, μέσω χειροκίνητης λίστας ωριαίων τιμών ή μέσω του `legacy default history_store.csv`, εφόσον έχει ενεργοποιηθεί η αντίστοιχη επιλογή.

Αρχικές παραδοχές του UI

Στην αρχή του αρχείου καταγράφονται οι βασικές παραδοχές της διεπαφής:

Final API/UI assumptions:

- # - The user does NOT provide `home_id`.
- # - `model_id="auto"` selects the correct default by scenario:
 - # `no_history` -> `rf_coldstart_default`
 - # `with_history` -> `lgbm_withhistory_default`
- # - Optional comparison models are selected through the new API registry.
- # - With-history requires recent hourly consumption history, default 168 hours.
- # - History can come from a user-selected single-home CSV path/upload,
 - # manual comma-separated values, or the legacy `history_store.csv` fallback.
- # - Weather can come from Open-Meteo, manual `t_min/t_max`, or manual 24h vector.

Οι παραδοχές αυτές δείχνουν ότι το UI έχει σχεδιαστεί για χρήση από τελικό χρήστη και όχι για εσωτερική αξιολόγηση συγκεκριμένων κατοικιών του dataset. Η επιλογή `model_id="auto"` επιτρέπει στο API να επιλέγει αυτόματα το κατάλληλο default μοντέλο ανά σενάριο. Για το `no_history` σενάριο χρησιμοποιείται ως default το Random Forest, ενώ για το `with_history` σενάριο χρησιμοποιείται το LightGBM generic with-history.

Η διεπαφή προσφέρει τρεις βασικές λειτουργίες:

Main actions:

- # Predict -> one selected model or auto default
- # Compare -> RF / XGB / LGBM comparison for selected scenario
- # Combined Prediction -> weighted ensemble of RF / XGB / LGBM predictions

Η λειτουργία Predict εκτελεί πρόβλεψη με ένα επιλεγμένο μοντέλο ή με το default μοντέλο. Η λειτουργία Compare συγκρίνει τις προβλέψεις των RF, XGB και LGBM. Η λειτουργία Combined Prediction δημιουργεί συνδυαστική πρόβλεψη ως σταθμισμένο ensemble των επιμέρους μοντέλων.

Ρυθμίσεις περιβάλλοντος πριν από την εισαγωγή του Gradio

Στην αρχή του script ορίζονται ορισμένες μεταβλητές περιβάλλοντος πριν γίνει import του Gradio και του requests:

```
import os
```

```
os.environ["NO_PROXY"] = "localhost,127.0.0.1,::1"
```

```
os.environ["no_proxy"] = "localhost,127.0.0.1,::1"
```

```
os.environ["GRADIO_ANALYTICS_ENABLED"] = "False"
```

Οι μεταβλητές `NO_PROXY` και `no_proxy` διασφαλίζουν ότι οι τοπικές κλήσεις προς το FastAPI backend στο localhost δεν θα περάσουν από proxy. Αυτό είναι σημαντικό όταν η εφαρμογή εκτελείται τοπικά. Η μεταβλητή `GRADIO_ANALYTICS_ENABLED=False` απενεργοποιεί τα analytics του Gradio.

Εισαγωγή βιβλιοθηκών

Στη συνέχεια εισάγονται οι βιβλιοθήκες που χρειάζεται το UI:

```
import datetime as dt
import shutil
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import gradio as gr
import matplotlib.pyplot as plt
import pandas as pd
import requests
```

Η `datetime` χρησιμοποιείται για χειρισμό ημερομηνιών, ώρας και υπολογισμό υπολειπόμενης κατανάλωσης όταν η ημερομηνία στόχος είναι η σημερινή. Η `shutil` χρησιμοποιείται για την αντιγραφή του επιλεγμένου CSV ιστορικού σε σταθερό φάκελο του project. Η `Path` χρησιμοποιείται για τη διαχείριση διαδρομών αρχείων. Η `gradio` δημιουργεί τη διεπαφή, η `matplotlib` χρησιμοποιείται για τα γραφήματα, η `pandas` για πίνακες και μετατροπές δεδομένων, ενώ η `requests` για τις HTTP κλήσεις προς το API.

Compatibility patch για Gradio 4.44.x

Το script περιλαμβάνει ειδικό compatibility patch για ορισμένες εκδόσεις του Gradio:

```
try:
    from gradio_client import utils as _gradio_client_utils

    _orig_json_schema_to_python_type = _gradio_client_utils.json_schema_to_python_type
    _gradio_client_utils.json_schema_to_python_type = _safe_json_schema_to_python_type

    def _safe_json_schema_to_python_type(schema, defs=None):
        if isinstance(schema, bool):
            return "Any"

        if isinstance(schema, dict):
            schema = dict(schema)
            if isinstance(schema.get("additionalProperties"), bool):
                schema["additionalProperties"] = {"type": "object"}

        return _orig_json_schema_to_python_type(schema, defs)

    _gradio_client_utils.json_schema_to_python_type = _safe_json_schema_to_python_type
```

```
except Exception as exc:
```

```
    print(f"[WARN] Gradio schema compatibility patch not applied: {exc}")
```

Το patch αυτό αντιμετωπίζει πρόβλημα συμβατότητας σε περιπτώσεις όπου το JSON schema περιέχει boolean τιμές. Αν δεν εφαρμοστεί, η εφαρμογή δεν σταματά υποχρεωτικά, αλλά εμφανίζεται προειδοποίηση. Η ύπαρξη αυτής της διόρθωσης δείχνει ότι η διεπαφή έχει προσαρμοστεί σε πρακτικά προβλήματα εκτέλεσης της συγκεκριμένης έκδοσης του Gradio.

Βασική παραμετροποίηση

Οι βασικές ρυθμίσεις του UI ορίζονται ως εξής:

```
API_BASE = "http://127.0.0.1:8000"
```

```
UI_HOST = "127.0.0.1"
```

```
UI_PORT = 7860
```

```
BASE_DIR = Path("C:/IDEAL_Programming")
```

```
UI_EXPORT_DIR = BASE_DIR / "processed" / "predictions" / "ui_exports"
```

```
UI_EXPORT_DIR.mkdir(parents=True, exist_ok=True)
```

```
HISTORY_UPLOAD_DIR = BASE_DIR / "processed" / "stores" /  
"selected_history_files"
```

```
HISTORY_UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
```

```
TIMEOUT = 180
```

Το API_BASE δηλώνει τη διεύθυνση του FastAPI backend. Το UI εκτελείται τοπικά στη θύρα 7860. Ο φάκελος UI_EXPORT_DIR χρησιμοποιείται για την αποθήκευση εξαγόμενων προβλέψεων σε CSV, ενώ ο φάκελος HISTORY_UPLOAD_DIR χρησιμοποιείται για την αποθήκευση των επιλεγμένων CSV ιστορικού. Το timeout των HTTP κλήσεων προς το API ορίζεται σε 180 δευτερόλεπτα, ώστε να υπάρχει επαρκής χρόνος για βαρύτερες προβλέψεις ή συγκρίσεις μοντέλων.

Οι διαθέσιμες πόλεις, οι τύποι κατοικίας και οι κατηγορίες αστικότητας ορίζονται επίσης στο UI:

```
DEFAULT_CITIES = ["Edinburgh", "Glasgow", "Dundee", "Aberdeen"]
```

```
DEFAULT_HOMETYPES = [  
    ("flat", "flat"),  
    ("house_or_bungalow", "house_or_bungalow"),  
]
```

```
URBAN_RURAL_OPTIONS = [  
    ("1 - Large urban area", "1"),  
    ("2 - Other urban area", "2"),  
    ("3 - Accessible small town / rural", "3"),  
]
```

```

    ("3+ - Remote small town / rural", "3+"),
]

```

Οι επιλογές αυτές αντιστοιχούν στα διαθέσιμα χαρακτηριστικά που μπορούν να δοθούν στο API για την περιγραφή της κατοικίας και του περιβάλλοντος.

Προεπιλεγμένα βάρη ensemble

Το script ορίζει προεπιλεγμένα βάρη για τη συνδυαστική πρόβλεψη:

```

DEFAULT_ENSEMBLE_WEIGHTS = {
    "rf": 0.40,
    "xgb": 0.30,
    "lgbm": 0.30,
}

```

Με βάση αυτή τη ρύθμιση, όταν ο χρήστης δεν δώσει custom βάρη, το ensemble χρησιμοποιεί 40% Random Forest, 30% XGBoost και 30% LightGBM. Η συνδυαστική πρόβλεψη δεν αποτελεί ξεχωριστό εκπαιδευμένο μοντέλο. Είναι σταθμισμένος μέσος όρος των προβλέψεων των επιμέρους μοντέλων.

Σε τεχνικό επίπεδο, αν σε νεότερη αξιολόγηση έχουν αποφασιστεί διαφορετικά τελικά βάρη ανά σενάριο, το συγκεκριμένο σημείο πρέπει να ενημερωθεί ώστε το UI να συμφωνεί με την τελική μεθοδολογία.

Έλεγχος κατάστασης API και δυναμική ανάκτηση μοντέλων

Η συνάρτηση `_api_health` ελέγχει αν το FastAPI backend είναι διαθέσιμο:

```

def _api_health() -> str:
    try:
        r = requests.get(f"{API_BASE}/health", timeout=3)
        if r.status_code == 200:
            js = r.json()
            ready = js.get("num_ready_models")
            nohist = js.get("default_no_history_model") or
js.get("default_coldstart_model")
            hist = js.get("default_with_history_model") or
js.get("default_withhistory_model")
            return f"✅ API: OK | Ready models: {ready} | No-history:
{nohist} | With-history: {hist}"
        return f"⚠️ API: HTTP {r.status_code}"
    except Exception:
        return f"❌ API: not reachable. Start FastAPI first."

```

Η συνάρτηση επιστρέφει μήνυμα κατάστασης που εμφανίζεται στο UI. Αν το API είναι διαθέσιμο, εμφανίζεται ο αριθμός των έτοιμων μοντέλων και τα default μοντέλα για τα δύο σενάρια. Αν το API δεν είναι διαθέσιμο, εμφανίζεται μήνυμα ότι πρέπει πρώτα να ξεκινήσει το FastAPI.

Η διεπαφή μπορεί επίσης να ανακτήσει δυναμικά τις διαθέσιμες επιλογές μοντέλων από το endpoint `/models`:

```
def _fetch_models_response(mode: Optional[str] = None) -> Dict[str, Any]:
    try:
        params = {"mode": mode} if mode else None
        r = requests.get(f"{API_BASE}/models", params=params, timeout=5)
        r.raise_for_status()
        return r.json()
    except Exception:
        return {}
```

Αν το API δεν είναι διαθέσιμο, η συνάρτηση επιστρέφει κενό dictionary. Σε αυτή την περίπτωση, η `_fetch_model_options` χρησιμοποιεί fallback επιλογές, ώστε το UI να μπορεί να εμφανιστεί ακόμη και πριν ξεκινήσει το API.

Διαχείριση επιλογών μοντέλων

Η συνάρτηση `_fetch_model_options` επιστρέφει τα διαθέσιμα μοντέλα ανά mode:

```
def _fetch_model_options(mode: Optional[str] = None) -> List[Dict[str, Any]]:
    js = _fetch_models_response(mode=mode)
    options = js.get("options_for_ui") or []

    if options:
        return options
```

Αν το API επιστρέψει επιλογές, αυτές χρησιμοποιούνται άμεσα. Αν όχι, υπάρχουν fallback επιλογές. Για το `with_history` mode, εμφανίζονται το default LightGBM και τα optional XGBoost και Random Forest. Για το `no_history` mode, εμφανίζονται το default Random Forest και τα optional LightGBM και XGBoost.

Η συνάρτηση `_option_to_choice` μετατρέπει τις επιλογές σε μορφή κατάλληλη για dropdown:

```
def _option_to_choice(option: Dict[str, Any]) -> str:
    value = str(option.get("value", "auto"))
    label = str(option.get("label", value))
    marker = ""

    if option.get("is_default") and value != "auto":
        marker = " [default]"
    elif option.get("is_optional"):
        marker = " [optional]"

    return f"{value} – {label}{marker}"
```

Η μορφή αυτή επιτρέπει στο UI να εμφανίζει φιλικό όνομα στον χρήστη, αλλά να μπορεί να εξάγει το πραγματικό `model_id` όταν σταλεί το αίτημα στο API.

Έλεγχος και κανονικοποίηση εισόδων

Το UI περιλαμβάνει βοηθητικές συναρτήσεις για τον έλεγχο των εισόδων. Για παράδειγμα, η `_validate_inputs` ελέγχει ότι η ημερομηνία έχει σωστή μορφή, ότι το εμβαδόν και ο αριθμός κατοίκων είναι θετικοί και ότι οι θερμοκρασίες έχουν λογική σχέση:

```
def _validate_inputs(
    target_date: str,
    total_floor_area_m2: float,
    residents: float,
    min_history_hours: int,
    t_min: Optional[float],
    t_max: Optional[float],
) -> Optional[str]:
    try:
        _ = _parse_date(target_date)
    except Exception:
        return "Σφάλμα: target_date πρέπει να είναι σε μορφή YYYY-MM-DD."
```

Αν δοθεί ελάχιστη και μέγιστη θερμοκρασία, ελέγχεται ότι η ελάχιστη είναι μικρότερη από τη μέγιστη:

```
if tmin >= tmax:
    return "Σφάλμα: t_min πρέπει να είναι μικρότερο από t_max."
```

Η συνάρτηση `_parse_csv_float_list` χρησιμοποιείται για πεδία όπου ο χρήστης μπορεί να δώσει comma-separated αριθμητικές τιμές, όπως 24ωρες θερμοκρασίες ή χειροκίνητο ιστορικό κατανάλωσης:

```
def _parse_csv_float_list(text: str, name: str, expected_len:
Optional[int] = None) -> Optional[List[float]]:
    if not text or not str(text).strip():
        return None

    try:
        vals = [float(x.strip()) for x in str(text).split(",") if
x.strip() != ""]
    except Exception:
        raise ValueError(f"Σφάλμα: {name} δεν είναι έγκυρη λίστα αριθμών
comma-separated.")
```

Αν αναμένεται συγκεκριμένο μήκος, όπως 24 τιμές θερμοκρασίας, γίνεται σχετικός έλεγχος. Με αυτόν τον τρόπο, το UI αποτρέπει πολλά λάθη πριν ακόμη σταλεί το αίτημα στο API.

Επιλογή και αντιγραφή CSV ιστορικού

Μία από τις σημαντικότερες λειτουργίες του UI είναι η επιλογή αρχείου ιστορικού. Η συνάρτηση `register_history_csv_upload` χειρίζεται το αρχείο που επιλέγει ο χρήστης:

```
def register_history_csv_upload(file_obj: Any) -> Tuple[str, str]:
    """Handle the UI history CSV selection button.
```

```

    The browser opens the normal Windows file picker. Gradio receives the
    selected CSV and stores it in a temporary location. We copy it to a
    stable
```

```
project folder and write that path into the History CSV path textbox,  
so  
  
the FastAPI process can read the same file reliably.  
"""
```

Η συνάρτηση εξάγει τη διαδρομή του αρχείου, ελέγχει ότι υπάρχει και ότι έχει κατάληξη .csv, και στη συνέχεια το αντιγράφει στον φάκελο:

```
HISTORY_UPLOAD_DIR = BASE_DIR / "processed" / "stores" /  
"selected_history_files"
```

Η αντιγραφή γίνεται με μοναδικό όνομα που περιλαμβάνει timestamp:

```
timestamp = dt.datetime.now().strftime("%Y%m%d_%H%M%S")  
  
safe_stem = "".join(ch if ch.isalnum() or ch in ("_", "-") else "_" for  
ch in src.stem)  
  
dst = HISTORY_UPLOAD_DIR / f"{safe_stem}_{timestamp}.csv"  
  
  
shutil.copy2(src, dst)
```

Αυτό είναι σημαντικό επειδή το Gradio συχνά αποθηκεύει τα uploaded αρχεία προσωρινά. Αν το API λάμβανε την προσωρινή διαδρομή, ενδέχεται το αρχείο να μη διατίθεται αξιόπιστα. Με την αντιγραφή σε σταθερό φάκελο του project, το FastAPI μπορεί να διαβάσει το ίδιο αρχείο με ασφάλεια.

Ανάλυση σταθερότητας ιστορικού και σύσταση max alpha

Το UI περιλαμβάνει λειτουργία ανάλυσης σταθερότητας ιστορικού, η οποία χρησιμοποιείται για να προταθεί κατάλληλη τιμή max_alpha για την adaptive history-aware correction. Η βασική συνάρτηση είναι η analyze_history_stability_recommendation:

```
def analyze_history_stability_recommendation(  
    history_csv_file: Any,  
    history_csv_path_text: str,  
    target_date: str,  
    history_correction_days: int,  
    min_history_hours: int,  
    history_consumption_text: str,  
):
```

Η συνάρτηση εξετάζει μόνο ιστορικές τιμές πριν από την ημέρα στόχου. Αυτό είναι απαραίτητο ώστε να μην υπάρχει διαρροή πληροφορίας από την ημέρα που πρόκειται να προβλεφθεί.

Το ιστορικό μπορεί να προέλθει είτε από CSV είτε από χειροκίνητες τιμές:

```
hist, source = _history_dataframe_from_inputs(  
    history_csv_file=history_csv_file,  
    history_csv_path_text=history_csv_path_text,  
    history_consumption_text=history_consumption_text,  
    target_date=target_date,  
    history_days=history_days,  
)
```

Η συνάρτηση ελέγχει το πρόσφατο παράθυρο ιστορικού, υπολογίζει την πληρότητα, συμπληρώνει προσωρινά κενά μόνο για τους διαγνωστικούς υπολογισμούς και δημιουργεί ημερήσια και ωριαία στατιστικά. Μεταξύ άλλων υπολογίζονται η μέση ημερήσια κατανάλωση, η διάμεση ημερήσια κατανάλωση, το εύρος ημερήσιας κατανάλωσης, ο συντελεστής μεταβλητότητας ημερήσιων totals, η σταθερότητα του ωριαίου προφίλ και η σταθερότητα των αιχμών.

Η μέση ημερήσια κατανάλωση υπολογίζεται στο τμήμα:

```
daily = (
    aligned
    .groupby("date", as_index=False)
    .agg(
        daily_kWh=("consumption_filled_Wh", lambda s:
float(pd.to_numeric(s, errors="coerce").sum() / 1000.0)),
        daily_mean_Wh=("consumption_filled_Wh", "mean"),
        daily_peak_Wh=("consumption_filled_Wh", "max"),
        observed_hours=("consumption_Wh", lambda s: int(pd.to_numeric(s,
errors="coerce").notna().sum()))),
        is_weekend=("is_weekend", "max"),
    )
)
```

Στη συνέχεια δημιουργείται συνολικός δείκτης σταθερότητας:

```
base_score = (
    0.30 * daily_score
    + 0.35 * hourly_score
    + 0.20 * peak_score
    + 0.15 * completeness_score
)

stability_score = _clamp01(base_score * (0.85 + 0.15 * day_type_score))
```

Ο δείκτης αυτός συνδυάζει την ημερήσια σταθερότητα, τη σταθερότητα του ωριαίου σχήματος, τη σταθερότητα των αιχμών και την πληρότητα του ιστορικού. Επιπλέον, λαμβάνει υπόψη αν υπάρχουν επαρκείς ημέρες ίδιου τύπου με την ημέρα στόχο, δηλαδή εργάσιμες ή Σαββατοκύριακα.

Με βάση το τελικό stability score, η συνάρτηση προτείνει ένα συντηρητικό recommended_alpha:

```
if completeness_score < 0.85:
    recommended_alpha = 0.05
    category = "Low"
elif stability_score >= 0.93:
    recommended_alpha = 0.60
    category = "Excellent"
elif stability_score >= 0.80:
```

```

recommended_alpha = 0.40
category = "Very High"
elif stability_score >= 0.60:
    recommended_alpha = 0.25
    category = "High"
elif stability_score >= 0.40:
    recommended_alpha = 0.10
    category = "Medium"
else:
    recommended_alpha = 0.05
    category = "Low"

```

Η πρόταση αυτή δεν εφαρμόζεται αυτόματα αν ο χρήστης δεν την επιλέξει. Η συνάρτηση επιστρέφει score, κατηγορία, προτεινόμενο max alpha, αναλυτική εξήγηση και μήνυμα κατάστασης.

Behavioral adjustment

Το UI περιλαμβάνει λειτουργία συμπεριφορικής προσαρμογής. Ο χρήστης μπορεί να δηλώσει ώρες ή χρονικά διαστήματα στα οποία αναμένει διαφορετική κατανάλωση και να εφαρμόσει έναν συντελεστή στις αντίστοιχες προβλέψεις.

Η συνάρτηση `_parse_behavior_hours` αναλύει τις ώρες που εισάγει ο χρήστης:

```

def _parse_behavior_hours(text: str) -> List[int]:
    if text is None:
        return []

    text = str(text).strip()
    if text == "":
        return []

```

Η είσοδος μπορεί να είναι είτε μεμονωμένες ώρες είτε διαστήματα, όπως 18-23. Οι ώρες πρέπει να βρίσκονται στο εύρος 0–23, ενώ τα διαστήματα πρέπει να έχουν σωστή σειρά.

Η εφαρμογή της προσαρμογής γίνεται στη συνάρτηση `_apply_behavior_adjustment_single_df`:

```

def _apply_behavior_adjustment_single_df(
    df: pd.DataFrame,
    hours: List[int],
    factor: float,
    value_col: str = "pred_consumption_Wh",
) -> pd.DataFrame:

```

Η συνάρτηση προσθέτει στήλες `behavior_adjusted` και `behavior_factor` και πολλαπλασιάζει τις προβλέψεις των επιλεγμένων ωρών με τον αντίστοιχο συντελεστή. Η λειτουργία αυτή δεν αλλάζει το μοντέλο και δεν επηρεάζει το backend inference. Είναι post-processing στο επίπεδο της διεπαφής και λειτουργεί ως εργαλείο σεναριακής ανάλυσης.

Συνδυαστική πρόβλεψη και βάρη ensemble

Η συνδυαστική πρόβλεψη βασίζεται σε σταθμισμένο μέσο όρο των τριών οικογενειών μοντέλων. Η συνάρτηση `_resolve_ensemble_weights` ελέγχει αν ο χρήστης χρησιμοποιεί custom βάρη:

```
def _resolve_ensemble_weights(
    use_custom_weights: bool,
    rf_weight_pct: Optional[float],
    xgb_weight_pct: Optional[float],
    lgbm_weight_pct: Optional[float],
) -> Dict[str, float]:
```

Αν δεν χρησιμοποιούνται custom βάρη, επιστρέφονται τα προεπιλεγμένα βάρη. Αν χρησιμοποιούνται, το UI ελέγχει ότι τα βάρη είναι αριθμητικά, μη αρνητικά και ότι αθροίζουν σε 100%:

```
total = rf + xgb + lgbm
if abs(total - 100.0) > 1e-6:
    raise ValueError(f"Τα ensemble weights πρέπει να αθροίζουν σε 100%. Τρέχον άθροισμα: {total:.2f}%.")
```

Η επιλογή των μοντέλων που θα συμμετέχουν στη σύγκριση ή στο ensemble γίνεται από τη συνάρτηση `_comparison_model_ids`:

```
def _comparison_model_ids(mode: str) -> Dict[str, str]:
    if mode == MODE_WITH_HISTORY:
        return {
            "rf": "rf_withhistory_optional",
            "xgb": "xgb_withhistory_optional",
            "lgbm": "lgbm_withhistory_default",
        }

    return {
        "rf": "rf_coldstart_default",
        "xgb": "xgb_coldstart_optional",
        "lgbm": "lgbm_coldstart_optional",
    }
```

Έτσι, στο `with_history` mode το ensemble χρησιμοποιεί τα αντίστοιχα with-history μοντέλα, ενώ στο `no_history` mode χρησιμοποιεί τα αντίστοιχα no-history μοντέλα.

Δημιουργία summaries και υπολειπόμενη κατανάλωση σημερινής ημέρας

Το UI δημιουργεί αναλυτικές περιλήψεις για κάθε πρόβλεψη. Η συνάρτηση `_build_summary_single` δημιουργεί σύνοψη για μία πρόβλεψη:

```
def _build_summary_single(
    out: Dict[str, Any],
    remaining_kwh_today: Optional[float] = None,
    cutoff_iso: Optional[str] = None,
    behavior_text: Optional[str] = None,
```

```
) -> str:
```

Η σύνοψη περιλαμβάνει το ζητούμενο και το τελικό model ID, το mode, το optimization, την πόλη, την ημερομηνία, τη συνολική ημερήσια κατανάλωση, την πηγή καιρού, την πηγή ιστορικού και το αν εφαρμόστηκε history correction.

Για τη σημερινή ημέρα, το UI υπολογίζει και την υπολειπόμενη κατανάλωση από την επόμενη πλήρη ώρα:

```
def _compute_remaining_kwh_today(df_full: pd.DataFrame, target_date: str)
-> Tuple[Optional[float], Optional[str]]:
    try:
        d = _parse_date(target_date)
    except Exception:
        return None, None

    if d != _today_date():
        return None, None
```

Αν η ημερομηνία στόχος είναι η σημερινή, η συνάρτηση κρατά μόνο τις προβλέψεις από την επόμενη πλήρη ώρα και μετά. Αυτό κάνει την εφαρμογή πιο πρακτική για χρήση μέσα στην τρέχουσα ημέρα, καθώς εμφανίζει και το αναμενόμενο υπόλοιπο κατανάλωσης.

Αποθήκευση αποτελεσμάτων σε CSV

Το UI μπορεί να αποθηκεύσει τις προβλέψεις σε CSV. Για μία πρόβλεψη χρησιμοποιείται η `_save_csv_full_day`:

```
def _save_csv_full_day(
    out: Dict[str, Any],
    preds_full: pd.DataFrame,
    temps24: Optional[List[float]],
    remaining_kwh_today: Optional[float],
    cutoff_iso: Optional[str],
) -> Path:
```

Η συνάρτηση προσθέτει στο CSV όχι μόνο τις ωριαίες προβλέψεις, αλλά και μεταδεδομένα όπως model ID, model name, mode, optimization, city, target date, weather source, history source, history correction, flags ημέρας και συνολική ημερήσια κατανάλωση.

Για combined prediction υπάρχει αντίστοιχη συνάρτηση `_save_csv_combined`, ενώ για σύγκριση μοντέλων υπάρχει η `_save_csv_compare`. Έτσι, τα αποτελέσματα του UI μπορούν να αποθηκευτούν και να χρησιμοποιηθούν αργότερα για τεκμηρίωση, ανάλυση ή σύγκριση.

Δημιουργία γραφημάτων

Τα γραφήματα δημιουργούνται με τη matplotlib. Για μία πρόβλεψη χρησιμοποιείται η `_build_single_chart`:

```
def _build_single_chart(df_show: pd.DataFrame):
    if df_show is None or df_show.empty:
        return None
```

```
df = df_show.copy()
df["hour"] = pd.to_datetime(df["timestamp"]).dt.hour
```

Το γράφημα εμφανίζει την ωριαία πρόβλεψη κατανάλωσης και μία οριζόντια γραμμή με τον ημερήσιο μέσο όρο. Για τη σύγκριση μοντέλων χρησιμοποιείται η `_build_compare_chart`, η οποία σχεδιάζει τις προβλέψεις των διαφορετικών μοντέλων στο ίδιο γράφημα.

Τα γραφήματα αυτά βοηθούν τον χρήστη να κατανοήσει όχι μόνο τη συνολική ημερήσια κατανάλωση, αλλά και την κατανομή της κατανάλωσης μέσα στο 24ωρο.

Προετοιμασία payload προς το API

Η συνάρτηση `_prepare_payload` είναι από τις πιο σημαντικές συναρτήσεις του UI:

```
def _prepare_payload(
    use_history: bool,
    model_choice: str,
    optimization: str,
    city: str,
    target_date: str,
    total_floor_area_m2: float,
    residents: float,
    hometype: str,
    urban_rural_class: str,
    num_electric_appliances: Optional[float],
    t_min: Optional[float],
    t_max: Optional[float],
    external_temperature_24h_text: str,
    internal_temperature_24h_text: str,
    history_csv_file: Any,
    history_csv_path_text: str,
    use_default_history_store: bool,
    min_history_hours: int,
    history_consumption_text: str,
    apply_history_correction: bool,
    history_correction_days: int,
    history_correction_max_alpha: float,
):
```

Η συνάρτηση αυτή μετατρέπει τις τιμές των UI components σε JSON payload κατάλληλο για το `/predict` endpoint του API. Αρχικά καθορίζει το model ID και το mode:

```
model_id = _choice_to_model_id(model_choice)
mode = MODE_WITH_HISTORY if use_history else MODE_NO_HISTORY
history_csv_path = _resolve_history_csv_path(history_csv_file,
history_csv_path_text)
```

Στη συνέχεια δημιουργεί το βασικό payload:

```

payload: Dict[str, Any] = {
    "model_id": model_id,
    "mode": mode,
    "optimization": optimization,
    "city": city,
    "target_date": target_date,
    "total_floor_area_m2": float(total_floor_area_m2),
    "residents": float(residents),
    "hometype": hometype,
    "urban_rural_class": str(urban_rural_class),
    "save_csv": False,
    "min_history_hours": int(min_history_hours),
}

```

Αν ο χρήστης έχει δώσει 24ωρη εξωτερική θερμοκρασία, αυτή προστίθεται στο payload. Διαφορετικά, αν υπάρχουν `t_min` και `t_max`, προστίθενται αυτές. Αν υπάρχει 24ωρη εσωτερική θερμοκρασία, προστίθεται επίσης.

Για το `with_history` mode, η συνάρτηση προσθέτει τις παραμέτρους `adaptive correction` και την πηγή ιστορικού:

```

if use_history:
    payload["apply_history_correction"] = bool(apply_history_correction)
    payload["history_correction_days"] = int(history_correction_days)
    payload["history_correction_max_alpha"] = float(history_correction_max_alpha)

    if history_vals is not None:
        payload["history_consumption_Wh"] = history_vals
        payload["use_default_history_store"] = False
        payload["use_proxy_history"] = False
    elif history_csv_path:
        payload["history_csv_path"] = history_csv_path
        payload["use_default_history_store"] = False
        payload["use_proxy_history"] = False
    else:
        payload["use_default_history_store"] = bool(use_default_history_store)
        payload["use_proxy_history"] = bool(use_default_history_store)

```

Με αυτή τη λογική, το UI ακολουθεί την ίδια σειρά προτεραιότητας ιστορικού με το API: χειροκίνητες τιμές, CSV path και τέλος `legacy default store`.

Κλήση του API

Η συνάρτηση `_post_predict` στέλνει το `payload` στο API:

```

def _post_predict(payload: Dict[str, Any]) -> Dict[str, Any]:

```

```

r = requests.post(f"{API_BASE}/predict", json=payload,
timeout=TIMEOUT)

```

Αν το API επιστρέψει σφάλμα, η συνάρτηση προσπαθεί να διαβάσει το μήνυμα σφάλματος. Αν το σφάλμα σχετίζεται με έλλειψη εξωτερικής θερμοκρασίας, επιστρέφει ειδικό `RuntimeError` με πρόθεμα `WEATHER_FALLBACK`, ώστε το UI να εμφανίσει τα πεδία `t_min` και `t_max`:

```

if r.status_code == 422 and _needs_weather_fallback(detail):
    raise RuntimeError(f"WEATHER_FALLBACK::{detail}")

```

Αν η πρόβλεψη είναι επιτυχής, επιστρέφεται το JSON αποτέλεσμα του API.

Λειτουργία Predict

Η λειτουργία Predict υλοποιείται από τη συνάρτηση `do_predict`:

```

def do_predict(
    use_history: bool,
    model_choice: str,
    optimization: str,
    city: str,
    target_date: str,
    ...
):

```

Η συνάρτηση κανονικοποιεί αρχικά το `behavioral adjustment` και επιλέγει την τελική τιμή `history_correction_max_alpha`. Αν ο χρήστης έχει ενεργοποιήσει τη χρήση προτεινόμενης τιμής, τότε χρησιμοποιείται η τιμή που προέκυψε από την ανάλυση σταθερότητας ιστορικού.

Στη συνέχεια δημιουργείται το `payload`, καλείται το API και οι προβλέψεις μετατρέπονται σε `dataframe`:

```

out = _post_predict(payload)
preds_full = _predictions_to_df(out)

```

Αν είναι ενεργό το `behavioral adjustment`, εφαρμόζεται μετά την πρόβλεψη:

```

if behavior_enabled:
    preds_full = _apply_behavior_adjustment_single_df(preds_full,
behavior_hours, behavior_factor_norm)

```

Η συνολική ημερήσια κατανάλωση ενημερώνεται με βάση το τελικό `dataframe` και δημιουργείται σύνοψη, πίνακας και γράφημα. Αν έχει ενεργοποιηθεί η αποθήκευση, το αποτέλεσμα αποθηκεύεται σε CSV.

Λειτουργία Compare

Η λειτουργία Compare υλοποιείται από τη συνάρτηση `do_compare`. Η συνάρτηση επιλέγει τα κατάλληλα `model IDs` για το επιλεγμένο `mode`:

```

mode = MODE_WITH_HISTORY if use_history else MODE_NO_HISTORY
compare_ids = _comparison_model_ids(mode)

```

Στη συνέχεια καλεί το API διαδοχικά για κάθε μοντέλο:

```

for key, model_id in compare_ids.items():

```

```

payload_mid = dict(payload)
payload_mid["model_id"] = model_id
out_mid = _post_predict(payload_mid)
results[key] = out_mid

```

Για κάθε μοντέλο υπολογίζεται η συνολική ημερήσια κατανάλωση και δημιουργείται συγκριτικός πίνακας. Παράλληλα, οι ωριαίες προβλέψεις συγχωνεύονται σε κοινό dataframe ώστε να δημιουργηθεί συγκριτικό γράφημα. Έτσι ο χρήστης μπορεί να συγκρίνει RF, XGB και LGBM στο ίδιο σενάριο πρόβλεψης.

Λειτουργία Combined Prediction

Η λειτουργία Combined Prediction χρησιμοποιεί τα ίδια μοντέλα με τη σύγκριση, αλλά υπολογίζει μία τελική συνδυαστική πρόβλεψη. Αρχικά επιλύονται τα βάρη:

```

weights = _resolve_ensemble_weights(use_custom_weights, rf_weight_pct,
xgb_weight_pct, lgbm_weight_pct)

```

Στη συνέχεια γίνονται κλήσεις στο API για κάθε μοντέλο και οι προβλέψεις συνδυάζονται με σταθμισμένο μέσο όρο. Η τελική πρόβλεψη δεν προκύπτει από νέο μοντέλο που έχει εκπαιδευτεί ξεχωριστά, αλλά από post-processing των προβλέψεων των τριών μοντέλων.

Η λειτουργία αυτή είναι χρήσιμη επειδή επιτρέπει στον χρήστη να συνδυάσει τις συμπεριφορές διαφορετικών αλγορίθμων και να δοκιμάσει εναλλακτικές σταθμίσεις, χωρίς να χρειάζεται νέα εκπαίδευση.

Δομή της γραφικής διεπαφής

Η διεπαφή δημιουργείται με gr.Blocks:

```

with gr.Blocks(title="IDEAL Forecasting UI") as demo:
    gr.Markdown("## IDEAL Load Forecasting – Generic API/UI Demo")
    gr.Markdown(
        "Συμπλήρωσε τα χαρακτηριστικά της κατοικίας και πάτα Predict"
        "για ένα μοντέλο, "
        "Compare για σύγκριση RF / XGB / LGBM ή Combined Prediction"
        "για weighted ensemble. "
    )

```

Στην κορυφή εμφανίζεται η κατάσταση του API και κουμπί ανανέωσης. Στη συνέχεια ο χρήστης επιλέγει αν θα χρησιμοποιηθεί ιστορικό:

```

use_history = gr.Checkbox(
    value=False,
    label="Πρόβλεψη με χρήση ιστορικών δεδομένων",
    info="Αν ενεργοποιηθεί, το API χρησιμοποιεί mode='with_history' και απαιτεί πρόσφατο ιστορικό κατανάλωσης.",
)

```

Η επιλογή αυτή αλλάζει δυναμικά και τα διαθέσιμα μοντέλα στο dropdown. Αν ο χρήστης ενεργοποιήσει το with_history, εμφανίζεται η ομάδα πεδίων ιστορικού και τα μοντέλα αλλάζουν στα διαθέσιμα with-history μοντέλα.

Το UI περιλαμβάνει επίσης πεδία για πόλη, ημερομηνία, εμβαδόν κατοικίας, αριθμό κατοίκων, τύπο κατοικίας, κατηγορία περιοχής, αριθμό ηλεκτρικών συσκευών,

θερμοκρασίες, ιστορικό CSV, adaptive correction, behavioral adjustment, ensemble weights και επιλογή αποθήκευσης CSV.

Στο κάτω μέρος υπάρχουν τα τρία βασικά κουμπιά:

```
btn_predict = gr.Button("Predict", variant="primary")
btn_compare = gr.Button("Compare")
btn_combined = gr.Button("Combined Prediction")
```

Τα αποτελέσματα εμφανίζονται σε summary textbox, πίνακα προβλέψεων, πίνακα θερμοκρασιών, πίνακα σύγκρισης μοντέλων και γράφημα.

Σύνδεση UI events με συναρτήσεις

Στο τέλος του αρχείου ορίζονται οι event handlers. Όταν ο χρήστης ανεβάζει CSV ιστορικού, καλείται η register_history_csv_upload:

```
history_csv_file.upload(
    fn=register_history_csv_upload,
    inputs=[history_csv_file],
    outputs=[history_csv_path_text, history_upload_status],
)
```

Όταν ο χρήστης πατά το κουμπί ανάλυσης ιστορικού, καλείται η analyze_history_stability_recommendation:

```
btn_analyze_history_stability.click(
    fn=analyze_history_stability_recommendation,
    inputs=[
        history_csv_file,
        history_csv_path_text,
        target_date,
        history_correction_days,
        min_history_hours,
        history_consumption_text,
    ],
    outputs=[
        history_stability_score,
        history_stability_category,
        recommended_history_correction_max_alpha,
        history_stability_explanation,
        history_stability_status,
    ],
)
```

Αν αλλάξει το αρχείο ιστορικού, η ημερομηνία στόχος ή οι παράμετροι της ανάλυσης, η προηγούμενη σύσταση max alpha χαρακτηρίζεται ως μη ενημερωμένη. Αυτό αποτρέπει τη χρήση παλιάς σύστασης σε νέο σενάριο.

Τα βασικά inputs και outputs οργανώνονται σε κοινές λίστες:

```

common_inputs = [
    use_history,
    model_choice,
    optimization,
    city,
    target_date,
    total_floor_area_m2,
    residents,
    hometype,
    urban_rural_class,
    num_electric_appliances,
    t_min,
    t_max,
    external_temperature_24h_text,
    internal_temperature_24h_text,
    history_csv_file,
    history_csv_path_text,
    use_default_history_store,
    min_history_hours,
    history_consumption_text,
    apply_history_correction,
    history_correction_days,
    history_correction_max_alpha,
    use_recommended_history_alpha,
    recommended_history_correction_max_alpha,
    enable_behavior_adjustment,
    high_consumption_hours_text,
    behavior_factor,
    use_custom_weights,
    rf_weight_pct,
    xgb_weight_pct,
    lgbm_weight_pct,
    save_csv,
]

```

Με αυτόν τον τρόπο, τα τρία κουμπιά Predict, Compare και Combined Prediction μπορούν να χρησιμοποιούν την ίδια ομάδα εισόδων:

```

btn_predict.click(fn=do_predict,                               inputs=common_inputs,
outputs=common_outputs)

btn_compare.click(fn=do_compare,                               inputs=common_inputs,
outputs=common_outputs)

```

```
btn_combined.click(fn=do_combined,  
outputs=common_outputs)
```

```
inputs=common_inputs,
```

Η εφαρμογή εκκινεί ως Gradio app στο τέλος του αρχείου:

```
if __name__ == "__main__":  
    demo.queue()  
    demo.launch(  
        server_name=UI_HOST,  
        server_port=UI_PORT,  
        share=False,  
        inbrowser=False,  
        show_error=True,  
        quiet=False,  
        show_api=False,  
    )
```

Η χρήση `demo.queue()` επιτρέπει καλύτερη διαχείριση αιτημάτων, ενώ το `demo.launch` εκκινεί τη διεπαφή τοπικά στη διεύθυνση 127.0.0.1:7860.